



Apple Platform Security



August 2026

Contents

Apple Platform Security	5
Intro to Apple platform security	5
Hardware security and biometrics	7
Hardware security overview	7
Apple SoC security	8
The Secure Enclave	10
Optic ID, Face ID, and Touch ID	18
Hardware microphone disconnect	28
Mac on-screen camera indicator light	29
Express Cards with power reserve	29
System security	30
System security overview	30
Secure boot	30
Signed system volume security	53
Secure software updates	55
Background Security Improvements	56
Operating system integrity	58
Device pairing and connection security	62
BlastDoor for Messages and IDS	67
Lockdown Mode security	67
Overview of additional macOS system security capabilities	68
System security for watchOS	78
Random number generation	81
Communicating emergency information using satellites	82
Apple Security Research Device	84
Encryption and Data Protection	86
Encryption and Data Protection overview	86
Quantum-secure cryptography with Apple devices	87

Passcodes and passwords	88
Data Protection	91
FileVault	104
How Apple protects users' personal data	108
Digital signing and encryption	111
App security	113
App security overview	113
App code signing process	114
App security in iOS, iPadOS, and visionOS	116
App security in macOS	119
Supporting extensions	125
Secure features in the Notes app	126
Secure features in the Shortcuts app	127
Services security	128
Services security overview	128
Apple Accounts and Managed Apple Accounts	128
iCloud	131
Passcode and password management	142
Apple Pay	151
Using Apple Wallet	168
iMessage	188
Secure Apple Messages for Business	192
FaceTime security	193
SharePlay security	194
Nearby sharing security	194
Find My	195
Continuity	198
Network security	201
Network security overview	201
TLS security	201
IPv6 security	203
VPN security	204
Wi-Fi security and privacy	205
Bluetooth security	210
Ultra Wideband security in iOS	212
Single Sign-on security	212
AirDrop security	215

Wi-Fi password sharing security	216
Firewall security in macOS	216
Developer kit security	217
Developer kit security overview	217
ARKit security	217
DriverKit security	218
EnergyKit data security	218
HomeKit security	218
NFC & SE Platform security	224
ReplayKit security	227
SiriKit security	228
WidgetKit security	228
Secure device management	230
Secure device management overview	230
Device management	230
Apple Configurator security	245
Screen Time security	245
Glossary	248
Document revision history	253
Document revision history	253
Copyright and trademarks	266

Apple Platform Security

Intro to Apple platform security

Apple designs security into the core of its platforms. Building on the experience of creating the world's most advanced mobile operating system, Apple has created security architectures that address the unique requirements of mobile, watch, desktop, and home.

Every Apple device combines *hardware*, *software*, and *services* designed to work together for maximum security and a transparent user experience in service of the ultimate goal of keeping personal information safe. For example, Apple designed silicon and security hardware powers critical security features, including those that protect both the operating system and third-party apps. Services provide a mechanism for secure and timely software updates, power a protected app ecosystem, and facilitate secure communications and payments. As a result, Apple devices protect not only the device and its data but the entire ecosystem, including everything users do locally on networks, and with key internet services.

Apple designs its products to be simple, intuitive, capable, and above all else, secure. Key security features, such as hardware-based device encryption, can't be turned off by mistake. Other features, such as Face ID and Touch ID, enhance the user experience by making it simpler and more intuitive to secure the device. And because many of these features are enabled by default, users or IT departments don't need to perform extensive configurations.

This documentation provides details about how security technology and features are implemented within Apple platforms. It also helps organizations combine Apple platform security technology and features with their own policies and procedures to meet their specific security needs.

Content is organized into the following topic areas:

- **Hardware security and biometrics:** The silicon and hardware that forms the foundation for security on Apple devices, including Apple silicon, the Secure Enclave, cryptographic engines, and biometric authentication
- **System security:** The integrated hardware and software functions that provide for the safe boot, update, and ongoing operation of Apple operating systems
- **Encryption and Data Protection:** The architecture and design that protects user data if the device is lost or stolen or if an unauthorized person or process attempts to use or modify it
- **App security:** The software and services that provide a safe app ecosystem and enable apps to run securely and without compromising platform integrity

- **Services security:** Apple's services for identification, password management, payments, communications, and finding lost devices
- **Network security:** Industry-standard networking protocols that provide secure authentication and encryption of data in transmission
- **Developer kit security:** Framework "kits" for secure and private management of home and health, as well as extension of Apple device and service capabilities to third-party apps
- **Secure device management:** Methods that allow management of Apple devices, help prevent unauthorized use, and enable remote wipe if a device is lost or stolen

A commitment to security

Apple is committed to helping protect customers with leading privacy and security technologies—designed to safeguard personal information—and comprehensive methods, to help protect corporate data in an enterprise environment. Apple rewards researchers for the work they do to uncover vulnerabilities by offering the Apple Security Bounty. Details of the program and bounty categories are available at <https://security.apple.com/bounty>.

We maintain a dedicated security team to support all Apple products. The team provides security auditing and testing for products, both under development and released. The Apple team also provides security tools and training, and actively monitors for threats and reports of new security issues. Apple is a member of the [Forum of Incident Response and Security Teams \(FIRST\)](#).

Apple continues to push the boundaries of what's possible in security and privacy. It uses custom Apple silicon across the product lineup—from Apple Watch, to iPad and iPhone, to the M-series chips in Mac—powering not only efficient computation but also security. For example, Apple silicon forms the foundation for secure boot, biometric authentication, and Data Protection. In addition, security features powered by Apple silicon—such as Kernel Integrity Protection, Pointer Authentication Codes, and Fast Permission Restrictions—help thwart common types of exploitation. Even if attacker code somehow executes, the damage it can do is dramatically reduced.

To make the most of the extensive security features built into our platforms, organizations are encouraged to review their IT and security policies to ensure that they are taking full advantage of the layers of security technology offered by these platforms.

To learn more about reporting issues to Apple and subscribing to security notifications, see [Report a security or privacy vulnerability](#).

Apple believes privacy is a fundamental human right and has numerous built-in controls and options that allow users to decide how and when apps use their information, as well as what information is being used. To learn more about Apple's approach to privacy, privacy controls on Apple devices, and the Apple privacy policy, see <https://www.apple.com/privacy>.

Note: Unless otherwise noted, this documentation covers the following operating system versions: iOS 26.5, iPadOS 26.5, macOS 26.5, tvOS 26.5, visionOS 26.5, and watchOS 26.5.

Hardware security and biometrics

Hardware security overview

For software to be secure, it needs to rest on hardware that has security built in. That's why Apple devices—with iOS, iPadOS, macOS, tvOS, visionOS, and watchOS—have security capabilities designed into silicon. These capabilities include a CPU that powers system security features, as well as additional silicon that's dedicated to security functions. Security-focused hardware follows the principle of supporting limited and discretely defined functions to minimize attack surface. Such components include a Boot ROM, which forms a hardware root of trust for secure boot, dedicated AES Engines for efficient and secure encryption and decryption, and a *Secure Enclave*. The Secure Enclave is a component on Apple [system on a chip \(SoC\)](#) that's included on all recent iPad, iPhone, Apple TV, Apple Vision Pro, Apple Watch, HomePod devices, and on a Mac with Apple silicon and those with the Apple T2 Security Chip. The Secure Enclave follows the same principle of design as the SoC does, containing its own discrete Boot ROM and the advanced encryption standard (AES) engine. The Secure Enclave also provides the foundation for the secure generation and storage of the keys necessary for encrypting data at rest, and it protects and evaluates the biometric data for Optic ID, Face ID, and Touch ID.

Storage encryption needs to be fast and efficient. At the same time, it can't expose the data (or *keying material*) it uses to establish cryptographic keying relationships. The AES hardware engine solves this problem by performing fast in-line encryption and decryption *as files are written or read*. A special channel from the Secure Enclave provides necessary keying material to the AES Engine without exposing this information to the Application Processor (or CPU) or overall operating system. This helps ensure that the Apple Data Protection and FileVault technologies protect user files without exposing long-lived encryption keys.

Apple has designed secure boot to protect the lowest levels of software against tampering, allowing only trusted operating system software from Apple to load at startup. Secure boot begins in immutable code called the [Boot ROM](#), which is laid down during Apple SoC fabrication and is known as the *hardware root of trust*. On a Mac with a T2 chip, trust for macOS secure boot begins with the T2. (Both the T2 chip and the Secure Enclave also execute their own secure boot processes using their own separate Boot ROM—this is an exact analogue to how the A-series and M series chips boot securely.)

The Secure Enclave also processes eye, face, and fingerprint data from Optic ID, Face ID, and Touch ID sensors in Apple devices. This provides secure authentication while keeping user biometric data private and secure. It also allows users to benefit from the security of longer and more complex passcodes and passwords with, in many situations, the convenience of swift authentication for access or purchases.

Apple SoC security

Apple-designed silicon forms a common architecture across all Apple products and powers iPad, iPhone, Mac, Apple TV, Apple Vision Pro, Apple Watch, and HomePod. For over a decade, Apple's world-class silicon design team has been building and refining Apple systems on a chip (SoCs). The result is a scalable architecture designed for all devices that leads the industry in security capabilities. This common foundation for security features is only possible from a company that designs its own silicon to work with its software.

Apple silicon has been designed and fabricated to specifically enable the system security features detailed below.

Feature	A10	A11, S3	A12-A14 S4-S10	A15-A18	M1	M2-M4	A19 M5
Kernel Integrity Protection	✓	✓	✓	✓	✓	✓	✓
Fast Permission Restrictions	✗	✓	✓	✓	✓	✓	✓
System Coprocessor Integrity Protection	✗	✗	✓	✓	✓	✓	✓
Pointer Authentication Codes	✗	✗	✓	✓	✓	✓	✓
Page Protection Layer	✗	✓	✓	✗ 1	✓ 2	✗	✗
Secure Page Table Monitor	✗	✗	✗	✓	✗	✓ 2	✓ 2
Memory Integrity Enforcement with Enhanced Memory Tagging Extension	✗	✗	✗	✗	✗	✗	✓

1: Secure Page Table Monitor (SPTM) is supported in the A15 or later and M2 or later SOC's and replaces Page Protection Layer on supported platforms.

2: Page Protection Layer (PPL) and Secure Page Table Monitor (SPTM) enforce the execution of signed and trusted code on all platforms with the exception of macOS (because macOS is designed to run any code). All of the other security properties, including the protection of page tables, are present across all supported platforms.

Apple-designed silicon also specifically enables the Data Protection capabilities detailed below.

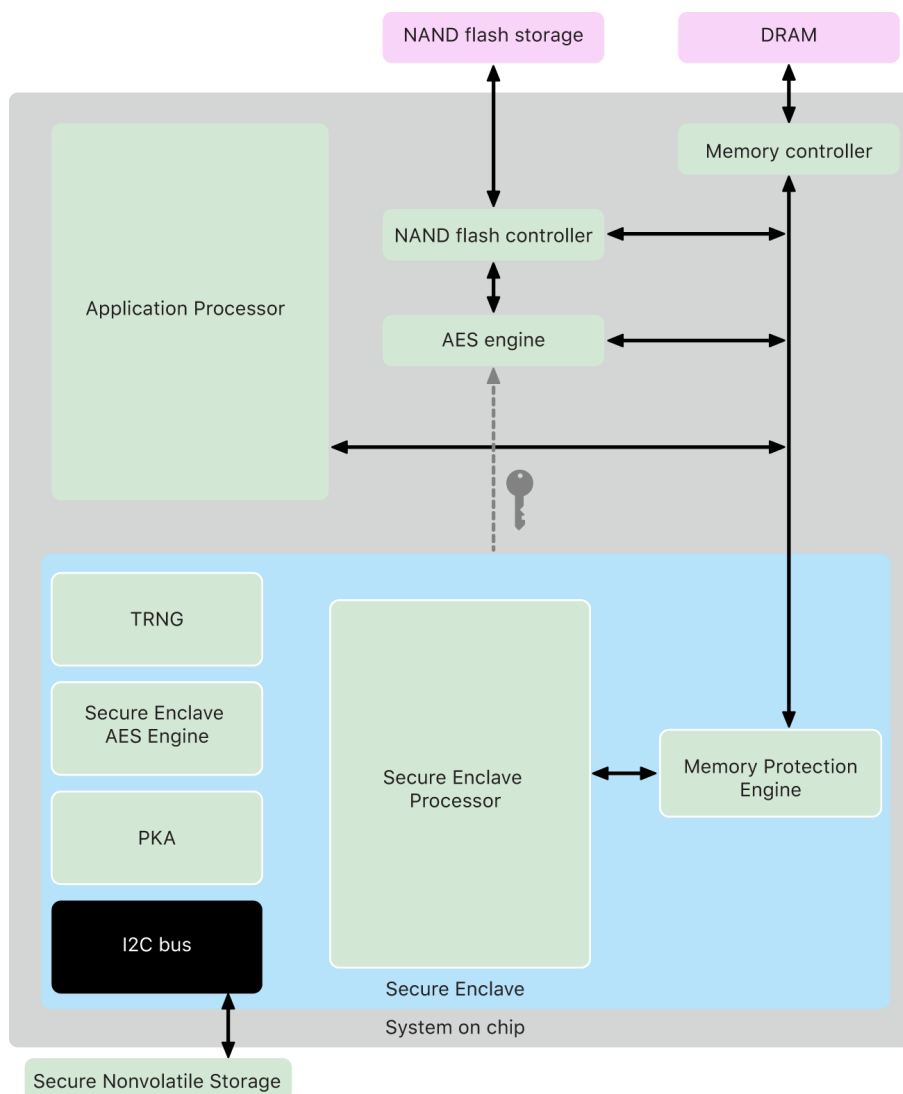
Feature	A10, A11 S3	A12 or later S4 or later M1 or later
Sealed Key Protection (SKP)	✓	✓
recoveryOS - All Data Protection Classes protected	✓	✓
Alternate boots of DFU, Diagnostics, and Update - Class A, B, and C data protected	✗	✓

The Secure Enclave

The Secure Enclave is a dedicated secure subsystem in the latest versions of iPad, iPhone, Mac, Apple TV, Apple Vision Pro, Apple Watch, and HomePod.

Overview

The Secure Enclave is a dedicated secure subsystem integrated into Apple (SoC). The Secure Enclave is isolated from the main processor to provide an extra layer of security and is designed to keep sensitive user data secure even when the Application Processor kernel becomes compromised. It follows the same design principles as the SoC does—a Boot ROM to establish a hardware root of trust, an AES Engine for efficient and secure cryptographic operations, and protected memory. Although the Secure Enclave doesn't include storage, it has a mechanism to store information securely on attached storage separate from the NAND flash storage that's used by the Application Processor and operating system.



The Secure Enclave is a hardware feature of most versions of iPad, iPhone, Mac, Apple TV, Apple Vision Pro, Apple Watch, and HomePod—namely:

- All iPad models starting with iPad Air or later
- All iPhone models starting with iPhone 5s or later
- All Mac computers with Apple silicon
- MacBook Pro computers with Touch Bar (2016 and 2017) that contain the Apple T1 Chip
- All Intel-based Mac computers that contain the Apple T2 Security Chip
- All Apple TV models starting with Apple TV HD or later
- All Apple Watch models starting with Apple Watch Series 1 or later
- Apple Vision Pro
- All HomePod models

The Secure Enclave Processor

The Secure Enclave Processor provides the main computing power for the Secure Enclave. To provide the strongest isolation, the Secure Enclave Processor is dedicated solely for Secure Enclave use. This helps prevent side-channel attacks that depend on malicious software sharing the same execution core as the target software under attack.

The Secure Enclave Processor runs an Apple-customized version of the L4 microkernel. It's designed to operate efficiently at a lower clock speed that helps to protect it against clock and power attacks. The Secure Enclave Processor, starting with the A11 and S4, includes a memory-protected engine and encrypted memory with anti-replay capabilities, secure boot, a dedicated random number generator, and its own AES Engine.

Memory Protection Engine

The Secure Enclave operates from a dedicated region of the device's DRAM memory. Multiple layers of protection isolate the Secure Enclave protected memory from the Application Processor.

When the device starts up, the Secure Enclave Boot ROM generates a random ephemeral memory protection key for the Memory Protection Engine. Whenever the Secure Enclave writes to its dedicated memory region, the Memory Protection Engine encrypts the block of memory using AES in MAC XEX (xor-encrypt-xor) mode, and calculates a Cipher-based Message Authentication Code (CMAC) authentication tag for the memory. The Memory Protection Engine stores the authentication tag alongside the encrypted memory. When the Secure Enclave reads the memory, the Memory Protection Engine verifies the authentication tag. If the authentication tag matches, the Memory Protection Engine decrypts the block of memory. If the tag doesn't match, the Memory Protection Engine signals an error to the Secure Enclave. After a memory authentication error, the Secure Enclave stops accepting requests until the system is rebooted.

Starting with the Apple A11 and S4 SoCs, the Memory Protection Engine adds replay protection for Secure Enclave memory. To help prevent replay of security-critical data, the Memory Protection Engine stores a unique one-off number, called an *anti-replay value*, for the block of memory alongside the authentication tag. The anti-replay value is used as an additional refinement for the CMAC authentication tag. The anti-replay values for all memory blocks are protected using an integrity tree rooted in dedicated SRAM within the Secure Enclave. For writes, the Memory Protection Engine *updates* the anti-replay value and each level of the integrity tree up to the SRAM. For reads, the Memory Protection Engine *verifies* the anti-replay value and each level of the integrity tree up to the SRAM. Anti-replay value mismatches are handled similarly to authentication tag mismatches.

On Apple A14, M1, or later SoCs, the Memory Protection Engine supports two ephemeral memory protection keys. The first is used for data private to the Secure Enclave, and the second is used for data shared with the Secure Neural Engine.

The Memory Protection Engine operates inline and transparently to the Secure Enclave. The Secure Enclave reads and writes memory as if it were regular unencrypted DRAM, whereas an observer outside the Secure Enclave sees only the encrypted and authenticated version of the memory. The result is strong memory protection without performance or software complexity tradeoffs.

The Secure Enclave Boot ROM

The Secure Enclave includes a dedicated Secure Enclave Boot ROM. Like the Application Processor Boot ROM, the Secure Enclave Boot ROM is immutable code that establishes the hardware root of trust for the Secure Enclave.

On system startup, [iBoot](#) assigns a dedicated region of memory to the Secure Enclave. Before using the memory, the Secure Enclave Boot ROM initializes the Memory Protection Engine to provide cryptographic protection of the Secure Enclave protected memory.

The Application Processor then sends the [sepOS](#) image to the Secure Enclave Boot ROM. After copying the sepOS image into the Secure Enclave protected memory, the Secure Enclave Boot ROM checks the cryptographic hash and signature of the image to verify that the sepOS is authorized to run on the device. If the sepOS image is properly signed to run on the device, the Secure Enclave Boot ROM transfers control to sepOS. If the signature isn't valid, the Secure Enclave Boot ROM is designed to prevent any further use of the Secure Enclave until the next chip reset.

On Apple A10 or later SoCs, the Secure Enclave Boot ROM locks a hash of the sepOS into a register dedicated to this purpose. The Public Key Accelerator uses this hash for operating-system-bound (OS-bound) keys.

The Secure Enclave Boot Monitor

On Apple A13 or later SoCs, the Secure Enclave includes a Boot Monitor designed to ensure stronger integrity on the hash of the booted sepOS.

At system startup, the Secure Enclave Processor's [System Coprocessor Integrity Protection \(SCIP\)](#) configuration helps prevent the Secure Enclave Processor from executing any code other than the Secure Enclave Boot ROM. The Boot Monitor helps prevent the Secure Enclave from modifying the SCIP configuration directly. To make the loaded sepOS executable, the Secure Enclave Boot ROM sends the Boot Monitor a request with the address and size of the loaded sepOS. On receipt of the request, the Boot Monitor resets the Secure Enclave Processor, hashes the loaded sepOS, updates the SCIP settings to allow execution of the loaded sepOS, and starts execution within the newly loaded code. As the system continues booting, this same process is used whenever new code is made executable. Each time, the Boot Monitor updates a running hash of the boot process. The Boot Monitor also includes critical security parameters in the running hash.

When boot completes, the Boot Monitor finalizes the running hash and sends it to the Public Key Accelerator to use for OS-bound keys. This process is designed to avoid bypassing operating system keybinding even with a vulnerability in the Secure Enclave Boot ROM.

True Random Number Generator

The True Random Number Generator (TRNG) is used to generate secure random data. The Secure Enclave uses the TRNG whenever it generates a random cryptographic key, random key seed, or other entropy. The TRNG is based on multiple ring oscillators post processed with CTR_DRBG (an algorithm based on block ciphers in Counter Mode).

Root Cryptographic Keys

The Secure Enclave includes a unique ID (UID) root cryptographic key. The UID is unique to each individual device and isn't related to any other identifier on the device.

A randomly generated UID is fused into the SoC at manufacturing time. Starting with A9 SoCs, the UID is generated by the Secure Enclave TRNG during manufacturing and written to the fuses using a software process that runs entirely in the Secure Enclave. This process protects the UID from being visible outside the device during manufacturing and therefore isn't available for access or storage by Apple or any of its suppliers.

sepOS uses the UID to protect device-specific secrets. The UID allows data to be cryptographically tied to a particular device. For example, the key hierarchy protecting the file system includes the UID, so if the internal SSD storage is physically moved from one device to another, the files are inaccessible. Other protected device-specific secrets include Optic ID, Face ID, or Touch ID data. On a Mac, only fully internal storage linked to the AES Engine receives this level of encryption. For example, neither external storage devices connected over USB nor PCIe-based storage added to the 2019 Mac Pro are encrypted in this fashion.

The Secure Enclave also has a device group ID (GID), which is common to all devices that use a given SoC (for example, all devices using the Apple A15 SoC share the same GID).

The UID and GID aren't available through [Joint Test Action Group \(JTAG\)](#) or other debugging interfaces.

The Secure Enclave AES Engine

The Secure Enclave AES Engine is a hardware block used to perform symmetric cryptography based on the AES cipher. The AES Engine is designed to resist leaking information by using timing and Static Power Analysis (SPA). Starting with the A9 SoC, the AES Engine also includes Dynamic Power Analysis (DPA) countermeasures.

The AES Engine supports hardware and software keys. Hardware keys are derived from the Secure Enclave UID or GID. These keys stay within the AES Engine and aren't made visible even to sepOS software. Although software can request encryption and decryption operations with hardware keys, it can't extract the keys.

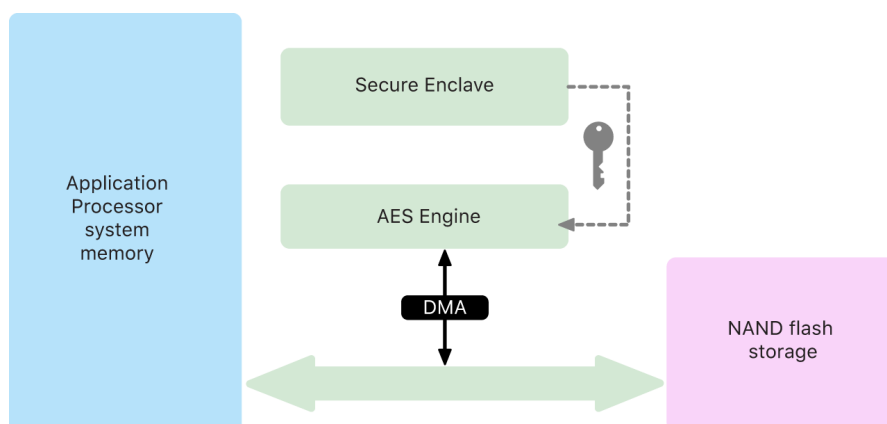
On Apple A10 and newer SoCs, the AES Engine includes lockable seed bits that diversify keys derived from the UID or GID. This allows data access to be conditioned on the device's mode of operation. For example, lockable seed bits are used to deny access to password-protected data when booting from Device Firmware Update (DFU) mode. For more information, see [Passcodes and passwords](#).

AES Engine

Every Apple device with a Secure Enclave also has a dedicated AES-256 crypto engine (the AES Engine) built into the direct memory access (DMA) path between the NAND (nonvolatile) flash storage and main system memory, making file encryption highly efficient. On A9 or later A-series processors, the flash storage subsystem is on an isolated bus that's granted access only to memory containing user data through the DMA crypto engine.

At boot time, sepOS generates an ephemeral wrapping key using the TRNG. The Secure Enclave transmits this key to the AES Engine using dedicated wires, designed to prevent it from being accessed by any software outside the Secure Enclave. sepOS can then use the ephemeral wrapping key to wrap file keys for use by the Application Processor file-system driver. When the file-system driver reads or writes a file, it sends the wrapped key to the AES Engine, which unwraps the key. The AES Engine never exposes the unwrapped key to software.

Note: The AES Engine is a separate component from both the Secure Enclave and the Secure Enclave AES Engine, but its operation is closely tied to the Secure Enclave, as shown below.



Public Key Accelerator

The Public Key Accelerator (PKA) is a hardware block used to perform asymmetric cryptography operations. The PKA supports RSA and Elliptic Curve Cryptography (ECC) signing and encryption algorithms. The PKA is designed to resist leaking information using timing and side-channel attacks such as SPA and DPA.

The PKA supports software and hardware keys. Hardware keys are derived from the Secure Enclave UID or GID. These keys stay within the PKA and aren't made visible even to sepOS.

Starting with A13 SoCs, the PKA's encryption implementations have been proved to be mathematically correct using formal verification techniques.

On Apple A10 or later SoCs, the PKA supports OS-bound keys, also referred to as [Sealed Key Protection \(SKP\)](#). These keys are generated using a combination of the device's UID and the hash of the sepOS running on the device. The hash is provided by the Secure Enclave Boot ROM, or by the Secure Enclave Boot Monitor on Apple A13 or later SoCs. These keys are also used to:

- Verify the sepOS version when making requests to certain Apple services.
- Improve the security of passcode-protected data by helping to prevent access to keying material if critical changes are made to the system without user authorization.

Secure nonvolatile storage

The Secure Enclave is equipped with a dedicated secure nonvolatile storage device. The secure nonvolatile storage is connected to the Secure Enclave using a dedicated I2C bus, so that it can only be accessed by the Secure Enclave. All user data encryption keys are rooted in entropy stored in the Secure Enclave nonvolatile storage.

In devices with A12, S4, or later SoCs, the Secure Enclave is paired with a Secure Storage Component for entropy storage. The Secure Storage Component is itself designed with immutable ROM code, a hardware random number generator, a per-device unique cryptographic key, cryptography engines, and physical tamper detection. The Secure Enclave and Secure Storage Component communicate using an encrypted and authenticated protocol that provides exclusive access to the entropy.

Devices first released in Fall 2020 or later are equipped with a 2nd-generation Secure Storage Component, which adds counter lockboxes. Each counter lockbox stores a 128-bit salt, a 128-bit passcode verifier, an 8-bit counter, and an 8-bit maximum attempt value. Access to the counter lockboxes is through an encrypted and authenticated protocol.

Counter lockboxes hold the entropy needed to unlock passcode-protected user data. To access the user data, the paired Secure Enclave needs to derive the correct passcode entropy value from the user's passcode and the Secure Enclave's UID. The user's passcode can't be learned using unlock attempts sent from a source other than the paired Secure Enclave. If the passcode attempt limit is exceeded (for example, 10 attempts on iPhone), the passcode-protected data is erased completely by the Secure Storage Component.

To create a counter lockbox, the Secure Enclave sends the Secure Storage Component the passcode entropy value and the maximum attempt value. The Secure Storage Component generates the salt value using its random number generator. It then derives a passcode verifier value and a lockbox entropy value from the provided passcode entropy, the Secure Storage Component's unique cryptographic key, and the salt value. The Secure Storage Component initializes the counter lockbox with a count of 0, the provided maximum attempt value, the derived passcode verifier value, and the salt value. The Secure Storage Component then returns the generated lockbox entropy value to the Secure Enclave.

To retrieve the lockbox entropy value from a counter lockbox later, the Secure Enclave sends the Secure Storage Component the passcode entropy. The Secure Storage Component first increments the counter for the lockbox. If the incremented counter exceeds the maximum attempt value, the Secure Storage Component completely erases the counter lockbox. If the maximum attempt count hasn't been reached, the Secure Storage Component attempts to derive the passcode verifier value and lockbox entropy value with the same algorithm used to create the counter lockbox. If the derived passcode verifier value matches the stored passcode verifier value, the Secure Storage Component returns the lockbox entropy value to the Secure Enclave and resets the counter to 0.

The keys used to access password-protected data are rooted in the entropy stored in counter lockboxes. For more information, see [Data Protection overview](#).

The secure nonvolatile storage is used for all anti-replay services in the Secure Enclave. Anti-replay services on the Secure Enclave are used for revocation of data over events that mark anti-replay boundaries including, but not limited to, the following:

- Passcode change
- Turning on, off, or resetting Optic ID, Face ID, or Touch ID
- Adding or removing an eye, face, or fingerprint, respectively
- Adding or removing an Apple Pay card
- Erase All Content and Settings

On architectures that don't feature a Secure Storage Component, electrically erasable programmable read-only memory (EEPROM) is utilized to provide secure storage services for the Secure Enclave. Just like the Secure Storage Components, the EEPROM is attached and accessible only from the Secure Enclave, but it doesn't:

- Contain dedicated hardware security features.
- Guarantee exclusive access to entropy (aside from its physical attachment characteristics) or counter lockbox functionality.

The Secure Neural Engine

On Apple Vision Pro, the Secure Neural Engine converts images into a mathematical representation of a user's eyes. On devices with Face ID (not Touch ID), the Secure Neural Engine converts 2D images and depth maps into a mathematical representation of a user's face.

On Apple A11 through A13 SoCs, the Secure Neural Engine is integrated into the Secure Enclave. The Secure Neural Engine uses DMA for high performance. An input-output memory management unit (IOMMU) under the sepOS kernel's control limits this direct access to authorized memory regions.

On Apple A14, M1, or later SoCs, the Secure Neural Engine is implemented as a secure mode in the Application Processor's Neural Engine. A dedicated hardware security controller switches between Application Processor and Secure Enclave tasks, resetting Neural Engine state on each transition to keep Optic ID, Face ID, or Touch ID data secure. A dedicated engine applies memory encryption, authentication, and access control. At the same time, it uses a separate cryptographic key and memory range to limit the Secure Neural Engine to authorized memory regions.

Power and clock monitors

All electronics are designed to operate within a limited voltage and frequency envelope. When operated outside this envelope, the electronics can malfunction and then security controls may be bypassed. To help ensure that the voltage and frequency stay in a safe range, the Secure Enclave is designed with monitoring circuits. These monitoring circuits are designed to have a much larger operating envelope than the rest of the Secure Enclave. If the monitors detect an illegal operating point, the clocks in the Secure Enclave automatically stop and don't restart until the next SoC reset.

Secure Enclave feature summary

A12, A13, S4, and S5 products first released in Fall 2020 have a 2nd generation Secure Storage Component, whereas while earlier products based on these SoCs have a 1st generation Secure Storage Component.

SoC	Memory Protection Engine	Secure Storage	AES Engine	PKA
A8	Encryption and authentication	EEPROM	✓	✗
A9	Encryption and authentication	EEPROM	DPA protection	✓
A10	Encryption and authentication	EEPROM	DPA protection and lockable seed bits	OS-bound keys
A11	Encryption, authentication, and replay prevention	EEPROM	DPA protection and lockable seed bits	OS-bound keys
A12 (Apple devices released before Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 1	DPA protection and lockable seed bits	OS-bound keys
A12 (Apple devices released after Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys
A13 (Apple devices released before Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 1	DPA protection and lockable seed bits	OS-bound keys and Boot Monitor
A13 (Apple devices released after Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys and Boot Monitor

SoC	Memory Protection Engine	Secure Storage	AES Engine	PKA
A14 or later	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys and Boot Monitor
S3	Encryption and authentication	EEPROM	DPA protection and lockable seed bits	✓
S4	Encryption, authentication, and replay prevention	Secure Storage Component gen 1	DPA protection and lockable seed bits	OS-bound keys
S5 (Apple devices released before Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 1	DPA protection and lockable seed bits	OS-bound keys
S5 (Apple devices released after Fall 2020)	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys
S6 or later	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys
T2	Encryption and authentication	EEPROM	DPA protection and lockable seed bits	OS-bound keys
M1 or later	Encryption, authentication, and replay prevention	Secure Storage Component gen 2	DPA protection and lockable seed bits	OS-bound keys and Boot Monitor

Optic ID, Face ID, and Touch ID

Biometric security

Passcodes and passwords are essential to the security of Apple devices. At the same time, users require convenient access to their devices, often more than a hundred times a day. Biometric authentication provides a way to retain the security of a strong passcode—or even strengthen the passcode or password because it doesn’t need to be entered manually—while providing the convenience of swiftly unlocking with a finger press or glance. Optic ID, Face ID, and Touch ID don’t replace a passcode or password, but in most situations they do make access faster.

Apple's biometric security architecture relies on a strict separation of responsibilities between the biometric sensor and the Secure Enclave, and a secure connection between the two. The sensor captures the biometric image and securely transmits it to the Secure Enclave. During enrollment, the Secure Enclave processes, encrypts, and stores the corresponding Optic ID, Face ID, and Touch ID template data. During matching, the Secure Enclave compares incoming data from the biometric sensor against the stored templates to determine whether to unlock the device or respond that a match is valid (for Apple Pay, in-app, and other uses of Optic ID, Face ID, and Touch ID). The architecture supports devices that include both the sensor and Secure Enclave (such as iPad, iPhone, many Mac computers, and Apple Vision Pro), as well as the ability to physically separate the sensor into a peripheral that's then securely paired to the Secure Enclave in a Mac with Apple silicon.

Optic ID security

Optic ID provides intuitive and secure authentication that uses the uniqueness of a person's iris, made possible by the high-performance eye-tracking system of LEDs and infrared cameras in Apple Vision Pro. It uses a spatiotemporally modulated eye-safe near-infrared light to illuminate the eye so that Apple Vision Pro eye cameras capture images of the iris. Image data is sent to and processed on the Secure Enclave and Secure Neural Engine, where it's transformed into a mathematical representation for enrollment.

Optic ID is designed to:

- Work with prescription soft contact lenses and ZEISS Optical Inserts for users who require vision correction.
- Confirm user attention using gaze tracking and provide robust authentication with a low false-match rate.
- Mitigate both digital and physical spoofing.

When a user unlocks their Apple Vision Pro using their eyes, Optic ID uses neural networks to determine matching and antispooofing. It automatically adapts to changes in conditions by updating the user's enrolled template after each successful authentication. This helps improve Optic ID performance in various lighting conditions, under which the visible portion of the iris and the size of the pupil might vary.

When a user unlocks Apple Vision Pro, the Optic ID icon is prominently displayed in the center of the user's line of sight. After successful unlock (or during an app authentication), Optic ID continually tracks the user's eye area with the eye cameras in Apple Vision Pro. While the user continues to wear the device, if the user needs to authenticate again, this continual tracking is used by Optic ID for authentication.

Face ID security

With a glance, Face ID securely unlocks supported Apple devices and is designed to confirm user attention, provide robust authentication with a low false-match rate, and mitigate both digital and physical spoofing. It provides intuitive and secure authentication enabled by the TrueDepth camera system, which uses advanced technologies to accurately map the geometry of a user's face. Face ID uses neural networks for determining attention, matching, and antispooofing, so a user can unlock their phone with a glance, even with a mask on when using supported devices. Face ID automatically adapts to changes in appearance, and carefully safeguards the privacy and security of a user's biometric data.

The TrueDepth camera automatically looks for the user's face when the user wakes an Apple device that features Face ID (by raising it or tapping the screen), as well as when those devices attempt to authenticate the user to display an incoming notification or when a supported app requests Face ID authentication. When a face is detected, Face ID confirms attention and intent to unlock by detecting that the user's eyes are open and their attention is directed at their device; for accessibility, the Face ID attention check is turned off when VoiceOver is activated and, if required, can be turned off separately. Attention detection is always required when using Face ID with a mask.

After the TrueDepth camera confirms the presence of an attentive face, it projects and reads thousands of infrared dots to form a depth map of the face along with a 2D infrared image. This data is used to create a sequence of 2D images and depth maps, which are digitally signed and sent to the Secure Enclave. To counter both digital and physical spoofs, the TrueDepth camera randomizes the sequence of 2D images and depth map captures, and projects a device-specific random pattern. A portion of the Secure Neural Engine—protected within the Secure Enclave—transforms this data into a mathematical representation and compares that representation to the enrolled facial data. This enrolled facial data is itself a mathematical representation of the user's face captured across a variety of poses.

Touch ID security

Touch ID is the fingerprint sensing system that makes secure access to supported Apple devices faster and easier. This technology reads fingerprint data from any angle and learns more about a user's fingerprint over time, with the sensor continuing to expand the fingerprint map as additional overlapping nodes are identified with each use.

Apple devices with a Touch ID sensor can be unlocked using a fingerprint. Touch ID doesn't replace the need for a device passcode or user password, which is still required after device startup, restart, or logout (on a Mac). In some apps, Touch ID can also be used in place of a device passcode or user password; for example, to unlock:

- Password-protected notes in the Notes app
- Keychain-protected websites
- Supported app passwords

However, a device passcode or user password is always required in some scenarios (for example, to change an existing device passcode or user password or to remove existing fingerprint enrollments or create new ones).

When the fingerprint sensor detects the touch of a finger, it initiates the advanced imaging array to scan the finger and sends the scan to the Secure Enclave. The channel used to secure this connection varies, depending on whether the Touch ID sensor is built into the device with the Secure Enclave or is located in a separate peripheral.

While the fingerprint scan is being vectorized for analysis, the raster scan is temporarily stored in encrypted memory within the Secure Enclave and then it's discarded. The analysis uses subdermal [ridge flow angle mapping](#), a lossy process that discards "finger minutiae data" that would be required to reconstruct the user's actual fingerprint. During enrollment, the resulting map of nodes is stored in an encrypted format that can be read only by the Secure Enclave as a template to compare against for future matches, but without any identity information. This data never leaves the device. It's not sent to Apple, nor is it included in device backups.

Built-in Touch ID channel security

Communication between the Secure Enclave and the built-in Touch ID sensor takes place over a serial peripheral interface bus. The processor forwards the data to the Secure Enclave but can't read it. It's encrypted and authenticated with a session key that's negotiated using a shared key provisioned for each Touch ID sensor and its corresponding Secure Enclave at the factory. For every Touch ID sensor, the shared key is strong, random, and different. The session key exchange uses AES [key wrapping](#), with both sides providing a random key that establishes the session key and uses transport encryption that provides both authentication and confidentiality (using AES-CCM).

Magic Keyboard with Touch ID

The Magic Keyboard with Touch ID (and the Magic Keyboard with Touch ID and Numeric Keypad) provides a Touch ID sensor in an external keyboard that can be used with any Mac with Apple silicon. The Magic Keyboard with Touch ID performs the role of the biometric sensor; it doesn't store biometric templates, perform biometric matching, or enforce security policies (for example, having to enter the password after 48 hours without an unlock). The Touch ID sensor in the Magic Keyboard with Touch ID needs to be securely paired to the Secure Enclave on the Mac before it can be used, and then the Secure Enclave performs the enrollment and matching operations and enforces security policies in the same way it would for a built-in Touch ID sensor. Apple performs the pairing process in the factory for a Magic Keyboard with Touch ID that's shipped with a Mac. Pairing can also be performed by the user if needed. A Magic Keyboard with Touch ID can be securely paired with only one Mac at a time, but a Mac can maintain secure pairings with up to five different Magic Keyboard with Touch ID keyboards.

The Magic Keyboard with Touch ID and built-in Touch ID sensors are compatible. If a finger that was enrolled on a built-in Mac Touch ID sensor is presented on a Magic Keyboard with Touch ID, the Secure Enclave in the Mac successfully processes the match and vice versa.

To support secure pairing and thus communication between the Mac Secure Enclave and the Magic Keyboard with Touch ID, the keyboard is equipped with a hardware Public Key Accelerator (PKA) block, to provide attestation, and with hardware-based keys, to perform the necessary cryptographic processes.

Secure pairing

Before a Magic Keyboard with Touch ID can be used for Touch ID operations, it needs to be securely paired to the Mac. To pair, the Secure Enclave on the Mac and the PKA block in the Magic Keyboard with Touch ID exchange public keys, rooted in the trusted Apple certificate authority (CA), and they use hardware-held attestation keys and [Elliptic Curve Diffie-Hellman Exchange Ephemeral \(ECDHE\)](#) to securely attest to their identity. On the Mac, this data is protected by the Secure Enclave; on the Magic Keyboard with Touch ID, this data is protected by the PKA block. After secure pairing, all Touch ID data communicated between the Mac and the Magic Keyboard with Touch ID is encrypted by AES-GCM with a key length of 256 bits, and with ephemeral ECDH keys using NIST P-256 curve based on the stored identities. For more information on using the keyboard in wireless mode, see [Bluetooth security](#).

Secure intent to pair

To perform some Touch ID operations for the first time, such as enrolling a new fingerprint, the user needs to physically confirm their intent to use a Magic Keyboard with Touch ID with the Mac. Physical intent is confirmed by pressing twice on the Mac power button when indicated by the user interface, or by successfully matching a fingerprint that had previously been enrolled with the Mac. For more information, see [Secure intent and connections to the Secure Enclave](#).

Apple Pay transactions can be authorized with a Touch ID match or by entering the macOS user password and pressing twice on the Touch ID button on the Magic Keyboard with Touch ID. The latter allows the user to confirm physical intent even without a Touch ID match.

Magic Keyboard with Touch ID channel security

To help ensure a secure communication channel between the Touch ID sensor in the Magic Keyboard with Touch ID and Secure Enclave on the paired Mac, the following are required:

- The secure pairing between the Magic Keyboard with Touch ID PKA block and the Secure Enclave as described above.
- A secure channel, between the Magic Keyboard with Touch ID sensor and its PKA block.

The secure channel between the Magic Keyboard with Touch ID sensor and its PKA block is established in the factory by using a unique key shared between the two. For Mac computers with Touch ID built-in, this is the same technique used to create the secure channel between the Secure Enclave on the Mac and its built-in sensor.

Optic ID, Face ID, Touch ID, passcodes, and passwords

To use Optic ID, Face ID, or Touch ID, the user needs to set up their device so that a passcode or password is required to unlock it. When one of them detects a successful match, the user's device unlocks without asking for the device passcode or password. This makes using a longer, more complex passcode or password far more practical because the user doesn't need to enter it as frequently. Optic ID, Face ID, and Touch ID don't replace the user's passcode or password; instead, they provide access to the device within thoughtful boundaries and time constraints. This is important because a strong passcode or password forms the foundation for how a user's iPad, iPhone, Mac, Apple Watch, or Apple Vision Pro cryptographically protects that user's data.

When a device passcode or password is required

Users can use their passcode or password anytime instead of Optic ID, Face ID, or Touch ID, but there are situations where biometrics aren't permitted. The following security-sensitive operations always require entry of a passcode or password:

- Updating the software
- Erasing the device
- Viewing or changing passcode settings
- Installing configuration profiles
- Unlocking the Privacy & Security pane in System Settings (macOS 13 or later) on Mac

- Unlocking the Security & Privacy pane in System Preferences (macOS 12 or earlier) on Mac
- Unlocking the Users & Groups pane in System Settings (macOS 13 or later) on Mac (if FileVault is turned on)
- Unlocking the Users & Groups pane in System Preferences (macOS 12 or earlier) on Mac (if FileVault is turned on)

A passcode or password is also required if the device is in any of the following states:

- The device has just been turned on or restarted (doesn't apply to Apple Vision Pro if the "Nearby iPhone Enables Optic ID" feature is turned on).
- The user has logged out of their Mac account (or hasn't yet logged in).
- The user hasn't unlocked their device for more than 48 hours.
- The user hasn't used their passcode or password to unlock their device for 156 hours (six and a half days), and the user hasn't used biometric authentication to unlock their device in four hours.
- The device has received a remote lock command.
- The user exited power off/Emergency SOS by pressing and holding either volume button and the Sleep/Wake button simultaneously for two seconds and then pressing Cancel.
- There were five unsuccessful biometric authentication match attempts (though for usability, the device might offer entering a passcode or password instead of using biometrics after a smaller number of failures).

When Face ID with a mask is enabled on an iPhone, it's available for the next 6.5 hours after one of the following user actions:

- Successful Face ID match attempt, even with or without a mask covering the nose and mouth
- Device passcode validation
- Device unlock with Apple Vision Pro
- Device unlock with Apple Watch

Any of these actions extends the period an additional 6.5 hours when performed.

When Optic ID, Face ID, or Touch ID is enabled on an iPad, iPhone, Mac laptop with Touch ID, or Apple Vision Pro, the device immediately locks:

- When the Sleep/Wake button is pressed (if applicable).
- Every time it goes to sleep.

Optic ID, Face ID, and Touch ID require a successful match—or optionally, use of the passcode or password—at every wake.

The probability that a random person in the population could unlock a user's iPad, iPhone, or Apple Vision Pro is less than 1 in 1 million with Optic ID or Face ID—including when Face ID with a mask is turned on. For a user's iPad, iPhone, Mac models with Touch ID, and those paired with a Magic Keyboard with Touch ID, it's less than 1 in 50,000. This probability increases with multiple enrolled fingerprints (up to 1 in 10,000 with five fingerprints) or appearances for Face ID (up to 1 in 500,000 with two appearances). For additional protection, Optic ID, Face ID, and Touch ID allow only five unsuccessful match attempts before a passcode or password is required to obtain access to the user's device or account. With Face ID, the probability of a false match is higher for:

- Twins and siblings who look like the user
- Children under the age of 13 (because their distinct facial features may not have fully developed)

The probability is further increased in these two cases when Face ID with a mask covering the nose and mouth is used. If a user is concerned about a false match, Apple recommends using a passcode to authenticate.

Optic ID matching security

Optic ID matches against detailed iris structure in the near-infrared domain, which reveals highly unique patterns independent of iris pigmentation. It's designed to protect against spoofing through the use of sophisticated neural networks that analyze the authenticity of the iris and surrounding region.

While a user is setting up Optic ID, spatiotemporally modulated eye-safe near-infrared light illuminates the eye, so that Apple Vision Pro eye cameras can capture images of the iris. This iris image data is sent to and processed on the Secure Enclave and Secure Neural Engine, where it's transformed into a mathematical representation for enrollment. When the user unlocks or authenticates with Optic ID, the match attempt uses the same process to compare the iris to the enrolled biometric data to determine whether there's a match.

Apple developed the neural networks and tuned algorithms for iris recognition using over a billion images. Studies conducted with the participants' informed consent are used to collect those near-infrared domain images. Apple worked with participants from around the world to include a representative group of people accounting for gender, age, ethnicity, and other factors. The studies were augmented as needed to provide a high degree of accuracy for a diverse range of users.

Optic ID works with prescription soft contact lenses and ZEISS Optical Inserts for users who require vision correction. An antispoofing neural network spots and resists spoofs—and defends against attempts to unlock the device with photos or masks. Optic ID data, including mathematical representations of a user's eye, is encrypted and available only to the Secure Enclave. This data never leaves the device. It's not sent to Apple, nor is it included in device backups.

The following Optic ID data is saved, encrypted only for use by the Secure Enclave, during normal operation:

- The mathematical representations of a user's eye and iris calculated during enrollment
- The mathematical representations of a user's eye and iris calculated during some unlock attempts if Optic ID deems them useful to augment future matching—for instance, improved visibility of iris or different pupil size

The Secure Enclave immediately discards eye images captured during normal operation after it calculates the mathematical representation for either enrollment in Optic ID or for matching with enrolled Optic ID data.

Facial matching security

Facial matching is performed within the Secure Enclave using neural networks trained specifically for that purpose. Apple developed the facial matching neural networks using over a billion images, including infrared (IR) and depth images collected in studies conducted with the participants' informed consent. Apple then worked with participants from around the world to include a representative group of people accounting for gender, age, ethnicity, and other factors. The studies were augmented as needed to provide a high degree of accuracy for a diverse range of users. Face ID is designed to work with hats, scarves, eyeglasses, contact lenses, and many types of sunglasses. Face ID also supports unlocking with a mask on iPhone devices starting with iPhone 12 and iOS 15.4 or later. Furthermore, it's designed to work indoors, outdoors, and even in total darkness. An additional neural network—that's trained to spot and resist spoofing—defends against attempts to unlock the device with photos or masks. Face ID data, including mathematical representations of a user's face, is encrypted and available only to the Secure Enclave. This data never leaves the device. It's not sent to Apple, nor is it included in device backups. The following Face ID data is saved, encrypted only for use by the Secure Enclave, during normal operation:

- The mathematical representations of a user's face calculated during enrollment
- The mathematical representations of a user's face calculated during some unlock attempts if Face ID deems them useful to augment future matching

The Secure Enclave immediately discards face images captured during normal operation after it calculates the mathematical representation for either enrollment in Face ID or for matching with enrolled Face ID data.

Improving Face ID matches

To improve match performance and keep pace with the natural changes of a face and look, Face ID augments its stored mathematical representation over time. Upon a successful match, Face ID may use the newly calculated mathematical representation—if its quality is sufficient—for a finite number of additional matches before that data is discarded. Conversely, if Face ID fails to recognize a face but the match quality is higher than a certain threshold and a user immediately follows the failure by entering their passcode, Face ID takes another capture and augments its enrolled Face ID data with the newly calculated mathematical representation. This new Face ID data is discarded if the user stops matching against it or after a finite number of matches; the new data is also discarded when the option to reset Face ID is selected. These augmentation processes allow Face ID to keep up with dramatic changes in a user's facial hair or makeup use while minimizing false acceptance.

Uses for Optic ID, Face ID, and Touch ID

How a device's Data Protection keys work varies depending on whether its Optic ID, Face ID, or Touch ID is turned on or off.

Unlocking a device or user account

With Optic ID, Face ID, or Touch ID turned off, when a device or account is locked, the keys for the highest class of [Data Protection](#)—which are held in the Secure Enclave—are discarded. The files and [keychain](#) items in that class are inaccessible until the user unlocks the device or account by entering their passcode or password.

With Optic ID, Face ID, or Touch ID turned on, the keys aren't discarded when the device or account locks; instead, they're wrapped with a key that's given to the Optic ID, Face ID, or Touch ID subsystem inside the Secure Enclave. When a user attempts to unlock the device or account, if the device detects a successful match, it provides the key for unwrapping the Data Protection keys, and the device or account is unlocked. This process provides additional protection by requiring cooperation between the Data Protection and Optic ID, Face ID, or Touch ID subsystems to unlock the device.

When the device restarts, the keys required for Optic ID, Face ID, or Touch ID to unlock the device or account are lost; they're discarded by the Secure Enclave after any condition is met that requires passcode or password entry.

Securing purchases with Apple Pay

The user can also use Optic ID, Face ID, and Touch ID with Apple Pay to make easy and secure purchases in stores, apps, and on the web:

- **Using Face ID in stores:** To authorize an in-store payment with Face ID, the user needs to first confirm intent to pay by double-clicking the side button. This double-click captures user intent using a physical gesture directly linked to the Secure Enclave and is resistant to forgery by a malicious process. The user then authenticates using Face ID before placing the device near the contactless payment reader. A different Apple Pay payment method can be selected after Face ID authentication, which requires reauthentication, but the user won't have to double-click the side button again.
- **Using Optic ID or Face ID in apps and on the web:** To make a payment within apps and on the web, the user confirms their intent to pay by double-clicking the side button (on iPad or iPhone) or the top button (on Apple Vision Pro), and then authenticates using Optic ID or Face ID to authorize the payment. If the Apple Pay transaction isn't completed within 60 seconds of double-clicking the side button or the top button, the user needs to reconfirm intent to pay by double-clicking again.
- **Using Touch ID:** For Touch ID, the intent to pay is confirmed using the gesture of activating the Touch ID sensor combined with successfully matching the user's fingerprint.

Using system-provided APIs

Third-party apps can use system-provided APIs to ask the user to authenticate using Optic ID, Face ID, Touch ID, or a passcode or password. Apps that support Touch ID automatically support Optic ID and Face ID without any changes. When using Optic ID, Face ID, or Touch ID, the app is notified only as to whether the authentication was successful; it can't access Optic ID, Face ID, or Touch ID or the data associated with the enrolled user.

Protecting keychain items

Keychain items can also be protected with Optic ID, Face ID, or Touch ID, to be released by the Secure Enclave only by a successful match or with the device passcode or account password. App developers have APIs to verify that a passcode or password has been set by the user before requiring Optic ID, Face ID, Touch ID, or a passcode or password to unlock [keychain](#) items. App developers can do any of the following:

- Require that authentication API operations don't fall back to an app password or the device passcode. They can query whether a user is enrolled, allowing Optic ID, Face ID, or Touch ID to be used as a second factor in security-sensitive apps.
- Generate and use Elliptic Curve Cryptography (ECC) keys inside the Secure Enclave that can be protected by Optic ID, Face ID, or Touch ID. Operations with these keys are always performed inside the Secure Enclave after it authorizes their use.

Making and approving purchases

Users can also configure Optic ID, Face ID, or Touch ID to approve purchases from the iTunes Store, the App Store, Apple Books, and more, so users don't have to enter their Apple Account password. When purchases are made, the Secure Enclave verifies that a biometric authorization occurred and then releases ECC keys used to sign the store request.

Authorizing access and car keys sharing

Users can use Optic ID, Face ID, and Touch ID to authorize access and car keys sharing.

Locking and hiding apps

On devices with iOS 18 or iPadOS 18 or later, Face ID and Touch ID can be used to access apps that the user decided to lock or hide.

Secure intent and connections to the Secure Enclave

Secure intent provides a way to confirm a user's intent without any interaction with the operating system or Application Processor. The connection is a physical link—from a physical button to the Secure Enclave—that's available in the following:

- All iPhone models starting with iPhone X or later
- All iPad models starting with iPad Air (4th generation) or later
- All Mac computers with Apple silicon
- All Apple Watch models starting with Apple Watch Series 1 or later
- Apple Vision Pro

With this link, users can confirm their intent to complete an operation in a way designed such that even software running with root privileges or in the kernel can't spoof.

This feature is used to confirm user intent during Apple Pay transactions and when finalizing pairing Magic Keyboard with Touch ID to a Mac with Apple silicon. A double-press on the following buttons—when prompted by the user interface—signals confirmation of user intent:

- The top button on Apple Vision Pro (for Optic ID)
- The side button on an iPad or iPhone (for Face ID)
- A fingerprint scan (for devices with Touch ID)

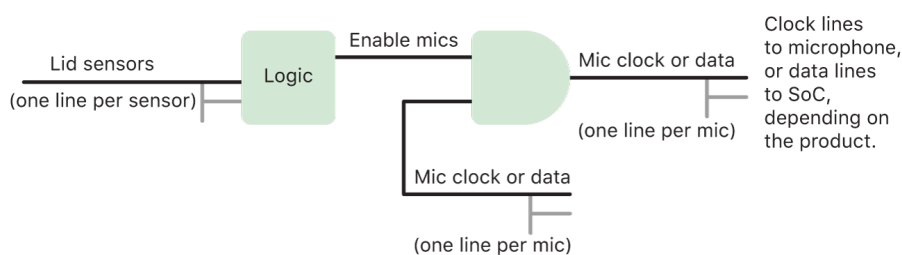
A similar mechanism—based on the Secure Enclave and T2 firmware—is supported on MacBook models with the Apple T2 Security Chip and no Touch Bar.

Hardware microphone disconnect

All Apple silicon-based Mac laptops and Intel-based Mac laptops with the Apple T2 Security Chip feature a hardware disconnect that turns off the microphone whenever the lid is closed. On all 13-inch MacBook Pro and MacBook Air laptops with the T2 chip, all MacBook laptops with a T2 chip from 2019 or later, and Mac laptops with Apple silicon, this disconnect is implemented in hardware alone. The disconnect is designed to prevent any software—even with root or kernel privileges in macOS, and even the software on the T2 chip or other firmware—from engaging the microphone when the lid is closed. (The camera isn't disconnected in hardware, because its field of view is completely obstructed with the lid closed.)

iPad models beginning in 2020 also feature the hardware microphone disconnect. When an MFi-compliant case (including those sold by Apple) is attached to the iPad and closed, the microphone is disconnected in hardware. This is designed to prevent microphone audio data being made available to any software—even with root or kernel privileges in iPadOS, or any device firmware.

The protections in this section are implemented directly with hardware logic, according to the following circuit diagram:



In each product with a hardware microphone cutoff, one or more lid sensors detect the physical closure of the lid or case using some physical property (for example, a Hall effect sensor or a hinge angle sensor) of the interaction. For sensors where calibration is necessary, parameters are set during production of the device and the calibration process includes a nonreversible hardware lock out of any subsequent changes to sensitive parameters on the sensor. These sensors emit a direct hardware signal that goes through a simple set of nonreprogrammable hardware logic. This logic provides debounce, hysteresis, and/or a delay of up to 500 ms before turning off the microphone. Depending on the product, this signal can be implemented either by turning off the lines transporting data between the microphone and the [system on a chip \(SoC\)](#) or by turning off one of the input lines to the microphone module that's allowing it to be active—for example, the clock line or a similar effective control.

Mac on-screen camera indicator light

MacBook Neo combines system software and dedicated silicon elements within A18 Pro to provide additional security for the camera feed. The architecture is designed to prevent any untrusted software—even with root or kernel privileges in macOS—from engaging the camera without also visibly lighting the on-screen camera indicator light.

Express Cards with power reserve

If iOS isn't running because iPhone needs to be charged, there may still be enough power in the battery to support Express Card transactions. Supported iPhone devices automatically support this feature with:

- A payment or transit card designated as the Express Transit card
- Access cards with Express Mode turned on

When the side button is pressed, the battery icon shows that power is low, and text indicates that Express Cards are available to use. The NFC controller performs Express Card transactions under the same conditions as when iOS is running, except that transactions are indicated only with haptic notifications (no visible notification is shown). On iPhone SE 2nd generation, completed transactions may take a few seconds to appear on screen. This feature isn't available when a standard user-initiated shutdown is performed.

System security

System security overview

Building on the unique capabilities of Apple hardware, system security is responsible for controlling access to system resources in Apple devices without compromising usability. System security encompasses the boot process, software updates, and protection of computer system resources such as CPU, memory, disk, software programs, and stored data.

The most recent versions of Apple operating systems are the most secure. An important part of Apple security is *secure boot*, which protects the system from malware infection at boot time. Secure boot begins in silicon and builds a chain of trust through software, where each step is designed to ensure that the next is functioning properly before handing over control. This security model supports not only the default boot of Apple devices but also the various modes for recovery and timely updates on Apple devices. Subcomponents like the Secure Enclave also perform their own secure boot to help ensure they only boot known-good code from Apple. The update system is designed to prevent downgrade attacks, so that devices can't be rolled back to an older version of the operating system (which an attacker knows how to compromise) as a method of stealing user data.

Apple devices also include boot and runtime protections so that they maintain their integrity during ongoing operation. Apple-designed silicon on iPad, iPhone, a Mac with Apple silicon, Apple TV, Apple Vision Pro, Apple Watch, and HomePod provide a common architecture for protecting operating system integrity. macOS also features an expanded and configurable set of protection capabilities in support of its differing computing model, as well as capabilities supported on all Mac hardware platforms.

Secure boot

Boot process for iPad and iPhone devices

Each step of the startup process contains components that are cryptographically signed by Apple to enable integrity checking so that boot proceeds only after verifying the chain of trust. These components include the bootloaders, the kernel, kernel extensions (kext), and cellular baseband firmware. This secure boot chain is designed to verify that the lowest levels of software aren't tampered with.

When an iPad and iPhone device is turned on, its Application Processor immediately executes code from read-only memory referred to as [Boot ROM](#). This immutable code, known as the *hardware root of trust*, is laid down during chip fabrication and is implicitly trusted. The Boot ROM code contains the Apple Root certificate authority (CA) public key—used to verify that the [iBoot](#) bootloader is signed by Apple before allowing it to load. This is the first step in the chain of trust, in which each step checks that the next is signed by Apple. When the iBoot finishes its tasks, it verifies and runs the iOS or iPadOS kernel. For devices with an A9 or earlier A-series processor, an additional [Low-Level Bootloader \(LLB\)](#) stage is loaded and verified by the Boot ROM and in turn loads and verifies iBoot.

A failure to load or verify following stages is handled differently depending on the hardware:

- *Boot ROM can't load LLB (older devices):* [Device Firmware Upgrade \(DFU\) mode](#)
- *LLB or iBoot:* [Recovery mode](#)

In either case, the device needs to be connected to the Finder (macOS 10.15 or later) or iTunes (in macOS 10.14 or earlier) through USB and restored to factory default settings.

The [Boot Progress Register \(BPR\)](#) is used by the Secure Enclave to limit access to user data in different modes and is updated before entering the following modes:

- *DFU mode:* Set by Boot ROM on devices with an Apple A12 or later SoCs
- *Recovery mode:* Set by iBoot on devices with Apple A10, S2, or later SoCs

On devices with cellular access, a cellular baseband subsystem performs additional secure booting using signed software and keys verified by the baseband processor.

The Secure Enclave also performs a secure boot that checks its software (sepOS) is verified and signed by Apple.

Memory safe iBoot implementation

On devices with iOS 14 and iPadOS 14 or later, Apple modified the C compiler toolchain used to build the iBoot bootloader to improve its security. The modified toolchain implements code designed to prevent memory- and type-safety issues that are typically encountered in C programs. For example, it helps prevent most vulnerabilities in the following classes:

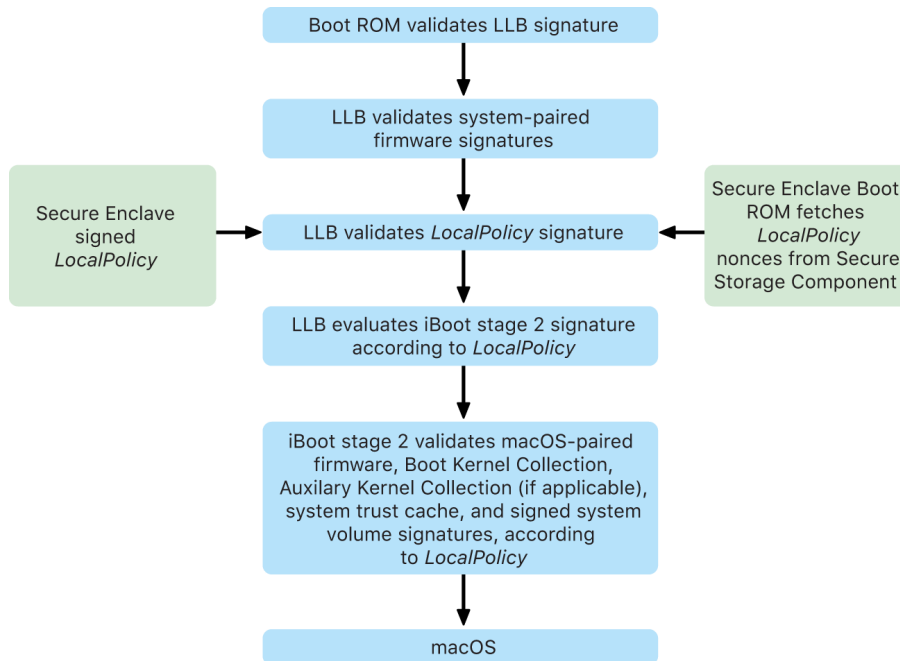
- Buffer overflows, by ensuring that all pointers carry bounds information that's verified when accessing memory
- Heap exploitation, by separating heap data from its metadata and accurately detecting error conditions such as double free errors
- Type confusion, by ensuring that all pointers carry runtime type information that's verified during pointer cast operations
- Type confusion caused by use after free errors, by segregating all dynamic memory allocations by static type

This technology is available on iPhone with the A13 Bionic chip or later, and on iPad with the A14 Bionic chip or later.

Mac computers with Apple silicon

Boot process for a Mac with Apple silicon

When a Mac with Apple silicon is turned on, it performs a boot process much like that of iPad and iPhone.



The chip executes code from the [Boot ROM](#) in the first step in the chain of trust. macOS secure boot on a Mac with Apple silicon verifies not only the operating system code itself, but also the security policies and even kexts (supported, though not recommended) configured by authorized users.

When the Low-Level Bootloader (LLB) is launched, it verifies the signatures and loads system-paired firmware for intra-SoC cores such as the storage, display, system management, and Thunderbolt controllers. LLB is also responsible for loading the LocalPolicy, which is a file signed by the Secure Enclave Processor. The LocalPolicy file describes the configuration that the user has chosen for the system boot and runtime security policies. The LocalPolicy has the same data structure format as all other boot objects, but it's signed locally by a private key that's available only within a particular computer's Secure Enclave instead of being signed by a central Apple server (like software updates).

To help prevent replay of any previous LocalPolicy, LLB needs to look up an anti-replay value from the Secure Enclave-attached Secure Storage Component. To do this, it uses the Secure Enclave Boot ROM and makes sure the anti-replay value in the LocalPolicy matches the anti-replay value in the Secure Storage Component. This helps prevent an old LocalPolicy—which could have been configured for lower security—from being reapplied to the system after security has been upgraded. The result is that secure boot on a Mac with Apple silicon helps protect not only against rollback of operating system versions but also against security policy downgrades.

The LocalPolicy file captures whether the operating system is configured for Full, Reduced, or Permissive security.

- *Full Security*: The system behaves like iOS and iPadOS, and allows only booting software which was known to be the latest that was available at install time.
- *Reduced Security*: LLB is directed to trust “global” signatures, which are bundled with the operating system. This allows the system to run older versions of macOS. Because older versions of macOS inevitably have unpatched vulnerabilities, this security mode is described as *reduced*. This is also the policy level required to support booting kexts.
- *Permissive Security*: The system behaves like Reduced Security in that it uses global signature verification for [iBoot](#) and beyond, but it also tells iBoot that it should accept some boot objects being signed by the Secure Enclave with the same key used to sign the LocalPolicy. This policy level supports users that are building, signing, and booting their own custom XNU kernels.

If the LocalPolicy indicates to LLB that the selected operating system is running in Full Security, LLB evaluates the personalized signature for iBoot. If it’s running in Reduced Security or Permissive Security, it evaluates the global signature. Any signature verification errors cause the system to boot to recoveryOS to provide repair options.

After LLB hands off to iBoot, it loads macOS-paired firmware such as that for the Secure Neural Engine, the Always On Processor, and other firmware. iBoot also looks at information about the LocalPolicy handed to it from LLB. If the LocalPolicy indicates that there should be an Auxiliary Kernel Collection (AuxKC), iBoot looks for it on the file system, verifies that it was signed by the Secure Enclave with the same key as the LocalPolicy, and verifies that its hash matches a hash stored in the LocalPolicy. If the AuxKC is verified, iBoot places it into memory with the Boot Kernel Collection before locking the full memory region covering the Boot Kernel Collection and AuxKC with the [System Coprocessor Integrity Protection \(SCIP\)](#). If the policy indicates that an AuxKC should be present but it isn’t found, the system continues to boot into macOS without it. iBoot is also responsible for verifying the root hash for the signed system volume (SSV), to check that the file system the kernel will mount is fully integrity verified.

Boot modes for a Mac with Apple silicon

A Mac with Apple silicon has the boot modes described below.

Mode	Key combo	Description
macOS	From a shutdown state, press and release the power button.	1. Boot ROM hands off to LLB. 2. LLB loads system-paired firmware and the LocalPolicy for the selected macOS. 3. LLB locks an indication into the Boot Progress Register (BPR) that it’s booting into macOS, and hands off to iBoot. 4. iBoot loads the macOS-paired firmware, the static trust cache, the device tree, and the Boot Kernel Collection. 5. If the LocalPolicy allows it, iBoot loads the Auxiliary Kernel Collection (AuxKC) of third-party kexts. 6. If the LocalPolicy didn’t make it inactive, iBoot verifies the root signature hash for the signed system volume (SSV).

Mode	Key combo	Description
Paired recoveryOS	From a shutdown state, press and hold the power button.	1. Boot ROM hands off to LLB. 2. LLB loads system-paired firmware and the LocalPolicy for the recoveryOS. 3. LLB locks an indication into the Boot Progress Register that it's booting into paired recoveryOS, and hands off to iBoot for paired recoveryOS. 4. iBoot loads the macOS-paired firmware, the trust cache, the device tree, and the Boot Kernel Collection. 5. If paired recoveryOS boot fails, booting into fallback recoveryOS is attempted.
Fallback recoveryOS	From a shutdown state, double-press and hold the power button.	1. Boot ROM hands off to LLB. 2. LLB loads system-paired firmware and the LocalPolicy for the recoveryOS. 3. LLB locks an indication into the Boot Progress Register that it's booting into paired recoveryOS, and hands off to iBoot for recoveryOS. 4. iBoot loads the macOS-paired firmware, the trust cache, the device tree, and the Boot Kernel Collection.
Safe mode	Boot into recoveryOS per the above, then hold Shift while selecting the startup volume.	1. Boots to recoveryOS as per the above. 2. Holding the Shift key while selecting a volume causes the BootPicker app to approve that macOS for booting, as normal; it also sets an nvram variable that tells iBoot to not load the AuxKC on the next boot. 3. System restarts and boots to the targeted volume, but iBoot doesn't load AuxKC.

Paired recoveryOS restrictions

On devices with macOS 12.0.1 or later, every new macOS installation also installs a paired version of recoveryOS into the corresponding APFS volume group. This design is familiar to users of Intel-based Mac computers, but it provides additional security and compatibility guarantees on a Mac with Apple silicon. Because every macOS installation now has a dedicated paired recoveryOS, this helps ensure that only that dedicated paired recoveryOS can perform security-downgrading operations. This helps protect installations of newer versions of macOS from tampering initiated from older versions of macOS and vice versa.

The pairing restrictions are enforced as follows:

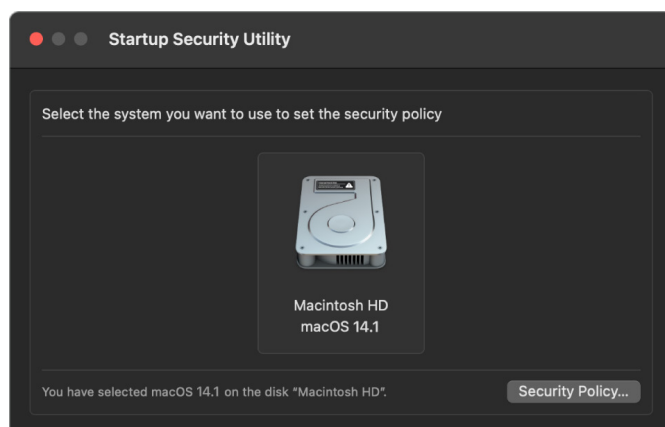
- All installations of macOS 11 are paired to the recoveryOS. If a macOS 11 installation is selected to boot by default, the recoveryOS is booted by holding down the power key at boot time on a Mac with Apple silicon. The recoveryOS can downgrade security settings of any macOS 11 installations, but not any installations of macOS 12.0.1.
- If a macOS 12.0.1 or later installation is selected to boot by default, its paired recoveryOS is booted by holding down the power key when the Mac starts up. The paired recoveryOS can downgrade security settings for the paired macOS installation, but not for any other macOS installation.

To boot into a paired recoveryOS for any macOS installation, that installation needs to be selected as the default, which is done using General > Startup Disk in System Settings (macOS 13 or later), Startup Disk in System Preferences (macOS 12 or earlier), or by starting any recoveryOS and holding Option while selecting a volume.

Note: Fallback recoveryOS can't perform downgrades for any macOS installation.

Startup Disk security policy control for a Mac with Apple silicon

Unlike security policies on an Intel-based Mac, security policies on a Mac with Apple silicon are for each installed operating system. This means that multiple installed macOS instances with different versions and security policies are supported on the same Mac. For this reason, an *operating system picker* has been added to Startup Security Utility.

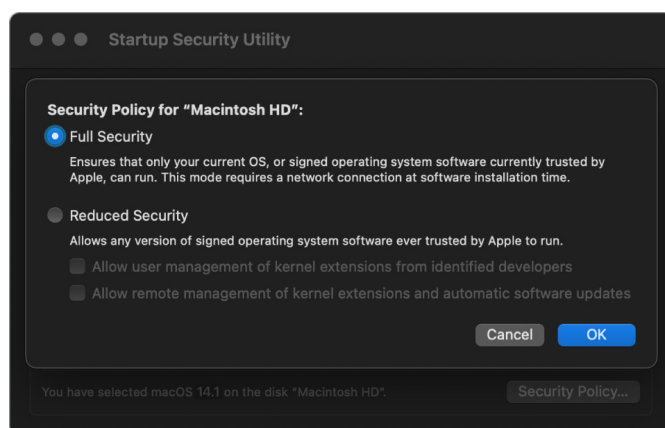


On a Mac with Apple silicon, System Security Utility indicates the overall user-configured security state of macOS, such as the booting of a kext or the configuration of System Integrity Protection (SIP). If changing a security setting would significantly degrade security or make the system easier to compromise, users need to enter into recoveryOS by holding the power button (so that malware can't initiate the signal, only a human with physical access can) to make the change. Because of this, an Apple-silicon based Mac also won't require (or support) a firmware password—all critical changes are already gated by user authorization. For more information on SIP, see [System Integrity Protection](#).

Full Security and Reduced Security can be set using Startup Security Utility from recoveryOS. But Permissive Security can be accessed only from command-line tools for users who accept the risk of making their Mac less secure.

Full Security policy

Full Security is the default, and it behaves like iOS and iPadOS. At the time software is downloaded and prepared to install, rather than using the global signature that comes with the software, macOS contacts the same Apple signing server used for iOS and iPadOS and requests a fresh, “personalized” signature. A signature is personalized when it includes the [Exclusive Chip Identification \(ECID\)](#)—a unique ID specific to the Apple CPU in this case—as part of the signing request. The signature given back by the signing server is then unique and usable only by that particular Apple CPU. When the Full Security policy is in effect, the Boot ROM and LLB helps ensure that a given signature isn’t just signed by Apple but is signed for this specific Mac, essentially tying that version of macOS to that Mac.



Using an online signing server also provides better protection against rollback attacks than typical global signature approaches. In a global signing system, the security epoch could have rolled many times, but a system that has never seen the latest firmware won't know this. For example, a computer that currently believes it's in security epoch 1 accepts software from security epoch 2, even if the current actual security epoch is 5. With an Apple silicon online signing system, the signing server can reject creating signatures for software that's in anything except the latest security epoch.

Additionally, if an attacker discovers a vulnerability after a security epoch change, they can't simply pick up the vulnerable software from a previous epoch off system A and apply it to system B to attack it. The fact that the vulnerable software from an older epoch was personalized to system A helps prevent it from being transferable and thus being used to attack a system B. All these mechanisms work together to provide much stronger guarantees that attackers can't purposely place vulnerable software on a Mac to circumvent the protections provided by the latest software. But a user that's in possession of an administrator user name and password for the Mac can always choose the security policy that works best for their use cases.

Reduced Security policy

Reduced Security is similar to Medium Security behavior on an Intel-based Mac with a T2 chip, in which a vendor (in this case, Apple) generates a digital signature for the code to assert it came from the vendor. This design helps prevent attackers from inserting unsigned code. Apple refers to this signature as a global signature because it can be used on any Mac, for any amount of time, for a Mac that currently has a Reduced Security policy set. Reduced security doesn't itself provide protection against rollback attacks (although unauthorized operating system changes can result in user data being rendered inaccessible. For more information, see [In a Mac with Apple silicon](#)).

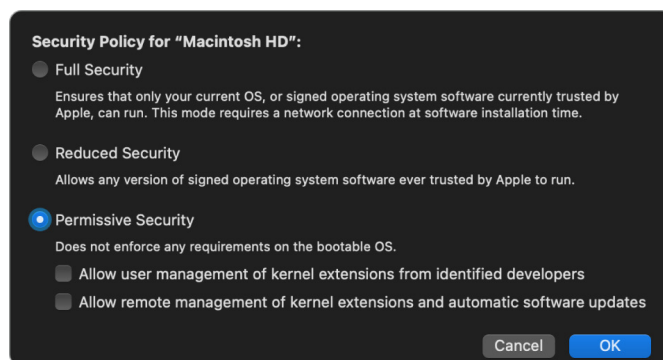


In addition to enabling users to run older versions of macOS, Reduced Security is required for other actions that can put a user's system security at risk, such as introducing third-party kernel extensions (kexts). Kexts have the same privileges as the kernel, and vulnerabilities in third-party kexts can compromise the entire operating system. This is why developers are being strongly encouraged to adopt system extensions before kext support is removed from macOS for future Mac computers with Apple silicon. Even when third-party kexts are enabled, they can't be loaded into the kernel on demand. Instead, the kexts are merged into an Auxiliary Kernel Collection (AuxKC)—whose hash is stored in the LocalPolicy—and thus they require a restart. For more information about AuxKC generation, see [Securely extending the kernel](#).

Permissive Security policy

Permissive Security is for users who accept the risk of putting their Mac into a much more insecure state. This mode differs from No Security mode on an Intel-based Mac with a T2 chip. With Permissive Security, signature verification is still performed along the entire secure boot chain, but setting the policy to Permissive signals to iBoot that it should accept locally Secure Enclave-signed boot objects, such as a user-generated Boot Kernel Collection built from a custom XNU kernel. In this way, Permissive Security also provides an architectural capability for running an arbitrary fully untrusted operating system kernel. When a custom Boot Kernel Collection or fully untrusted operating system is loaded on the system, some decryption keys become unavailable. This is designed to prevent a fully untrusted operating systems from accessing data from trusted operating systems.

Important: Apple doesn't provide or support custom XNU kernels.



There's another way that Permissive Security differs from No Security on an Intel-based Mac with a T2 chip: It's a prerequisite for some security downgrades that in the past have been independently controllable. Most notably, to turn off SIP on a Mac with Apple silicon, a user needs to acknowledge that they're putting the system into Permissive Security. This is required because turning off SIP has always put the system into a state that makes the kernel much easier to compromise. In particular, turning off SIP on a Mac with Apple silicon inactivates kext signature enforcement during AuxKC generation time, allowing any arbitrary kext to be loaded into kernel memory. Another improvement to SIP on a Mac with Apple silicon is that the policy store has moved out of NVRAM and into the LocalPolicy. So now, turning off SIP requires authentication by a user with access to the LocalPolicy signing key from recoveryOS (reached by pressing and holding the power button). This makes it significantly more difficult for a software-only attacker, or even a physically present attacker, to turn off SIP.

It isn't possible to downgrade to Permissive Security from the Startup Security Utility app. Users can downgrade only by running command-line tools in recoveryOS, such as `csrutil` (to turn off SIP). After the user downgrade—Startup Security Utility reflects this fact, and so the user can then set the security to a more secure mode.

Note: A Mac with Apple silicon doesn't require or support a specific media boot policy, because technically all boots are performed locally. If a user chooses to boot from external media, that operating system version first needs to be personalized using an authenticated restart from recoveryOS. This restart creates a LocalPolicy file on the internal drive that's used to perform a trusted boot from the operating system stored on the external media. This means the configuration of starting from external media is always explicitly enabled on a per operating system basis, and already requires user authorization, so no additional secure configuration is necessary.

LocalPolicy signing-key creation and management

Creation

When macOS is first installed in the factory, or when a tethered erase-install is performed, the Mac runs code from temporary restore RAM disk to initialize the default state. During this process, the restore environment creates a new pair of public and private keys which are held in the Secure Enclave. The private key is referred to as the *Owner Identity Key (OIK)*. If any OIK already exists, it's destroyed as part of this process. The restore environment also initializes the key used for Activation Lock; the *User Identity Key (UIK)*. Part of that process which is unique to a Mac with Apple silicon is when UIK certification is requested for Activation Lock, a set of requested constraints to be enforced at validation-time on the LocalPolicy are included. If the device can't get a UIK certified for Activation Lock (for example, because the device is currently associated with a Find My Mac account and reported as lost), it's unable to proceed further to create a Local Policy. If a device is issued a *User identity Certificate (ucrt)*, that ucrt contains server imposed policy constraints and user requested policy constraints in an X.509 v3 extension.

When an Activation Lock/ucrt is successfully retrieved, it's stored in a database on the server side and also returned to the device. After the device has a ucrt, a certification request for the public key which corresponds to the OIK is sent to the *Basic Attestation Authority (BAA)* server. BAA verifies the OIK certification request using the public key from the ucrt stored in the BAA accessible database. If the BAA can verify the certification, it certifies the public key, returning the *Owner Identity Certificate (OIC)* which is signed by the BAA and contains the constraints stored in ucrt. The OIC is sent back to the Secure Enclave. From then on, whenever the Secure Enclave signs a new LocalPolicy, it attaches the OIC to the Image4. LLB has built-in trust in the BAA root certificate, which causes it to trust the OIC, which causes it to trust the overall LocalPolicy signature.

RemotePolicy constraints

All Image4 files, not just Local Policies, contain constraints on Image4 manifest evaluation. These constraints are encoded using special object identifiers (OIDs) in the leaf certificate. The Image4 verification library looks up the special certificate constraint OID from a certificate during signature evaluation and then mechanically evaluates the constraints specified in it. The constraints are of the form:

- x must exist
- x must not exist
- x must have a specific value

So, for instance, for "personalized" signatures, the certificate constraints will contain "ECID must exist," and for "global" signatures, it will contain "ECID must not exist." These constraints are designed to ensure that all Image4 files signed by a given key need to conform to certain requirements to avoid erroneous signed Image4 manifest generation.

In the context of each LocalPolicy, these Image4 certificate constraints are referred to as the *RemotePolicy*. A different RemotePolicy can exist for different boot environments' LocalPolicies. The RemotePolicy is used to restrict the recoveryOS LocalPolicy so that when recoveryOS is booted it can only ever behave as if it's booting with Full Security. This increases trust in the integrity of the recoveryOS boot environment as a place where policy can be changed. The RemotePolicy restricts the LocalPolicy to contain the ECID of the Mac on which the LocalPolicy was generated, and the specific Remote Policy Nonce Hash (rpnh) stored in the Secure Storage Component on that Mac. The rpnh, and therefore the RemotePolicy, change only when actions are taken for Find My Mac and Activation Lock, such as enrollment, unenrollment, remote lock, and remote wipe. Remote Policy constraints are determined and specified at User Identity Key (UIK) certification time and are signed in to the issued User identity Certificate (ucrt). Some Remote Policy constraints, such as ECID, ChipID, and BoardID, are determined by the server. This is designed to prevent one device from signing LocalPolicy files for another device. Other Remote Policy constraints may be specified by the device to help prevent Security downgrade of the Local Policy without providing both the local authentication required to access the current OIK and remote authentication of the account to which the device is Activation Locked.

Contents of a LocalPolicy file for a Mac with Apple silicon

The LocalPolicy is an Image4 file signed by the Secure Enclave. Image4 is an ASN.1 (Abstract Syntax Notation One) DER-encoded data structure format that's used to describe information about secure boot chain objects on Apple platforms. In an Image4-based secure boot model, security policies are requested at software installation time initiated by a signing request to a central Apple signing server. If the policy was acceptable, the signing server returns a signed Image4 file containing a variety of four-character code (4CC) sequences. These signed Image4 files and 4CCs are evaluated at startup by software like the Boot ROM or LLB.

Ownership handoff between operating systems

Access to the Owner Identity Key (OIK) is referred to as "Ownership." Ownership is required to allow users to resign the LocalPolicy after making policy or software changes. The OIK is protected with the same key hierarchy as described in [Sealed Key Protection \(SKP\)](#), with the OIK being protected by the same Key encryption key (KEK) as the Volume encryption key (VEK). This means it's normally protected by both user passwords and measurements of the operating system and policy. There's only a single OIK for all operating systems on the Mac. Therefore, when installing a second operating system, explicit consent is required from the users on the first operating system to hand off Ownership to the users on the second operating system. However, users don't yet exist for the second operating system, when the installer is running from the first operating system. Users in operating systems aren't normally generated until the operating system is booted and the Setup Assistant is running. Two new actions are required when installing a second operating system on a Mac with Apple silicon:

- Creating a LocalPolicy for the second operating system
- Preparing an "Install User" for handing off Ownership

When running an Install Assistant and targeting installation for a secondary blank volume, a prompt asks the user if they'd like to copy a user from the current volume to be the first user of the second volume. If the user says yes, the Install User which is created is, in reality, a KEK which is derived from the selected user's password and hardware keys, which is then used to encrypt the OIK as it's being handed to the second operating system. Then from within the second operating system Install Assistant, it prompts for that user's password to allow it to access the OIK in the Secure Enclave for the new operating system. If users opt not to copy a user, the Install User is created the same way, but an empty password is used instead of a user's password. This second flow exists for certain system administration scenarios. However, users who want to have multivolume installs and want to perform Ownership handoff in the most secure fashion should always opt to copy a user from the first operating system to the second operating system.

LocalPolicy on a Mac with Apple silicon

For a Mac with Apple silicon, local security policy control has been delegated to an application running in the Secure Enclave. This software can use the user's credentials and the boot mode of the primary CPU to determine who can change the security policy and from a specific boot environment. This helps prevent malicious software from using the security policy controls against the user by downgrading them to gain more privileges.

LocalPolicy manifest properties

The LocalPolicy file contains some architectural 4CCs that are found in most all Image4 files—such as a board or model ID (BORD), indicating a particular Apple chip (CHIP), or [Exclusive Chip Identification \(ECID\)](#). But the 4CCs below focus only on the security policies that users can configure.

Note: Apple uses the term *Paired One True recoveryOS (1TR)* to indicate a boot into the paired recoveryOS initiated by a physical power button single-press-and-hold. This differs from a normal recoveryOS boot, which NVRAM or a physical power button double-press-and-hold, or which errors on startup may initiate. A physical button press of a specific kind increases trust that a software-only attacker who has broken into macOS can't reach the boot environment.

LocalPolicy Nonce Hash (lpth)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The lpth is used for anti-replay of the LocalPolicy. This is an SHA-384 hash of the LocalPolicy Nonce (LPN), which is stored in the Secure Storage Component and accessible using the Secure Enclave Boot ROM or Secure Enclave. The raw anti-replay value is never visible to the Application Processor, only to the sepOS. An attacker wanting to convince LLB that a previous LocalPolicy they had captured was valid would need to place a value into the Secure Storage Component, which hashes to the same lpth value found in the LocalPolicy they want to replay. Normally there is a single LPN valid on the system—except during software updates, when two are simultaneously valid—to allow for the possibility of falling back to booting the old software in the event of an update error. When any LocalPolicy for any operating system is changed, all policies are re-signed with the new lpth value corresponding to the new LPN found in the Secure Storage Component. This change happens when the user changes security settings or creates new operating systems with a new LocalPolicy for each.

Remote Policy Nonce Hash (rpnh)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The rpnh behaves the same way as the lpnh but is updated only when the remote policy is updated, such as when changing the state of Find My enrollment. This change happens when the user changes the state of Find My on their Mac.

recoveryOS Nonce Hash (ronh)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The ronh behaves the same way as the lpnh, but is found exclusively in the LocalPolicy for system recoveryOS. It's updated when the system recoveryOS is updated, such as on software updates. A separate anti-replay value from the lpnh and rpnh is used so that when a device is put into an inactive state by Find My, existing operating systems can be made inactive (by removing their LPN and RPN from the Secure Storage Component), while still leaving the system recoveryOS bootable. In this way, the operating systems can be reenabled when the system owner proves their control over the system by putting in their iCloud password used for the Find My account. This change happens when a user updates the system recoveryOS or creates new operating systems.

Next Stage Image4 Manifest Hash (nsih)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The nsih field represents an SHA-384 hash of the Image4 manifest data structure that describes the booted macOS. The macOS Image4 manifest contains measurements for all the boot objects—such as iBoot, the static trust cache, device tree, Boot Kernel Collection, and signed system volume (SSV) volume root hash. When LLB is directed to boot a given macOS, it's designed to ensure that the hash of the macOS Image4 manifest attached to iBoot matches what's captured in the nsih field of the LocalPolicy. In this way, the nsih captures the user intention of what operating system the user has created a LocalPolicy for. Users change the nsih value implicitly when they perform a software update.

Cryptex1 Image4 Manifest Hash (spih)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The spih field represents an SHA-384 hash of the Cryptex1 Image4 manifest data structure. The Cryptex1 Image4 manifest contains measurements of its cryptexes, their file system seals, and their associated trust cache. When macOS is starting up, the XNU kernel and Page Protection Layer ensure that the hash of the Cryptex1 Image4 manifest matches what iBoot published from the spih field of the LocalPolicy. Users change the spih value implicitly when they install a Background Security Improvement or perform a software update. The Cryptex1 Image4 Manifest Hash can be updated independently of the Next Stage Image4 Manifest Hash.

Cryptex1 Generation (stng)

- *Type:* 64 bit unsigned integer
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The stng field is a counter value representing when the Cryptex1 Image4 Manifest Hash was last updated in a LocalPolicy. It provides an anti-replay value in place of lpnH during the Page Protection Layer's evaluation of the local policy for applying the Incoming Cryptex. Users increase the stng value implicitly when they install a Background Security Improvement or a software update.

Auxiliary Kernel Collection (AuxKC) Policy Hash (auxp)

- *Type:* OctetString (48)
- *Mutable environments:* macOS
- *Description:* The auxp is an SHA-384 hash of the user-authorized kext list (UAKL) policy. This is used at AuxKC generation time to help ensure that only user-authorized kexts are included in the AuxKC. smb2 is a prerequisite for setting this field. Users change the auxp value implicitly when they change the UAKL by approving a kext from Privacy & Security in System Settings (macOS 13 or later), or the Security & Privacy pane in System Preferences (macOS 12 or earlier).

Auxiliary Kernel Collection (AuxKC) Image4 Manifest Hash (auxi)

- *Type:* OctetString (48)
- *Mutable environments:* macOS
- *Description:* After the system verifies that the UAKL hash matches what's found in the auxp field of the LocalPolicy, it requests that the AuxKC be signed by the Secure Enclave processor application that's responsible for LocalPolicy signing. Next, an SHA-384 hash of the AuxKC Image4 manifest signature is placed into the LocalPolicy to avoid the potential for mixing and matching previously signed AuxKCs to an operating system at boot time. If iBoot finds the auxi field in the LocalPolicy, it attempts to load the AuxKC from storage and validate its signature. It also verifies that the hash of the Image4 manifest attached to the AuxKC matches the value found in the auxi field. If the AuxKC fails to load for any reason, the system continues to boot without this boot object and without any third-party kexts loaded. The auxp field is a prerequisite for setting the auxi field in the LocalPolicy. Users change the auxi value implicitly when they change the UAKL by approving a kext from Privacy & Security in System Settings (macOS 13 or later), or the Security & Privacy pane in System Preferences (macOS 12 or earlier).

Auxiliary Kernel Collection (AuxKC) Receipt Hash (auxr)

- *Type:* OctetString (48)
- *Mutable environments:* macOS
- *Description:* The auxr is an SHA-384 hash of the AuxKC receipt, which indicates the exact set of kexts that were included into the AuxKC. The AuxKC receipt can be a subset of the UAKL, because kexts can be excluded from the AuxKC even if they're user authorized if they're known to be used for attacks. In addition, some kexts that can be used to break the user-kernel boundary may lead to decreased functionality, such as an inability to use Apple Pay or play 4K and HDR content. Users who want these capabilities opt in to a more restrictive AuxKC inclusion. The auxp field is a prerequisite for setting the auxr field in the LocalPolicy. Users change the auxr value implicitly when they build a new AuxKC from Privacy & Security in System Settings (macOS 13 or later), or the Security & Privacy pane in System Preferences (macOS 12 or earlier).

CustomOS Image4 Manifest Hash (coih)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR
- *Description:* The coih is an SHA-384 hash of CustomOS Image4 manifest. The payload for that manifest is used by iBoot (instead of the XNU kernel) to transfer control. Users change the coih value implicitly when they use the kmutil configure-boot command-line tool in 1TR.

APFS volume group UUID (vuid)

- *Type:* OctetString (16)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The vuid indicates the volume group the kernel should use as root. This field is primarily informational and isn't used for security constraints. This vuid is set by the user implicitly when creating a new operating system install.

Key encryption key (KEK) Group UUID (kuid)

- *Type:* OctetString (16)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The kuid indicates the volume that was booted. The key encryption key has typically been used for Data Protection. For each LocalPolicy, it's used to protect the LocalPolicy signing key. The kuid is set by the user implicitly when creating a new operating system install.

Paired recoveryOS Trusted Boot Policy Measurement (prot)

- *Type:* OctetString (48)
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* A paired recoveryOS Trusted Boot Policy Measurement (TBPM) is a special iterative SHA-384 hash calculation over the Image4 manifest of a LocalPolicy, excluding anti-replay values, to give a consistent measurement over time (because anti-replay values like lpmh are frequently updated). The prot field, which is found only in each macOS LocalPolicy, provides a pairing to indicate the recoveryOS LocalPolicy that corresponds to the macOS LocalPolicy.

Has Secure Enclave Signed recoveryOS Local Policy (hr1p)

- *Type:* Boolean
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The hr1p indicates whether the prot value (above) is the measurement of a Secure Enclave–signed recoveryOS LocalPolicy. If not, then the recoveryOS LocalPolicy is signed by the Apple online signing server, which signs things such as macOS Image4 files.

Local Operating System Version (losv)

- *Type:* Boolean
- *Mutable environments:* 1TR, recoveryOS, macOS
- *Description:* The losv indicates the OS version that the LocalPolicy is created for. The version is obtained from the next state manifest during LocalPolicy creation and is used to enforce recoveryOS pairing restrictions.

Secure Multi-Boot (smb0)

- *Type:* Boolean
- *Mutable environments:* 1TR, recoveryOS
- *Description:* If smb0 is present and true, LLB allows the next stage Image4 manifest to be globally signed instead of requiring a personalized signature. Users can change this field with Startup Security Utility or bputil to downgrade to Reduced Security.

Secure Multi-Boot (smb1)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If smb1 is present and true, iBoot allows objects such as a custom kernel collection to be Secure Enclave signed with the same key as the LocalPolicy. Presence of smb0 is a prerequisite for presence of smb1. Users can change this field using command-line tools such as csrutil or bputil to downgrade to Permissive Security.

Secure Multi-Boot (smb2)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If smb2 is present and true, iBoot allows the Auxiliary Kernel Collection to be Secure Enclave signed with the same key as the LocalPolicy. The presence of smb0 is a prerequisite for the presence of smb2. Users can change this field using Startup Security Utility or bputil to downgrade to Reduced Security and enable third-party kexts.

Secure Multi-Boot (smb3)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If smb3 is present and true, a user on the device has opted in to a device management service to control their system. Presence of this field makes the LocalPolicy-controlling Secure Enclave processor application accept device management authentication instead of requiring local user authentication. Users can change this field using Startup Security Utility or bputil to enable managed control over third-party kexts and software updates. (In macOS 11.2 or later, a device management service can also initiate an update to the latest macOS version if the current security mode is Full Security.)

Secure Multi-Boot (smb4)

- *Type:* Boolean
- *Mutable environments:* macOS
- *Description:* If smb4 is present and true, the device has opted in to a device management service to control of the operating system using [Apple School Manager](#) or [Apple Business](#). Presence of this field makes the LocalPolicy-controlling Secure Enclave application accept device management authentication instead of requiring local user authentication. This field is changed by the device management service when it detects that a device's serial number appears in any of those three services.

System Integrity Protection (sip0)

- *Type:* 64 bit unsigned integer
- *Mutable environments:* 1TR
- *Description:* The sip0 holds the existing System Integrity Protection (SIP) policy bits that previously were stored in NVRAM. New SIP policy bits are added here (instead of using LocalPolicy fields like the below) if they're used only in macOS and not used by LLB. Users can change this field using csrutil from 1TR to turn off SIP and downgrade to Permissive Security.

System Integrity Protection (sip1)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If sip1 is present and true, iBoot allows failures to verify the SSV volume root hash. Users can change this field using csrutil or bputil from 1TR.

System Integrity Protection (sip2)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If sip2 is present and true, iBoot won't lock the *Configurable Text Read-only Region (CTRR)* hardware register that marks kernel memory as non-writable. Users can change this field using csrutil or bputil from 1TR.

System Integrity Protection (sip3)

- *Type:* Boolean
- *Mutable environments:* 1TR
- *Description:* If sip3 is present and true, iBoot won't enforce its built-in allow list for the boot-args NVRAM variable, which would otherwise filter the options passed to the kernel. Users can change this field using `csrutil` or `bputil` from 1TR.

Certificates and RemotePolicy

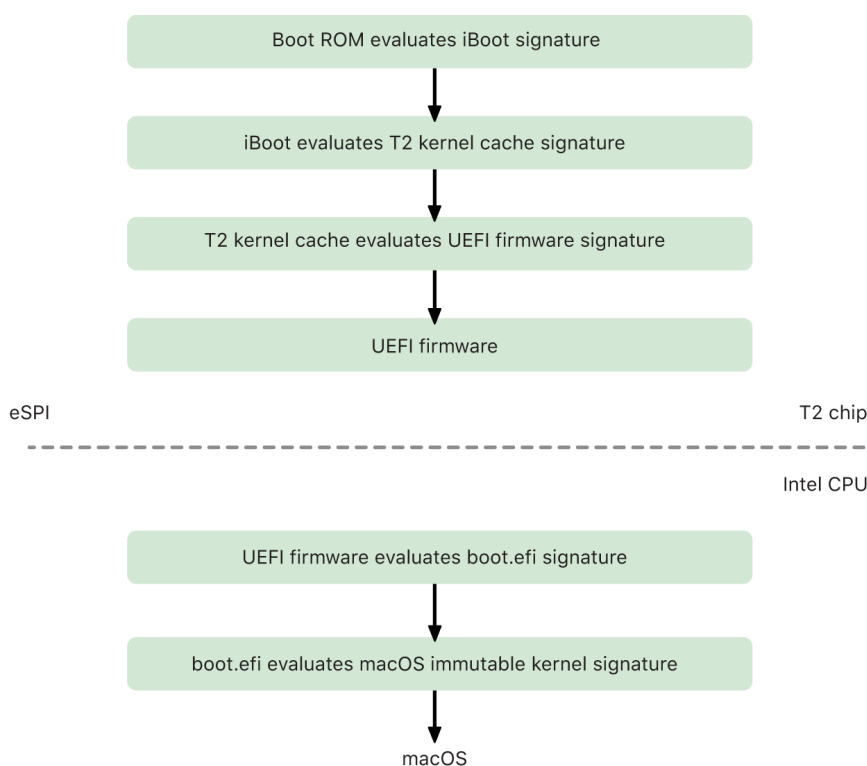
As described in [LocalPolicy signing-key creation and management](#), the LocalPolicy Image4 also contains the Owner Identity Certificate (OIC) and the embedded RemotePolicy.

Intel-based Mac computers

Boot process for an Intel-based Mac

Intel-based Mac with an Apple T2 Security Chip

When an Intel-based Mac computer with the Apple T2 Security Chip is turned on, the chip performs a secure boot from its [Boot ROM](#) in the same fashion as iPad, iPhone, and a Mac with Apple silicon. This verifies the [iBoot](#) bootloader and is the first step in the chain of trust. iBoot checks the kernel and kernel extension code on the T2 chip, which then checks the [Unified Extensible Firmware Interface \(UEFI\) firmware](#). The UEFI firmware and the associated signature are initially available only to the T2 chip.



After verification, the UEFI firmware image is mapped into a portion of the T2 chip memory. This memory is made available to the Intel CPU through the enhanced Serial Peripheral Interface (eSPI). When the Intel CPU first boots, it fetches the UEFI firmware through the eSPI from the integrity-checked, memory-mapped copy of the firmware located on the T2 chip.

The evaluation of the chain of trust continues on the Intel CPU, with the UEFI firmware evaluating the signature for the macOS bootloader (`Boot.efi`). The Intel-resident macOS secure boot signatures are stored in the same Image4 format used for iOS, iPadOS, and T2 chip secure boot, and the code that parses the Image4 files is the same hardened code from the current iOS and iPadOS secure boot implementation. `Boot.efi` in turn verifies the signature of a new file, called `immutablekernel`. When secure boot is enabled, the `immutablekernel` file represents the complete set of Apple kernel extensions required to boot macOS. The secure boot policy terminates at the handoff to the `immutablekernel`, and after that, macOS security policies (such as System Integrity Protection and signed kernel extensions) take effect.

If there are any errors or failures in this process, the Mac enters [Recovery mode](#), Apple T2 Security Chip Recovery mode, or Apple T2 Security Chip [Device Firmware Upgrade \(DFU\) mode](#).

Microsoft Windows on an Intel-based Mac with a T2 chip

By default, an Intel-based Mac that supports secure boot trust only content signed by Apple. However, to improve the security of Boot Camp installations, Apple also supports secure booting for Windows. The UEFI firmware includes a copy of the following Microsoft Bootloader certificates, which are used to authenticate the Microsoft bootloaders:

- Microsoft Corporation KEK 2K CA 2023
- Windows UEFI CA 2023
- Microsoft UEFI CA 2023
- Microsoft Option ROM UEFI CA 2023

Note: It's a known issue that Windows doesn't report the presence of these bootloader certificates when updating to a newer macOS updates.

Intel-based Mac computers without a T2 chip

Because an Intel-based Mac without a T2 chip doesn't support secure boot, the UEFI firmware loads the `boot.efi` from the file system without verification and the booter loads the kernel (prelinked kernel) from the file system without verification. To protect the integrity of the boot chain, users should enable all of the following security mechanisms:

- *System Integrity Protection (SIP)*: Enabled by default, this protects the booter and kernel against malicious writes from within a running macOS.
- *FileVault*: This can be enabled in two ways: by the user or by a [device management service](#) administrator. This protects against a physically present attacker using target disk mode to overwrite the booter.

- *Firmware Password:* This can be enabled in two ways: by the user or by a device management service administrator. This helps prevent a physically present attacker from launching alternate boot modes such as recoveryOS, Single User Mode, or target disk mode from which the booter can be overwritten. This also helps prevent booting from alternate media, by which an attacker could run code to overwrite the booter.



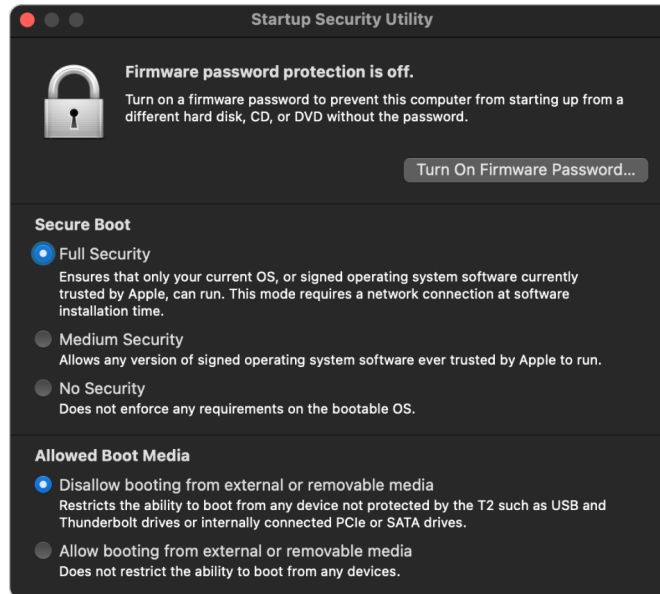
Boot modes of an Intel-based Mac with an Apple T2 Security Chip

An Intel-based Mac with an Apple T2 Security Chip has a variety of boot modes that can be entered at boot time by pressing key combinations, which are recognized by the UEFI firmware or booter. Some boot modes, such as Single User Mode, won't work unless the security policy is changed to No Security in Startup Security Utility.

Mode	Key combo	Description
macOS boot	None	The UEFI firmware hands off to the macOS booter (a UEFI application), which hands off to the macOS kernel. On standard booting of a Mac with FileVault enabled, the macOS booter presents the Login Window interface, which takes the password to decrypt the storage.
Startup Manager	Option (⌥)	The UEFI firmware launches the built-in UEFI application that presents the user with a boot device selection interface.
Target Disk Mode (TDM)	T	The UEFI firmware launches the built-in UEFI application that exposes the internal storage device as a raw, block-based storage device over FireWire, Thunderbolt, USB, or any combination of the three (depending on the Mac model).
Single User Mode	Command-S (⌘-S)	The macOS kernel passes the <code>-s</code> flag in the <code>launchd</code> argument vector, then <code>launchd</code> creates the single-user shell in the <code>Console</code> app's <code>tty</code> . <i>Note:</i> If the user exits the shell, macOS continues boot to the Login Window.
recoveryOS	Command-R (⌘-R)	The UEFI firmware loads a minimal macOS from a signed disk image (<code>.dmg</code>) file on the internal storage device.
Internet recoveryOS	Option-Command-R (⌥-⌘-R)	The signed disk image is downloaded from the internet using HTTP.
Diagnostics	D	The UEFI firmware loads a minimal UEFI diagnostic environment from a signed disk image file on the internal storage device.
Internet Diagnostics	Option-D (⌥-D)	The signed disk image is downloaded from the internet using HTTP.
Windows boot	None	If Windows has been installed using Boot Camp, the UEFI firmware hands off to the Windows booter, which hands off to the Windows kernel.

Startup Security Utility on a Mac with an Apple T2 Security Chip

On an Intel-based Mac with an Apple T2 Security Chip, Startup Security Utility handles a number of security policy settings. Users access the utility by booting into recoveryOS and choosing Startup Security Utility, and it protects supported security settings from easy manipulation by an attacker.



Critical policy changes require authentication, even in [Recovery mode](#). When Startup Security Utility is first opened, it prompts the user to enter an administrator password from the primary macOS installation associated with the currently booted recoveryOS. If no administrator exists, one needs to be created before the policy can be changed. Before the T2 chip allows a policy change, the Mac computer needs to boot into recoveryOS and the user needs to authenticate with a Secure Enclave-backed credential. Security policy changes have two implicit requirements. recoveryOS needs to:

- Boot from a storage device directly connected to the T2 chip, because partitions on other devices don't have Secure Enclave-backed credentials bound to the internal storage device.
- Reside on an [APFS](#)-based volume, because there is support only for storing the Authentication in Recovery credentials sent to the Secure Enclave on the "Preboot" APFS volume of a drive. HFS plus-formatted volumes can't use secure boot.

This policy is shown only in Startup Security Utility on an Intel-based Mac with a T2 chip. Although most use cases shouldn't require changes to the secure boot policy, users are ultimately in control of their device's settings and may choose, depending on their needs, to turn off or downgrade the secure boot functionality on their Mac.

Secure boot policy changes made from within this app apply only to the evaluation of the chain of trust being verified on the Intel processor. The option "Secure boot the T2 chip" is always in effect.

Secure boot policy can be configured to one of three settings: Full Security, Medium Security, and No Security. No Security completely turns off secure boot evaluation on the Intel processor and allows the user to boot whatever they want.

Full Security boot policy

Full Security is the default boot policy, and it behaves much like iOS and iPadOS, or Full Security on a Mac with Apple silicon. At the time that software is downloaded and prepared to install, it is personalized with a signature that includes the [Exclusive Chip Identification \(ECID\)](#)—a unique ID specific to the T2 chip in this case—as part of the signing request. The signature given back by the signing server is then unique and usable only by that particular T2 chip. The [Unified Extensible Firmware Interface \(UEFI\) firmware](#) is designed to ensure that when the Full Security policy is in effect, a given signature isn't just signed by Apple but is signed for this specific Mac, essentially tying that version of macOS to that Mac. This helps prevent rollback attacks as described for Full Security on a Mac with Apple silicon.

Medium Security boot policy

Medium Security boot policy is somewhat like a traditional UEFI secure boot, in which a vendor (here, Apple) generates a digital signature for the code to assert it came from the vendor. In this way, attackers are prevented from inserting unsigned code. We refer to this signature as a “global” signature because it can be used on any Mac, for any amount of time, for a Mac that currently has a Medium Security policy set. Neither iOS, iPadOS, nor the T2 chip itself support global signatures. This setting doesn't attempt to prevent rollback attacks.

Media boot policy

Media boot policy exists only on an Intel-based Mac with a T2 chip and is independent from the secure boot policy. So even if a user turns off secure boot, this doesn't change the default behavior of preventing anything other than the storage device directly connected to the T2 chip to boot the Mac. (Media boot policy isn't required on a Mac with Apple silicon. For more information, see [Startup Disk security policy control](#).)

Firmware password protection in an Intel-based Mac

macOS on Intel-based Mac computers with an Apple T2 Security Chip supports the use of a Firmware Password to help prevent unintended modifications of firmware settings on a specific Mac. The Firmware Password is designed to prevent selecting alternate boot modes such as booting into recoveryOS or Single User Mode, booting from an unauthorized volume, or booting into target disk mode.

Note: The firmware password isn't required on a Mac with Apple silicon, because the critical firmware functionality it restricted has been moved into the recoveryOS and (when FileVault is enabled) recoveryOS requires user authentication before its critical functionality can be reached.

The most basic mode of firmware password can be reached from the recoveryOS Firmware Password Utility on an Intel-based Mac *without* a T2 chip, and from the Startup Security Utility on an Intel-based Mac *with* a T2 chip. Advanced options (such as the ability to prompt for the password at every boot) are available from the `firmwarepasswd` command-line tool in macOS.

Setting a Firmware Password is especially important to reduce the risk of attacks on Intel-based Mac computers without a T2 chip from a physically present attacker. The Firmware Password can help prevent an attacker from booting to recoveryOS, from where they could otherwise turn off System Integrity Protection (SIP). By restricting boot of alternative media, an attacker can't execute privileged code from another operating system to attack peripheral firmwares.

A firmware password reset mechanism exists to help users who forget their password. Users press a key combination at startup, and are presented with a model-specific string to provide to AppleCare. AppleCare digitally signs a resource that's signature checked by the [Uniform Resource Identifier \(URI\)](#). If the signature is validated and the content is for the specific Mac, the UEFI firmware removes the firmware password.

For users who want no one but themselves to remove their firmware password by software means, the `-disable-reset-capability` option has been added to the `firmwarepasswd` command-line tool in macOS 10.15. Before setting this option, users need to acknowledge that if the password is forgotten and needs removal, the user needs to bear the cost of the logic board replacement necessary to achieve this. Organizations that want to protect their Mac computers from external attackers and from employees need to set a firmware password on organization-owned systems. This can be accomplished on the device in any of the following ways:

- At provisioning time, by manually using the `firmwarepasswd` command-line tool
- With third-party management tools that use the `firmwarepasswd` command-line tool
- Using device management

recoveryOS and diagnostics environments for an Intel-based Mac

recoveryOS

The recoveryOS is completely separate from the main macOS, and the entire contents are stored in a disk image file named `BaseSystem.dmg`. There is also an associated `BaseSystem.chunklist`, which is used to verify the integrity of the `BaseSystem.dmg`. The chunklist is a series of hashes for 10 MB chunks of the `BaseSystem.dmg`. The [Unified Extensible Firmware Interface \(UEFI\) firmware](#) evaluates the signature of the chunklist file and then evaluates the hash one chunk at a time from the `BaseSystem.dmg`. This helps ensure that it matches the signed content present in the chunklist. If any of these hashes don't match, booting from the local recoveryOS is aborted and the UEFI firmware attempts to boot from Internet recoveryOS instead.

If the verification is successfully completed, the UEFI firmware mounts the `BaseSystem.dmg` as a RAM disk and launches the `boot.efi` file that's in it. There's no need for the UEFI firmware to do a specific check of the `boot.efi`, nor for the `boot.efi` to do a check of the kernel, because the completed contents of the operating system (of which these elements are only a subset) have already been integrity checked.

Apple Diagnostics

The procedure for booting the local diagnostic environment is mostly the same as launching the recoveryOS. Separate `AppleDiagnostics.dmg` and `AppleDiagnostics.chunklist` files are used, but they're verified in the same way as the `BaseSystem` files are. Instead of launching `boot.efi`, the UEFI firmware launches a file inside the disk image (`.dmg` file) named `diags.efi`, which is in turn responsible for invoking a variety of other UEFI drivers that can interface with and check for errors in the hardware.

Internet recoveryOS and diagnostic environment

If an error has occurred in the launching of the local recovery or diagnostic environments, the UEFI firmware attempts to download the images from the internet instead. (A user can also specifically request the images to be fetched from the internet using special key sequences held at boot.) The integrity validation of the disk images and chunklists downloaded from the OS Recovery Server is performed the same way as with images retrieved from a storage device.

While the connection to the OS Recovery Server is done using HTTP, the complete downloaded contents are still integrity checked as previously described, and as such are protected against manipulation by an attacker with control of the network. In the event that an individual chunk fails integrity verification, it is re-requested from the OS Recovery Server 11 times, before giving up and displaying an error.

When Apple added internet recovery and diagnostic modes to Mac computers in 2011, it was decided to use the simpler HTTP transport and handle content authentication using the chunklist mechanism, rather than implement the more complicated HTTPS functionality in the UEFI firmware and as a result, increase the firmware's attack surface.

Signed system volume security

In macOS 10.15, Apple introduced the read-only system volume, a dedicated, isolated volume for system content. macOS 11 or later adds strong cryptographic protections to system content with a *signed system volume* (SSV). SSV features a kernel mechanism that verifies the integrity of the system content at runtime and rejects any data—code and noncode—without a valid cryptographic signature from Apple. On devices with iOS 15 and iPadOS 15 or later, the system volume on an iPhone or iPad also gains the cryptographic protection of a signed system volume.

SSV not only helps prevent tampering with any Apple software that's part of the operating system, it also makes macOS software update more reliable and much safer. And because SSV uses [Apple File System \(APFS\)](#) snapshots, if an update can't be performed, the old system version can be restored without reinstallation.

Since its introduction, APFS has provided file-system metadata integrity using noncryptographic checksums on the internal storage device. SSV strengthens the integrity mechanism by adding cryptographic hashes, thus extending it to encompass every byte of file data. Data from the internal storage device (including file system metadata) is cryptographically hashed in the read path, and the hash is then compared with an expected value in the file-system metadata. In case of mismatch, the system assumes the data has been tampered with and won't return it to the requesting software.

Each SSV SHA-256 hash is stored in the main file-system metadata tree, which is itself hashed. Because each node of the tree recursively verifies the integrity of the hashes of its children—similar to a binary hash (Merkle) tree—the root node's hash value, called a *seal*, encompasses every byte of data in the SSV. The cryptographic signature then covers the entire system volume.

During macOS installation and update, the seal is recomputed from the file system on-device and that measurement is verified against the measurement Apple signed. On a Mac with Apple silicon, the bootloader verifies the seal before transferring control to the kernel. On an Intel-based Mac with an Apple T2 Security Chip, the bootloader forwards the measurement and signature to the kernel, which then verifies the seal directly before mounting the root file system. In either case, if the verification fails, the startup process halts and the user is prompted to reinstall macOS. This procedure is repeated at every boot unless the user has elected to enter a lower security mode and has separately chosen to make the signed system volume unavailable.

During iOS and iPadOS software updates, the system volume is prepared and recomputed in a similar fashion. The iOS and iPadOS bootloaders verify that the seal is intact and that it matches an Apple-signed value before allowing the device to start the kernel. Mismatches at boot prompt the user to update the system software on the device. Users aren't allowed to turn off the protection of a signed system volume on iOS and iPadOS.

SSV and code signing

Code signing is still present and enforced by the kernel. The signed system volume provides protection when any bytes at all are read from the internal storage device. In contrast, code signing provides protection when Mach objects are memory mapped as executable. Both SSV and code signing protect executable code on all read and execute paths.

SSV and FileVault

In macOS 11 or later, equivalent at-rest protection for system content is provided by the SSV, and therefore the system volume no longer needs to be encrypted. Any modifications made to the file system while it's at rest are detected by the file system when they're read. If the user has turned on FileVault, the user's content on the data volume is still encrypted with a user-provided secret.

If the user chooses to turn off the SSV, the system at rest becomes vulnerable to tampering, and this tampering could enable an attacker to extract encrypted user data when the system next starts up. Therefore the system won't permit the user to turn off the SSV if FileVault is turned on. Protection while at rest needs to be turned on or off for both volumes in a consistent manner.

In macOS 10.15 or earlier, FileVault protects operating system software while at rest by encrypting user and system content with a key protected by a user-provided secret. This protects against an attacker with physical access to the device from accessing or effectively modifying the file system containing system software.

SSV and a Mac with an Apple T2 Security Chip

On a Mac with an Apple T2 Security Chip, only macOS itself is protected by the SSV. The software that runs on the T2 chip and verifies macOS is protected by secure boot.

Secure software updates

Security is a process and isn't enough to reliably boot the operating system version installed at the factory—there needs to also exist a mechanism to quickly and securely obtain the latest security updates. Apple regularly releases software updates to address emerging security concerns. Users of iPad and iPhone devices receive update notifications on the device. Mac users find available updates in System Settings (macOS 13 or later), or System Preferences (macOS 12 or earlier). Updates are delivered wirelessly, for rapid adoption of the latest security fixes.

Update process security

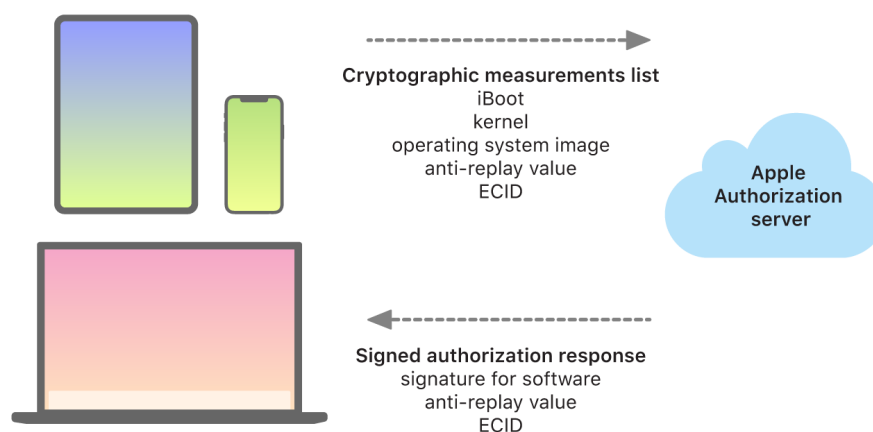
The update process uses the same hardware-based root of trust that secure boot uses that's designed to install only Apple-signed code. The update process also uses [system software authorization](#) to check that only copies of operating system versions that are actively being signed by Apple can be installed on iPad and iPhone devices, or on a Mac with the Full Security setting configured as the secure boot policy in Startup Security Utility. With these secure processes in place, Apple can stop signing older operating system versions with known vulnerabilities and help prevent downgrade attacks.

For greater software update security, when the device to be upgraded is physically connected to a Mac, a full copy of iOS or iPadOS is downloaded and installed. But for over-the-air (OTA) software updates, *only the components required to complete an update are downloaded*, improving network efficiency by not downloading the entire operating system. What's more, software updates can be cached on a Mac with macOS 10.13 or later with Content Caching turned on, so that iPad and iPhone devices don't need to redownload the necessary update over the internet. (They still need to contact Apple servers to complete the update process.)

Personalized update process

During upgrades and updates, certain information is made available to the Apple installation authorization server, which includes a list of cryptographic measurements for each part of the installation bundle to be installed (for example, iBoot, the kernel, and the operating system image), a random anti-replay value, and the device's unique [Exclusive Chip Identification \(ECID\)](#).

The authorization server checks the presented list of measurements against versions whose installation is permitted and, if it finds a match, adds the ECID to the measurement and signs the result. The server passes a complete set of signed data to the device as part of the upgrade process. Adding the ECID “personalizes” the authorization for the requesting device. By authorizing and signing only for known measurements, the server helps ensure that the update takes place exactly as Apple provided.



The boot-time chain-of-trust evaluation verifies that the signature comes from Apple and that the measurement of the item loaded from the storage device, combined with the device's ECID, matches what was covered by the signature. These steps are designed to ensure that, on devices that support personalization, the authorization is for a specific device and that an older operating system or firmware version from one device can't be copied to another. The anti-replay value helps prevent an attacker from saving the server's response and using it to tamper with a device or otherwise alter the system software.

The personalization process is why a network connection to Apple is always required to update any device with Apple-designed silicon, including an Intel-based Mac with the Apple T2 Security Chip.

On devices with the Secure Enclave, that hardware similarly uses [system software authorization](#) to check the integrity of its software and is designed to prevent downgrade installations.

Background Security Improvements in Apple operating systems

Background Security Improvements are supported and enabled for future releases starting with iOS 26.1, iPadOS 26.1, and macOS 26.1. This enables rapid delivery of ongoing and regular lightweight security releases between software updates for components such as the Safari browser, WebKit framework stack, and other system libraries that benefit from smaller, ongoing security patches. In rare instances of compatibility issues, Background Security Improvements may be temporarily removed and then enhanced in a subsequent software update.

Background Security Improvement security content is published in the Apple Support article [Apple security releases](#).

Background Security Improvements differ from software updates in the following ways:

- On devices with macOS, Safari security improvements delivered through a Background Security Improvement become active as soon as Safari is relaunched, even before the whole operating system restarts.
- Background Security Improvements are activated instantly upon restart; the system volume doesn't need to be cryptographically resealed, and therefore the device doesn't need to cycle through a RAM disk.
- Background Security Improvements require a much lower battery state of charge than is required to install a software update.
- A Background Security Improvement can be removed, which reverts the device to the baseline software update state with no Background Security Improvements applied.
- A Background Security Improvement can be reapplied after removal.

The system volume in iOS, iPadOS, and macOS has been reorganized to support Background Security Improvements. Content that can be patched using the Background Security Improvement mechanism has been moved into cryptexes, which are optimized, cryptographically sealed disk images that reside on the preboot volume alongside other boot firmware. Cryptexes have different subtypes for operating system framework components and apps, and they can be updated by applying a binary patch to their backing disk image file.

Cryptex content is bootstrapped after the kernel has booted. The measurements of the cryptexes, their file system seals, and their associated trust caches are all represented in a separate Image4 ticket, which is cryptographically bound to the device on which it resides. When a Background Security Improvement is being applied, the device sends a request to Apple's trusted signing service to obtain a corresponding Cryptex1Image4 manifest; the existing AP boot ticket isn't updated.

On devices with macOS, a Background Security Improvement may offer the user the option to apply the changes in the Background Security Improvement to the Safari web browser by quitting and relaunching it. After Safari has been relaunched, it uses the framework and library content from the new cryptexes. The rest of the operating system remains unaffected and doesn't make use of the new content until the system restarts.

Removing Background Security Improvements

Background Security Improvements are intended to be removable in case a critical regression associated with the Background Security Improvement is discovered. Users can also elect to remove all Background Security Improvements currently applied on their device. In addition, in the rare event that a Background Security Improvement affects software compatibility or quality, Apple may remove the most recently applied Background Security Improvement from users' devices using the Automatic Software Update mechanism. To facilitate removal, Background Security Improvements ship both a patch and an antipatch to the device.

If the user removes Background Security Improvements, the removal action removes all currently installed Background Security Improvements (returning all cryptexes to their base state) and returns patched system binaries to the version from the last installed software update. The user needs to restart the device to complete Background Security Improvement removal. For more information, see the Apple Support article [If you need to remove a Background Security Improvement](#).

Removal recommendations

If Apple observes that a Background Security Improvement is contributing to a potentially higher rate of application crashes, the identities of the affected applications are published by a service that devices regularly query. Devices having that Background Security Improvement installed then use an on-device analytics system to see whether one of the affected applications has crashed significantly more often after installing the Background Security Improvement. If the user encounters such a crash, the operating system lets them know the Background Security Improvement may be contributing to the issue, and the user is offered the opportunity to remove all Background Security Improvements and restore the device to its last software update.

Analytics related to removal recommendations (for example, the app that triggered the recommendation or the fact that the recommendation was displayed) are sent to Apple only if the user has agreed to share this information using the following settings:

- *iPad*: Settings > Privacy & Security > Analytics & Improvements > Share iPad Analytics
- *iPhone*: Settings > Privacy & Security > Analytics & Improvements > Share iPhone & Watch Analytics
- *Mac*: Settings > Privacy & Security > Analytics & Improvements > Share Mac Analytics

Operating system integrity

Apple designs its operating system software with security at its core. This design includes a hardware root of trust that enables secure boot and a secure software update process that's quick and safe. Apple's operating systems also use their purpose-built silicon-based hardware capabilities to help prevent exploitation as the operating system runs. These runtime features protect the integrity of trusted code as it executes. Apple's operating system software helps mitigate attack and exploit techniques—whether those originate from a malicious app, from the web, or through any other channel. These protections are available on devices with supported Apple-designed SoCs, including iOS, iPadOS, macOS on a Mac with Apple silicon, tvOS, visionOS, and watchOS.

Feature	A10	A11, S3	A12-A14 S4-S10	A15-A18	M1	M2-M4	A19 M5
Kernel Integrity Protection	✓	✓	✓	✓	✓	✓	✓
Fast Permission Restrictions	✗	✓	✓	✓	✓	✓	✓
System Coprocessor Integrity Protection	✗	✗	✓	✓	✓	✓	✓
Pointer Authentication Codes	✗	✗	✓	✓	✓	✓	✓

Page Protection Layer	✗	✓	✓	✗ 1	✓ 2	✗	✗
Secure Page Table Monitor	✗	✗	✗	✓	✗	✓ 2	✓ 2
Memory Integrity Enforcement with Enhanced Memory Tagging Extension	✗	✗	✗	✗	✗	✗	✓

1: Secure Page Table Monitor (SPTM) is supported in the A15 or later and M2 or later SOCs and replaces Page Protection Layer on supported platforms.

2: Page Protection Layer (PPL) and Secure Page Table Monitor (SPTM) enforce the execution of signed and trusted code on all platforms with the exception of macOS (because macOS is designed to run any code). All of the other security properties, including the protection of page tables, are present across all supported platforms.

Kernel Integrity Protection

After the operating system kernel completes initialization, Kernel Integrity Protection (KIP) is enabled to help prevent modifications of kernel and driver code. The [memory controller](#) provides a protected physical memory region that [iBoot](#) uses to load the kernel and kernel extensions. After startup is complete, the memory controller denies writes to the protected physical memory region. The Application Processor's Memory Management Unit (MMU) is configured to help prevent mapping privileged code from physical memory outside the protected memory region and to help prevent writeable mappings of physical memory within the kernel memory region.

To prevent reconfiguration, the hardware used to enable KIP is locked after the boot process is complete.

Fast Permission Restrictions

Starting with the Apple A11 Bionic and S3 SoCs, a new hardware primitive was introduced. This primitive, Fast Permission Restrictions, includes a CPU register that quickly restricts permissions per thread. With Fast Permission Restrictions (also known as APRR registers), supported operating systems can remove execute permissions from memory without the overhead of a system call and a page table walk or flush. These registers provide one more level of mitigation for attacks from the web, particularly for code compiled at runtime (just-in-time compiled)—because memory can't be effectively executed at the same time it's being read from and written to.

System Coprocessor Integrity Protection

Coprocessor firmware handles many critical system tasks—for example, the Secure Enclave, the image sensor processor, and the motion coprocessor. Therefore its security is a key part of the security of the overall system. To prevent modification of coprocessor firmware, Apple uses a mechanism called *System Coprocessor Integrity Protection (SCIP)*.

SCIP works much like KIP: At boot time, iBoot loads each coprocessor's firmware into a protected memory region, one that's reserved and separate from the KIP region. iBoot configures each coprocessor's memory unit to help prevent:

- Executable mappings outside its part of the protected memory region
- Writeable mappings inside its part of the protected memory region

Also at boot time, to configure SCIP for the Secure Enclave, the Secure Enclave operating system is used. After the boot process is complete, the hardware used to enable SCIP is locked. This is designed to prevent reconfiguration.

Pointer Authentication Codes

Pointer Authentication Codes (PACs) are used to protect against exploitation of memory corruption bugs. System software and built-in apps use PAC to help prevent modification of function pointers and return addresses (code pointers). PAC uses five secret 128-bit values to sign kernel instructions and data, and each user space process has its own B keys. Items are salted and signed as indicated below.

Item	Key	Salt
Function Return Address	IB	Storage address
Function Pointers	IA	0
Block Invocation Function	IA	Storage address
Block Descriptor Pointer	DA	Storage address + 0xC0BB
Objective-C Method Cache	IB	Storage address + Class + Selector
Objective-C Isa Pointer	DA	Storage address + 0x6AE1
Objective-C Super Pointer	DA	Storage address + 0xB5AB
Selector Typed Objective-C ivars	DB	Storage address + 0x57C2
Objective-C Read-only Class Data Pointer	DA	Storage address + 0x61F8
C++ V-Table Entries	IA	Storage address + Hash (mangled method name)
C++ V-Table Pointers	DA	Storage address + Hash (mangled base V-Table name)
Computed Goto Label	IA	Hash (function name)
Kernel Thread State	GA	•
User Thread State Registers	IA	Storage address

The signature value is stored in the unused padding bits at the top of the 64-bit pointer. The signature is verified before use, and the padding is restored to help ensure a functioning pointer address. Failure to verify results in an abort. This verification increases the difficulty of many attacks, such as a return-oriented programming (ROP) attack, which attempts to trick the device into executing existing code maliciously by manipulating function return addresses stored on the stack.

Page Protection Layer

Page Protection Layer (PPL) in iOS, iPadOS, visionOS, and watchOS is designed to prevent user space code from being modified after code signature verification is complete. Building on KIP and Fast Permission Restrictions, PPL manages the page table permission overrides to make sure only the PPL can alter protected pages containing user code and page tables. The system provides a massive reduction in attack surface by supporting systemwide code integrity enforcement, even in the face of a compromised kernel. This protection isn't offered in macOS because PPL is only applicable on systems where all executed code needs to be signed.

Secure Page Table Monitor and Trusted Execution Monitor

Secure Page Table Monitor (SPTM) and Trusted Execution Monitor (TXM) in iOS, iPadOS, macOS, and visionOS are designed to work together to help protect page tables for both user and kernel processes against modification. This includes when attackers have kernel write capabilities and can bypass control flow protections. SPTM does this by utilizing a higher privilege level than the kernel, and utilizing the lower privileged TXM to actually enforce the policies that govern code execution. This system is designed so that a TXM compromise doesn't automatically translate to an SPTM bypass due to this privilege separation and the governing of trust between them. In the A15 or later and M2 or later SOCs, SPTM (in combination with TXM) replaces the PPL, providing a smaller attack surface that doesn't rely on trust of the kernel, even during early boot. SPTM relies on new silicon primitives that are an evolution of the Fast Permission Restrictions that PPL utilizes, and are available only on the processors listed in the table above.

Memory Integrity Enforcement

Memory Integrity Enforcement (MIE), is a comprehensive memory safety defense for Apple platforms available on A19 and M5 processors or later. MIE is built on the robust foundation provided by Apple's secure memory allocators, coupled with Enhanced Memory Tagging Extension (EMTE) in synchronous mode, and supported by extensive Tag Confidentiality Enforcement policies. MIE is built in to Apple silicon and offers unparalleled, always-on memory safety protection for key attack surfaces including the kernel, while maintaining the power and performance that users expect. For more information, see [Memory Integrity Enforcement: A complete vision for memory safety in Apple devices](#) on Apple Security Research blog.

Memory Tagging Extension

Arm published the Memory Tagging Extension (MTE) specification in 2019 as a tool for hardware to help find memory corruption bugs. MTE is a memory tagging and tag-checking system, where every memory allocation is tagged with a secret. The hardware guarantees that later requests to access memory are granted only if the request contains the correct secret. If the secrets don't match, the app crashes, and the event is logged. This allows developers to identify memory corruption bugs immediately as they occur.

Enhanced Memory Tagging Extension

EMTE closes the holes that prevent MTE from being an active defense, including only supporting the more secure synchronous mode. In addition, accessing nontagged memory from a tagged memory region requires knowing that region's tag, making it significantly harder for attackers to turn out-of-bounds bugs in dynamic tagged memory into a way to sidestep EMTE by directly modifying nontagged allocations.

Tag Confidentiality Enforcement

Tag Confidentiality Enforcement protects the implementation of the secure allocators from technical threats and guards the confidentiality of EMTE tags—including against side-channel and speculative-execution attacks. The [Secure Page Table Monitor](#) protects the kernel allocator backing store and tag storage. The system also ensures that when the kernel accesses memory on behalf of an application, it's subject to the same tag-checking rules as userspace. Tag Confidentiality Enforcement also is designed to mitigate against tag leakage due to timing or speculative attacks, and even includes a protection against Spectre V1.

Device pairing and connection security

Physical pairing model security for iPad and iPhone

iOS and iPadOS use a pairing model to control access to a device from a host computer. This establishes a trust relationship between the device and its connected host, signified by public key exchange. iOS and iPadOS also use this sign of trust to enable additional functionality with the connected host, such as data syncing. On an iPhone with iOS 9 or later or an iPad with iPadOS 13.1 or later, services:

- That require pairing can't be started until after the device has been unlocked by the user
- Won't start unless the device has been recently unlocked
- May (such as with photo syncing) require the device to be unlocked to begin

The pairing process requires the user to unlock the device and accept the pairing request from the host. On an iPhone with iOS 9 or later or an iPad with iPadOS 13.1 or later, the user is also required to enter their passcode, after which the host and device exchange and save 2048-bit RSA public keys. The host is then given a 256-bit key that can unlock an escrow keybag stored on the device. The exchanged keys are used to start an encrypted SSL session, which the device requires before it sends protected data to the host or starts a service Finder or Music app syncing, file transfers, Xcode development, and so on. To use this encrypted session for all communication, the device requires connections from a host over Wi-Fi, so it needs to have been previously paired over a physical connection (Thunderbolt or USB). Pairing also enables several diagnostic capabilities. On devices with iOS 11 and iPadOS 13.1, or later, if a pairing record hasn't been used for more than 30 days, it expires.

Certain diagnostic services, including `com.apple.mobile.pcapd`, are restricted to work only over USB. Additionally, the `com.apple.file_relay` service requires an Apple-signed configuration profile to be installed. On devices with iOS 11 and iPadOS 13.1, or later, Apple TV can use the Secure Remote Password protocol to wirelessly establish a pairing relationship.

A user can clear the list of trusted hosts with the Reset Network Settings or Reset Location & Privacy options.

Activating data connections securely

On iPad and iPhone devices and portable Mac computers with Apple silicon, if no data connection has been established recently, users need to use Face ID, Touch ID, or a passcode to activate data connections through a Thunderbolt, USB, Lightning, Smart Connector, or—in macOS 13.3 or later—the SD Extended Capacity (SDXC) cards interface. This limits the attack surface against physically connected devices, such as malicious chargers, while still enabling usage of other accessories within reasonable time constraints. If more than an hour has passed since the iPad or iPhone has locked or since an accessory's data connection has been terminated, the device won't allow any new data connections to be established until the device is unlocked. During this hour period, only data connections from accessories that have been previously connected to the device while in an unlocked state will be allowed. These accessories are remembered for 30 days after the last time they were connected. Attempts by an unknown accessory to open a data connection during this period inactivates all accessory data connections over those connections until the device is unlocked again. This hour period:

- Helps ensure that frequent users of connections to a Mac or PC, to accessories, or wired to CarPlay won't need to enter their passcodes every time they attach their device
- Is necessary because the accessory ecosystem doesn't provide a cryptographically reliable way to identify accessories before establishing a data connection

In addition, if it's been more than three days since a data connection has been established with an accessory, the device will disallow new data connections immediately after it locks. This is to increase protection for users that don't often make use of such accessories. These data connections are also inactive whenever the device is in a state where it requires a passcode to reenable biometric authentication.

The user can choose to reenable always-on data connections in Settings (setting up some assistive devices does this automatically).

Verifying accessories for Apple devices and services

The MFi licensing program provides vetted accessory manufacturers access to the iPod Accessories Protocol (iAP) and the necessary supporting hardware components.

When an MFi accessory communicates with an iPad, iPhone, or Apple Watch, the accessory needs to prove to Apple that it's been vetted. (The accessory-device connection is with Thunderbolt, Lightning, Bluetooth, or for specific devices, USB-C.) As proof of authorization, the accessory sends an Apple-provided certificate to the device, which the device then verifies. The device then sends a challenge, which the accessory needs to answer with a signed response. This process is entirely handled by a custom [integrated circuit \(IC\)](#) that Apple provides to approved accessory manufacturers and is transparent to the accessory itself.

Verified MFi accessories can request access to different transport methods and functionality—for example, access to digital audio streams over the Thunderbolt cable, or location information provided over Bluetooth. An authentication IC is designed to help ensure that only approved MFi accessories are granted full access to the device. If an accessory doesn't support authentication, its access is limited to analog audio and a small subset of serial (UART) audio playback controls.

AirPlay also uses the authentication IC to verify that receivers have been approved by Apple. AirPlay audio and CarPlay video streams use the MFi-SAP (Secure Association Protocol), which encrypts communication between the accessory and device using AES-128 in counter (CTR) mode. Ephemeral keys are exchanged using ECDH key exchange (Curve25519) and signed using the authentication IC's 1024-bit RSA key as part of the Station-to-Station (STS) protocol.

Automatically unlock Apple devices

For greater convenience when using multiple Apple devices, some devices can automatically unlock others in certain situations. Automatic unlocking can be done with the following:

- An Apple Watch can be unlocked by an iPhone.
- A Mac can be unlocked by an Apple Watch.
- An iPhone can be unlocked by an Apple Watch when a user is detected with their nose and mouth covered.
- An iPhone can be unlocked by an Apple Vision Pro.
- An iPhone can be unlocked and viewed on a Mac using iPhone Mirroring.

All use cases are built upon the same basic foundation: a mutually authenticated Station-to-Station (STS) protocol, with Long-Term Keys exchanged at time of feature enablement and unique ephemeral session keys negotiated for each request. Regardless of the underlying communication channel, the STS tunnel is negotiated directly between the Secure Enclaves in both devices, and all cryptographic material is kept within that secure domain (with the exception of Mac computers without a Secure Enclave, which terminate the STS tunnel in the kernel).

To unlock one device with another, both devices need to be signed into the same Apple Account using two-factor authentication, and the user needs to enable each specific unlocking relationship between the two devices.

Unlocking

A complete unlock sequence can be broken down in two phases.

- The device being unlocked (the *target*) generates a cryptographic unlock secret and sends it to the device performing the unlock (the *initiator*).
- The initiator performs the unlock using the previously generated secret.

To prepare for devices to automatically unlock, the devices connect to each other using a Bluetooth Low Energy (BLE) connection. Then a 32-byte unlock secret randomly generated by the target device is sent to the initiator over the STS tunnel. During the next biometric or passcode unlock, the target device wraps its [passcode-derived key \(PDK\)](#) with the unlock secret and discards the unlock secret from its memory.

To perform the unlock, the devices initiate a new BLE connection and then use peer-to-peer Wi-Fi to securely approximate the distance between each other. If the devices are within the specified range and the required security policies are met, the initiator sends its unlock secret to the target through the STS tunnel. The target then generates a new 32-byte unlock secret and returns it to the initiator. If the current unlock secret sent by the initiator successfully decrypts the unlock record, the target device is unlocked and the PDK is rewrapped with a new unlock secret. Finally, the new unlock secret and PDK are then discarded from the target's memory.

Mac unlock using Apple Watch

The unlocking flow described above is used when using an Apple Watch to unlock a paired Mac, and can also be used to approve app requests—like viewing passwords or downloading an app—without having to enter a password. When Apple Watch successfully unlocks a paired iPhone, the watch displays a notification and plays an associated haptic.

Successfully unlocking a paired Mac from Apple Watch requires that all of the following criteria be met:

- The Mac needs to be unlocked using another method at least once after the associated Apple Watch was placed on wrist and unlocked.
- Distance measured between the Mac and Apple Watch needs to be 2–3 meters or less.
- Apple Watch needs to be unlocked.
- Apple Watch can't be in bedtime mode.

iPhone unlock using Apple Watch

Additional security policies apply to iPhone unlock using Apple Watch. If the user taps the Lock iPhone button in the notification, the watch sends the iPhone a lock command over BLE. When the iPhone receives the lock command, it locks and disallows both Face ID and unlock using other devices. The next iPhone unlock needs to be performed with the iPhone passcode. Apple Watch can't be used in place of Face ID on iPhone for other operations, such as Apple Pay or app authorizations. When Apple Watch successfully unlocks a paired iPhone, the watch displays a notification and plays an associated haptic.

Successfully unlocking a paired iPhone from Apple Watch (when enabled) requires that all of the following criteria be met:

- iPhone needs to have been unlocked:
 - Using another method at least once after the associated Apple Watch was placed on wrist and unlocked.
 - At least once in the past 6.5 hours.
- Apple Watch or iPhone needs to have been unlocked recently, or Apple Watch needs to have experienced physical motion indicating that the wearer is active (for example, not asleep).
- Sensors need to be able to detect that the nose and mouth are covered.
- Distance measured between iPhone and Apple Watch needs to be 2–3 meters or less.
- Apple Watch can't be in bedtime mode.
- iPhone needs to be in a state where Face ID is allowed to perform a device unlock. (For more information, see [Optic ID, Face ID, Touch ID, passcodes, and passwords](#).)

iPhone unlock using Apple Vision Pro

Similar security policies apply to iPhone unlock with Apple Vision Pro. The user can pair their Apple Vision Pro with an iPhone to enable automatic unlock of that iPhone by Apple Vision Pro, and for in-app authentication using Apple Vision Pro in supported iPhone apps. When Apple Vision Pro successfully unlocks a paired iPhone, the Apple Vision Pro displays a notification. If the user taps the Lock iPhone button in the notification, the Apple Vision Pro sends the iPhone a lock command over BLE. When the iPhone receives the lock command, it locks and disallows both Face ID and unlock using other devices. The next iPhone unlock needs to be performed with the iPhone passcode. Apple Vision Pro can't be used in place of Face ID on iPhone for Apple Pay.

Successfully unlocking a paired iPhone from Apple Vision Pro (when enabled) requires that all of the following criteria be met:

- iPhone needs to be unlocked using another method at least once since it booted.
- Apple Vision Pro needs to be unlocked and in use.
- Apple Vision Pro needs to optically detect the iPhone is within approximately one meter or less of the user, and that the user is looking at said iPhone.
- Distance measured between iPhone and Apple Vision Pro needs to be approximately one meter or less.
- iPhone needs to be in a state where Face ID is allowed to perform a device unlock. (For more information, see [Optic ID](#), [Face ID](#), [Touch ID](#), [passcodes](#), and [passwords](#).)

Apple Watch unlock using iPhone

For added convenience, Apple Watch can be unlocked by an iPhone directly after initial startup, without requiring the user to enter the passcode on the Apple Watch itself. To achieve this, the random unlock secret (generated during the very first unlock sequence after enablement of the feature) is used to create a long-term escrow record, which is stored in the Apple Watch keybag. The escrow record secret is stored in the iPhone keychain and used to bootstrap a new session after each Apple Watch restart.

iPhone Mirroring security

iPhone Mirroring allows a user to use their iPhone from their nearby Mac. While being remotely used on the Mac the iPhone remains locked and users see a persistent notification on the iPhone's Lock Screen. A banner is shown the first time the iPhone is unlocked after a session has ended.

Notification forwarding

iPhone Mirroring lets users forward notifications from their iPhone to a Mac using the same Apple Account. Users signed in to devices with the same Apple Account exchange cryptographic identities using a local peer-to-peer protocol, encrypted using keys stored in iCloud using end-to-end encryption. When the user enables iPhone Mirroring and enters their passcode on the iPhone, the current cryptographic identity for the Mac is recorded. The private key for this identity is protected in the Secure Enclave. This identity is pinned so that if it changes, notifications aren't forwarded to the Mac. Notifications are encrypted in transit using end-to-end encryption.

Remote unlock

Remote unlock for iPhone Mirroring uses the same remote unlock protocol as iPhone unlock using Apple Watch but is initiated by the user launching the iPhone Mirroring app on their paired Mac. Secure ranging isn't required for iPhone Mirroring.

When users set up iPhone Mirroring for the first time, they're prompted to choose to either "Automatically authenticate," or to "Ask every time." The Secure Enclave on the Mac enforces this user choice and prompts the user to authenticate using their Mac password (or Touch ID if supported). After the authentication policy is completed, the Mac connects to the iPhone using a local wireless peer-to-peer connection and unlocks the iPhone keybag to enable remote access for the duration of the remote session.

BlastDoor for Messages and IDS

Devices with iOS 14, iPadOS 14, macOS 11, watchOS 7, or later include a security feature called *BlastDoor*. The goal of BlastDoor is to help protect the system by corralling attackers—increasing the complexity of their efforts to exploit Messages and Apple Identity Services (IDS). BlastDoor isolates, parses, transcodes, and validates untrusted data arriving in Messages, IDS and other vectors to help prevent attacks.

BlastDoor does this by employing sandbox restrictions and memory safe validation of output which creates a significant obstacle for attackers to overcome before reaching other parts of the operating system. It's designed to drastically improve user protection against attacks, particularly "0-click" attacks—those that don't require user interaction.

Finally, Messages treats traffic from known senders differently than traffic from unknown senders, offering a different set of functionality to each group and segmenting known versus unknown data into distinct BlastDoor instances.

Lockdown Mode security for Apple devices

Lockdown Mode is an optional, extreme protection that's designed for the very few individuals who, because of who they are or what they do, might be personally targeted by some of the most sophisticated digital threats such as targeted mercenary spyware. Most people are never targeted by attacks of this nature.

When Lockdown Mode is turned on, a device doesn't function the way it typically does. To reduce the attack surface that potentially could be exploited, certain apps, websites, and features are strictly limited for security, and some experiences might not be available at all.

Lockdown Mode is available in iOS 16, iPadOS 16, macOS 13, watchOS 10, or later. Additional protections are available in iOS 17, iPadOS 17, macOS 14, and updates to watchOS 10.1, or later. To take advantage of additional features of Lockdown Mode, devices should be updated to the latest operating system. For more information, see the Apple Support article [About Lockdown Mode](#).

Lockdown Mode makes tradeoffs for increased security at the expense of functionality, performance, or both. These tradeoffs impact:

- Background services
- Connectivity

- Device management
- FaceTime
- GameCenter
- Mail
- Messages
- Photos
- Safari
- System Settings
- WebKit

Overview of additional macOS system security capabilities

Additional macOS system security capabilities

macOS operates on a broader set of hardware (for example, Intel-based CPUs, Intel-based CPUs in combination with the Apple T2 Security Chip, and Apple silicon-based SoCs) and supports a range of general-purpose computing use cases. Whereas some users use only the preinstalled apps or those available from the App Store, others are kernel hackers who need to make platform protections inactive so they can run and test their code with the highest levels of trust. Most fall somewhere between, and many of those have peripherals and software that require varying levels of access.

Apple designed the macOS platform with an integrated approach to hardware, software, and services—a platform that provides security by design and makes it simple to configure, deploy, and manage but that retains the configurability that users expect. macOS also includes the key security technologies that an IT professional needs to help protect corporate data and integrate within secure enterprise networking environments.

The following capabilities support and help secure the varied needs of macOS users. They include:

- Signed system volume security
- System Integrity Protection
- Trust caches
- Protection for peripherals
- Rosetta 2 (automatic translation) support and security for a Mac with Apple silicon
- DMA support and protections
- Kernel extension (kext) support and security
- Option ROM support and security
- UEFI firmware security for Intel-based Mac computers

System Integrity Protection

macOS utilizes kernel permissions to limit writability of critical system files with a feature called System Integrity Protection (SIP). This feature is separate and in addition to the hardware-based Kernel Integrity Protection (KIP) available on a Mac with Apple silicon, which protects modification of the kernel in memory. Mandatory access control technology is leveraged to provide this and a number of other kernel level protections, including sandboxing and Data Vault.

Mandatory access controls

macOS uses mandatory access controls—policies that set security restrictions, created by the developer, that can't be overridden. This approach is different from discretionary access controls, which permit users to override security policies according to their preferences.

Mandatory access controls aren't visible to users, but they're the underlying technology that helps enable several important features, including sandboxing, parental controls, managed preferences, extensions, and SIP.

Restricting components

SIP restricts components to read-only in specific critical file system locations to help prevent malicious code from modifying them. SIP is a computer-specific setting that's on by default when a user upgrades to OS X 10.11 or later. On an Intel-based Mac, turning off SIP removes protection for all partitions on the physical storage device. macOS applies this security policy to every process running on the system, regardless of whether that process is running sandboxed or with administrative privileges.

Trust caches

One of the objects included in the Secure Boot chain is the *static trust cache*, a trusted record of all the Mach-O binaries that are assembled into the signed system volume. Each Mach-O is represented by a code directory hash. For efficient searching, these hashes are sorted before being inserted into the trust cache. The code directory is the result of the signing operation performed by `codesign(1)`. To enforce the trust cache, SIP needs to remain active. To deactivate trust cache enforcement on a Mac with Apple silicon, secure boot needs to be configured to Permissive Security.

When a binary runs (whether as part of spawning a new process or mapping executable code into an existing process), its code directory is extracted and hashed. If the resulting hash is found in the trust cache, the executable mappings created for the binary will be granted platform privileges—that is, they may possess any entitlement and execute without further verification as to the authenticity of the signature. This is in contrast to an Intel-based Mac, where platform privileges are conveyed to operating system content by the Apple certificate that signs the binaries. (This certificate doesn't constrain which entitlements the binary may possess.)

Nonplatform binaries (for example, notarized third-party code) need to have valid certificate chains to execute, and the entitlements they may possess are constrained by the signing profile issued to the developer by the Apple Developer Program.

All binaries shipped within macOS are signed with a *platform identifier*. On a Mac with Apple silicon, this identifier is used to indicate that even though the binary is signed by Apple, its code directory hash needs to be present in the trust cache to execute. On an Intel-based Mac, the platform identifier is used to perform targeted revocation of binaries from an older release of macOS; this targeted revocation helps prevent those binaries from executing on newer versions.

The static trust cache completely locks a set of binaries to a given version of macOS. This behavior helps prevent legitimately Apple-signed binaries from older operating systems from being introduced into newer ones for an attacker to gain advantage.

Platform code shipped outside the operating system

Apple ships some binaries—for example, Xcode and the development tools stack—that aren't signed with a platform identifier. They're still permitted to execute with platform privileges on a Mac with Apple silicon and a Mac with a T2 chip. Because this platform software is shipped independently of macOS, it isn't subject to the revocation behaviors imposed by the static trust cache.

Loadable trust caches

Apple ships certain software packages with *loadable trust caches*. These caches have the same data structure as the static trust cache. Although there's only one static trust cache—and its contents are always guaranteed to be locked into read-only ranges after the kernel's early initialization is complete—loadable trust caches are added to the system at runtime.

These trust caches are authenticated either through the same mechanism that authenticates boot firmware (personalization using the Apple trusted signing service) or as globally signed objects (whose signatures don't bind them to a particular device).

One example of a personalized trust cache is the cache, shipped with the disk image that's used to perform field diagnostics on a Mac with Apple silicon. This trust cache is personalized, along with the disk image, and loaded into the subject Mac computer's kernel while it's booted into a diagnostic mode. The trust cache allows the software within the disk image to run with platform privilege.

An example of a globally signed trust cache is shipped with macOS software updates. This trust cache permits a chunk of code within the software update—the *update brain*—to run with platform privilege. The update brain performs any work to stage the software update that the host system lacks the capacity to perform in a consistent fashion across versions.

Peripheral processor security in Mac computers

All modern computing systems have many built-in peripheral processors dedicated to tasks such as networking, graphics, power management, and more. These peripheral processors are often single-purpose and are much less powerful than the primary CPU. Built-in peripherals that don't implement sufficient security become an easier target for attackers to exploit, through which they can persistently infect the operating system. Having infected a peripheral processor firmware, an attacker could target software on the primary CPU or directly capture sensitive data (For example, an Ethernet device could see the contents of packets that aren't encrypted.)

Whenever possible, Apple works to reduce the number of peripheral processors necessary and avoid designs that require firmware. When separate processors with their own firmware are required, efforts are taken to help ensure an attacker can't persist on that processor by verifying the processor in one of two ways:

- Running the processor so that it downloads verified firmware from the primary CPU on startup
- Having the peripheral processor implement its own secure boot chain, to verify the peripheral processor firmware every time the Mac starts up

Apple works with vendors to audit their implementations and enhance their designs to include desired properties such as:

- Ensuring minimum cryptographic strengths
- Ensuring strong revocation of known bad firmware
- Turning off debug interfaces
- Signing the firmware with cryptographic keys that are stored in Apple-controlled hardware security modules (HSMs)

In recent years, Apple has worked with some external vendors to adopt the same “Image4” data structures, verification code, and signing infrastructure used by Apple silicon.

When storage-free operation or storage plus secure boot is an option, the design mandates that firmware updates be cryptographically signed and verified before the persistent storage can be updated.

Rosetta 2 on a Mac with Apple silicon

A Mac with Apple silicon is capable of running code compiled for the x86_64 instruction set using a translation mechanism called Rosetta 2. There are two types of translation offered: just-in-time and ahead-of-time.

Just-in-time translation

In the just-in-time (JIT) translation pipeline, an x86_64 Mach object is identified early in the image execution path. When these images are encountered, the kernel transfers control to a special Rosetta translation stub rather than to the dynamic link editor, `dyld(1)`. The translation stub then translates x86_64 pages during the image's execution. This translation takes place entirely within the process. The kernel still verifies the code hashes of each x86_64 page against the code signature attached to the binary as the page is faulted in. In the event of a hash mismatch, the kernel enforces the remediation policy appropriate for that process.

Ahead-of-time translation

With the ahead-of-time (AOT) translation path, x86_64 binaries are read from storage at times the system deems optimal for responsiveness of that code. The translated artifacts are written to storage as a special type of Mach object file. That file is similar to an executable image, but it's marked to indicate it's the translated product of another image.

In this model, the AOT artifact derives its identity information from the original x86_64 executable image. To enforce this binding, a privileged userspace entity signs the translation artifact using a device-specific key that's managed by the Secure Enclave. This key is released only to the privileged userspace entity, which is identified as such using a restricted entitlement. The code directory created for the translation artifact includes the code directory hash of the original x86_64 executable image. The signature on the translation artifact itself is known as the *supplemental signature*.

The AOT pipeline begins similarly to the JIT pipeline, with the kernel transferring control to the Rosetta runtime rather than to the dynamic link editor, `dyld(1)`. But the Rosetta runtime then sends an interprocess communication (IPC) query to the Rosetta system service, which asks whether there's an AOT translation available for the current executable image. If found, the Rosetta service provides a handle to that translation, and it's mapped into the process and executed. During execution, the kernel enforces the code directory hashes of the translation artifact which are authenticated by the signature rooted in the device-specific signing key. The original x86_64 image's code directory hashes aren't involved in this process.

Translated artifacts are stored in a Data Vault which isn't runtime-accessible by any entity except the Rosetta service. The Rosetta service manages access to its cache by distributing read-only file descriptors to individual translation artifacts; this limits access to the AOT artifact cache. This service's interprocess communication and dependent footprint are kept intentionally very narrow to limit its attack surface.

If the code directory hash of the original x86_64 image doesn't match with the one encoded into the AOT translation artifact's signature, this result is considered the equivalent of an invalid code signature, and appropriate enforcement action is taken.

If a remote process queries the kernel for the entitlements or other code identity properties of an AOT-translated executable, the identity properties of the original x86_64 image are returned to it.

Static trust cache content

macOS 11 or later ships with Mach "fat" binaries that contain slices of x86_64 and arm64 computer code. On a Mac with Apple silicon, the user may decide to execute the x86_64 slice of a system binary through the Rosetta pipeline—for example, to load a plug-in that has no native arm64 variant. To support this approach, the static trust cache that ships with macOS contains three code directory hashes per Mach object file:

- A code directory hash of the arm64 slice
- A code directory hash of the x86_64 slice
- A code directory hash of the AOT translation of the x86_64 slice

The Rosetta AOT translation procedure is deterministic in that it reproduces identical output for any given input, irrespective of when the translation was performed or on what device it was performed.

During the macOS build, every Mach object file is run through the Rosetta AOT translation pipeline associated with the version of macOS being built, and the resulting code directory hash is recorded into the trust cache. For efficiency, the actual translated products don't ship with the operating system and are reconstituted on demand when the user requests them.

When an x86_64 image is being executed on a Mac with Apple silicon, if that image's code directory hash is in the static trust cache, the resulting AOT artifact's code directory hash is *also* expected to be in the static trust cache. Such products aren't signed by the device-specific key, because the signing authority is rooted in the Apple secure boot chain.

Unsigned x86_64 code

A Mac with Apple silicon doesn't permit native arm64 code to execute unless a valid signature is attached. This signature can be as simple as an ad hoc code signature (cf. `codesign(1)`) that doesn't bear any actual identity from the secret half of an asymmetric key pair (it's simply an unauthenticated measurement of the binary).

For binary compatibility, translated x86_64 code is permitted to execute through Rosetta with no signature information at all. No specific identity is conveyed to this code through the device-specific Secure Enclave signing procedure, and it executes with precisely the same limitations that native unsigned code executing on an Intel-based Mac.

Direct memory access protections for Mac computers

To achieve high throughput on high-speed interfaces like PCIe, FireWire, Thunderbolt, and USB, computers need to support direct memory access (DMA) from peripherals. That is, they need to be able to read and write to RAM without continuous involvement of the CPU. Since 2012, Mac computers have implemented numerous technologies to protect DMA, resulting in the best and most comprehensive set of DMA protections on any PC.

Protections for a Mac with Apple silicon

Apple systems on chip contain an [Input/Output Memory Management Unit \(IOMMU\)](#) for each DMA agent in the system, including PCIe and Thunderbolt ports. Because each IOMMU has its own set of address translation tables to translate DMA requests, peripherals connected by PCIe or Thunderbolt can access only memory that has been explicitly mapped for their use. Peripherals can't access memory belonging to other parts of the system—such as the kernel or firmware—memory assigned to other peripherals. If an IOMMU detects an attempt by a peripheral to access memory that isn't mapped for that peripheral's use, it institutes a kernel panic.

Protections for an Intel-based Mac

Intel-based Mac computers with Intel Virtualization Technology for Directed I/O (VT-d) initialize the IOMMU, enabling DMA remapping and interrupt remapping very early in the boot process to mitigate various classes of security vulnerabilities. The Apple IOMMU hardware begins operation with a default-deny policy, so the instant when the system is powered on, it automatically begins blocking DMA requests from peripherals. After being initialized by software, the IOMMUs begin allowing DMA requests from peripherals to memory regions that have been explicitly mapped for their use.

Note: Interrupt remapping for PCIe isn't necessary on a Mac with Apple silicon, because each IOMMU handles MSIs for its own peripherals.

Starting in macOS 11, all Mac computers with an Apple T2 Security Chip run UEFI drivers that facilitate DMA in a restricted ring 3 environment when these drivers are pairing with external devices. This property helps mitigate security vulnerabilities that may occur when a malicious device interacts with a UEFI driver in an unexpected way at boot time. In particular, it reduces the impact of vulnerabilities in a drivers handling of DMA buffers.

Securely extending the kernel in macOS

Starting with macOS 11, if third-party kernel extensions (kexts) are enabled, they can't be loaded into the kernel on demand. Instead, they're merged into an Auxiliary Kernel Collection (AuxKC), which is loaded during the boot process. For a Mac with Apple silicon, the measurement of the AuxKC is signed into the LocalPolicy (for previous hardware, the AuxKC resided on the data volume). Rebuilding the AuxKC requires the user's approval and restarting of the macOS to load the changes into the kernel, and it requires that the secure boot be configured to Reduced Security.

Important: Kexts are no longer recommended for macOS. Kexts risk the integrity and reliability of the operating system, and Apple recommends users select solutions that don't require extending the kernel.

In a Mac with Apple silicon

Kexts need to be explicitly enabled for a Mac with Apple silicon by holding the power button at startup to enter into One True Recovery (1TR) mode, then downgrading to Reduced Security and checking the box to enable kernel extensions. This action also requires entering an administrator password to authorize the downgrade. The combination of the 1TR and password requirement makes it difficult for software-only attackers starting from within macOS to inject kexts into macOS, which they can then exploit to gain kernel privileges.

After a user authorizes kexts to load, the above User-Approved Kernel Extension Loading flow is used to authorize the installation of kexts. The authorization used for the above flow is also used to capture an SHA-384 hash of the user-authorized kext list (UAKL) in the LocalPolicy. The kernel management daemon (kmd) is then responsible for validating only those kexts found in the UAKL for inclusion into the AuxKC:

- If System Integrity Protection (SIP) is turned on, the signature of each kext is verified before being included in the AuxKC.
- If SIP is turned off, the kext signature isn't enforced.

This approach allows Permissive Security flows for developers or users who aren't part of the Apple Developer Program to test kexts before they are signed.

After the AuxKC is created, its measurement is sent to the Secure Enclave to be signed and included in an Image4 data structure that can be evaluated by iBoot at startup. As part of the AuxKC construction, a kext receipt is also generated. This receipt contains the list of kexts that were actually included in the AuxKC, because the set could be a subset of the UAKL if banned kexts were encountered. An SHA-384 hash of the AuxKC Image4 data structure and the kext receipt are included in the LocalPolicy. The AuxKC Image4 hash is used for extra verification by iBoot at startup to help ensure that it isn't possible to start up an older Secure Enclave-signed AuxKC Image4 file with a newer LocalPolicy. The kext receipt is used by subsystems such as Apple Pay to determine whether there are any kexts currently loaded that could interfere with the trustworthiness of macOS. If there are, then Apple Pay capabilities may not be possible.

System extensions

macOS 10.15 allows developers to extend the capabilities of macOS by installing and managing system extensions that run in user space rather than at the kernel level. By running in user space, system extensions increase the stability and security of macOS. Even though kexts inherently have full access to the entire operating system, extensions running in user space are granted only the privileges necessary to perform their specified function.

Developers can use frameworks, including DriverKit, EndpointSecurity, and NetworkExtension, to write USB and human interface drivers, endpoint security tools (like data loss prevention or other endpoint agents), and VPN and network tools, all without needing to write kexts. Third-party security agents should be used only if they take advantage of these APIs or have a robust road map to transition to them and away from kernel extensions.

User-Approved Kernel Extension Loading

To improve security, user consent is required to load kernel extensions installed with or after installing macOS 10.13. This process is known as *User-Approved Kernel Extension Loading*. Administrator authorization is required to approve a kernel extension. Kernel extensions don't require authorization if they:

- Were installed on a Mac when with macOS 10.12 or earlier
- Are replacing previously approved extensions
- Are allowed to load without user consent by using the `spctl` command-line tool available when a Mac was booted from recoveryOS
- Are allowed to load using a [device management service](#) configuration

On devices with macOS 10.13.2 or later, users can use a device management service to specify a list of kernel extensions that load without user consent. This option requires a Mac that's enrolled in a device management service—through [Apple School Manager](#), [Apple Business](#), or device management enrollment done by the user.

Option ROM security in macOS

Option ROM security in a Mac with the Apple T2 Security Chip

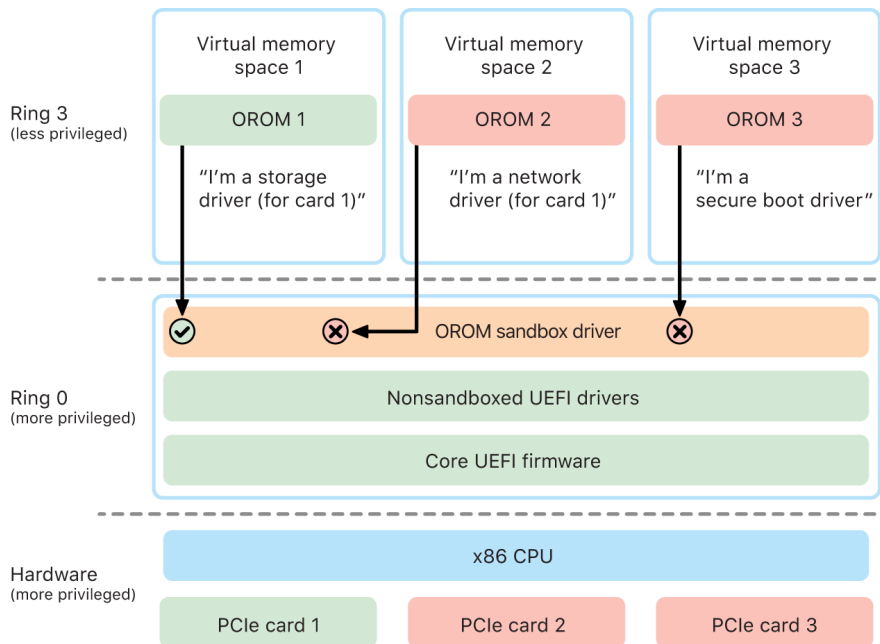
Both Thunderbolt and PCIe devices can have an *Option ROM (OROM)* physically attached to the device. (This is typically not a true ROM but instead a rewritable chip that stores firmware.) On UEFI-based systems, that firmware is typically a UEFI driver, which is read in by the UEFI firmware and ran. The running code is supposed to initialize and configure the hardware it was retrieved from, so that the hardware can be made usable by the rest of the firmware. This capability is required so that specialized third-party hardware can load and operate during the earliest startup phases—for example, to start up from external RAID arrays.

However, because OROMs are generally rewritable, if an attacker overwrites the OROM of a legitimate peripheral, the attacker's code executes early in the boot process and is able to tamper with the execution environment and violate the integrity of software that's loaded later. Likewise, if the attacker introduces their own malicious device to the system, they're also able to execute malicious code.

In macOS 10.12.3, the behavior of Mac computers sold after 2011 changed to not run OROMs by default at the time the Mac booted unless a special key combination was pressed. This key combination protected against malicious OROMs being inadvertently introduced into the macOS boot sequence. The default behavior of the Firmware Password Utility was also changed so that when the user set a firmware password, OROMs couldn't execute even if the key combination was pressed. This protected against a physically present attacker intentionally introducing a malicious OROM. For users who still need to run OROMs while they have a firmware password set, a nondefault option can be configured using the `firmwarepasswd` command-line tool in macOS.

OROM sandbox security

In macOS 10.15, UEFI firmware was updated to contain a mechanism for sandboxing OROMs and for stripping privileges from them. UEFI firmware typically executes all code, including OROMs, at the maximum CPU privilege level, called ring 0, and has a single shared virtual memory space for all code and data. Ring 0 is the privilege level where the macOS kernel runs, whereas the lower privilege level, ring 3, is where apps run. The OROM sandbox deprived OROMs by making use of virtual memory separation like the kernel does and then making the OROMs run in ring 3.



The sandbox further significantly restricts both the interfaces that the OROMs can call (much like system call filtering in kernels) and the type of device that an OROM can register as (much like app approval.) The benefit of this design is that malicious OROMs can no longer directly write anywhere within ring 0 memory. Instead, they are limited to a very narrow and well-defined sandbox interface. This limited interface significantly reduces attack surface and forces attackers to first escape the sandbox and escalate privilege.

Note: Option ROMs aren't currently supported on a Mac with Apple silicon.

UEFI firmware security in an Intel-based Mac

An Intel-based Mac with an Apple T2 Security Chip offers security using UEFI (Intel) firmware.

Since 2006, Mac computers with an Intel-based CPU use an Intel firmware based on the Extensible Firmware Interface (EFI) and Extensible Development Kit (EDK) version 1 or version 2. EDK2-based code conforms to the Unified Extensible Firmware Interface (UEFI) specification. This section refers to the Intel firmware as the UEFI firmware. The UEFI firmware was the first code to execute on the Intel chip.

For an Intel-based Mac without the Apple T2 Security Chip, the root of trust for the UEFI firmware is the chip where the firmware is stored. UEFI firmware updates are digitally signed by Apple and verified by the firmware before updating the storage. To help prevent rollback attacks, updates need to always have a version newer than the existing one. However, an attacker with physical access to the Mac could potentially use hardware to attach to the firmware storage chip and update the chip to contain malicious content. Likewise, if vulnerabilities are found in the early boot process of the UEFI firmware (before it write-restricts the storage chip), this could also lead to persistent infection of the UEFI firmware. This is a hardware architectural limitation common in most Intel-based PCs and present in all Intel-based Mac computers without the T2 chip.

To help prevent physical attacks that subvert UEFI firmware, Mac computers were rearchitected to root the trust in the UEFI firmware in the T2 chip. On these Mac computers, the root of trust for the UEFI firmware is specifically the T2 firmware, as described in [Boot process](#).

Intel Management Engine (ME) subcomponent

One subcomponent stored within the UEFI firmware is the Intel Management Engine (ME) firmware. The ME—a separate processor and subsystem within Intel chips—is used primarily for audio and video copyright protection on a Mac that has only Intel-based graphics. To reduce this subcomponent’s attack surface, an Intel-based Mac runs a custom ME firmware from which most components have been removed. Because the resulting Mac ME firmware is smaller than the default minimal build that Intel makes available, many components that have been the subject of public attacks by security researchers in the past are no longer present.

System Management Mode (SMM)

Intel processors have a special execution mode that’s distinct from normal operation. Called System Management Mode (SMM), it was originally introduced to handle time-sensitive operations such as power management. However, to perform such actions, Mac computers have historically used a discrete microcontroller called the System Management Controller (SMC). No longer a separate microcontroller, the SMC has been integrated into the T2 chip.

System security for watchOS

Apple Watch uses many of the same hardware-based platform security capabilities that iOS and iPadOS use. For example, Apple Watch:

- Performs secure boot and secure software updates
- Maintains operating system integrity
- Helps protect data, both on the device and when communicating with a paired iPhone or the internet

Supported technologies include those listed in System Security (for example, KIP, SKP, and SCIP) as well as [Data Protection](#), keychain, and network technologies.

Updating watchOS

watchOS can be configured to update overnight. For more information on how the Apple Watch passcode gets stored and used during the update, see [Keybags](#).

Wrist detection

If wrist detection is turned on, the device locks automatically soon after it's removed from the user's wrist. If wrist detection is turned off, Control Center provides an option for locking Apple Watch. When Apple Watch is locked, Apple Pay can be used only by entering the passcode on the Apple Watch. Wrist detection is turned off using the Watch app on iPhone. This setting can also be enforced using a [device management service](#).

Activation Lock

When Find My is turned on for an iPhone, its paired Apple Watch can also use Activation Lock. Activation Lock makes it harder for anyone to use or sell an Apple Watch that's been lost or stolen. Activation Lock requires the user's Apple Account and password to unpair, erase, or reactivate an Apple Watch. For more information, see [Activation Lock security](#).

Secure pairing with iPhone

Apple Watch can be paired with only one iPhone at a time. When Apple Watch is unpaired, iPhone communicates instructions to erase all content and settings from the watch.

Pairing Apple Watch with iPhone is secured using a secret encoded in an animated pattern displayed by Apple Watch, which is captured by the camera on iPhone. A six-digit PIN is also available as a fallback pairing method, if necessary. The way the secret or the PIN is used depends on which operating system version is running on the Apple Watch and iPhone.

When Apple Watch with watchOS 26 or later is paired to iPhone with iOS 26 or later, pairing is performed by exchanging keys over a secure IKEv2 connection. This connection is either authenticated using standard PSK authentication with the secret encoded in the animated pattern or by a connection-specific secret derived from the PIN using SPAKE2+. ML-KEM-1024 is used to provide quantum security in addition to the security provided by elliptic-curve Diffie-Hellman.

After the connection is established, each device generates random Ed25519 public-private key pairs, and the public keys are exchanged. The private keys are rooted in the Secure Enclave on Apple Watch. This isn't possible on iPhone because a user restoring their iCloud Backup to the same iPhone preserves the existing Apple Watch pairing without requiring migration. Each device also generates and exchanges secrets for BLE 4.1 out-of-band pairing.

When Apple Watch and iPhone are running older software versions, the secret encoded in the animated pattern is used for BLE 4.1 out-of-band pairing, and the six-digit PIN is used for pairing standard BLE Passkey Entry. After the BLE session is established and encrypted using the highest security protocol available in the Bluetooth Core Specification, iPhone and Apple Watch exchange keys using either:

- A process adapted from [Apple Identity Service \(IDS\)](#) as described in the [iMessage security overview](#).

- A key exchange using IKEv2/IPSec. The initial key exchange is authenticated using either the Bluetooth session key (for pairing scenarios) or the IDS keys (for operating system update scenarios). Each device generates Ed25519 public-private key pair, and during the initial key exchange process, the public keys are exchanged. When an Apple Watch with watchOS 10 or later is first paired, the private keys are rooted in its Secure Enclave.

On an iPhone with iOS 17 or later, the private keys aren't rooted in the Secure Enclave, because a user restoring their iCloud Backup to the same iPhone preserves the existing Apple Watch pairing without requiring migration.

Note: The mechanism used for key exchange and encryption varies, depending on which operating system versions are on the iPhone and Apple Watch. An iPhone with iOS 13 or later when paired with an Apple Watch with watchOS 6 or later, use only IKEv2/IPSec for key exchange and encryption.

After keys have been exchanged:

- The Bluetooth session key is discarded and all communications between iPhone and Apple Watch are encrypted using one of the methods listed above—with the encrypted Bluetooth, Wi-Fi, and cellular links providing a secondary encryption layer.

The BLE device address is also rotated at 15-minute intervals to reduce the risk of the device being locally tracked if someone broadcasts a persistent identifier.

- (IKEv2/IPsec only) The keys are stored in the System keychain and used for authenticating future IKEv2/IPsec sessions between the devices. Encryption between devices depends on the hardware and operating systems:
 - An iPhone with iOS 26 or later paired with an Apple Watch with watchOS 26 or later utilizes ML-KEM-768 for quantum security in addition to the security provided by elliptic-curve Diffie-Hellman.
 - An iPhone with iOS 15 or later paired with an Apple Watch Series 4 or later with watchOS 8 or later is encrypted and integrity protected using AES-256-GCM.
 - Older devices or devices with older operating system versions ChaCha20-Poly1305 with 256-bit keys.

To support apps that need streaming data, encryption is provided with methods described in [FaceTime security](#), using either the Apple Identity Service (IDS) provided by the paired iPhone or a direct internet connection.

Apple Watch implements hardware-encrypted storage and class-based protection of files and keychain items. Access-controlled [keybags](#) for keychain items are also used. Keys used to communicate between Apple Watch and iPhone are also secured using class-based protection. For more information, see [Keybags for Data Protection](#).

Approve in macOS with Apple Watch

When Auto Unlock with Apple Watch is enabled, the Apple Watch can be used in place, or together with Touch ID, to approve authorization and authentication prompts from:

- macOS and Apple apps that request authorization
- Third-party apps that request authentication
- Saved Safari passwords
- Secure Notes

Secure use of Wi-Fi, cellular, iCloud, and Gmail

When Apple Watch isn't within Bluetooth range, Wi-Fi or cellular can be used instead. Apple Watch automatically joins Wi-Fi networks that have already been joined on the paired iPhone and whose credentials have synced to the Apple Watch while both devices were in range. This Auto-Join behavior can then be configured on a per-network basis in the Wi-Fi section of the Apple Watch Settings app. Wi-Fi networks that have never been joined before on either device can be manually joined in the Wi-Fi section of the Apple Watch Settings app.

When Apple Watch and iPhone are out of range, Apple Watch connects directly to iCloud and Gmail servers to fetch mail, as opposed to syncing mail data with the paired iPhone over the internet. For Gmail accounts, the user needs to authenticate to Google in the Mail section of the Watch app on iPhone. The OAuth token received from Google is sent over to Apple Watch in encrypted format over Apple Identity Service (IDS) so that it can be used to fetch mail. This OAuth token is never used for connectivity with the Gmail server from the paired iPhone.

Random number generation

Cryptographic pseudorandom number generators (CPRNGs) are an important building block for secure software. To this end, Apple provides a trusted software CPRNG running in the iOS, iPadOS, macOS, tvOS, visionOS, and watchOS kernels. It's responsible for aggregating raw entropy from the system and providing secure random numbers to consumers in both the kernel and user space.

Entropy sources

The kernel CPRNG is seeded from multiple entropy sources during boot and over the lifetime of the device. These include (contingent on availability):

- The Secure Enclave hardware TRNG
- Timing-based jitter collected during boot
- Entropy collected from hardware interrupts
- A seed file used to persist entropy across boots
- Intel random instructions—for example, RDSEED and RDRAND (only on an Intel-based Mac)

The kernel CPRNG

The kernel CPRNG is a Fortuna-derived design targeting a 256-bit security level. It provides high-quality random numbers to user-space consumers using the following APIs:

- The `getentropy(2)` system call
- The random device (`/dev/random`)

The kernel CPRNG accepts user-supplied entropy through writes to the random device.

Communicating emergency information using satellites

Users with iPhone 14 or later can communicate over satellite links to provide various services. Satellite links impose significant bandwidth constraints and require security protocols with minimal bandwidth overhead. Regardless of communication type (Emergency SOS, SMS, or iMessage), Apple establishes a secure session between the off-grid device and an Apple datacenter. Devices preregister public key material with the server over cellular or Wi-Fi to save bandwidth while off-grid.

Core security architecture

When a device requests a satellite service, it establishes a secure connection with the Apple datacenter by referencing a truncated hash of its public key for key exchange. Each public-private key pair supports only a single session, with a maximum validity of eight hours. The server generates a corresponding public-private key pair for every client key pair in advance and shares the public key with the client.

After the session expires or finishes, Apple servers and the device remove the corresponding private keys, providing forward secrecy. Apple and devices register additional keys at regular intervals and after coming back online following a private key use, providing post-compromise security for satellite communications. Observers can't identify users from session identifiers, as each device rotates identifiers on every session to prevent linking.

Emergency and Roadside Assistance over satellite

When users employ SOS or Roadside Assistance over Satellite, the secure connection to Apple (described above) protects the communication content. An additional encryption layer ensures that the content remains encrypted all the way to the application server, which forwards these messages to Apple emergency services and road assistance partners with transport encryption.

iMessage over satellite

When exchanging iMessages over Wi-Fi or cellular, devices exchange key material along with messages if any device in the sender's or receiver's group supports satellite service. Through this mechanism, devices receive regularly updated keys that enable them to encrypt and decrypt messages when cellular connectivity is unavailable and the device connects to satellite. Due to key lifetime limitations, users need to exchange messages within the last 30 days to communicate off-grid.

Each device generates symmetric key material and distributes it using iMessage encryption to encrypt its own outgoing messages. Each message uses a unique symmetric key derived through a Hash-based Key Derivation Function (HKDF) ratchet for its corresponding message counter. Upon receiving a message, the recipient's devices look up the sender's symmetric key material to derive the corresponding message key. When the device comes back online, new symmetric keys are distributed for subsequent sessions.

The sending device sends iMessages using Satellite only after it receives a signed payload from the recipient confirming that the device can receive messages over satellite.

Note: iMessage using satellite supports one-to-one messaging only, not groups.

SMS over satellite

When users send or receive SMS messages using satellite, the messages go to or come from a network Interworking Function (IWF). The IWF receives messages destined to the off-grid client from the sender's carrier (the client appears roaming while off-grid, as it isn't on the carrier's local network). Similarly, the IWF delivers outgoing messages from the off-grid client to the recipient's home carrier. Encryption and authentication protect SMS messages exchanged between the off-grid client and the IWF, preventing both Apple and those with satellite signal access from accessing or modifying the messages.

A Diffie-Hellman key exchange over the NIST P256 curve begins while a client is on-grid and completes when the client goes off-grid and establishes the initial satellite connection. Using HKDF-SHA-256, Apple and the device derive several keys from this key exchange. One key symmetrically encrypts (using AES-GCM with a 256-bit key) client information necessary for proper SMS routing (IMSI and SMSC). Another set provides bidirectional encryption of messages between the client and the IWF, also using AES-GCM with a 256-bit key.

Because satellite connections have lower bandwidth and higher latency than traditional cellular communications, not all incoming SMS messages reach the client immediately when it connects off-grid to prevent spam or low-value messages from delaying critical SMS retrieval. Instead, the system transmits inbound messages to the client only if either:

1. The client, while off-grid, has sent a message to the sender within the recent 24-hour period.
2. The sender is an approved sender (in the user's emergency contacts or iCloud family members).

Otherwise, Apple stores the message encrypted on its servers until the client comes back online and retrieves the messages over traditional networks. To maintain user privacy, Apple ensures that the IWF doesn't learn the user's approved sender list.

The client uses HMAC-SHA-256 with a key derived from the session's primary key to generate random pseudonyms for each phone number in each session. On session establishment, the client sends the pseudonyms for each phone number on the allow list to the Apple server. Similarly, it generates and sends a pseudonym to Apple each time the client sends an SMS to a phone number. This allows Apple to maintain a list of permitted sender pseudonyms for the session.

When someone sends the client an SMS after session establishment, the IWF encrypts the message and calculates the sender's phone number pseudonym. It communicates this along with the encrypted message to Apple's servers. The server compares the incoming message's sender pseudonym against the allow list and forwards the encrypted message if there is a match.

Apple Security Research Device

The Apple Security Research Device is a specially-fused iPhone that allows security researchers to perform research on iOS without having to defeat or inactivate the platform security features. With this device, a researcher can side-load content that runs with platform-equivalent permissions and perform research on a platform that more closely models that of a production device.

To help ensure that user devices aren't affected by the security research device execution policy, the policy changes are implemented in a variant of iBoot and in the Boot Kernel Collection, which fail to boot on a production device. The research iBoot checks for a new fusing state and enters a panic loop if it's being run on nonresearch-fused hardware.

The cryptex subsystem allows a researcher to load a personalized [trust cache](#) and a disk image containing corresponding content. A number of defense in-depth measures have been implemented that are designed to ensure that this subsystem doesn't allow execution on user devices:

- launchd doesn't load the cryptexd property list if it detects a normal customer device.
- cryptexd aborts if it detects a normal customer device.
- AppleImage4 doesn't vend the anti-replay value used for verifying a research cryptex on a normal customer device.
- The signing server refuses to personalize a cryptex disk image for a device not on an explicit allow list.

To respect the privacy of the security researcher, only the measurements (for example, hashes) of the executables or kernel cache and the security research device identifiers are sent to Apple during personalization. Apple doesn't receive the content of the cryptex being loaded onto the device.

To avoid having a malicious party attempt to masquerade a research device as a user device to trick a target into using it for everyday usage, the security research device has the following differences:

- The security research device starts up only while charging. This can be using a Lightning cable or a Qi-compatible charger. If the device isn't charging during startup, the device enters Recovery mode. If the user starts charging and restarts the device, it starts up as normal. As soon as XNU starts, the device doesn't need to be charging to continue operation.
- The words *Security Research Device* are displayed below the Apple logo during iBoot startup.
- The XNU kernel boots in verbose mode.
- The device is etched on the side with the message "Property of Apple. Confidential and Proprietary. Call +1 877 595 1125."

The following are additional measures that are implemented in software that appears after boot:

- The words *Security Research Device* are displayed during device setup.
- The words *Security Research Device* are displayed on the Lock Screen and in the Settings app.

The Security Research Device researchers:

- Side-load executable code onto the device with arbitrary entitlements at the same permission level as Apple operating system components
- Start services at startup.
- Persist content across restarts.
- Use the `research.com.apple.license-to-operate` entitlement to permit a process to debug any other process on the system, including system processes.

The `research.` namespace is respected only by the RESEARCH variant of the AppleMobileFileIntegrity kernel extension; any process with this entitlement is terminated on a customer device during signature validation.

- Personalize and restore a custom kernel cache.

Encryption and Data Protection

Encryption and Data Protection overview

The secure boot chain, system security, and app security capabilities all help to verify that only trusted code and apps run on a device. Apple devices have additional encryption features to safeguard user data, even when other parts of the security infrastructure have been compromised (for example, if a device is lost or is running untrusted code). All of these features benefit both users and IT administrators, protecting personal and corporate information and providing methods for instant and complete remote wipe in the case of device theft or loss.

iPad, iPhone, and Apple Vision Pro use a file encryption methodology called *Data Protection*, whereas the data on an Intel-based Mac is protected with a volume encryption technology called *FileVault*. A Mac with Apple silicon uses a hybrid model that supports Data Protection, with two caveats: The lowest protection level Class (D) isn't supported, and the default level (Class C) uses a volume key and acts just like the FileVault on an Intel-based Mac. In all cases, key management hierarchies are rooted in the dedicated silicon of the Secure Enclave, and a dedicated AES Engine supports line-speed encryption and helps ensure that long-lived encryption keys aren't exposed to the kernel operating system or CPU (where they might be compromised). (An Intel-based Mac with a T1 or lacking a Secure Enclave doesn't use dedicated silicon to protect its FileVault encryption keys.)

Besides using Data Protection and FileVault to help prevent unauthorized access to data, Apple uses *operating system kernels* to enforce protection and security. The kernel uses access controls to sandbox apps (which restricts what data an app can access) and a mechanism called a [Data Vault](#) (which rather than restricting the calls an app can make, restricts access to the data of an app from all other requesting apps).

Quantum-secure cryptography in Apple operating systems

Overview

Historically, communications protocols have used classical public key cryptography, such as RSA, Elliptic Curve Diffie-Hellman key exchange and Elliptic Curve signature, to establish secure connections between devices or between a device and a server. All these algorithms rest on mathematical problems that computers have long considered too computationally intensive to solve, even when accounting for Moore's law. However, the rise of quantum computing threatens to change the equation. A sufficiently powerful quantum computer could solve these classical mathematical problems in fundamentally different ways, and—in theory—do so fast enough to threaten the security of end-to-end encrypted communications.

Although quantum computers with this capability don't exist yet, extremely well-resourced attackers can already prepare for their arrival by exploiting the steep decrease in modern data storage costs. The premise is simple: attackers can collect large amounts of today's encrypted data and file it all away for future reference. Even though they can't decrypt any of this data today, they can retain it until they acquire a quantum computer that *can* decrypt it in the future, an attack scenario known as *Harvest Now, Decrypt Later*.

To mitigate risks from future quantum computers, the cryptographic community has been working on post-quantum cryptography (PQC): new public key algorithms that provide the building blocks for quantum-secure protocols don't require a quantum computer—protocols that can run on the classical, non-quantum computers in use today, but that will remain secure from known threats posed by future quantum computers.

Apple's approach to quantum-secure cryptography

When deploying quantum-secure cryptography, Apple is adopting hybrid cryptography, which combines classic algorithms and new post-quantum algorithms so that updates can't make systems less secure than before. Hybrid cryptography is critical because it lets Apple continue taking advantage of field-tested classic algorithm implementations, which Apple has hardened against key recovery attacks exploiting CPU signals during algorithm execution such as side-channel attacks.

Apple has deployed quantum-secure cryptography across a wide range of protocols, prioritizing applications involving sensitive user information where attackers could harvest encrypted communications at scale:

- *iMessage*: Apple deployed iMessage PQ3 in iOS 17.4, iPadOS 17.4, macOS 14.4, and watchOS 10.4, which advanced the state-of-the-art in quantum-secure messaging at scale. For more information, see [iMessage with PQ3: The new state of the art in quantum-secure messaging at scale](#) on the Apple Security Blog.

- *TLS and HTTPS:* Apple supports quantum-secure encryption in TLS in the developer networking APIs URLSession and Network frameworks in iOS 26, iPadOS 26, macOS 26, tvOS 26, and watchOS 26. These APIs enable it by default for all systems services and apps that use them. This is an especially important protocol because it protects a vast amount of personal data like internet browsing and email, transiting across networks where attackers could harvest data. For more information, see [TLS security](#).
- *VPN:* Apple added quantum-secure encryption in the native VPN client support as well as the developer IKEv2 APIs, which also makes it easy for third-party VPN solutions to enable quantum-secure encryption with iOS 26, iPadOS 26, macOS 26, tvOS 26, and watchOS 26. For more information, see [VPN security](#).
- *SSH:* Apple upgraded this protocol (commonly used on Mac to log in remotely and transfer files) with a quantum-secure encryption key exchange in macOS 26. For more information, see [Allow a remote computer to access a Mac](#) in the Mac User Guide.
- *Apple Watch:* Apple enabled quantum-secure encryption between iPhone and Apple Watch in iOS 26 and watchOS 26 by adding additional key exchanges using ML-KEM. For more information, see [System security for watchOS](#).
- *Developer cryptographic APIs:* To enable developers to leverage Apple's native implementation and transition their own protocols to post-quantum cryptography, support was added in the Apple CryptoKit framework in iOS 26, iPadOS 26, macOS 26, tvOS 26, and watchOS 26. For secure encryption, ML-KEM offers two parameters: ML-KEM 768 and ML-KEM 1024. For quantum-secure authentication, developers can use ML-DSA-65 and ML-DSA-87. Although these algorithms are robust in isolation, developers need to use them in well-analyzed protocols to ensure they correctly use and combine them to achieve the application's security needs. For more information, see [Apple CryptoKit](#) on the Apple Developer website.

Important: Systems support quantum encryption only when they connect to supporting servers.

Passcodes and passwords

To protect user data from malicious attack, Apple uses passcodes in iOS, iPadOS, and visionOS, and passwords in macOS. The longer a passcode or password is, the stronger it is—and the easier it is to discourage brute-force attacks. To further discourage attacks, Apple enforces time delays (iOS, iPadOS, and visionOS) and a limited number of password attempts (Mac).

On an iPad, iPhone, and Apple Vision Pro, setting up a device passcode or password, the user automatically enables [Data Protection](#). Data Protection is also enabled on other devices that feature an Apple [system on a chip \(SoC\)](#)—such as a Mac with Apple silicon, Apple TV, and Apple Watch. On devices with macOS, Apple uses the built-in volume encryption program FileVault.

Increasing security with strong passcodes and passwords

iOS, iPadOS, and visionOS support six-digit, four-digit, and arbitrary-length alphanumeric passcodes. Besides unlocking the device, a passcode or password provides entropy for certain encryption keys. This means an attacker in possession of a device can't get access to data in specific protection classes without the passcode.

The passcode or password is [entangled](#) with the device’s UID, so brute-force attempts need to be performed on the device under attack. A large iteration count is used to make each attempt slower. The iteration count is calibrated so that one attempt takes approximately 80 milliseconds. In fact, it would take more than five and one-half years to try all combinations of a six-character alphanumeric passcode with lowercase letters and numbers.

The stronger the user passcode is, the stronger the encryption key becomes. And by using Optic ID, Face ID and Touch ID, the user can establish a much stronger passcode than would otherwise be practical. The stronger passcode increases the effective amount of entropy protecting the encryption keys used for Data Protection, without adversely affecting the user experience of unlocking a device multiple times a day.

If a passcode contains only numbers, a numeric keypad is displayed at the Lock Screen. A longer numeric passcode may be easier to enter than a shorter alphanumeric password, while providing similar security.

Users can specify a longer alphanumeric password by selecting Custom Alphanumeric Code in the Passcode Options in Settings > [Optic ID], [Face ID], or [Touch ID] & Passcode). If a password is alphanumeric, a full keypad is displayed at the Lock Screen.

Escalating time delays discourage brute-force attacks

On iPad, iPhone, Mac, and Apple Vision Pro, to further discourage brute-force passcode attacks, there are escalating time delays after the entry of an invalid passcode, password, or PIN (depending on the device and the state the device is in), as shown in the table below.

Attempts	3	4	5	6	7	8	9	10 or more
iOS and iPadOS Lock Screen	None	1 minute	5 minutes	15 minutes	1 hour	3 hours	8 hours	Device is locked and needs to connect to a Mac or PC
watchOS Lock Screen	None	1 minute	5 minutes	15 minutes	1 hour	3 hours	8 hours	Device is locked and needs to connect to an iPhone
FileVault Login Window and Lock Screen	None	1 minute	5 minutes	15 minutes	1 hour	3 hours	8 hours	8 hours
macOS Recovery Mode	None	1 minute	5 minutes	15 minutes	1 hour	3 hours	8 hours	See “How escalating time delays discourage brute-force attacks in macOS” below

Attempts	3	4	5	6	7	8	9	10 or more
FileVault with recovery key (Personal, Institutional, or iCloud)	None	1 minute	5 minutes	15 minutes	1 hour	3 hours	8 hours	See “How escalating time delays discourage brute-force attacks in macOS” below
macOS Remote lock PIN code	1 minute	5 minutes	15 minutes	30 minutes	1 hour	1 hour	1 hour	1 hour

If the Erase Data option is turned on for iPad, iPhone, or Apple Vision Pro (in Settings > [Optic ID], [Face ID], or [Touch ID] & Passcode), after 10 consecutive incorrect attempts to enter the passcode, all content and settings are removed from storage. Consecutive attempts of the same incorrect passcode don’t count toward the limit. This setting is also available as an administrative policy through a device management service that supports this feature and through Microsoft Exchange ActiveSync, and can be set to a lower threshold.

The delays are enforced by the Secure Enclave. If the device is restarted during a timed delay, the delay is still enforced, with the timer starting over for the current period.

Escalating time delays discourage brute-force attacks in macOS

To help prevent brute-force attacks, when a Mac starts up, no more than 10 password attempts are allowed at the Login Window, and escalating time delays are imposed after a certain number of incorrect attempts. The delays are enforced by the Secure Enclave. If a Mac is restarted during a timed delay, the delay is still enforced, with the timer starting over for the current period.

To help prevent malware from causing permanent data loss by trying to attack the user’s password, these limits aren’t enforced after the user has successfully logged in to the Mac, but they are reimposed after a restart. If the 10 attempts are exhausted, 10 more attempts are available after restarting into recoveryOS. And if those are also exhausted, then 10 additional attempts are available for the configured FileVault recovery mechanism (iCloud recovery, FileVault recovery key, and institutional key), for a maximum of 30 additional attempts. After those additional attempts are exhausted, the Secure Enclave no longer processes any requests to decrypt the volume or verify the password, and the data on the drive becomes unrecoverable.

To help protect data in an enterprise setting, IT should define and enforce FileVault configuration policies using a device management service. Organizations have several options for managing encrypted volumes, including institutional recovery keys, personal recovery keys (that can optionally be stored with a device management service for escrow), or a combination of both. Key rotation can also be set as a policy in a device management service.

On a Mac with the Apple T2 Security Chip, the password serves a similar function except that the key generated is used for FileVault encryption rather than Data Protection.

Data Protection

Data Protection overview

Apple uses a technology called [Data Protection](#) to protect data stored in flash storage on the devices that feature an Apple SoC—such as iPad, iPhone, a Mac with Apple silicon, Apple TV, Apple Vision Pro, and Apple Watch. With Data Protection, a device can respond to common events such as incoming phone calls while at the same time providing a high level of encryption for user data. Certain system apps (such as Calendar, Contacts, Mail, Messages, Photos) and Health data values use Data Protection by default. Third-party apps receive this protection automatically.

Implementation

Data Protection is implemented by constructing and managing a hierarchy of keys and builds on the hardware encryption technologies built into Apple devices. Data Protection is controlled on a per-file basis by assigning each file to a class; accessibility is determined according to whether the class keys have been unlocked. [Apple File System \(APFS\)](#) allows the file system to further subdivide the keys into a per-extent basis (where portions of a file can have different keys).

Every time a file on the data volume is created, Data Protection creates a new 256-bit key (the *per-file key*) and gives it to the hardware AES Engine, which uses the key to encrypt the file as it's being written to flash storage.

- On A14 through A18 and M1 or later devices, the encryption uses AES-256 in XTS mode, where the 256-bit per-file-key goes through a Key Derivation Function (NIST Special Publication 800-108) to derive a 256-bit value and a 256-bit cipher key.
- On A9 through A13 and S5 or later devices, the encryption uses AES-128 in XTS mode, where the 256-bit per file key is split to provide a 128-bit value and a 128-bit cipher key.

On a Mac with Apple silicon, Data Protection defaults to Class C (see [Data Protection classes](#)) but uses a volume key rather than a per-extent or per-file key—effectively recreating the security model of FileVault for user data. On a Mac with macOS 26.4 or later, FileVault is turned on by default, which automatically provides users the full protection of entangling the encryption key hierarchy with their password. Users can opt out if desired. Developers can also opt in to a higher protection class that uses a per-file or per-extent key.

Data Protection in Apple devices

On Apple devices with Data Protection, each file is protected with a unique per-file (or per-extent) key. The key, wrapped using the NIST AES key wrap algorithm, is further wrapped with one of several class keys, depending on how the file is meant to be accessed. The wrapped [per-file key](#) is then stored in the file's metadata.

Devices with APFS format may support cloning of files (zero-cost copies using copy-on-write technology). If a file is cloned, each half of the clone gets a new key to accept incoming writes so that new data is written to the media with a new key. Over time, the file may become composed of various extents (or fragments), each mapping to different keys. However, all of the extents that comprise a file are guarded by the same class key.

When a file is opened, the [file system key](#) decrypts its metadata, revealing the wrapped per-file key and the protection class. The per-file (or per-extent) key is unwrapped with the class key and then supplied to the hardware AES Engine, which decrypts the file as it's read from flash storage. All wrapped file key handling occurs in the Secure Enclave; the file key is never directly exposed to the Application Processor. At startup, the Secure Enclave negotiates an ephemeral key with the AES Engine. When the Secure Enclave unwraps a file's keys, they're rewrapped with the ephemeral key and sent back to the Application Processor.

All file metadata is encrypted with a random value key, created when the operating system is first installed or when the user completes Erase All Content and Settings. This key is encrypted and wrapped by a key-wrapping key that's known only to the Secure Enclave for long-term storage. The key-wrapping key changes every time a user erases their device. On A9 or later SoCs, Secure Enclave relies upon entropy, backed by anti-replay systems, to achieve effaceability and to protect its key wrapping key, among other assets. For more information, see [Secure nonvolatile storage](#).

Just like per-file or per-extent keys, the metadata key of the data volume is never directly exposed to the Application Processor; the Secure Enclave provides an ephemeral, per-boot version instead. When stored, the encrypted file system key is additionally wrapped by an effaceable key stored in [Effaceable Storage](#) or using a media key-wrapping key, protected by Secure Enclave anti-replay mechanism. This key doesn't provide additional data confidentiality. Instead, it's designed to be quickly erased on demand (by the user with the "Erase All Content and Settings" option, or by a user or administrator issuing a remote wipe command from a [device management service](#), Microsoft Exchange ActiveSync, or iCloud).

Important: Erasing the key in this manner renders all files cryptographically inaccessible.

The contents of a file may be encrypted with one or more per-file (or per-extent) keys that are wrapped with a class key and stored in a file's metadata, which in turn is encrypted with the file system key. The class key is protected with the hardware UID and, for some classes, the user's passcode. This hierarchy provides both flexibility and performance. For example, changing a file's class only requires rewrapping its per-file key, and a change of passcode just rewraps the class key.

Data Protection classes

When a new file is created on devices supporting Data Protection, it's assigned a class by the app that creates it. Each class uses different policies to determine when the data is accessible. The basic classes and policies are described in the following sections. Apple silicon-based Mac computers don't support Class D: No Protection, and a security boundary is established around logging in and out (not locking or unlocking as on iPad and iPhone).

Class	Protection type
Class A: Complete Protection	<code>NSFileProtectionComplete</code>
Class B: Protected Unless Open	<code>NSFileProtectionCompleteUnlessOpen</code>
Class C: Protected Until First User Authentication <i>Note:</i> macOS uses a volume key to recreate FileVault protection characteristics.	<code>NSFileProtectionCompleteUntilFirstUserAuthentication</code>
Class D: No Protection <i>Note:</i> Not supported in macOS.	<code>NSFileProtectionNone</code>

Complete Protection

NSFileProtectionComplete

The class key is protected with a key derived from the user passcode or password and the device UID. Shortly after the user locks a device (10 seconds, if the Require Password setting is Immediately), the decrypted class key is discarded, rendering all data in this class inaccessible until the user enters the passcode again or unlocks (logs in to) the device using Optic ID, Face ID, or Touch ID.

On devices with macOS, shortly after the last user is logged out, the decrypted class key is discarded, rendering all data in this class inaccessible until a user enters the passcode again or logs into the device using Touch ID.

Protected Unless Open

NSFileProtectionCompleteUnlessOpen

Some files may need to be written while the device is locked or the user is logged out. A good example of this is a mail attachment downloading in the background. This behavior is achieved by using asymmetric elliptic curve cryptography (ECDH) over Curve25519. The usual [per-file key](#) is protected by a key derived using One-Pass Diffie-Hellman Key Agreement as described in NIST SP 800-56A.

The ephemeral public key for the Agreement is stored alongside the wrapped per-file key. The KDF is Concatenation Key Derivation Function (Approved Alternative 1) as described in 5.8.1 of NIST SP 800-56A. AlgorithmID is omitted. PartyUInfo and PartyVInfo are the ephemeral and static public keys, respectively. SHA-256 is used as the hashing function. As soon as the file is closed, the per-file key is wiped from memory. To open the file again, the shared secret is re-created using the Protected Unless Open class's private key and the file's ephemeral public key, which are used to unwrap the per-file key that is then used to decrypt the file.

On devices with macOS, the private part of `NSFileProtectionCompleteUnlessOpen` is accessible as long as any users on the system are logged in or are authenticated.

Protected Until First User Authentication

NSFileProtectionCompleteUntilFirstUserAuthentication

This class behaves in the same way as Complete Protection, except that the decrypted class key isn't removed from memory when the device is locked or the user logged out. The protection in this class has similar properties to desktop full-volume encryption, and protects data from attacks that involve a reboot. This is the default class for all third-party app data not otherwise assigned to a [Data Protection](#) class.

On Mac computers, this class uses a volume key—accessible as long as the volume is mounted—that functions just like FileVault.

No Protection

NSFileProtectionNone

This class key is protected only with the UID, and is kept in [Effaceable Storage](#). Because all the keys needed to decrypt files in this class are stored on the device, the encryption only affords the benefit of fast remote wipe. If a file isn't assigned a Data Protection class, it is still stored in encrypted form (as is all data on an iPad, iPhone, and Apple Vision Pro). This isn't supported in macOS.

Note: On devices with macOS, for volumes that don't correspond to a booted operating system, all data protection classes are accessible as long as the volume is mounted. The default data protection class is *NSFileProtectionCompleteUntilFirstUserAuthentication*. Per-extent key functionality is available to both Rosetta 2 and native apps.

Keybags for Data Protection

The keys for both file and keychain [Data Protection](#) classes are collected and managed in [keybags](#) in iOS, iPadOS, macOS, tvOS, visionOS, and watchOS. These operating systems use the following keybags: user, device, backup, escrow, and iCloud Backup.

User keybag

The user keybag is where the wrapped class keys used in normal operation of the device are stored. For example, when a passcode is entered, *NSFileProtectionComplete* is loaded from the user keybag and unwrapped. It is a binary property list (.plist) file stored in the No Protection class.

For devices with SoCs earlier than the A9, the .plist file contents are encrypted with a key held in [Effaceable Storage](#). To give forward security to keybags, this key is wiped and regenerated each time a user changes their passcode.

For devices with the A9 or later, S series, and M series SoCs (other than Mac computers), the .plist file contains a key which indicates that the keybag is stored in a locker protected by the Secure Enclave-controlled anti-replay value.

For Mac computers, the user's keybag is stored in their Keychains folder inside a folder named with the hardware UUID. For example:

```
~/Library/Keychains/123ABCDE-45FG-67HI-8910-1JK234L56M78/user.kb
```

For devices with the Secure Enclave, it manages the user keybag and can be queried regarding a device's lock state. It reports that the device is unlocked only if all the class keys in the user keybag:

- Are accessible
- Have been unwrapped successfully

Device keybag

The device keybag is used to store the wrapped class keys used for operations involving device-specific data.

iOS, iPadOS, and visionOS don't support cryptographic separation of per-user file system content, which means the system uses class keys from the device keybag to wrap [per-file keys](#). The keychain, however, uses class keys from the user keybag to protect items in the user keychain. On iPad and iPhone devices configured for use by a single user (the default configuration), the device keybag and the user keybag are one and the same, and are protected by the user's passcode.

Note: iPad devices configured in Shared iPad mode sometimes require access to credentials before any user has signed in. In this case, a keybag that isn't protected by the user's passcode is required.

Backup keybag

The backup keybag is created when an encrypted backup is made by the Finder (macOS 10.15 or later) or iTunes (in macOS 10.14 or earlier) and stored on the computer to which the device is backed up. A new keybag is created with a new set of keys, and the backed-up data is reencrypted to these new keys. As explained, nonmigratory keychain items remain wrapped with the UID-derived key, allowing them to be restored to the device they were originally backed up from but rendering them inaccessible on a different device.

The keybag—protected with the password set—is run through 10 million iterations of the key derivation function PBKDF2. Despite this large iteration count, there's no tie to a specific device, and therefore a brute-force attack parallelized across many computers could theoretically be attempted on the backup keybag. This threat can be mitigated with a sufficiently strong password.

If a user chooses not to encrypt the backup, the files aren't encrypted regardless of their Data Protection class but the keychain remains protected with a UID-derived key. This is why keychain items migrate to a new device only if a backup password is set.

Escrow keybag

The escrow keybag is used for syncing with the Finder or iTunes through USB and a [device management service](#). This keybag allows the Finder or iTunes to back up and sync without requiring the user to enter a passcode, and it allows a device management service to remotely clear a user's passcode. It is stored on the computer that's used to sync with the Finder or iTunes, or on the device management service that remotely manages the device.

The escrow keybag improves the user experience during device syncing, which potentially requires access to all classes of data. When a passcode-locked device is first connected to the Finder or iTunes, the user is prompted to enter a passcode. The device then creates an escrow keybag containing the same class keys used on the device, protected by a newly generated key. The escrow keybag and the key protecting it are split between the device and the host or server, with the data stored on the device in the Protected Until First User Authentication class. This is why the device passcode needs to be entered before the user backs up with the Finder or iTunes for the first time after a reboot.

In the case of an over-the-air (OTA) software update, the user is prompted for their passcode when initiating the update. This is used to securely create a one-time unlock token, which unlocks the user keybag after the update. This token can't be generated without entering the user's passcode, and any previously generated token is invalidated if the user's passcode changed.

One-time unlock tokens are either for attended or unattended installation of a software update. They're encrypted with a key derived from the current value of a monotonic counter in the Secure Enclave, the UUID of the keybag, and the Secure Enclave UID.

On A9 (or later) SoCs, one-time Unlock token no longer relies on counters or Effaceable Storage. Instead, it's protected by Secure Enclave-controlled anti-replay value.

The one-time unlock token for attended software updates expires after 20 minutes. In iOS 13.1, iPadOS 13, visionOS 1, or later, the token is stored in a locker protected by the Secure Enclave. Prior to iOS 13, this token was exported from the Secure Enclave and written to [Effaceable Storage](#) or was protected by the Secure Enclave anti-replay mechanism. A policy timer incremented the counter if the device hadn't rebooted within 20 minutes.

Unattended software updates occur when the system detects an update is available and when one of the following is true:

- Automatic updates are configured in iOS 12 or later.
- The user chooses Install Later when notified of the update.

After the user enters their passcode, a one-time unlock token is generated and can remain valid in Secure Enclave for up to 16 hours. If the update hasn't yet occurred, this one-time unlock token is destroyed on every lock and re-created on every subsequent unlock. Each unlock restarts the 16-hour window. After 16 hours a policy timer invalidates the one-time unlock token.

iCloud Backup keybag

The iCloud Backup keybag is similar to the backup keybag. All the class keys in this keybag are asymmetric (using Curve25519, like the Protected Unless Open Data Protection class does). An asymmetric keybag is also used to protect the backed-up keychain for iCloud Keychain recovery.

Protecting keys in alternate boot modes

Data Protection is designed to provide access to user data only after successful authentication, and only to the authorized user. Data protection classes are designed to support a variety of use cases, such as the ability to read and write some data even when a device is locked (but after first unlock). Additional steps are taken to protect access to user data during alternate boot modes such as those used for Device Firmware Update (DFU) mode, Recovery mode, Apple Diagnostics, or even during software update. These capabilities are based on a combination of hardware and software features, and have been expanded as Apple-designed silicon has evolved.

Feature	A10	A11 or later S3 or later M1 or later
Recovery: All Data Protection Classes protected	✓	✓
Alternate boots of DFU mode, Recovery, and software updates: Class A, B, and C data protected	✗	✓

The Secure Enclave AES Engine is equipped with lockable [software seed bits](#). When keys are created from the UID, these seed bits are included in the key derivation function to create additional key hierarchies. How the seed bit is used varies according to the [system on a chip \(SoC\)](#):

- Starting with the Apple A10 and S3 SoCs, a seed bit is dedicated to distinguish keys protected by the user's passcode. The seed bit is set for keys that require the user's passcode (including Data Protection Class A, Class B, and Class C keys), and cleared for keys that don't require the user's passcode (including the file system metadata key and Class D keys).
- In iOS 13.1, iPadOS 13, visionOS 1, or later, on devices with an A10 or later, all user data is rendered cryptographically inaccessible when devices are booted into Diagnostics mode. This is achieved by introducing an additional seed bit whose setting governs the ability to access the media key, which itself is needed to access the metadata (and therefore contents of all files) on the data volume encrypted with Data Protection. This protection encompasses files protected in all classes (A, B, C, and D), not just those that required the user's passcode.
- On devices with A12 SoCs, the Secure Enclave [Boot ROM](#) locks the passcode seed bit if the Application Processor has entered [Device Firmware Upgrade \(DFU\) mode](#) or [Recovery mode](#). When the passcode seed bit is locked, no operation to change it is allowed. This is designed to prevent access to data protected with the user's passcode.

Restoring a device after it enters DFU mode returns it to a known good state with the certainty that only unmodified Apple-signed code is present. DFU mode can be entered manually.

See the following Apple Support articles on how to place a device in DFU mode:

Device	Apple Support article
iPad, iPhone	If you forgot your iPhone passcode or your iPhone is disabled
Apple TV	If you see a warning symbol on Apple TV
A Mac with Apple silicon	How to revive or restore Mac firmware

Protecting user data in the face of attack

Attackers attempting to extract user data often try a number of techniques: extracting the encrypted data to another medium for brute-force attack, manipulating the operating system version, or otherwise changing or weakening the security policy of the device to facilitate attack. Attacking data on a device often requires communicating with the device using physical interfaces like Thunderbolt, Lightning, or USB-C. Apple devices include features to help prevent such attacks.

Apple devices support a technology called *Sealed Key Protection (SKP)* that's designed to ensure that cryptographic material is rendered unavailable off device, or that's used if manipulations are made to operating system versions or security settings without appropriate user authorization. This feature isn't provided by the Secure Enclave; instead, it's supported by hardware registers that exist at an even lower layer to provide an additional protection to the keys necessary to decrypt user data independent of the Secure Enclave.

SKP is available only on devices with the following Apple-designed SoCs:

- A11 or later
- S3 or later
- M1 or later

iPad and iPhone devices can also be configured to only activate data connections in conditions more likely to indicate the device is still under the physical control of the authorized owner.

Automatic Restart

Automatic Restart is a security mechanism in iOS 18.1 iPadOS 18.1 and or later that leverages the Secure Enclave to monitor device unlock events. If a device remains locked for a prolonged period, it automatically restarts, transitioning from an After First Unlock state to a Before First Unlock state. During the restart, the device purges sensitive security keys and transient data from memory.

For additional control, on devices with iOS 18.4, iPadOS 18.4, or later, the `IdleRebootAllowed` setting to allow device management administrators to turn on or turn off Automatic Restart. With this setting, administrators can programmatically to turn on or turn off Automatic Restart behavior to align with organizational security protocols and operational requirements.

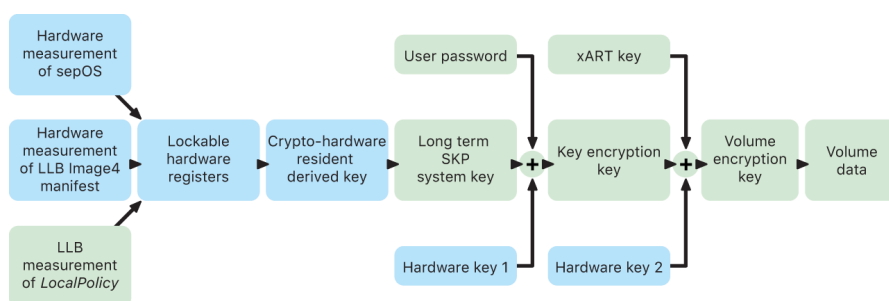
Note: Automatic Restart is turned off by default on supervised devices.

Although Automatic Restart enhances security, it can inadvertently cause devices to lose their Wi-Fi connection upon restart. This loss of connectivity may disrupt device management service operations, especially in environments where persistent network access is critical.

Sealed Key Protection (SKP)

On Apple devices that support Data Protection, the key encryption key (KEK) is protected (or sealed) with measurements of the software on the system, as well as being tied to the UID available only from the Secure Enclave. On a Mac with Apple silicon, the protection of the KEK is further strengthened by incorporating information about security policy on the system, because macOS supports critical security policy changes (for example, turning off secure boot or SIP) that are unsupported on other platforms. On a Mac with Apple silicon, this protection encompasses [FileVault](#) keys, because FileVault is implemented using Data Protection (Class C).

The key that results from entangling the user password, long-term SKP key, and Hardware key 1 (the UID from Secure Enclave) is called the *password-derived key*. This key is used to protect the user keybag (on all supported platforms) and KEK (in macOS only), and then enable biometric unlock or auto unlock with other devices such as Apple Watch.



The Secure Enclave Boot Monitor captures the measurement of the Secure Enclave OS that's loaded. When the Application Processor Boot ROM measures the Image4 manifest attached to LLB, that manifest contains a measurement of all other system-paired firmware that's loaded as well. The LocalPolicy contains the core security configurations for the macOS which are loaded. The LocalPolicy also contains the `nsih` field which is a hash of the macOS Image4 manifest. The macOS Image4 manifest contains measurements of all of the macOS-paired firmware and core macOS boot objects such as the Boot Kernel Collection or signed system volume (SSV) root hash.

If an attacker can unexpectedly change any of the above measured firmware, software, or security configuration components, it modifies the measurements stored in the hardware registers. The modification of the measurements causes the crypto-hardware-derived *system measurement root key (SMRK)* to derive to a different value, effectively breaking the seal on the key hierarchy. That causes the *system measurement device key (SMDK)* to be inaccessible, which in turn causes the KEK, and the data, to be inaccessible.

However, when the system isn't under attack, it needs to accommodate legitimate software updates that change the firmware measurements and the `nsih` field in the LocalPolicy to point at new macOS measurements. In other systems that attempt to incorporate firmware measurements but that don't have a known good source of truth, the user is required to stop security protections, update firmware, and then start the protections again so that a new measurement baseline can be captured. This significantly increases the risk that the attacker could tamper with firmware during a software update. The system is helped by the fact that the Image4 manifest contains all the measurements needed. The hardware that decrypts the SMDK with the SMRK when the measurements match during a normal boot can also encrypt the SMDK to a proposed future SMRK. By specifying the measurements that are expected after a software update, the hardware can encrypt an SMDK, which is accessible in a current operating system, so that it remains accessible in a future operating system. Similarly, when a customer legitimately changes their security settings in the LocalPolicy, the SMDK needs to be encrypted to the future SMRK based on the measurement for the LocalPolicy, which LLB computes on the next restart.

Role of Apple File System

Apple File System (APFS) is a proprietary file system designed with encryption in mind. APFS works across all Apple's platforms—for iPad, iPhone, Mac, Apple TV, Apple Vision Pro, and Apple Watch. Optimized for Flash/SSD storage, it features strong encryption, copy-on-write metadata, space sharing, cloning for files and directories, snapshots, fast directory sizing, atomic safe-save primitives, and improved file system fundamentals, as well as a unique copy-on-write design that uses I/O coalescing to deliver maximum performance while ensuring data reliability.

Space sharing

APFS allocates storage space on demand. When a single APFS container has multiple volumes, the container's free space is shared and can be allocated to any of the individual volumes as needed. Each volume uses only part of the overall container, so the available space is the total size of the container, minus the space used in all volumes in the container.

Multiple volumes

In macOS 10.15 or later, an APFS container used to start up the Mac needs to contain at least five volumes, the first three of which are hidden from the user:

- *Preboot volume*: This volume is unencrypted and contains data needed for booting each system volume in the container.
- *VM volume*: This volume is unencrypted and is used by macOS for storing encrypted swap files.
- *Recovery volume*: This volume is unencrypted and needs to be available without unlocking a system volume to start up in recoveryOS.
- *System volume*: Contains the following:
 - All the necessary files to start up the Mac
 - All apps installed natively by macOS (apps that used to reside in the `/Applications` folder now reside in `/System/Applications`)

Note: By default, no process can write to the System volume, even Apple system processes.

- *Data volume:* Contains data that's subject to change, such as:
 - Any data inside the user's folder, including photos, music, videos, and documents
 - Apps the user installed, including AppleScript and Automator applications
 - Custom frameworks and daemons installed by the user, organization, or third-party apps
 - Other locations owned and writable by the user, as /Applications, /Library, /Users, /Volumes, /usr/local, /private, /var, and /tmp

A data volume is created for each additional system volume. The preboot, VM, and recovery volumes are all shared and not duplicated.

On devices with macOS 11 or later, the system volume is captured in a snapshot. The operating system boots from a snapshot of the system volume, not just from a read-only mount of the mutable system volume.

On devices with iOS, iPadOS, and visionOS, storage is divided into at least two APFS volumes:

- System volume
- Data volume

Keychain data protection

Many apps need to handle passwords and other short but sensitive bits of data, such as keys and login tokens. The [keychain](#) provides a secure way to store these items. The various Apple operating systems use differing mechanisms to enforce the guarantees associated with the different keychain protection classes. On devices with macOS, Data Protection isn't used directly to enforce these guarantees.

Overview

Keychain items are encrypted using two different AES-256-GCM keys: a table key (metadata) and a per-row key (secret key). Keychain metadata (all attributes other than `kSecValue`) is encrypted with the metadata key to speed searches, and the secret value (`kSecValueData`) is encrypted with the secret key. The metadata key is protected by the Secure Enclave but is cached in the Application Processor to allow fast queries of the keychain. The secret key always requires a round trip through the Secure Enclave.

The keychain is implemented as a SQLite database, stored on the file system. There is only one database, and the `securityd` daemon determines which keychain items each process or app can access. Keychain Access APIs result in calls to the daemon, which queries the app's Keychain-access-groups, application-identifier, and application-group entitlements. Rather than limiting access to a single process, access groups allow keychain items to be shared between apps.

Keychain items can be shared only between apps from the same developer. To share keychain items, third-party apps use access groups with a prefix allocated to them through the Apple Developer Program in their application groups. The prefix requirement and application group uniqueness are enforced through code signing, [provisioning profiles](#), and the [Apple Developer Program](#).

Keychain data is protected using a class structure similar to the one used in file [Data Protection](#). These classes have behaviors equivalent to file Data Protection classes but use distinct keys and functions.

Availability	File data protection	Keychain data protection
When unlocked	NSFileProtectionComplete	kSecAttrAccessibleWhenUnlocked
While locked	NSFileProtectionCompleteUnlessOpen	✗
After first unlock	NSFileProtectionCompleteUntilFirstUserAuthentication	kSecAttrAccessibleAfterFirstUnlock
Always	NSFileProtectionNone	kSecAttrAccessibleAlways
Passcode enabled	✗	kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly

Apps that use background refresh services can use `kSecAttrAccessibleAfterFirstUnlock` for keychain items that need to be accessed during background updates.

The class `kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly` behaves the same as `kSecAttrAccessibleWhenUnlocked`; however, it's available only when the device is configured with a passcode. This class exists only in the system [keybag](#); they:

- Don't sync to iCloud Keychain
- Aren't backed up
- Aren't included in escrow keybags

If the passcode is removed or reset, the items are rendered useless by discarding the class keys.

Other keychain classes have a "This device only" counterpart, which is always protected with the UID when being copied from the device during a backup, rendering it useless if it's restored to a different device. Apple has carefully balanced security and usability by choosing keychain classes that vary according to the type of information being secured and when it's needed by iOS, iPadOS, and visionOS.

Keychain data class protections

The class protections listed below are enforced for keychain items.

Item	Accessible
Handoff advertisement encryption keys	After first unlock
iCloud token	After first unlock
iMessage keys	After first unlock
LDAP, CalDAV, CardDAV	After first unlock
Mail accounts	After first unlock
Microsoft Exchange ActiveSync accounts	After first unlock

Item	Accessible
Social network account tokens	After first unlock
VPN passwords	After first unlock
Wi-Fi passwords	After first unlock
Home sharing password	When unlocked
Safari bookmarks	When unlocked
Safari passwords	When unlocked
Finder/iTunes backup	When unlocked, nonmigratory
VPN certificates	After first unlock, nonmigratory
Apple Push Notification service (APNs) token	Always, nonmigratory
Bluetooth keys	Always, nonmigratory
iCloud certificates and private key	Always, nonmigratory
SIM PIN	Always, nonmigratory
Find My token	Always
Voicemail	Always

On macOS, all keychain items installed by configuration profiles are *always* available after unlocking FileVault. On iOS, iPadOS, and visionOS, keychain items installed by a configuration profile have different accessibility depending on their type, how they're referenced, and when they were installed. By default, keychain items installed using configuration profiles are *available after first unlock and nonmigratory*. However, a keychain item installed by a configuration profile is *always* available if it:

- Was installed before upgrading to iOS 15 or iPadOS 15 or later
- Is a certificate (not an identity)
- Is an identity referenced by the IdentityCertificateUUID in a `com.apple.mdm` payload

Keychain access control

Keychains can use access control lists (ACLs) to set policies for accessibility and authentication requirements. Items can establish conditions that require user presence by specifying that they can't be accessed unless authenticated using Optic ID, Face ID, Touch ID, or by entering the device's passcode or password. Access to items can also be limited by specifying that Optic ID, Face ID, or Touch ID enrollment hasn't changed since the item was added. This limitation helps prevent an attacker from adding their own fingerprint to access a keychain item. ACLs are evaluated inside the Secure Enclave and are released to the kernel only if their specified constraints are met.

Keychain architecture in macOS

macOS also provides access to the keychain to conveniently and securely store user names and passwords, digital identities, encryption keys, and secure notes. It can be accessed by opening the Keychain Access app in `/Applications/Utilities`. Using a keychain eliminates the requirement to enter—or even remember—the credentials for each resource. An initial default keychain is created for each Mac user, though users can create other keychains for specific purposes.

In addition to relying on user keychains, macOS relies on a number of system-level keychains that maintain authentication assets that aren't user specific, such as network credentials and public key infrastructure (PKI) identities. One of these keychains, System Roots, is immutable and stores internet PKI root certificate authority (CA) certificates to facilitate common tasks like online banking and e-commerce. The user can similarly deploy internally provisioned CA certificates to managed Mac computers to help validate internal sites and services.

FileVault

Volume encryption with FileVault in macOS

Mac computers offer FileVault, a built-in encryption capability, to secure all data at rest. FileVault uses the AES-XTS data encryption algorithm to protect full volumes on internal and removable storage devices.

FileVault on a Mac with Apple silicon is implemented using Data Protection Class C with a volume key. On a Mac with Apple silicon and a Mac with an Apple T2 Security Chip, encrypted internal storage devices directly connected to the Secure Enclave leverage its hardware security capabilities as well as that of the AES Engine. After a user turns on FileVault on a Mac, their credentials are required during the boot process.

For Mac computers:

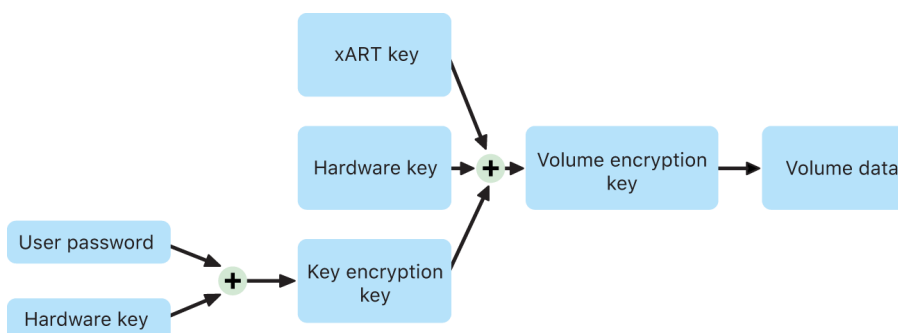
- Prior to those with a T2 chip
- With internal storage that didn't originally ship with the Mac
- With attached external storage

After FileVault is turned on, all existing files and any further data written are encrypted. Data that was added and then deleted before turning on FileVault isn't encrypted and may be recoverable with forensic data recovery tools.

Internal storage with FileVault turned on

Without valid login credentials or a cryptographic recovery key, the internal APFS volumes remain encrypted and are protected from unauthorized access even if the physical storage device is removed and connected to another computer. In macOS 10.15, this includes both the system volume and the data volume. In macOS 11 or later, the system volume is protected by the signed system volume (SSV) feature, but the data volume remains protected by encryption. Internal volume encryption on a Mac with Apple silicon as well as those with the T2 chip is implemented by constructing and managing a hierarchy of keys, and builds on the hardware encryption technologies built into the chip. This hierarchy of keys is designed to simultaneously achieve four goals:

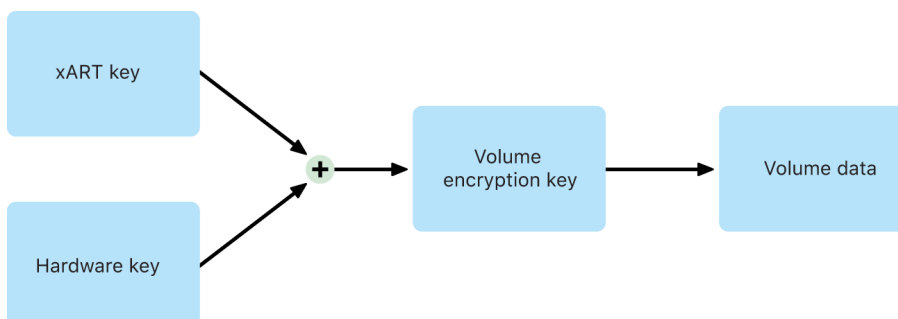
- Require the user's password for decryption
- Protect the system from a brute-force attack directly against storage media removed from Mac
- Provide a swift and secure method for wiping content via deletion of necessary cryptographic material
- Enable users to change their password (and in turn the cryptographic keys used to protect their files) without requiring reencryption of the entire volume



On a Mac with Apple silicon and those with the T2 chip, all FileVault key handling occurs in the Secure Enclave; encryption keys are never directly exposed to the CPU. All APFS volumes are created with a volume encryption key by default. Volume and metadata contents are encrypted with this volume encryption key, which is wrapped with a key encryption key (KEK). The KEK is protected by a combination of the user's password and hardware UID when FileVault is turned on.

Internal storage with FileVault turned off

If FileVault isn't turned on in a Mac with Apple silicon or a Mac with the T2 chip during the initial Setup Assistant process, the volume is still encrypted but the volume encryption key is protected only by the hardware UID in the Secure Enclave.



If FileVault is turned on later—a process that's immediate because the data has already been encrypted—an anti-replay mechanism helps prevent the old key (based on hardware UID only) from being used to decrypt the volume. The volume is then protected by a combination of the user password with the hardware UID as previously described.

Deleting FileVault volumes

When deleting a volume, its volume encryption key is securely deleted by the Secure Enclave. This helps prevent future access with this key even by the Secure Enclave. In addition, all volume encryption keys are wrapped with a media key. The media key doesn't provide additional confidentiality of data; instead, it's designed to enable swift and secure deletion of data because without it decryption is impossible.

On a Mac with Apple silicon and a Mac with the T2 chip, the media key is guaranteed to be erased by the [Secure Enclave](#) supported technology—for example by remote device management commands. Erasing the media key in this manner renders the volume cryptographically inaccessible.

FileVault recovery

macOS offers an additional password recovery option if a user has lost their account password. When FileVault is turned on, a recovery key is generated. The recovery key is a sequence of 24 random numbers and letters. It can be viewed in System Settings under Privacy & Security > FileVault, and is stored in the keychain so that it can be retrieved using the Passwords app. Additional recovery key considerations are:

- When using iCloud Keychain, the recovery key is securely synchronized along with other user passwords.
- When not using iCloud, the user is presented with the FileVault recovery key which should be stored in a safe location.
- The recovery key can be used in recoveryOS or at the Login Window when pressing Shift-Option-Return instead of the user password to unlock FileVault.
- On managed Mac computers, the organization's device management service can optionally escrow the key. For more information, see [Managing FileVault](#).

Removable storage devices

Encryption of removable storage devices doesn't utilize the security capabilities of the Secure Enclave and instead is performed in the same manner as an Intel-based Mac without the T2 chip.

Managing FileVault in macOS

On devices with macOS, organizations can manage FileVault using SecureToken or Bootstrap Token.

Using Secure Token

Apple File System (APFS) in macOS 10.13 or later changes how FileVault encryption keys are generated. In previous versions of macOS on CoreStorage volumes, the keys used in the FileVault encryption process were created when a user or organization turned on FileVault on a Mac. On devices with macOS on APFS volumes, the keys are generated either during user creation, setting the first user's password, or during the first login by a user of the Mac. This implementation of the encryption keys, when they're generated, and how they're stored are all part of a feature known as *Secure Token*. Specifically, a Secure Token is a wrapped version of a key encryption key (KEK) protected by a user's password.

When deploying FileVault on APFS, the user can continue to:

- Use existing tools and processes, such as a personal recovery key (PRK) that can be stored with a [device management service](#) for escrow
- Defer enablement of FileVault until a user logs into or out of the Mac
- Create and use an institutional recovery key (IRK)

In macOS 11, setting the initial password for the very first user on the Mac results in that user being granted a Secure Token. In some workflows, that may not be the desired behavior, as previously, granting the first Secure Token would have required the user account to log in. To prevent this from happening, add `;DisabledTags;SecureToken` to the programmatically created user's `AuthenticationAuthority` attribute prior to setting the user's password, as shown below:

```
sudo dscl . append /Users/<user name> AuthenticationAuthority  
";DisabledTags;SecureToken"
```

Using Bootstrap Token

macOS 10.15 introduced the *Bootstrap Token* to help with granting a Secure Token to both mobile accounts and the optional device enrollment-created administrator account ("managed administrator"). Using the Bootstrap Token feature of macOS 10.15 or later requires:

- Mac enrollment in a device management service using Apple School Manager or Apple Business, which makes the Mac supervised
- Device management service developer support

In macOS 10.15.4 or later, a Bootstrap Token is generated and escrowed to a device management service on the first login by any user who is Secure Token-enabled if the device management service supports the feature. A Bootstrap Token can also be generated and escrowed to a device management service using the `profiles` command-line tool, if needed.

In macOS 11 or later, a Bootstrap Token:

- Can grant a Secure Token to any user logging in to a Mac computer, including local user accounts.
- On a Mac with Apple silicon, a Bootstrap Token can be used to authorize the installation of both kernel extensions and software updates when managed using a device management service.

Institutional versus personal recovery keys

FileVault on both CoreStorage and APFS volumes supports using an institutional recovery key (IRK, previously known as a *FileVault Master identity*) to unlock the volume. Though an IRK is useful for command-line operations to unlock a volume or turn off FileVault altogether, its utility for organizations is limited, especially in recent versions of macOS. And on a Mac with Apple silicon, IRKs provide no functional value for two primary reasons: IRKs can't be used to access recoveryOS, and because target disk mode is no longer supported, the volume can't be unlocked by connecting it to another Mac.

Important: For those reasons and more, *the use of an IRK is no longer recommended for institutional management of FileVault on Mac computers*. Instead, a personal recovery key (PRK) should be used.

Unlock FileVault using SSH

On a Mac with Apple silicon with macOS 26 or later, FileVault can be unlocked over SSH after a restart if Remote Login is turned on and a network connection is available.

How Apple protects users' personal data

Protecting app access to user data

In addition to encrypting data at rest, Apple devices help prevent apps from accessing a user's personal information without permission using various technologies including [Data Vault](#). In Settings in iOS, iPadOS, and visionOS, and in macOS System Settings (macOS 13 or later) or System Preferences (macOS 12 or earlier), users can see which apps they've permitted to access certain information as well as grant or revoke any future access.

Access is enforced in the following cases:

- *iOS, iPadOS, macOS, visionOS*: Calendars, Camera, Contacts, Microphone, Photos, Reminders, and Speech recognition
- *iOS, iPadOS, visionOS*: Bluetooth, Home, Media, Media apps and Apple Music, Motion and fitness
- *iOS, watchOS*: Health
- *macOS*: Input monitoring (for example, keyboard strokes), Prompt, Screen recording (for example, static screen shots and video), and System Settings (macOS 13 or later) or System Preferences (macOS 12 or earlier)

On devices with iOS 13.4 or later and iPadOS 13.4 or later, all third-party apps automatically have their data protected in a [Data Vault](#). Data Vault helps protect against unauthorized access to the data, even from processes that aren't themselves sandboxed. Additional classes in iOS 15 and iPadOS 15 or later include Local Network, Nearby Interactions, Research Sensor & Usage Data, and Focus.

If the user signs in to iCloud, apps in iOS and iPadOS are granted access by default to iCloud Drive. Users may control each app's access under iCloud in Settings. iOS and iPadOS also provide restrictions designed to prevent data movement between apps and accounts installed by a [device management service](#) and those installed by the user.

Protecting access to user's health data

HealthKit provides a central repository for health and fitness data on iPhone and Apple Watch. HealthKit also works directly with health and fitness devices, such as compatible Bluetooth Low Energy (BLE) heart rate monitors and the motion coprocessor built into many iOS devices. All HealthKit interaction with health and fitness apps, healthcare institutions, and health and fitness devices require permission of the user. This data is stored in the [Data Protection](#) class Protected Unless Open. Access to the data is relinquished 10 minutes after the device locks, and data becomes accessible the next time user enters their passcode or uses Face ID or Touch ID to unlock the device. For certain workout-related features, an app with user-granted HealthKit data access can temporarily display workout-related data on the Lock Screen. To support this experience, when an app is using the `HKWorkoutSession` API, HealthKit data is accessible for the duration of the workout session while the device is locked. Users can manage the display of workout-related data on the Lock Screen through Settings > [Face ID][Touch ID] & Passcode > Workout Health Data.

Collecting and storing health and fitness data

HealthKit also collects and stores management data, such as access permissions for apps, names of devices connected to HealthKit, and scheduling information used to launch apps when new data is available. This data is stored in the Data Protection class Protected Until First User Authentication. Temporary journal files store health records that are generated when the device is locked, such as when the user is exercising. These are stored in the Data Protection class Protected Unless Open. When the device is unlocked, the temporary journal files are imported into the primary health databases, then deleted when the merge is completed.

Health data can be stored in iCloud. End-to-end encryption for Health data requires iOS 12 or later and two-factor authentication. Otherwise, the user's data is still encrypted in storage and transmission but isn't encrypted end-to-end. After the user turns on two-factor authentication and updates to iOS 12 or later, the user's health data is migrated to end-to-end encryption.

If the user backs up their device using the Finder (macOS 10.15 or later) or iTunes (in macOS 10.14 or earlier), health data is stored only if the backup is encrypted.

Clinical health records

Users can sign in to supported health systems within the Health app to obtain a copy of their clinical health records. When connecting a user to a health system, the user authenticates using OAuth 2 client credentials. After connecting, clinical health record data is downloaded directly from the health institution using a TLS 1.3 protected connection. Once downloaded, clinical health records are securely stored alongside other health data.

Health data authenticity

Data stored in the database includes metadata to track the provenance of each data record. This metadata includes an app identifier that identifies which app stored the record. Additionally, an optional metadata item can contain a digitally signed copy of the record. This is intended to provide data authenticity for records generated by a trusted device. The format used for the digital signature is the Cryptographic Message Syntax (CMS) specified in [RFC 5652](#).

Health data access by third-party apps

Access to the HealthKit API is controlled with entitlements, and apps need to conform to restrictions about how the data is used. For example, apps aren't allowed to use health data for advertising. Apps are also required to provide users with a privacy policy that details its use of health data.

Access to health data by apps is controlled by the user's Privacy settings. Users are asked to grant access when apps request access to health data, similar to Contacts, Photos, and other iOS data sources. However, with health data, apps are granted separate access for reading and writing data, as well as separate access for each type of health data. Users can view, and revoke, permissions they've granted for accessing health data under Settings > Health > Data Access & Devices.

If granted permission to write data, apps can also read the data they write. If granted permission to read data, apps can read data written by all sources. However, apps can't determine access granted to other apps. In addition, apps can't conclusively tell whether they've been granted read access to health data. When an app doesn't have read access, all queries return no data—the same response that an empty database would return. This is designed to prevent apps from inferring the user's health status by learning which types of data the user is tracking.

Medical ID for users

The Health app gives users the option of filling out a Medical ID form with information that could be important during a medical emergency. The information is entered or updated manually and isn't synced with the information in the health databases.

The Medical ID information is viewed by tapping the Emergency button on the Lock Screen. The information is stored on the device using the Data Protection class No Protection so that it's accessible without having to enter the device passcode. Medical ID is an optional feature that lets users decide how to balance both safety and privacy concerns. This data is backed up in iCloud Backup in iOS 13 or earlier. In iOS 14 or later, Medical ID is synced between devices using CloudKit and has the same encryption characteristics as the rest of health data.

Health sharing

On an iPhone with iOS 15 or later, the Health app gives users the option of sharing their Health data with other users. Health data is shared between the two users using end-to-end iCloud encryption, and Apple can't access data that's sent through Health sharing. To use the feature, both the sending and receiving users need to use iOS 15 or later and have two-factor authentication turned on.

Users can also choose to share their Health data with their healthcare provider using the Share with Provider feature in the Health app. Data shared using this feature is made available only to the health institutions selected by the user using end-to-end encryption, and Apple doesn't maintain or have access to the encryption keys to decrypt, view, or otherwise access the Health data shared through the Share with Provider feature. Further details about how the design of this service protects users' Health data can be found in the [Security and Privacy section](#) of the Apple Registration Guide for Healthcare Organizations.

Digital signing and encryption

Access control lists

Keychain data is partitioned and protected with access control lists (ACLs). As a result, credentials stored by third-party apps can't be accessed by apps with different identities unless the user explicitly approves them. This protection provides a mechanism for securing authentication credentials in Apple devices across a range of apps and services within the organization.

Mail

In the Mail app, users can send messages that are digitally signed and encrypted. Mail automatically discovers appropriate [RFC 5322](#) case-sensitive email address subject or subject alternative names on digital signing and encryption certificates on attached Personal Identification Verification (PIV) tokens in compatible smart cards. If a configured email account matches an email address on a digital signing or encryption certificate on an attached PIV token, Mail automatically displays the signing button in the toolbar of a new message window. If Mail has the recipient's email encryption certificate or can discover it in the Microsoft Exchange global address list (GAL), an unlocked icon appears in the new message toolbar. A locked lock icon indicates the message is sent encrypted with the recipient's public key.

Per-message S/MIME

iOS, iPadOS, macOS, and visionOS support per-message S/MIME. This means that S/MIME users can choose to always sign and encrypt messages by default or to selectively sign and encrypt individual messages.

Identities used with S/MIME can be delivered to Apple devices using a configuration profile, a [device management service](#), the Simple Certificate Enrollment Protocol (SCEP), or Microsoft Active Directory Certificate Authority.

Smart cards

On a Mac with macOS 10.12 or later, native support for PIV cards is included. These cards are widely used in commercial and government organizations for two-factor authentication, digital signing, and encryption.

Smart cards include one or more digital identities that have a pair of public and private keys and an associated certificate. Unlocking a smart card with the personal identification number (PIN) provides access to the private keys used for authentication, encryption, and signing operations. The certificate determines what a key can be used for, what attributes are associated with it, and whether it's validated (signed) by a certificate authority (CA) certificate.

Smart cards can be used for two-factor authentication. The two factors needed to unlock a card are “something the user has” (the card) and “something the user knows” (the PIN). macOS 10.12 or later also has native support for smart card Login Window authentication and client certificate authentication to websites on Safari. It also supports Kerberos authentication using key pairs (PKINIT) for Single Sign-on to Kerberos-supported services. To learn more about smart cards and macOS, see [Intro to smart card integration](#) in Apple Platform Deployment.

Encrypted disk images

On a Mac, encrypted disk images serve as secure containers in which users can store or transfer sensitive documents and other files. Encrypted disk images are created using Disk Utility, located in /Applications/Utilities. Disk images can be encrypted using either 128-bit or 256-bit AES encryption. Because a mounted disk image is treated as a local volume connected to a Mac, users can copy, move, and open files and folders stored in it. As with FileVault, the contents of a disk image are encrypted and decrypted in real time. With encrypted disk images, users can safely exchange documents, files, and folders by saving an encrypted disk image to removable media, sending it as a mail message attachment, or storing it on a remote server. For more information on encrypted disk images, see the [Disk Utility User Guide](#).

App security

App security overview

Today, apps are among the most critical elements of a security architecture. Even as apps provide productivity benefits for users, they also have the potential to negatively impact system security, stability, and user data if they're not handled properly.

Because of this, Apple provides layers of protection to help ensure that apps are free of known malware and haven't been tampered with. Additional protections enforce that access from apps to user data is carefully mediated. These security controls provide a stable, secure platform for apps, enabling thousands of developers to deliver hundreds of thousands of apps for iOS, iPadOS, macOS, tvOS, visionOS, and watchOS—all without impacting system integrity. And users can access these apps on their Apple devices without undue fear of viruses, malware, or unauthorized attacks.

On iPad and iPhone, the design principle focuses on centralized distribution, code signing, and strict sandboxing to provide the tightest controls. To reflect the Digital Market Act's requirements, users in the European Union (EU) can install apps from alternative app marketplaces and directly from an authorized developer's website, which introduces additional risks. Apple introduced protections, including (but not limited to):

- Notarization for apps
- An authorization for marketplace developers
- Disclosures on alternative payments
- Install confirmations that provide the user Apple-verified information about the app

These help to reduce risks and deliver the best, most secure experience possible for users in the EU. Even with these safeguards in place, many risks remain including a greater prevalence of malware, fraud and scams, illicit and harmful content, and other privacy and security threats. For more information, see [Update on apps distributed in the European Union](#) on the Apple Developer website.

On Mac, many apps are obtained from the App Store, but Mac users also download and use apps from the internet. To safely support internet downloading, macOS layers additional controls. First, by default in macOS 10.15 or later, all Mac apps need to be notarized by Apple to launch. This requirement helps ensure that these apps are free of known malware, without requiring that the apps be provided through the App Store. Second, macOS includes state-of-the-art antivirus protection to block—and if necessary remove—malware.

As an additional control across platforms, sandboxing helps protect user data from unauthorized access by apps. And in macOS, data in critical areas is itself protected—which helps ensure that users remain in control of access to files in Desktop, Documents, Downloads, and other areas from all apps, whether the apps attempting access are themselves sandboxed or not.

Native capability	Third-party equivalent
App notarization	Built into macOS
Kext exclude list	Built into macOS
Mandatory Access Controls	Built into macOS
Mandatory app code signing	Built into macOS
System Integrity Protection	Built into macOS
Gatekeeper	Endpoint protection; enforces code signing on apps to help ensure that only trusted software runs
Application firewall	Endpoint protection; firewalling
efiheck (Necessary for a Mac without an Apple T2 Security Chip)	Endpoint protection; rootkit detection
Packet Filter (pf)	Firewall solutions
File Quarantine	Virus/Malware definitions
Plug-in unapproved list, Safari extension unapproved list	Virus/Malware definitions
XProtect/YARA signatures	Virus/Malware definitions; endpoint protection

App code signing process in iOS, iPadOS, tvOS, visionOS, and watchOS

On devices with iOS, iPadOS, tvOS, visionOS, and watchOS, Apple offers app security through such things as mandatory code signing, strict developer sign-in, and more.

Mandatory code signing

After the kernel has started, it controls which user processes and apps can be run. To help ensure that all apps come from a known and approved source and haven't been tampered with, these operating systems require that all executable code be signed using an Apple-issued certificate. Apps provided with the device, like Mail and Safari, are signed by Apple. Third-party apps also need to be validated and signed using an Apple-issued certificate. Mandatory code signing extends the concept of chain of trust from the operating system to apps and helps prevent third-party apps from loading unsigned code resources or using self-modifying code.

How developers sign their apps

Developers can sign their apps through certificate validation (through the Apple Developer Program). They can also embed frameworks inside their apps and have that code validated with an Apple-issued certificate (through a team identifier string).

- *Certificate validation:* To develop and install apps on iPad, iPhone, Apple TV, Apple Vision Pro, and Apple Watch devices, developers need to register with Apple and join the Apple Developer Program. The real-world identity of each developer, whether an individual or a business, is verified by Apple before their certificate is issued. This certificate allows developers to sign apps and submit them to the App Store for distribution. As a result, all apps in the App Store have been submitted by an identifiable person or organization, serving as a deterrent to the creation of malicious apps. They have also been reviewed by Apple to help ensure they generally operate as described and don't contain obvious bugs or other notable problems. In addition to the technology already discussed, this curation process gives users confidence in the quality of the apps they buy.
- *Code signature validation:* iOS, iPadOS, tvOS, visionOS, and watchOS allow developers to embed frameworks inside of their apps, which can be used by the app itself or by extensions embedded within the app. To protect the system and other apps from loading third-party code inside of their address space, the system performs a code signature validation of all the dynamic libraries that a process links against at launch time. This verification is accomplished through the team identifier (Team ID), which is extracted from an Apple-issued certificate. A team identifier is a 10-character alphanumeric string—for example, 1A2B3C4D5F. A program may link against any platform library that ships with the system or any library with the same team identifier in its code signature as the main executable. Because the executables shipping as part of the system don't have a team identifier, they can only link against libraries that ship with the system itself.

Verifying proprietary in-house apps

Eligible businesses have the ability to write proprietary in-house apps for use within their organization and to distribute them to their employees. Businesses and organizations can apply to the Apple Developer Enterprise Program (ADEP). For more information and to review eligibility requirements, see the [Apple Developer Enterprise Program website](#). After an organization becomes a member of ADEP, it can register to obtain a [provisioning profile](#) that permits proprietary in-house apps to run on devices it authorizes.

Users need to have the provisioning profile installed to run these apps. This helps ensure that only the organization's intended users are able to load the apps onto their iPad, iPhone, or Apple Vision Pro. Apps installed through a [device management service](#) are implicitly trusted because the relationship between the organization and the device is already established. Otherwise, users have to approve the app's provisioning profile in Settings. Organizations can also restrict users from approving apps from unknown developers. On first launch of any proprietary in-house app, the device needs to receive positive confirmation from Apple that the app is allowed to run.

App security in iOS, iPadOS, and visionOS

Intro to app security for iOS, iPadOS, and visionOS

Unlike other mobile platforms, iOS, iPadOS, and visionOS don't allow users to install potentially malicious unsigned apps from websites or to run untrusted apps. Instead, (outside the EU) all apps need to be downloaded from the App Store, where all apps come from identified developers and need to pass automated and human review. At runtime, code signature checks of all executable memory pages are checked as they're loaded to help ensure that an app hasn't been modified since it was installed or last updated.

After an app is verified to be from an approved source, iOS, iPadOS, and visionOS enforce security measures designed to prevent it from compromising other apps or the rest of the system.

About App Store security

The App Store is a trusted place where users can safely discover and download apps. On the App Store, apps come from identified developers who have agreed to follow Apple guidelines, and are securely distributed to users with cryptographic guarantees against modification. Every single app and each app update is reviewed to evaluate whether it meets requirements for privacy, security, and safety. This process, which is being constantly improved, is designed to protect users by keeping malware, cybercriminals, and scammers out of the App Store. In addition, apps designed for children need to follow strict guidelines around data collection and security designed to keep children safe, and need to be tightly integrated with iOS, iPadOS, and visionOS parental control features.

App Store security protections include:

- *Automated scans for known malware:* To help prevent it from ever making it onto the App Store and thus ever reaching or harming users.
- *Human review by a team of experts:* To review the app description—including marketing text and screenshots—for accuracy. This creates a high barrier against the most common scams used to distribute malware: misrepresenting the malware as a popular app, or claiming to offer enticing features that aren't actually provided.
- *Manual checks:* To check that the app doesn't unnecessarily request access to sensitive data and extra evaluation of apps targeted at children to help ensure they comply with stringent data collection and safety rules.
- *Trustworthy, centralized user reviews:* To help surface issues and significantly reduce the attacker's prospects of misleading many users. Even if a malicious app were able to completely hide its behavior during the review process, users of the app who encounter and report issues alert others—and Apple—thereby providing another avenue for detection. The App Store aggressively combats fraudulent reviews to improve the value of this signal.
- *Processes for correction and removal:* In a case where an app makes it into the App Store but is later discovered to violate guidelines, Apple works with the developer to quickly resolve the issue. In dangerous cases, involving fraud and malicious activity, the app is immediately removed from the App Store and users who downloaded the apps can be notified of the app's malicious behavior.

Security of apps on iOS, iPadOS, and visionOS relies on a combination of all layers—robust App Review to help prevent the installation of malicious apps, and robust platform protections to limit the damage malicious apps can inflict. The security designed into iOS, iPadOS, and visionOS provides users with powerful protections that are the best of any consumer device, but those protections aren't engineered to protect against choices a user might be tricked into making. App Review enforces the App Store policies designed to protect users from apps that may attempt to harm them or trick them into granting access to sensitive data. And, in the very serious instances of malicious apps trying to bypass on-device protections, App Review makes it harder for them to get on users' devices in the first place.

Though App Store security measures alone can never be perfect, as part of a defense-in-depth strategy for platform security they contribute to making widespread attacks against iOS, iPadOS, and visionOS users impractical and uneconomical for financially-driven attackers. By reviewing every app before it becomes available on the App Store to help ensure it's free of malware and accurately represented to users, and by swiftly removing apps from distribution if they are found to be harmful and limiting the spread of future variants, Apple protects the security of the ecosystem and provides peace of mind to customers.

Security of runtime process in iOS, iPadOS, and visionOS

iOS, iPadOS, and visionOS help ensure runtime security by using a *sandbox*, declared entitlements, and Address Space Layout Randomization (ASLR).

Sandboxing

All third-party apps are sandboxed. Sandboxing is designed to prevent apps from gathering or modifying information stored by other apps or from making changes to the device. Each app has a unique home directory for its files, which is randomly assigned when the app is installed. If a third-party app needs to access information other than its own, it does so only by using services explicitly provided by iOS, iPadOS, and visionOS.

System files and resources are also shielded from the users' apps. Most iOS, iPadOS, and visionOS system files and resources run as the nonprivileged user *mobile*, as do all third-party apps. The entire operating system partition is mounted as read-only. Unnecessary tools, such as remote login services, aren't included in the system software, and APIs don't allow apps to escalate their own privileges to modify other apps or iOS, iPadOS, and visionOS.

Use of entitlements

Access by third-party apps to user information, and to features such as iCloud and extensibility, is controlled using declared entitlements. Entitlements are key-value pairs that are signed in to an app and allow authentication beyond runtime factors, like UNIX user ID. Since entitlements are digitally signed, they can't be changed. Entitlements are used extensively by system apps and daemons to perform specific privileged operations that would otherwise require the process to run as root. This greatly reduces the potential for privilege escalation by a compromised system app or daemon.

In addition, apps can only perform background processing through system-provided APIs. This allows apps to continue to function without degrading performance or dramatically impacting battery life.

Address Space Layout Randomization

Address Space Layout Randomization (ASLR) helps protect against the exploitation of memory corruption bugs. Built-in apps use ASLR to help randomize all memory regions upon launch. In addition to work upon launch, ASLR randomly arranges the memory addresses of executable code, system libraries, and related programming constructs, further reducing the likelihood of many exploits. For example, a return-to-libc attack attempts to trick a device into executing malicious code by manipulating memory addresses of the stack and system libraries. Randomizing the placement of these makes the attack more difficult to execute, especially across multiple devices. Xcode, and the iOS or iPadOS development environments, automatically compile third-party programs with ASLR support turned on.

Execute Never feature

Further protection is provided by iOS and iPadOS using ARM's Execute Never (XN) feature, which marks memory pages as nonexecutable. Memory pages marked as both writable and executable can be used only by apps under tightly controlled conditions: The kernel checks for the presence of the Apple-only dynamic code-signing entitlement. Even then, only a single mmap call can be made to request an executable and writable page, which is given a randomized address. Safari uses this functionality for its JavaScript just-in-time (JIT) compiler.

App protection and app groups in iOS, iPadOS, and visionOS

With iPad, iPhone, and Apple Vision Pro, organizations can protect apps securely by using the iOS SDK and by joining an App Group at the [Apple Developer website](#).

Adopting Data Protection in apps

The Apple Software Development Kits (SDK) offer a full suite of APIs that make it easy for third-party and in-house developers to adopt [Data Protection](#) and help ensure the highest level of protection in their apps. Data Protection is available for file and database APIs, including Core Data, NSData, NSFileManager, and SQLite.

The Mail app database (including attachments), managed books, Safari bookmarks, app launch images, and location data are also stored through encryption, with keys protected by the user's passcode on their device. Calendar (excluding attachments), Contacts, Reminders, Notes, Messages, and Photos implement the Data Protection entitlement *Protected Until First User Authentication*.

User-installed apps that don't opt in to a specific Data Protection class receive Protected Until First User Authentication by default.

Joining an App Group

Apps and extensions owned by a given developer account can share content when configured to be part of an App Group. It's up to the developer to create the appropriate groups on the Apple Developer website and include the desired set of apps and extensions. Once configured to be part of an App Group, apps have access to the following:

- A shared on-volume container for storage, which stays on the device as long as at least one app from the group is installed

- Shared preferences
- Shared [keychain](#) items

The Apple Developer website helps ensure that app [group IDs \(GIDs\)](#) are unique across the app ecosystem.

App security in macOS

Intro to app security for macOS

App security in macOS consists of a number of overlapping layers, the first of which is the option to run only signed and trusted apps from the App Store. In addition, macOS layers protections to help ensure that apps downloaded from the internet are free of known malware.

macOS offers technologies to detect and remove malware, and offers additional protections designed to prevent untrusted apps from accessing user data. Apple services such as Notarization and XProtect updates are designed to help prevent malware installation. When necessary, these services locate malware that may have at first avoided detection and then quickly and efficiently remove it. Ultimately, macOS users are free to operate within the security model that makes sense for them, including running completely unsigned and untrusted code.

App code signing process in macOS

All apps from the App Store are signed by Apple. This signing is designed to ensure that they haven't been tampered with or altered. Apple signs any apps provided with Apple devices.

On devices with macOS 10.15 or later, all apps distributed outside the App Store need to be signed by the developer using an Apple-issued Developer ID certificate (combined with a private key) and notarized by Apple to run under the default Gatekeeper settings. Apps developed in-house should also be signed with an Apple-issued Developer ID so that users can validate their integrity.

On devices with macOS, code signing and notarization work independently—and can be performed by different actors—for different goals. Code signing is performed by the developer using their Developer ID certificate (issued by Apple). Verification of this signature proves to the user that a developer's software hasn't been tampered with since the developer built and signed it. Notarization can be performed by anyone in the software distribution chain and proves that Apple has been provided a copy of the code to check for malware and no known malware was found. The output of Notarization is a ticket, which is stored on Apple servers and can be optionally stapled to the app (by anyone) without invalidating the signature of the developer.

Mandatory Access Controls (MACs) require code signing to enable entitlements protected by the system. For example, apps requiring access through the firewall need to be code signed with the appropriate MAC entitlement.

Gatekeeper and runtime protection in macOS

macOS offers the Gatekeeper technology and runtime protection to help ensure that only trusted software runs on a user's Mac.

Gatekeeper

macOS includes a security technology called *Gatekeeper*, which is designed to help ensure that only trusted software runs on a user's Mac. When a user downloads and opens an app, a plug-in, or an installer package from outside the App Store, Gatekeeper verifies that the software is from an identified developer, is notarized by Apple to be free of known malicious content, and hasn't been altered. Gatekeeper also requests user approval before opening downloaded software for the first time to make sure the user hasn't been tricked into running executable code they believed to simply be a data file. Gatekeeper also tracks the provenance of files written by downloaded software.

By default, Gatekeeper helps ensure that all downloaded software has been signed by the App Store or signed by a registered developer and notarized by Apple. Both the App Store review process and the notarization pipeline are designed to ensure that apps contain no known malware. Therefore by default, *all software in macOS is checked for known malicious content the first time it's opened, regardless of how it arrived on the Mac.*

Users and organizations have the option to allow only software installed from the App Store. Alternatively, users can override Gatekeeper policies to open any software unless restricted by a [device management service](#). This includes allowing software signed with alternate identities. Gatekeeper can also be completely turned off, if necessary.

Gatekeeper also protects against the distribution of malicious plug-ins with benign apps. Here, using the app triggers the loading of a malicious plug-in without the user's knowledge. When necessary, Gatekeeper opens apps from randomized, read-only locations. This is designed to prevent the automatic loading of plug-ins distributed alongside the app.

Runtime protection

System files, resources, and the kernel are shielded from a user's app space. All apps from the App Store are sandboxed to restrict access to data stored by other apps. If an app from the App Store needs to access data from another app, it can do so only by using the APIs and services provided by macOS.

Protecting against malware in macOS

Apple operates a threat intelligence process to quickly identify and block malware.

Three layers of defense

Malware defenses are structured in three layers:

1. *Prevent launch or execution of malware:* App Store, or Gatekeeper combined with Notarization
2. *Block malware from running on customer systems:* Gatekeeper, Notarization, and XProtect
3. *Remediate malware that has executed:* XProtect

The first layer of defense is designed to inhibit the distribution of malware, and prevent it from launching even once—this is the goal of the App Store, and Gatekeeper combined with Notarization.

The next layer of defense is to help ensure that if malware appears on any Mac, it's quickly identified and blocked, both to halt spread and to remediate the Mac systems it's already gained a foothold on. XProtect adds to this defense, along with Gatekeeper and Notarization.

Finally, XProtect acts to remediate malware that has managed to successfully execute.

These protections, further described below, combine to support best-practice protection from viruses and malware. There are additional protections, particularly on a Mac with Apple silicon, to limit the potential damage of malware that does manage to execute. See [Protecting app access to user data](#) for ways that macOS can help protect user data from malware, and [Operating system integrity](#) for ways macOS can limit the actions malware can take on the system.

Notarization

Notarization is a malware scanning service provided by Apple. Developers who want to distribute apps for macOS outside the App Store submit their apps for scanning as part of the distribution process. Apple scans this software for known malware and, if none is found, issues a Notarization ticket. Typically, developers staple this ticket to their app so Gatekeeper can verify and launch the app, even offline.

Apple can also issue a revocation ticket for apps known to be malicious—even if they've been previously notarized. macOS regularly checks for new revocation tickets so that Gatekeeper has the latest information and can block launch of such files. This process can very quickly block malicious apps because checks for revocation tickets happen in the background much more frequently than even the background updates that push new XProtect signatures. In addition, this protection can be applied to both apps that have been previously notarized and those that haven't.

XProtect

macOS includes built-in antivirus technology called *XProtect* for the signature-based detection and removal of malware. The system uses YARA signatures, a tool used to conduct signature-based detection of malware, which Apple updates regularly. Apple monitors for new malware infections and strains, and updates signatures automatically—independent from system updates—to help defend a Mac from malware infections. XProtect automatically detects and blocks the execution of known malware. In macOS 10.15 or later, XProtect checks for known malicious content whenever:

- An app is first launched
- An app has been changed (in the file system)
- XProtect signatures are updated

When XProtect detects known malware, it blocks it and moves it to the Trash. Then it alerts the user in the Finder. Users might be asked to share malware samples with Apple to improve macOS security. If they agree, XProtect uploads only the malware executable or, if it's in an app bundle, the entire bundle. Nothing else is shared.

Note: Notarization is effective against known files (or file hashes) and can be used on apps that have been previously launched. The signature-based rules of XProtect are more generic than a specific file hash, so it can find variants that Apple hasn't seen. XProtect scans only apps that have been changed or apps at first launch.

Should malware make its way onto a Mac, XProtect also includes technology to remediate infections. For example, it includes an engine that remediates infections based on updates automatically delivered from Apple (as part of automatic updates of system data files and security updates). This system removes malware upon receiving updated information, and it continues to periodically check for infections; however, XProtect doesn't automatically restart the Mac. In addition, XProtect contains an advanced engine to detect unknown malware based on behavioral analysis. Information about malware detected by this engine, including what software was ultimately responsible for downloading it, is used to improve XProtect signatures and macOS security.

Automatic XProtect security updates

Apple issues the updates for XProtect automatically based on the latest threat intelligence available. By default, macOS checks for these updates daily. Notarization updates, which are distributed using CloudKit sync are much more frequent.

How Apple responds when new malware is discovered

When new malware is discovered, a number of steps may be performed:

- Any associated Developer ID certificates are revoked.
- Notarization revocation tickets are issued for all files (apps and associated files).
- XProtect signatures are developed and released.

These signatures are also applied retroactively to previously notarized software, and any new detections can result in one or more of the previous actions occurring.

Ultimately, a malware detection launches a series of steps over the next seconds, hours, and days that follow to propagate the best protections possible to Mac users.

Gatekeeper bypass and XProtect events

macOS offers a secure API—the [Endpoint Security API](#)—for developers to create security software. For a Mac with macOS 15 or later, third-party developers can now receive events when a user bypasses Gatekeeper to run a program. This helps developers log these events centrally. It's especially useful for security software and administrators, because it signals when untrusted software is running on endpoints. Moreover, users can view this event in the eslogger utility, which is built into macOS.

The Endpoint Security API also provides insights into XProtect malware detections. Developers can see:

- Specific files that XProtect detected
- The XProtect signature associated with the detection event

This additional information can be collected by third-party developers and stored for later incident response and central logging.

Terminal and script protections

Overview

As anti-malware protections on macOS become more effective, threat actors have increasingly turned to new techniques, including social engineering—persuading users to copy and paste malicious commands into a terminal, or to download and execute malicious scripts. On a Mac with macOS 26.4 or later, a set of three security enhancements were designed to address such threats. While each operates independently, together they form a layered defense against common social engineering campaigns and attacks targeting command-line execution.

Terminal paste protection

Terminal paste protection helps protect users from social engineering attacks that instruct them to copy and paste malicious commands into Terminal. A warning prompt appears when a user pastes text into the Terminal app from a common attack vector application—such as web browsers, messaging apps, and other common communication software. This warning is shown regardless of the contents of the paste being copied.

This warning is designed to help users who don't routinely interact with the Terminal app. The warning is shown only when all of the following conditions are met:

- The user hasn't opened the Terminal app in more than 30 days.
- No common developer tooling is detected on the system, such as Xcode and other common third-party IDEs.
- The paste originates from a specific app, including popular web browsers, Messages, Mail, Preview, and common third-party messaging apps.

When these conditions are met and the user pastes content into the Terminal app, a dialog explains the potential risk and asks the user to confirm before the pasted text is executed.

Note: A 24-hour grace period after the initial system setup allows users to configure their systems without encountering the prompt. System software updates don't reset this grace period.

Pasteboard command blocking

Pasteboard command blocking is designed to help prevent malware delivered through pasted terminal commands. When a user pastes text into any terminal emulator—including third-party terminal applications—XProtect traces the resulting process tree and inspects its behavior. Network artifacts associated with pasted commands are checked against Apple's Safe Browsing Service. If a known malicious source is detected, the process is blocked and the user is notified. XProtect also evaluates behavioral patterns across the process tree, and can block activity consistent with known malware techniques.

AppleScript scanning

AppleScript scanning for AppleScript and JavaScript for Automation (JXA) scripts uses XProtect technology. Because the scanning is integrated into the OpenScripting framework, all AppleScript executions on macOS are subject to local inspection, whether the script is loaded from storage or from memory.

When a script matches a known malicious signature, macOS blocks execution and presents the user with a notification. If the blocked script is associated with a file stored on one of the following, the user can choose to override the block through the existing malware protection override in Finder:

- Internal storage
- External device
- Cloud-based service
- Mounted file server

Scripts executed without an associated file—such as those run directly from memory—can't be overridden. The user needs to save the script first before requesting an override.

Controlling app access to files in macOS

Apple believes that users should have full transparency, consent, and control over what apps are doing with their data. In macOS 10.15 or later, this model is enforced by the system to help ensure that all apps have obtained user consent before accessing files in Documents, Downloads, Desktop, iCloud Drive, and network volumes.

In macOS 10.13 or later, apps that require access to the full storage device need to be explicitly added in System Settings (macOS 13 or later) or System Preferences (macOS 12 or earlier). In addition, accessibility and automation capabilities require user permission to help ensure that they don't circumvent other protections. Depending on the access policy, users may be asked, or required, to change the setting in:

- In macOS 13 or later: System Settings > Privacy & Security > Privacy
- In macOS 12 or earlier: System Preferences > Security & Privacy > Privacy

Item	User prompted by app	User needs to edit system privacy settings
Accessibility	✗	✓
Full internal storage access	✗	✓
Files and folders <i>Note: Includes Desktop, Documents, Downloads, network volumes, and removable volumes</i>	✓	✗
Automation (Apple events)	✓	✗

A user who turns on FileVault on a Mac is asked to provide valid credentials before continuing the boot process and gain access to specialized startup modes. Without valid login credentials or a recovery key, the entire volume remains encrypted and is protected from unauthorized access, even if the physical storage device is removed and connected to another computer.

To protect data in an enterprise setting, IT should define and enforce FileVault configuration policies using a device management service. Organizations have several options for managing encrypted volumes, including institutional recovery keys, personal recovery keys (that can optionally be stored with a device management service for escrow), or a combination of both. Key rotation can also be set as a policy in a device management service.

Supporting extensions in iOS, iPadOS, macOS, and visionOS

iOS, iPadOS, macOS, and visionOS allow apps to provide functionality to other apps by providing extensions. Extensions are special-purpose signed executable binaries packaged within an app. During installation, the system automatically detects extensions and makes them available to other apps using a matching system.

Extension points

A system area that supports extensions is called an *extension point*. Each extension point provides APIs and enforces policies for that area. The system determines which extensions are available based on extension point–specific matching rules. The system automatically launches extension processes as needed and manages their lifetime. Entitlements can be used to restrict extension availability to particular system apps. For example, a Today view widget appears only in Notification Center, and a sharing extension is available only from the Sharing pane. Examples of extension points are Today widgets, Share, Actions, Photo Editing, File Provider, and Custom Keyboard.

How extensions communicate

Extensions run in their own address space. Communication between the extension and the app from which it was activated uses interprocess communications mediated by the system framework. They don't have access to each other's files or memory spaces. Extensions are designed to be isolated from each other, from their containing apps, and from the apps that use them. They are sandboxed like any other third-party app and have a container separate from the containing app's container. However, they share the same access to privacy controls as the container app. If a user grants an app access to Contacts, this grant is extended to the extensions that are embedded within the app but not to the extensions activated by the app.

How custom keyboards are used

Custom keyboards are a special type of extension, one that's enabled by the user for the entire system. After a keyboard extension is enabled, it's used for any text field except the passcode input and any secure text view. To restrict the transfer of user data, custom keyboards run by default in a very restrictive sandbox that blocks access to the network, to services that perform network operations on behalf of a process, and to APIs that would allow the extension to exfiltrate typing data. Developers of custom keyboards can request that their extension have Open Access, which lets the system run the extension in the default sandbox after getting consent from the user.

Device management and extensions

For devices enrolled in a [device management service](#), document and keyboard extensions obey Managed Open In rules. For example, the device management service can help prevent users from exporting a document from a managed app to an unmanaged Document Provider, or help prevent them from using an unmanaged keyboard with a managed app. Additionally, app developers can prevent the use of third-party keyboard extensions within their app.

Secure features in the Notes app

The Notes app includes a secure notes feature—on iPad, iPhone, Mac, Apple Vision Pro, and the iCloud website—that allows users to lock notes to protect the contents of specific notes. The user can choose to either use the device passcode to unlock the note, or they can set a separate password specifically for that note. Users can also securely share notes with others.

Secure notes

Secure notes are end-to-end encrypted using a user-provided passphrase that's required to view the notes on supported devices, and the iCloud website. Each Apple Account (including "On my" device accounts) can have a separate passphrase.

When a user secures a note, a 16-byte key is derived from the user's passphrase using PBKDF2 and SHA-256. The note and all of its attachments are encrypted using AES with Galois/Counter Mode (AES-GCM). New records are created in Core Data and CloudKit to store the encrypted note, attachments, tag, and initialization vector. After the new records are created, the original unencrypted data is deleted. Attachments that support encryption include images, sketches, tables, maps, and websites. Notes containing other types of attachments can't be encrypted, and unsupported attachments can't be added to secure notes.

To view a secure note, the user needs to enter their passphrase or authenticate using Optic ID, Face ID or Touch ID. After successfully authenticating the user, whether to view or create a secure note, Notes opens a secure session. While the secure session is open, the user can view or secure other notes without additional authentication. However, the secure session applies only to notes protected with the passphrase. The user still needs to authenticate for notes protected by a different passphrase. The secure session is closed when:

- The user taps the Lock Now button in Notes
- The iPad, iPhone, or Apple Vision Pro locks
- Notes is switched to the background for more than three minutes on an iPad, iPhone, or Apple Vision Pro
- Notes is switched to the background for more than eight minutes on a Mac

To change the passphrase on a secure note, the user needs to enter the current passphrase, because Optic ID, Face ID and Touch ID aren't available when changing the passphrase. After choosing a new passphrase, the Notes app rewraps, in the same account, the keys of all existing notes that are encrypted by the previous passphrase.

If a user mistypes the passphrase three times in a row, Notes shows a user-supplied hint if one was provided by the user at setup. If the user still doesn't remember their passphrase, they can reset it in Notes settings. This feature allows users to create new secure notes with a new passphrase, but it won't allow them to see previously secured notes. The previously secured notes can still be viewed if the old passphrase is remembered. Resetting the passphrase requires the user's Apple Account passphrase.

Shared notes

Notes that aren't end-to-end encrypted with a passphrase can be shared with others. Shared notes still use the CloudKit encrypted data type for any text or attachments that the user puts in a note. Assets are always encrypted with a key that's encrypted in the [CKRecord](#). Metadata, such as the creation and modification dates, aren't encrypted. CloudKit manages the process by which participants can encrypt and decrypt each other's data.

Secure features in the Shortcuts app

In the Shortcuts app, shortcuts are optionally synced across Apple devices using iCloud. Shortcuts can also be shared with other users through iCloud. Shortcuts are stored locally in an encrypted format.

Custom shortcuts are versatile—they're similar to scripts or programs. When downloading shortcuts from the internet, the user is warned that the shortcut hasn't been reviewed by Apple and is given the opportunity to inspect the shortcut. To protect against malicious shortcuts, updated malware definitions are downloaded to identify malicious shortcuts at runtime.

Custom shortcuts can also run user-specified JavaScript on websites in Safari when invoked from the share sheet. To protect against malicious JavaScript that, for example, tricks the user into running a script on a social media website that harvests their data, the JavaScript is validated against the aforementioned malware definitions. The first time a user runs JavaScript on a domain, the user is prompted to allow shortcuts containing JavaScript to run on the current webpage for that domain.

Services security

Services security overview

Apple has built a robust set of services to help users get even more utility and productivity out of their devices. These services provide powerful capabilities for cloud storage, sync, password storage, authentication, payment, messaging, communications, and more, all while protecting users' privacy and the security of their data.

This chapter covers security technologies used in iCloud, Sign in with Apple, Apple Pay, Apple Wallet, iMessage, Apple Messages for Business, FaceTime, Find My, and Continuity.

Note: Not all Apple services and content are available in all countries or regions.

Apple Accounts and Managed Apple Accounts

Apple Account security overview

An Apple Account is the account used to sign in to Apple services. It's important for users to keep their Apple Accounts secure to help prevent unauthorized access to their accounts. To help with this, Apple Accounts require strong passwords that:

- Need to be at least eight characters in length
- Need to contain both letters and numbers
- Need to not contain three or more consecutive identical characters
- Can't be a commonly used password

Users are encouraged to exceed these guidelines by adding extra characters and punctuation marks to make their passwords even stronger.

Apple also notifies users in email or push notifications or both when important changes are made to their account—for example, if a password or billing information has been changed or the Apple Account has been used to sign in on a new device. If anything looks unfamiliar, users are instructed to change their Apple Account password immediately.

In addition, Apple employs a variety of policies and procedures designed to protect user accounts. These include limiting the number of retries for sign-in and password reset attempts, active fraud monitoring to help identify attacks as they occur, and regular policy reviews that allow Apple to adapt to any new information that could affect user security.

Two-factor authentication

To help users further secure their accounts, by default Apple uses *two-factor authentication*—an extra layer of security for Apple Accounts. It's designed to ensure that only the account's owner can access the account, even if someone else knows the password. With two-factor authentication, a user's account can be accessed on only trusted devices, such as the user's iPad, iPhone, or Mac, or on other devices after completing a verification from one of these trusted devices or a trusted phone number. To sign in for the first time on any new device, two pieces of information are required—the Apple Account password and a six-digit verification code that's displayed on the user's trusted devices or sent to a trusted phone number. By entering the code, the user confirms that they trust the new device and that it's safe to sign in. Because a password alone is no longer enough to access a user's account, two-factor authentication improves the security of the user's Apple Account and all the personal information they store with Apple. It's integrated directly into iOS, iPadOS, macOS, tvOS, visionOS, watchOS, and the authentication systems used by Apple websites.

When a user signs in to an Apple website using a web browser, a second factor request is sent to all trusted devices associated with the user's Apple Account, requesting approval of the web session. If the user is signing in to an Apple website from a browser on a trusted device, they see the verification code displayed locally on the device they're using. When the user enters the code on that device, the web session is approved.

Password reset and account recovery

If an Apple Account password is forgotten, a user can reset it on a trusted device. If a trusted device isn't available and the password is known, a user can use a trusted phone number can be used to authenticate through SMS verification. In addition, to provide immediate recovery for an Apple Account, a previously used passcode can be used to reset in conjunction with SMS. If these options aren't possible, the account recovery process needs to be followed. For more information, see the Apple Support article [How to use account recovery when you can't reset your Apple Account password](#).

Managed Apple Account security

Managed Apple Accounts function much like an Apple Account but are owned and controlled by enterprise or educational organizations. These organizations can reset passwords, turn off communications such as FaceTime and iMessage, and set up custom roles and role-based permissions for employees, staff members, teachers, and students.

For Managed Apple Accounts, some services aren't available (for example, Find My, Health, and HomeKit).

Access management for Managed Apple Accounts

Organizations can use access management available in Apple School Manager or Apple Business to define where Managed Apple Accounts can be used and what services are available to them.

With access management, organizations can define whether users can sign in with a Managed Apple Account on any device, on managed devices only, or on managed and supervised devices only. Also, users with the role of Administrator in Apple School Manager or Organization Administrator in Apple Business can configure whether users are allowed to sign in to iCloud on the web. This allows organizations to use the management state of the device as a factor to decide if access to organizational data should be granted.

Additionally, administrators can define what iCloud services are available to their users. This includes defining access to Apple Developer Programs, and the AppleSeed for IT beta program, and determining whether users are allowed to access the Apple Privacy portal at privacy.apple.com.

Managed Apple Accounts also support collaboration on documents using Keynote, Notes, Numbers, Pages, and Reminders, as well as communication using FaceTime and iMessage. For those services, organizations can define whether users can collaborate with anyone or just with accounts created within the same Apple School Manager or Apple Business organization.

If access management rules change, they are reflected on devices the user is signed in to with their Managed Apple Account. If requirements for the management state of a device are changed, a Managed Apple Account is automatically signed out of a device if the device state doesn't meet the new requirements.

Note: The Managed Apple Account password policy of users with the role of Student can be set by a user with the role of administrator in [Apple School Manager](#) if the account was created manually or connected with a Student Information System (SIS).

Inspecting Managed Apple Accounts

Managed Apple Accounts created in Apple School Manager also support *inspection*, which allows organizations to comply with legal and privacy regulations. A user with the role of Administrator, Site Manager, People Manager, or Instructor can inspect specific Managed Apple Account accounts.

Inspectors can monitor only accounts that are below them in the organization's hierarchy. For example, teachers can monitor students, managers can inspect teachers and students, and administrators can inspect managers, teachers, and students.

When inspecting credentials are requested using Apple School Manager, a special account is issued that has access to only the Managed Apple Account for which inspecting was requested. The inspector can then read and modify the user's content stored in iCloud or in CloudKit-enabled apps. Every request for auditing access is logged in Apple School Manager. The logs show who the inspector was, the Managed Apple Account the inspector requested access to, the time of the request, and whether the inspection was performed.

iCloud

iCloud security overview

iCloud stores a user's contacts, calendars, photos, documents, and more and keeps the information up to date across all of their devices automatically. iCloud can also be used by third-party apps to store and sync documents as well as key values for app data as defined by the developer. Users set up iCloud by signing in with an Apple Account and choosing which services they would like to use. Certain iCloud features, such as iCloud Drive and iCloud Backup, can be turned off by IT administrators using configuration profiles from a [device management service](#).

iCloud uses strong security methods and employs strict policies to protect user data. Most iCloud data is first encrypted on the user's device, using device-generated iCloud keys, before being uploaded to iCloud servers. For data that isn't end-to-end encrypted, the user's device securely uploads these iCloud keys to iCloud Hardware Security Modules in Apple data centers. This allows Apple to help the user with data recovery, and decrypt the data on the user's behalf whenever they need it, (for example, when they sign in on a new device, restore from a backup, or access their iCloud data on the web). Data moving between the user's devices and iCloud servers is separately encrypted in transit with TLS, and iCloud servers store user data with an additional layer of encryption at rest.

Encryption keys, when available to Apple, are secured in Apple data centers. When processing data stored in a third-party data center, these encryption keys are accessed only by Apple software running on secure servers, and only while conducting the necessary processing. For additional privacy and security, many Apple services use end-to-end encryption, which means the user's iCloud data can be accessed only by the users themselves, and only from the trusted devices where they are signed in with their Apple Account.

Apple offers users two options to encrypt and protect the data they store in iCloud:

- *Standard data protection (the default setting)*: The user's iCloud data is encrypted, the encryption keys are secured in Apple data centers, and Apple can assist with data and account recovery. Only certain iCloud data—14 data categories, including Health data and passwords in iCloud Keychain—is end-to-end encrypted.
- *Advanced Data Protection for iCloud*: An optional setting that offers Apple's highest level of cloud data security. If a user chooses to turn on Advanced Data Protection, their trusted devices retain sole access to the encryption keys for the majority of their iCloud data, thereby protecting it using end-to-end encryption. When users turn on Advanced Data Protection, the number of data categories that use end-to-end encryption rises to 23 and includes their iCloud Backup, Photos, Notes, and more.

The specific categories of iCloud data protected with end-to-end encryption are listed in the Apple Support article [iCloud data security overview](#).

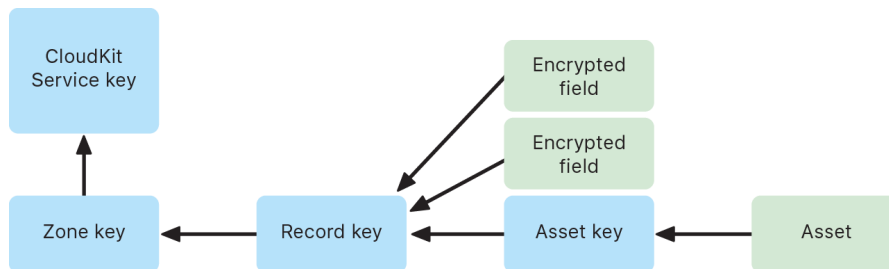
iCloud encryption

Data encryption in iCloud is closely tied to the data storage model, starting with the CloudKit frameworks and APIs that allow apps and system software to store data in iCloud on behalf of the user, and keep everything up-to-date across devices and on the web.

CloudKit encryption

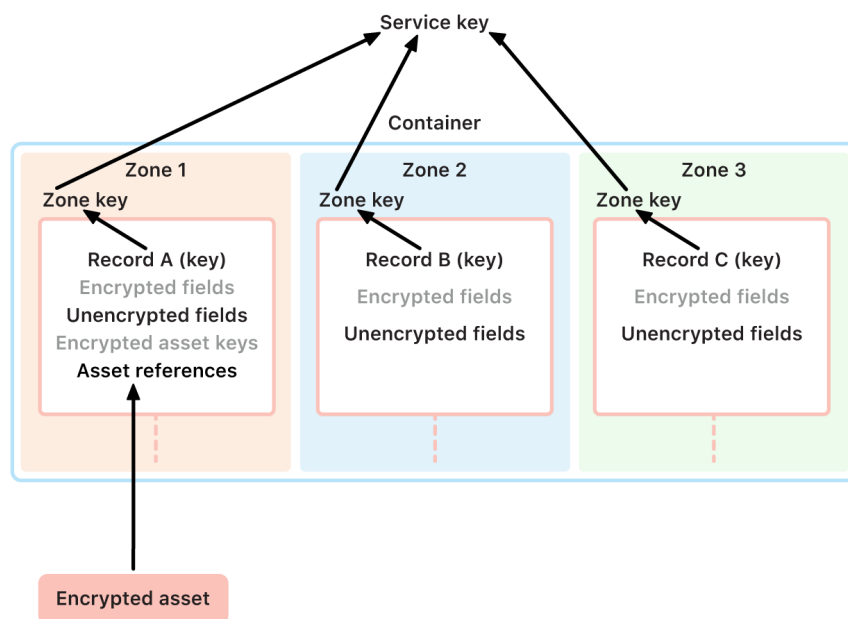
[CloudKit](#) is a framework that lets app developers store key-value data, structured data, and assets (large data stored separately from the database, such as images or videos) in iCloud. CloudKit supports both public and private databases, grouped in containers. Public databases are globally shared, typically used for generic assets, and aren't encrypted. Private databases store each user's iCloud data.

CloudKit uses a hierarchy of keys that matches the structure of the data. Each container's private database is protected by a key hierarchy, rooted in an asymmetric key called a *CloudKit Service key*. These keys are unique to each iCloud user and generated on their trusted device. When data is written to CloudKit, all record keys are generated on the user's trusted device and wrapped to the appropriate key hierarchy before any data is uploaded.



Many Apple services, listed in the Apple Support article [iCloud data security overview](#), use end-to-end encryption with a CloudKit service key that's protected in the same way as iCloud Keychain syncing. For these CloudKit containers, the service keys are available only on the user's trusted devices and can't be accessed by Apple or any third party. These keys are synchronized between a user's devices even if the user chooses not to use iCloud Keychain to sync their passwords, passkeys, and other user data. In the event of device loss, users can recover their iCloud Keychain data through the use of [Secure iCloud Keychain recovery](#), [Account Recovery Contacts](#), or an Account Recovery Key.

Encryption key management



The security of encrypted data in CloudKit relies on the security of the corresponding encryption keys. CloudKit service keys are divided into two categories: end-to-end encrypted and available-after-authentication.

- **End-to-end encrypted service keys:** For end-to-end encrypted iCloud services, the relevant CloudKit service private keys are never made available to Apple servers. Service key pairs, including the private keys, are created locally on a user's trusted device and transferred to the user's other devices using [iCloud Keychain security](#). Although iCloud Keychain recovery and synchronization flows are mediated by Apple servers, these servers are cryptographically prevented from accessing any of the user's keychain data. In the worst case of losing access to iCloud Keychain and all of its recovery mechanisms, the end-to-end encrypted data in CloudKit is lost. Apple can't help recover this data.
- **Available-after-authentication service keys:** For other services, such as Photos and iCloud Drive, the service keys are stored in iCloud Hardware Security Modules in Apple data centers, and can be accessed by some Apple services. When a user signs in to iCloud on a new device and authenticates their Apple Account, these keys can be accessed by Apple servers without further user interaction or input. For example, after signing into iCloud.com, the user can immediately view their photos online. These service keys are *available-after-authentication* keys.

Advanced Data Protection for iCloud

Advanced Data Protection for iCloud is an optional setting that offers Apple's highest level of cloud data security. When a user turns on Advanced Data Protection, their trusted devices retain sole access to the encryption keys for the majority of their iCloud data, thereby protecting it with *end-to-end encryption*. For users who turn on Advanced Data Protection, the total number of data categories protected using end-to-end encryption rises from 14 to 23 and includes iCloud Backup, Photos, Notes, and more.

Note: This feature may not be available in all countries or regions.

Conceptually, Advanced Data Protection is simple: All CloudKit Service keys that were generated on device and later uploaded to the *available-after-authentication* iCloud Hardware Security Modules (HSMs) in Apple data centers are deleted from those HSMs, and instead kept entirely within the Apple Account's iCloud Keychain protection domain. They are handled like the existing end-to-end encrypted service keys, which means Apple can no longer read or access these keys.

Advanced Data Protection also automatically protects CloudKit fields that third-party developers choose to mark as encrypted, and all CloudKit assets.

Enabling Advanced Data Protection

When the user turns on Advanced Data Protection, their trusted device performs two actions: First, it communicates the user's intent to turn on Advanced Data Protection to their other devices that participate in end-to-end-encryption. It does so by writing a new value, signed by device-local keys, into its iCloud Keychain device metadata. Apple servers can't remove or modify this attestation while it gets synchronized with the user's other devices.

Second, the device initiates the removal of the *available-after-authentication* service keys from Apple data centers. As these keys are protected by iCloud HSMs, this deletion is immediate, permanent, and irrevocable. After the keys are deleted, Apple can no longer access *any* of the data protected by the user's service keys. At this time, the device begins an asynchronous key rotation operation, which creates a new service key for each service whose key was previously available to Apple servers. If the key rotation fails, due to network interruption or any other error, the device retries the key rotation until it's successful.

After the service key rotation is successful, new data written to the service can't be decrypted with the old service key. It's protected with the new key which is controlled solely by the user's trusted devices, and was never available to Apple.

Advanced Data Protection and iCloud.com web access

When a user first turns on Advanced Data Protection, web access to their data at iCloud.com is automatically turned off. This is because iCloud web servers no longer have access to the keys required to decrypt and display the user's data. The user can choose to turn on web access again, and use the participation of their trusted device to access their encrypted iCloud data on the web.

After turning on web access, the user needs to authorize the web sign-in on one of their trusted devices each time they visit iCloud.com. The authorization arms the device for web access. For the next hour, this device accepts requests from specific Apple servers to upload individual service keys, but only those corresponding to an allow list of services normally accessible on iCloud.com. In other words, even after the user authorizes a web sign-in, a server request is unable to induce the user's device to upload service keys for data that isn't intended to be viewed on iCloud.com, (such as Health data or passwords in iCloud Keychain). Apple servers request only the service keys needed to decrypt the specific data that the user is requesting to access on the web. Every time a service key is uploaded, it is encrypted using an ephemeral key bound to the web session that the user authorized, and a notification is displayed on the user's device, showing the iCloud service whose data is temporarily being made available to Apple servers.

Preserving the user's choices

The Advanced Data Protection and iCloud.com web access settings can be modified only by the user. These values are stored in the user's iCloud Keychain device metadata and can only be changed from one of the user's trusted devices. Apple servers can't modify these settings on behalf of the user, nor can they roll them back to a previous configuration.

Security implications of sharing and collaboration

In most cases, when users share content to collaborate with each other—for example, with shared Notes, shared Reminders, shared folders in iCloud Drive, or iCloud Shared Photo Library—and all the users have Advanced Data Protection turned on, Apple servers are used only to establish sharing but don't have access to the encryption keys for the shared data. The content remains end-to-end encrypted and accessible only on participants' trusted devices. For each sharing operation, a title and representative thumbnail may be stored by Apple with standard data protection to show a preview to the receiving users.

Selecting the "anyone with a link" option when enabling collaboration will make the content available to Apple servers under standard data protection, as the servers need to be able to provide access to anyone who opens the URL.

Apple's collaborative productivity apps—Keynote, Numbers, and Pages, and the Shared Albums feature in Photos don't support Advanced Data Protection. When users collaborate on a Keynote, Numbers, or Pages document, or open one of those documents from a shared folder in iCloud Drive, the encryption keys for the document are securely uploaded to servers in Apple data centers. This is because real-time collaboration in Keynote, Numbers, and Pages requires server-side mediation to coordinate document changes between participants. Photos added to Shared Albums are stored with standard data protection, as the feature permits albums to be publicly shared on the web.

Turn off Advanced Data Protection

The user can turn off Advanced Data Protection at any time. If they decide to do so:

1. The user's device first records their new choice in iCloud Keychain participation metadata, and this setting is securely synchronized to all their devices.
2. The user's device securely uploads the service keys for all *available-after-authentication* services to the iCloud HSMs in Apple data centers. This never includes keys for services that are end-to-end encrypted under standard data protection, such as iCloud Keychain and Health.

The device uploads both the original service keys, generated before Advanced Data Protection had been turned on, and the new service keys that were generated after the user turned on the feature. This makes all data in these services accessible after authentication and returns the account to standard data protection, where Apple can once again help the user recover most of their data should they lose access to their account.

iCloud data not covered by Advanced Data Protection

Because of the need to interoperate with the global email, contacts, and calendar systems, iCloud Calendar, Contacts, and Mail aren't end-to-end encrypted.

iCloud stores some data without the protection of user-specific CloudKit service keys, even when Advanced Data Protection is turned on. CloudKit Record fields need to be explicitly declared as *encrypted* in the container's schema to be protected, and reading and writing encrypted fields requires the use of dedicated [APIs](#). Dates and times when a file or object was modified are used to sort a user's information, and checksums of file and photo data are used to help Apple de-duplicate and optimize the user's iCloud and device storage—all without having access to the files and photos themselves. Details about how encryption is used for specific data categories is available in the Apple Support article [iCloud data security overview](#).

Decisions such as the use of checksums for data de-duplication—a well-known technique called *convergent encryption*—were part of the original design of iCloud services when they launched. This metadata is always encrypted, but the encryption keys are stored by Apple with standard data protection. To continue to strengthen security protections for all users, Apple is committed to ensuring more data, including this kind of metadata, is end-to-end encrypted when Advanced Data Protection is turned on.

Advanced Data Protection requirements

The requirements to turn on Advanced Data Protection for iCloud include the following:

- The user's account needs to support end-to-end encryption. End-to-end encryption requires two-factor authentication for their Apple Account and a passcode or password set on their trusted devices. For more information, see the Apple Support article [Two-factor authentication for Apple Account](#).
- Devices where the user is signed in with their Apple Account needs to be updated to iOS 16.2, iPadOS 16.2, macOS 13.1, tvOS 16.2, watchOS 9.2, or later, and the latest version of iCloud for Windows. This requirement prevents a previous version of iOS, iPadOS, macOS, tvOS, or watchOS from mishandling the newly-created service keys by re-uploading them to the *available-after-authentication* HSMs in a misguided attempt to repair the account state.
- The user needs to set up at least one alternative recovery method—one or more recovery contacts or a recovery key—which they can use to recover their iCloud data if they lose access to their account.

If the recovery methods fail, such as if the recovery contact's information is out of date, or the user forgets them, Apple can't help recover the user's end-to-end encrypted iCloud data.

Advanced Data Protection for iCloud can be turned on only for Apple Accounts. Managed Apple Accounts and child accounts (varies by country or region) aren't supported.

iCloud Backup security

iCloud backs up information—including device settings, app data, photos, and videos in the Camera Roll, and conversations in the Messages app—daily over Wi-Fi. iCloud Backup occurs only when the device is locked, connected to a power source, and has Wi-Fi access to the internet. Mindful of the storage encryption used in iOS and iPadOS, iCloud Backup is designed to keep data secure while allowing incremental, unattended backup and restoration to occur. By default, the iCloud Backup service key is securely backed up to iCloud Hardware Security Modules in Apple data centers, and is part of the available-after-authentication data category. For users who turn on Advanced Data Protection for iCloud, the iCloud Backup service key is protected with end-to-end encryption, and available only to users on their trusted devices.

When files are created in [Data Protection](#) classes that aren't accessible when the device is locked, their [per-file keys](#) are encrypted, using the class keys from the iCloud Backup [keybag](#) and backing the files up to iCloud in their original, encrypted state. All files are encrypted during transport and, when stored, encrypted using account-based keys, as described in [CloudKit encryption](#).

The iCloud Backup keybag contains asymmetric (Curve25519) keys for Data Protection classes that aren't accessible when the device is locked. The backup set is stored in the user's Apple Account and consists of a copy of the user's files and the iCloud Backup keybag. The iCloud Backup keybag is protected by a random key, which is also stored with the backup set. The user's iCloud password isn't used for encryption, so changing the iCloud password won't invalidate existing backups.

On restore, the backed-up files, iCloud Backup keybag, and the key for the keybag are retrieved from the user's Apple Account. The iCloud Backup keybag is decrypted using its key, then the per-file keys in the keybag are used to decrypt the files in the backup set, which are written as new files to the file system, thus reencrypting them according to their Data Protection class.

The following content is backed up using iCloud Backup:

- Records for purchased music, movies, TV shows, apps, and books. A user's iCloud Backup includes information about purchased content present on the user's device, but not the purchased content itself. When the user restores from an iCloud Backup, their purchased content is automatically downloaded from the iTunes Store, the App Store, the Apple TV app, or Apple Books. Some types of content aren't downloaded automatically in all countries or regions, and previous purchases may be unavailable if they have been refunded or are no longer available in their respective store. Full purchase history is associated with a user's Apple Account.
- Photos and videos on a user's devices. Note that if a user turns on iCloud Photos in iOS 8.1, iPadOS 13.1, or OS X 10.10.3, or later, their photos and videos are already stored in iCloud, so they aren't included in the user's iCloud Backup.
- Contacts, calendar events, reminders, and notes
- Device settings
- App data
- Home Screen and app organization
- HomeKit configuration
- Medical ID data

- Voice Memos password (if necessary, requires the physical SIM card that was in use during backup)
- Message, Apple Messages for Business, text (SMS), and MMS messages (if necessary, requires the physical SIM card that was in use during backup)

iCloud Backup is also used to back up the local device keychain, encrypted with a key derived from the Secure Enclave UID root cryptographic key of the device. This key is unique to the device and not known to Apple. This allows the database to be restored only to the same device from which it originated, and it means no one else, including Apple, can read it. For more information, see [The Secure Enclave](#).

Messages in iCloud

Messages in iCloud keeps a user's entire message history updated and available on all devices.

With standard data protection, Messages in iCloud is end-to-end encrypted when iCloud Backup is turned off. When iCloud Backup is turned on, the backup includes a copy of the Messages in iCloud encryption key so Apple can help the user recover their messages even if they have lost access to iCloud Keychain and their trusted devices. If the user turns off iCloud Backup, a new key is generated on their device to protect future Messages in iCloud. The new key is stored only in iCloud Keychain, only accessible to the user on their trusted devices, and new data written to the container can't be decrypted with the old container key.

With Advanced Data Protection, Messages in iCloud is always end-to-end encrypted. When iCloud Backup is turned on, everything inside it is end-to-end encrypted, including the Messages in iCloud encryption key. The iCloud Backup service key, as well as the Messages in iCloud container key are both rotated when the user turns on Advanced Data Protection. For more information, see the Apple Support article [iCloud data security overview](#).

Search on iCloud.com security

Overview

Search on iCloud.com allows iCloud web applications—such as iCloud Drive and Photos—to search a user's synced iCloud files from a web session on iCloud.com. This search uses the local index on a user's iPhone, iPad, or Mac, enabling search from iCloud.com without needing to make an index available on iCloud servers. Search on iCloud.com uses multiple layers of cryptographic authentication, session binding, and user consent to protect access to the on-device index, and is further protected by the following policies:

- No search query will be executed on a device without the user explicitly opting in.
- No results are returned unless the device independently authorizes the requesting search on iCloud.com session.
- The connection is scoped to access only index data for files synced across iCloud. Local-only files aren't accessible.

Search on iCloud.com requires iCloud two-factor authentication, and isn't available for Managed Apple Accounts.

Mutual authentication

Before a Search on iCloud.com session can communicate with a device, a cryptographic pairing key needs to be established between the web browser used for the session and the device that grants access to the index. A device accepts a pairing key only after the user approves on the device. Apple's servers can't authorize a pairing on the user's behalf, and never learn the pairing key that results. In the browser, the pairing key is held as a non-extractable key object in secure browser storage, which helps prevent any code on the page from reading the raw key material. On the device, the pairing key is stored locally, isn't synchronized to iCloud, and is excluded from device backups.

After a pairing key is established, the web browser and the device perform a mutual cryptographic handshake, proving to each other that they hold the same key. Until this handshake succeeds, the device won't process any search requests. The pairing key is bound to the browser session it was created for and to the user's Apple Account, so it can't be used from a different browser or web session, or if either side signs in to a different Apple Account.

The web browser periodically requests a token from Apple's servers that confirms the session is signed in to the user's Apple Account and is the same session the pairing key was bound to. The web browser sends this token to the device, which verifies it with Apple's servers before answering that session's search requests for the next five minutes. After the web browser signs out, Apple's servers no longer issue tokens for it, so the device soon stops answering.

If the user has chosen to trust the browser, the pairing key can be reused across sign-ins for as long as that browser remains trusted.

Note: All pairing keys expire after 90 days.

Message transport

All messages are sent using existing IDS protections including end-to-end encryption. On the device, incoming messages are subject to additional validation against the user's Apple Account, checked for freshness to help prevent replay, and rate-limited to help prevent flooding.

Device-side security controls

The search daemon operates in a sandbox on iOS, iPadOS, and macOS with entitlement-based access control. Before processing any search request, the daemon validates the following:

- The Apple Account uses two-factor authentication.
- The user has explicitly turned on web search in their account settings.
- The device has been unlocked at least once since it was restarted.
- The device isn't in Lockdown Mode.

Search queries are executed against files synced to iCloud only, and only metadata—such as file names and types—is returned. Files are then fetched from iCloud servers. The device displays a notification when the first web search—within a 24 hour period—is performed. This provides users awareness of when their device is being used to fulfill searches. Every query is logged to a local history that users can review and export in iCloud settings. and the user can also turn off web search from their device at anytime.

Important: Apple servers never have access to the content of queries or results, as all communication between the browser and device is end-to-end encrypted.

iCloud Private Relay security

iCloud Private Relay helps protect users primarily when browsing the web with Safari, but it also includes all DNS name resolution requests. This helps ensure that no single party, not even Apple, can correlate the user's IP address and their browsing activity. It does this by using different proxies, an ingress proxy, managed by Apple and an egress proxy, managed by a content provider. To use iCloud Private Relay, the user needs to have iOS 15, iPadOS 15, macOS 12.0.1, visionOS 1, or later, and be signed in to their iCloud+ account with their Apple Account. iCloud Private Relay can then be turned on in Settings > iCloud or System Settings > iCloud.

For more information, see [iCloud Private Relay Overview](#).

Account recovery contact security

Users can add up to five people they trust as account recovery contacts to help them recover their Apple Account and data, including all of their end-to-end encrypted data, whether or not they have turned on Advanced Data Protection. Neither Apple nor the recovery contact have the necessary information individually to recover the user's end-to-end encrypted iCloud data.

Recovery Contact is designed with user privacy in mind. A user's chosen recovery contacts aren't known to Apple. Apple servers only learn information about a recovery contact late in the course of a recovery attempt after the user asks the contact for help and their contact begins actually assisting with the recovery. That information isn't retained after the recovery completes.

Recovery contact security process

When a user sets up an Account Recovery Contact, a key associated with that contact is generated. This key protects access to the user's iCloud data—including end-to-end encrypted CloudKit Data. Next, a random 256-bit AES key is generated and used to encrypt the Recovery Contact key to create a Recovery Contact Packet. The encrypted packet is sent to the Recovery Contact for safekeeping, and the random AES key is stored with Apple. Neither the AES key nor the packet provides any information about the underlying key by itself. At recovery time, after the user's device successfully obtains both the Recovery Contact Packet from their Recovery Contact and the AES key from Apple, it can combine the two to recover the original key and access the user's iCloud data.

To set up an Account Recovery Contact, the user's device communicates with Apple servers to upload the share of the keying information Apple will hold (the AES key mentioned above). It then establishes an end-to-end encrypted CloudKit container with the recovery contact to share the portion the recovery contact needs (the Recovery Contact Packet encrypted using the AES key). An authorization secret, created by Apple, is also shared with the recovery contact. This will be used to recover the account and help reset the password on the account. The communication to invite and accept recovery contacts takes place through a mutually authenticated IDS channel. The recovery contact automatically stores the received information in their iCloud Keychain. Apple can't access the contents of the CloudKit container, nor the iCloud Keychain that stores this information. When the sharing is performed, Apple servers view only an anonymous ID for the recovery contact.

Later, when a user needs to recover their account and iCloud data, they can request help from their recovery contact. At that time a recovery code is generated by the recovery contact's device, which the recovery contact then provides to the user out of band (for example in person or over a phone call). The user then enters the recovery code on their device to establish a secure connection between devices using the SPAKE2+ protocol, the contents of which isn't accessible by Apple. This interaction is orchestrated by Apple servers, but Apple can't initiate the recovery process.

After the secure connection is established and all required security checks are completed, the recovery contact's device returns their portion of the keying information and the previously established authorization secret back to the user requesting recovery. The user presents this authorization secret to an Apple server, which grants access to the keying information Apple is keeping. Providing the authorization secret also authorizes the account password reset to restore account access.

Finally, the user's device recombines the keying information received from Apple and the Account Recovery Contact, and then uses it to decrypt and recover their iCloud data.

There are safeguards in place to prevent a recovery contact from initiating recovery without the user's consent, which include a liveness check on the user's account. If the account is in active use, recovery using a Recovery Contact also requires knowledge of an iCloud Security Code.

Legacy Contact security

If a user wants their data to be accessible to designated beneficiaries after their death, they can set up Legacy Contacts on their account. A Legacy Contact is established much like a Recovery Contact, except that the keying information used by a beneficiary doesn't encompass the information necessary to decrypt the decedent's iCloud Keychain. The key structure used is the same as for [account recovery contact](#), except that in this case Apple stores the encrypted packet, and the beneficiary keeps the AES key. This allows the portion the beneficiary receives to be shorter—and thus easier to print out if necessary—while still providing the same property that neither part provides any information about the underlying key by itself.

The keying information a beneficiary receives is referred to as an access key in user-facing documentation. The access key is saved automatically on supported devices, but it can also be printed and stored offline for use. For more information, see the Apple support article [How to add a Legacy Contact for your Apple Account](#).

After the user's death, Legacy Contacts sign in to the [Apple claim website](#) to initiate access. This requires a death certificate and is authorized in part with the authorization secret mentioned in the previous section. After all the security checks are completed, Apple issues a user name and password for the new account and releases the necessary keying information to the Legacy Contact.

To more easily input the access key when needed, it's presented as an alphanumeric code with an associated QR code. After it's entered, access to the decedent's iCloud data is restored. This can be performed on a device, or access can be established online. For more information, see the Apple Support article [Request access to an Apple account as a Legacy Contact](#).

Passcode and password management

Password security overview

iOS, iPadOS, macOS, and visionOS make it easy for users to authenticate to third-party apps and websites that use passwords. The best way to manage passwords is not to have to use one. Sign in with Apple lets users sign in to third-party apps and websites without having to create and manage an additional account or password while protecting the sign-in with their two-factor authentication for Apple Account. For sites that don't support Sign in with Apple, the Automatic Strong Password feature enable a user's devices to automatically create, sync, and enter unique strong passwords for sites and apps. On iPad, iPhone, and Apple Vision Pro, passwords are saved to a special Password AutoFill keychain that's user controlled and manageable in Settings > Passwords.

On a Mac, saved passwords can be managed in Safari Passwords preferences. This sync system can also be used to sync passwords that are manually created by the user.

Sign in with Apple security

Sign in with Apple is a privacy-friendly alternative to other Single Sign-on systems. It provides the convenience and efficiency of one-tap sign-in while giving the user more transparency and control over their personal information.

Sign in with Apple allows users to set up an account and sign in to apps and websites using the Apple Account they already have, and it gives them more control over their personal information. Apps can only ask for the user's name and email address when setting up an account, and the user always has a choice: They can share their personal email address with an app or choose to keep their personal email private and use the new Apple private email relay service instead. This email relay service shares a unique, anonymized email address that forwards to the user's personal address so they can still receive useful communication from the developer while maintaining a degree of privacy and control over their personal information.

Sign in with Apple is built for security. Every Sign in with Apple user is required to have two-factor authentication enabled for their Apple Account. Two-factor authentication helps secure not only the user's Apple Account but also the accounts they establish with their apps. Furthermore, Apple has developed and integrated a privacy-friendly antifraud signal into Sign in with Apple. This signal gives developers confidence that the new users they acquire are real people and not bots or scripted accounts.

Automatic strong passwords

When iCloud Keychain is turned on, iOS, iPadOS, macOS, and visionOS create strong, random, unique passwords when users sign up for or change their password on a website in Safari. Automatic strong password generation is also available in apps. Users need to opt out of using strong passwords. Generated passwords are saved in the keychain and kept up to date across devices with iCloud Keychain, when it's enabled.

By default, passwords generated are 20 characters long. They contain one digit, one uppercase character, two hyphens, and 16 lowercase characters. These generated passwords are strong, containing 71 bits of entropy.

Passwords are generated based on heuristics that determine whether a password-field experience is for password creation. If the heuristic fails to recognize a context-specific password being used at password creation, app developers can set `UITextContentType.newPassword` on their text field and web developers can set `autocomplete="new-password"` on their `<input>` elements.

To help ensure that generated passwords are compatible with the relevant services, apps and websites can provide rules. Developers provide these rules using `UITextInputPasswordRules` or the `passwordrules` attribute on their input elements. Devices then generate the strongest password they can that fulfills these rules.

Password AutoFill security

Password AutoFill automatically fills credentials stored in the keychain. The iCloud Keychain password manager and Password AutoFill provide the following features:

- Filling in credentials in apps and websites
- Generating strong passwords
- Saving passwords in both apps and websites in Safari
- Sharing passwords securely to a users' contacts
- Providing passwords to a nearby Apple TV that's requesting credentials

Generating and saving passwords within apps, as well as providing passwords to Apple TV, are available only in iOS, iPadOS, and visionOS.

Password AutoFill in apps

iOS, iPadOS, and visionOS allow users to input saved user names and passwords into credential-related fields in apps, similar to the way Password AutoFill works in Safari. On iPad and iPhone, users tap a key affordance in the software keyboard's QuickType bar. On Mac, for apps built with Mac Catalyst, a Passwords drop-down menu appears below credential-related fields.

When an app is strongly associated with a website that uses the same app-website association mechanism and that's powered by the same apple-app-site-association file, the iOS and iPadOS QuickType bar and macOS drop-down menu directly suggest credentials for the app, if any are saved to the Password AutoFill Keychain. This allows users to choose to disclose Safari-saved credentials to apps with the same security properties, without those apps having to adopt an API.

Password AutoFill exposes no credential information to an app until a user consents to release a credential to the app. The credential lists are drawn from or presented out of the app's process.

When an app and website have a trusted relationship and a user submits credentials within an app, iOS, iPadOS, and visionOS may prompt the user to save those credentials to the Password AutoFill keychain for later use.

App access to saved passwords

iOS, iPadOS, macOS, and visionOS apps can request the Password AutoFill keychain's help with signing a user in using `ASAuthorizationPasswordProvider` and `SecAddSharedWebCredential`. The password provider and its request can be used in conjunction with Sign in with Apple, so that the same API is called to help users sign into an app, regardless of whether the user's account is password based or was created using Sign in with Apple.

Apps can access saved passwords only if the app developer and website administrator have given their approval and the user has given consent. App developers express their intent to access Safari saved passwords by including an entitlement in their app. The entitlement lists the fully qualified domain names of associated websites, and the websites need to place a file on their server listing the unique app identifiers of apps approved by Apple.

When an app with the `com.apple.developer.associated-domains` entitlement is installed, iOS, iPadOS, and visionOS make a TLS request to each listed website, requesting one of the following files:

- `apple-app-site-association`
- `.well-known/apple-app-site-association`

If the file lists the app identifier of the app being installed, then iOS, iPadOS, or visionOS mark the website and app as having a trusted relationship. Only with a trusted relationship will calls to these two APIs result in a prompt to the user, who needs to agree before any passwords are released to the app, updated, or deleted.

Password security recommendations

The Password AutoFill passwords list in iOS, iPadOS, macOS, and visionOS indicates which of a user's saved passwords will be *reused* with other websites, passwords that are considered *weak*, and passwords that have been compromised by a *data leak*.

Overview

Using the same password for more than one service may leave those accounts vulnerable to a credential-stuffing attack. If a service is breached and passwords are leaked, attackers may try the same credentials on other services to compromise additional accounts.

- Passwords are marked *reused* if the same password is seen used for more than one saved password across different domains.

- Passwords are marked *weak* if they may be easily guessed by an attacker. iOS, iPadOS, macOS, and visionOS detect common patterns used to create memorable passwords, such as using words found in a dictionary, common character substitutions (such as using p4ssw0rd instead of password), patterns found on a keyboard (such as q12we34r from a QWERTY keyboard), or repeated sequences (such as 123123). These patterns are often used to create passwords that satisfy minimum password requirements for services, but are also commonly used by attackers attempting to obtain a password using brute force.

Because many services specifically require a four- or six-digit PIN code, these short passcodes are evaluated with different rules. PIN codes are considered weak if they are one of the most common PIN codes, if they are an increasing or decreasing sequence such as 1234 or 8765, or if they follow a repetition pattern, such as 123123 or 123321.

- Passwords are marked *leaked* if the Password Monitoring feature can claim they have been present in a data leak. For more information, see [Password Monitoring](#).

Weak, reused, and leaked passwords are either indicated in the list of passwords (macOS) or present in the dedicated Security Recommendations interface (iOS, iPadOS, and visionOS). If the user logs in to a website in Safari using a previously saved password that's very weak or that's been compromised by a data leak, they're shown an alert strongly encouraging them to upgrade to an automatic strong password.

Upgrading account authentication security in iOS, iPadOS, and visionOS

Apps that implement an Account Authentication Modification Extension (in the Authentication Services framework) can provide easy, tap-of-a-button upgrades for password-based accounts—namely, they can switch to using Sign in with Apple or an automatic strong password. This extension point is available in iOS, iPadOS, and visionOS.

If an app has implemented the extension point and is installed on device, users see extension upgrade options when viewing Security Recommendations for credentials associated with the app in the iCloud Keychain password manager in Settings. The upgrades are also offered when users sign in to the app with the at-risk credential. Apps have the ability to tell the system not to prompt users with upgrade options after signing in. Using new AuthenticationServices API, apps can also invoke their extensions and perform upgrades themselves, ideally from an account settings or account management screen in the app.

Apps can choose to support strong password upgrades, Sign in with Apple upgrades, or both. In a strong password upgrade, the system generates an automatic strong password for the user. If necessary, the app can provide custom password rules to follow when generating the new password. When a user switches an account from using a password to using Sign in with Apple, the system provides a new Sign in with Apple credential to the extension to associate the account with. The user's Apple Account email isn't provided as part of the credential. After a successful Sign in with Apple upgrade, the system deletes the previously used password credential from the user's keychain if it's saved there.

Account Authentication Modification Extensions have the opportunity to perform additional user authentication before performing an upgrade. For upgrades initiated within the password manager or after signing in to an app, the extension provides the user name and password for the account to upgrade. For in-app upgrades, only the user name is provided. If the extension requires further user authentication, it can request to show a custom user interface before moving on with the upgrade. The intended use case for showing this user interface is to have the user enter a second factor of authentication to authorize the upgrade.

Password Monitoring

Password Monitoring is a feature that matches passwords stored in the user's Password AutoFill keychain against a continuously updated and curated list of passwords known to have been exposed in leaks from different online organizations. If the feature is turned on, the monitoring protocol continuously matches the user's Password AutoFill keychain passwords against the curated list.

How monitoring works

The user's device continuously performs round-robin checks on a user's passwords, querying an interval that's independent of the user's passwords or their password manager usage patterns. This helps ensure that verification states remain up to date with the current curated list of leaked passwords. To help prevent leakage of information related to how many unique passwords a user has, requests are batched and performed in parallel. A fixed number of passwords are verified in parallel on each check, and if a user has fewer than this number, random passwords are generated and added to the queries to make up the difference.

How passwords are matched

Passwords are matched in a two-part process. The most commonly leaked passwords are contained within a local list on the user's device. If the user's password occurs on this list, the user is immediately notified without any external interaction. This is designed to ensure that no information is leaked about the passwords a user has that are most at risk due to a password breach.

If the password isn't contained on the most frequent list, it's matched against less frequently leaked passwords.

Comparing users' passwords against a curated list

To verify whether a password not present in the local list is a match involves some interaction with Apple servers. To help ensure that legitimate users' passwords aren't sent to Apple, a form of cryptographic *private set intersection* is deployed that compares the users' passwords against a large set of leaked passwords. This is designed to ensure that for passwords less at risk of breach, little information is shared with Apple. For a user's password, this information is limited to a 15-bit prefix of a cryptographic hash. The removal of the most frequently leaked passwords from this interactive process, using the local list of most commonly leaked passwords, reduces the delta in relative frequency of passwords in the web services buckets, making it impractical to infer user passwords from these lookups.

The underlying protocol partitions the list of curated passwords, which contained approximately 1.5 billion passwords at the time of this writing, into 2^{15} different buckets. The bucket a password belongs to is based on the first 15 bits of the SHA-256 hash value of the password. Additionally, each leaked password, pw , is associated with an elliptic curve point on the NIST P256 curve: $P_{pw} = \alpha \cdot H_{SWU}(pw)$, where α is a secret random key known only to Apple and H_{SWU} is a random oracle function that maps passwords to curve points based on the Shallue-van de Woestijne-Ulas method. This transformation is designed to computationally hide the values of passwords and helps prevent revealing newly leaked passwords through Password Monitoring.

To compute the private set intersection, the user's device determines the bucket the user's password belongs to using λ , the 15-bit prefix of $SHA-256(upw)$, where upw is one of the user's passwords. The device generates their own random constant, β , and sends the point $P_c = \beta \cdot H_{SWU}(upw)$ to the server, along with a request for the bucket corresponding to λ . Here β hides information about the user's password and limits to λ the information exposed from the password to Apple. Finally, the server takes the point sent by the user's device, computes $\alpha P_c = \alpha \beta \cdot H_{SWU}(upw)$, and returns it, along with the appropriate bucket of points— $B_\lambda = \{ P_{pw} \mid SHA-256(pw) \text{ begins with prefix } \lambda \}$ —to the device.

The returned information allows the device to compute $B'_\lambda = \{ \beta \cdot P_{pw} \mid P_{pw} \in B_\lambda \}$. If $\alpha P_c \in B'_\lambda$, the user's password has been leaked.

Sending passwords to other users or Apple devices

Apple sends passwords securely to other users or Apple devices with AirDrop and on Apple TV.

Saving credentials to another device with AirDrop

When iCloud is enabled, users can use AirDrop to send a saved credential to another device. The credential includes the user's name and password and the websites it's saved for. Sending credentials with AirDrop always operates in Contacts Only mode, regardless of the user's settings. On the receiving device, after user consent, the credentials are stored in the user's Password AutoFill Keychain.

Filling in credentials in apps on Apple TV

Password AutoFill is available to fill credentials in apps on Apple TV. When the user focuses on a user name or password text field in tvOS, Apple TV begins advertising a request for Password AutoFill over Bluetooth Low Energy (BLE).

Any nearby iPad or iPhone displays a prompt inviting the user to share a credential with Apple TV. Here's how the encryption method is established:

- If the device and Apple TV uses the same Apple Account, encryption between the devices happens automatically.
- If the device is signed in to an Apple Account other than the one used by Apple TV, the user is prompted to establish an encrypted connection through use of a PIN code. To receive this prompt, the iPhone needs to be unlocked and in close proximity to the Siri Remote paired to that Apple TV.

After the encrypted connection is made using BLE link encryption, the credential is sent to Apple TV and is automatically filled in to the relevant text fields on the app.

Credential provider extensions

On devices with iOS, iPadOS, and macOS, users can designate a participating third-party app as a credential provider for Password AutoFill in Passwords settings (iOS and iPadOS) or in Extensions settings in System Settings (macOS 13 or later) or System Preferences (macOS 12 or earlier). This mechanism is built on app extensions. The credential provider extension *needs to provide* a view for choosing credentials. The extension *can optionally provide* metadata about saved credentials so they can be offered directly on the QuickType bar (iOS and iPadOS) or in an autocomplete suggestion (macOS). The metadata includes the website of the credential and the associated user name, but not its password. iOS, iPadOS, and macOS communicate with the extension to get the password when the user chooses to fill a credential into an app or a website in Safari. Credential metadata is stored inside the credential provider app's container and is automatically removed when an app is uninstalled.

iCloud Keychain

iCloud Keychain security overview

iCloud Keychain allows users to securely sync their passwords and passkeys between iPad, iPhone, Mac, Apple Watch, and Apple Vision Pro without exposing them to Apple. In addition to strong privacy and security, other goals for the design and architecture of iCloud Keychain were ease of use, and the ability to recover keychain contents even if all of a user's devices are inaccessible. iCloud Keychain consists of two services: keychain syncing and keychain recovery.

iCloud Keychain and keychain recovery are designed such that a user's passwords and passkeys are still protected under the following conditions:

- A user's Apple Account is compromised.
- iCloud is compromised by an external attacker or employee.
- A third party accesses user accounts.

Password manager integration with iCloud Keychain

iOS, iPadOS, macOS, and visionOS can automatically generate cryptographically strong random strings to use as account passwords in Safari. iOS, iPadOS, and visionOS can also generate strong passwords for apps. Generated passwords are stored in the [keychain](#) and synced to other devices. Keychain items are transferred from device to device, traveling through Apple servers, but are encrypted end-to-end so that Apple and other devices can't read their contents.

Secure keychain syncing

When a user turns on iCloud Keychain for the first time on a two-factor authentication account, the device establishes and creates a syncing identity for itself. The syncing identity consists of asymmetric elliptical keys (using P-384), which are stored in the device's keychain. Each device maintains its own list of syncing identities of the user's other devices, and signs this list using one of its identity keys. These lists are stored in CloudKit, allowing the user's devices to reach consensus on how to securely sync keychain data between themselves.

For compatibility with older iCloud devices, a similar syncing circle of trust is created and another syncing identity is formed. The public key of the syncing identity is put in the circle, and the circle is signed twice: first by the private key of the syncing identity, and then again with an asymmetric elliptical key (using P-256) derived from the user's Apple Account password. Also stored with the circle are the parameters (random salt and iterations) used to create the key that's based on the user's Apple Account password.

iCloud storage of the syncing circle

For two-factor authentication accounts, each device's list of trusted devices is stored in CloudKit. The lists can't be read without knowing the user's iCloud password, and they can't be modified without having the private keys of the owning device.

Similarly, the signed syncing circle is stored in the user's iCloud key-value storage area, can't be read without knowing the user's iCloud password, and is unable to be validly modified without having the private key of the syncing identity of its member.

How a user's other devices are added to the syncing circle

New devices, as they sign into iCloud, join the iCloud Keychain syncing circle in one of two ways: either by pairing with and being sponsored by an existing iCloud Keychain device, or by using iCloud Keychain recovery.

During the pairing flows, the applicant device creates new syncing identities for both the syncing circle and the syncing lists (for two-factor authentication accounts) and presents them to the sponsor. The sponsor adds the public key of the new member to the syncing circle and signs it again with both its syncing identity and the key derived from the user's iCloud password. The new syncing circle is placed in iCloud, where it's similarly signed by the new member of the circle. In two-factor authentication accounts, the sponsor device also provides the joining device with a *voucher* signed by its identity keys, showing that the applicant device should be trusted. It then updates its individual list of trusted syncing identities to include the applicant.

There are now two members of the signing circle, and each member has the public key of its peer. They now begin to exchange individual keychain items through CloudKit, or iCloud key-value storage, whichever is most appropriate for the situation. If both circle members have updates to the same item, one or the other is chosen, resulting in eventual consistency. Each item that's synced is encrypted so that it can be decrypted only by a device within the user's circle of trust; it can't be decrypted by any other devices or by Apple.

As new devices join the syncing circle, this "join process" is repeated. For example, when a third device joins, it can be paired with either of the existing devices. As new peers are added, each peer syncs with the new one. This is designed to ensure that all members have the same keychain items.

Only certain items are synced

Some keychain items are device specific, such as iMessage keys, and so they need to stay on the device. To prevent unexpected data transport, every item that will sync needs to be explicitly marked with the `kSecAttrSynchronizable` attribute.

Apple sets this attribute for Safari user data (including user names, passwords, and credit card numbers) as well as for Wi-Fi passwords, HomeKit encryption keys, and other keychain items supporting end-to-end iCloud encryption.

Additionally, by default, keychain items added by third-party apps don't sync. Developers need to set the `kSecAttrSynchronizable` attribute when adding items to the keychain.

Secure iCloud Keychain recovery

iCloud Keychain escrows users' keychain data with Apple without allowing Apple to read the passwords and other data it contains. Even if the user has only a single device, keychain recovery provides a safety net against data loss. This is particularly important when Safari is used to generate random, strong passwords or passkeys for web accounts, because the only record of those passwords is in the keychain.

A cornerstone of keychain recovery is secondary authentication and a secure escrow service, created by Apple specifically to support this feature. The user's keychain is encrypted using a strong passcode, and the escrow service provides a copy of the keychain only if a strict set of conditions are met.

Use of secondary authentication

There are several ways to establish a strong passcode:

- If two-factor authentication is turned on for the user's account, the device passcode is used to recover an escrowed keychain.
- If two-factor authentication isn't set up, the user is asked to create an iCloud security code by providing a six-digit passcode. Alternatively, without two-factor authentication, users can specify their own, longer code, or they can let their devices create a cryptographically random code that they can record and keep on their own.

Keychain escrow process

After the passcode is established, the keychain is escrowed with Apple. The device first exports a copy of the user's keychain and then encrypts it wrapped with keys in an asymmetric [keybag](#) and places it in the user's iCloud key-value storage area. The keybag is wrapped with the user's iCloud security code and with the public key of the [hardware security module \(HSM\)](#) cluster that stores the escrow record. This becomes the user's *iCloud escrow record*. For two-factor authentication accounts, the keychain is also stored in CloudKit and wrapped to intermediate keys that are recoverable only with the contents of the iCloud escrow record, thereby providing the same level of protection.

The contents of the escrow record also allow the recovering device to rejoin iCloud Keychain, proving to any existing devices that the recovering device successfully performed the escrow process and thus is authorized by the account's owner.

Note: Besides establishing a security code, users need to register a phone number for their Apple Account. This provides a secondary level of authentication during keychain recovery. The user receives an SMS message that need to be replied to for the recovery to proceed.

Escrow security for iCloud Keychain

iCloud provides a secure infrastructure for [keychain](#) escrow to help ensure that only authorized users and devices can perform a recovery. Topographically positioned behind iCloud are clusters of [hardware security modules \(HSMs\)](#) that guard the escrow records. As described previously, each has a key that's used to encrypt the escrow records under their watch.

To recover a keychain, users need to authenticate with their Apple Account and password and respond to an SMS sent to their registered phone number. After this is done, users need to enter their iCloud security code. The HSM cluster verifies that a user knows their iCloud security code using the Secure Remote Password (SRP) protocol; the code itself isn't sent to Apple. Each member of the cluster independently verifies that the user hasn't exceeded the maximum number of attempts allowed to retrieve their record, as discussed below. If a majority agree, the cluster unwraps the escrow record and sends it to the user's device.

The device uses the escrowed data to unwrap the random keys used to encrypt the user's keychain. With that key, the keychain—retrieved from CloudKit and iCloud key-value storage—is decrypted and restored onto the device. The escrow service allows only 10 attempts to authenticate and retrieve an escrow record. After several failed attempts, the record is locked and the user needs to call Apple Support to be granted more attempts. After the 10th failed attempt, the HSM cluster destroys the escrow record and the keychain is lost forever. This provides protection against a brute-force attempt to retrieve the record, at the expense of sacrificing the keychain data in response.

These policies are coded in the HSM firmware. The administrative access cards that permit the firmware to be changed have been destroyed. Any attempt to alter the firmware or access the private key causes the HSM cluster to delete the private key. Should this occur, the owner of each keychain protected by the cluster receives a message informing them that their escrow record has been lost. They can then choose to reenroll.

Apple Pay

Apple Pay security overview

With Apple Pay, users can use supported iPad, iPhone, Mac, Apple Watch, and Apple Vision Pro devices to pay in an easy, secure, and private way in stores, apps, and on the web in Safari. Users can also add Apple Pay-enabled transit, student ID, and access cards to Apple Wallet. It's simple for users, and it's built with integrated security in both hardware and software.

Apple Pay is also designed to protect the user's personal information. Apple Pay doesn't collect any transaction information that can be tied back to the user. Payment transactions are between the user, the merchant, and the card issuer.

Apple Pay component security

Apple Pay uses several hardware and software features to provide secure, reliable purchases.

Secure Element

The Secure Element is an industry-standard, certified chip running the Java Card platform, which is compliant with financial industry requirements for electronic payments. The Secure Element IC and the Java Card platform are certified in accordance with the EMVCo Security Evaluation process. After the successful completion of the security evaluation, EMVCo issues unique IC and platform certificates.

The Secure Element IC has been certified based on the Common Criteria standard.

NFC controller

The NFC controller handles near field communication protocols and routes communication between the Application Processor and the Secure Element, and between the Secure Element and the point-of-sale terminal.

Apple Wallet

The Apple Wallet app is used to add and manage credit, debit, and store cards and to make payments with Apple Pay. Users can view their cards and may be able to view additional information provided by their card issuer, such as their card issuer's privacy policy, recent transactions, and more in Apple Wallet. Users can also add cards to Apple Pay in:

- Setup Assistant and Settings in iOS, iPadOS, macOS, and visionOS
- The Watch app for Apple Watch
- Wallet & Apple Pay in System Settings (macOS 13 or later) or System Preferences (macOS 12 or earlier) for Mac computers with Touch ID

In addition, Apple Wallet allows users to add and manage transit cards, rewards cards, boarding passes, tickets, gift cards, student ID cards, access cards, and more.

Secure Enclave

The Secure Enclave manages the authentication process and allows authorized transactions to proceed. Users can authorize using the following methods:

- Biometric authentication
- On Apple Watch, the device needs to be unlocked, and the user needs to double-click the side button or enter the passcode.
- Password or passcode

The intent is detected and passed directly to the Secure Element or Secure Enclave, where available, without going through the Application Processor.

Apple Pay servers

The Apple Pay servers manage the setup and provisioning of credit, debit, transit, student ID, and access cards in Apple Wallet. The servers also manage the Device Account Numbers stored in the Secure Element. They communicate both with the device and with the payment network or card issuer servers. The Apple Pay servers are also responsible for reencrypting payment credentials for payments within apps or on the web.

How Apple Pay keeps users' purchases protected

Secure Element

The Secure Element hosts a specially designed applet to manage Apple Pay. It also includes applets certified by payment networks or card issuers. Credit, debit, or prepaid card data is sent from the payment network or card issuer encrypted to these applets using keys that are known only to the payment network or card issuer and the applets' security domain. This data is stored within these applets and protected using the Secure Element's security features. During a transaction, the terminal communicates directly with the Secure Element through the near-field-communication (NFC) controller over a dedicated hardware bus.

NFC controller

As the gateway to the Secure Element, the NFC controller helps ensure that all contactless payment transactions are conducted using a point-of-sale terminal that's in close proximity to the device. Only payment requests arriving from an in-field terminal are marked by the NFC controller as contactless transactions.

Payment with a credit, debit, or prepaid card (including store cards) is authorized by the cardholder after using one of the following methods:

- Biometric authentication
- Device passcode or password
- Double-clicking the side button of an unlocked Apple Watch

The payment applets in the Secure Element prepare contactless responses. The controller then routes them to the NFC field. In this way, the payment details stay within the NFC field and never reach the Application Processor. In contrast, payment details for in-app and web payments first go to the Application Processor. However, they are encrypted by the Secure Element before reaching the Apple Pay server.

Credit, debit, and prepaid cards

Card provisioning security overview

When a user adds a credit, debit, or prepaid card (including store cards) to Apple Wallet, Apple securely sends the card information, along with other information about user's account and device, to the card issuer or card issuer's authorized service provider (usually the payment network). Using this information, the card issuer (or its service provider) determines whether to approve the user's request to add the card to Apple Wallet. As part of the card provisioning process, Apple Pay uses three server-side calls to send and receive communication with the card issuer or payment network:

- Required Fields
- Check Card
- Link and Provision

The card issuer or payment network uses these calls to enable the card issuer to verify, approve, and add cards to Apple Wallet. These client-server sessions use TLS 1.2 or later to transfer the data.

Full card numbers aren't stored on the device or on Apple Pay servers. Instead, a unique Device Account Number is created by the card issuer, sent encrypted to Apple, and then stored in the Secure Element. This unique Device Account Number is encrypted in such a way that Apple can't access it. The Device Account Number is unique and different from most credit or debit card numbers, in that the card issuer or payment network can prevent its use on a magnetic stripe card, over the phone, or on websites. The Device Account Number in the Secure Element is never stored on Apple Pay servers or backed up to iCloud, and it's isolated from:

- Devices that use biometric authentication
- Apple Watch
- Mac computers with Apple silicon that use the Magic Keyboard with Touch ID

Users can add cards to Apple Watch for Apple Pay using either the Watch app on their iPhone or the card issuer's app. To add a card to Apple Watch:

- *When paired with an iPhone:* The watch needs to be within Bluetooth communications range
- *When set up without an iPhone:* The watch needs to have internet access using Wi-Fi

Cards are specifically enrolled for use with Apple Watch and have their own Device Account Numbers, which are stored within the Secure Element on the Apple Watch.

When credit, debit, or prepaid cards (including store cards) are added, they appear in a list of cards during Setup Assistant on devices that are signed in to the same Apple Account. These cards remain in this list for as long as they are active on at least one device. Cards are removed from this list after they have been removed from all devices for seven days. This feature requires two-factor authentication to be enabled on the respective Apple Account.

When a user provisions an eligible payment card, they can also provision the card to other Apple Pay-capable devices using the same Apple Account. No information is copied from the original device. The other devices provision the card using the same flow they would use during device setup.

Adding credit or debit cards to Apple Pay

Credit cards can be manually added to Apple Pay in Apple devices.

Adding credit or debit cards manually

To add a card manually, users need to enter the name, card number, expiration date, and CVV. They can do this through Setup Assistant, Settings, Apple Wallet, or the Watch app. The device's camera can also capture this information. Apple next tries to fill in the name, card number, and expiration date. The photo is never saved to the device or stored in the photo library. After all the fields are filled in, the Check Card process verifies the fields other than the CVV. They are then encrypted and sent to the Apple Pay server for routing to the card issuer or payment network.

If a terms and conditions ID is returned by the card issuer or payment network with the Check Card process, Apple downloads and displays the terms and conditions of the card issuer to the user. If the user accepts the issuer's terms and conditions, Apple sends the ID of the terms that were accepted as well as the CVV to the Link and Provision process. Additionally, as part of the Link and Provision process, Apple shares information from the device with the card issuer or network. This includes information about:

- The user's iTunes and App Store account activity (for example, whether the user has a long history of transactions within iTunes)
- The user's device (for example, the phone number, name, and model of the user's device plus any companion Apple device necessary to set up Apple Pay)
- The user's approximate location at the time the user adds their card (if the user has Location Services enabled).

Using this information, the card issuer determines whether to approve adding the card to Apple Pay

As the result of the Link and Provision process, two things occur:

- The device begins to download the Apple Wallet pass file representing the credit or debit card.
- The device begins to bind the card to the Secure Element.

The pass file contains URLs to download card art, metadata about the card such as contact information, the related issuer's app, and supported features. It also contains the pass state, which includes information such as whether the personalizing of the Secure Element has completed, whether the card is currently suspended by the card issuer, or whether additional verification is required before the card can make payments with Apple Pay.

Adding credit or debit cards from an iTunes Store account

For a credit or debit card on file with iTunes, the user may be required to reenter their Apple Account password. The card number is retrieved from iTunes, and the Check Card process is initiated. If the card is eligible for Apple Pay, the device downloads and displays terms and conditions of the card issuer, then sends along the term's ID and the card security code to the Link and Provision process. Additional verification may occur for iTunes account cards on file.

Adding credit or debit cards from a card issuer's app

When an app is registered for use with Apple Pay, keys are established for the app and for the card issuer's server. These keys are used to encrypt the card information that's sent to the card issuer. This is designed to prevent the information from being read by the Apple device. The provisioning flow is similar to that used for manually added cards, described previously, except one-time passwords are used in lieu of the CVV.

Adding credit or debit cards from a card issuer's website

Some card issuers provide the ability to initiate the card provisioning process for Apple Wallet directly from their websites. In this case, the user initiates the task by selecting a card to provision on the card issuer's website. The user is then redirected to a self-contained Apple sign-in experience (contained within Apple's domain) and is asked to sign in with their Apple Account. Upon successfully signing in, the user then chooses one or more devices to provision the card to and is required to confirm the provisioning result on each respective target device.

Adding credit or debit cards with Tap to Add Card

On an iPhone with iOS 18 or later with the Tap to Add Card feature, users can add eligible credit or debit cards to Apple Wallet by tapping their card to the back of their iPhone before completing CVV authentication with their card issuers.

On an iPhone with iOS 26 or later, users can complete the authentication with card issuers using their PIN (if supported by the card issuer).

Note: The Tap to Add Card feature isn't available in all countries or regions.

Adding additional verification

A card issuer can decide whether a credit or debit card requires additional verification. Depending on what's offered by the card issuer, the user may be able to choose between different options to complete the verification—for example, with a text message, email, a customer service call, or a method in the app of an approved card issuer. In the case of text messages or email, the user is presented an option to select from contact information the issuer already holds on file. A code is sent, which needs to be entered into Apple Wallet, Settings, or the Apple Watch app. In the case of customer service or verification using an app, the issuer performs their own communication process.

Payment authorization with Apple Pay

Overview

A payment can be made only after the Secure Element receives authorization from the Secure Enclave. This involves verifying that the user has confirmed their intent to pay and that the user has authenticated themselves using one of the following methods:

- Biometric authentication
- Device passcode or password
- Double-clicking the side button of an unlocked Apple Watch

Biometric authentication, if available, is the default method, but the passcode or password can be used at any time and is automatically offered after three unsuccessful attempts to match a fingerprint, or two unsuccessful attempts to match a face. After five unsuccessful attempts, the passcode or password is required.

A passcode or password is also required when biometric authentication isn't configured or turned on for Apple Pay.

Using a shared pairing key

The Secure Enclave and Secure Element communicate through a serial interface. The Secure Element links to the NFC controller, which connects to the Application Processor. Although the Secure Element and Secure Enclave don't connect directly, they can, however, communicate securely using a secret created during runtime. In the factory, both are set up with each other's long-term public keys. The Secure Enclave's public key comes from its UID key and the Secure Element's identifier. The corresponding private key is stored in hardware, hidden from software. During runtime, the long-term public keys create a shared secret using the Elliptic Curve Diffie-Hellman (ECDH) key agreement protocol. This shared secret ensures secure communication.

Authorizing a secure transaction

When the user authorizes a transaction, which includes a physical gesture communicated directly to the Secure Enclave, the Secure Enclave then sends signed data about the type of authentication and details about the type of transaction (contactless or e-commerce) to the Secure Element, tied to an Authorization Random (AR) value. The AR value is generated in the Secure Enclave when a user first provisions a credit card and persists while Apple Pay is enabled, protected by the Secure Enclave encryption and anti-rollback mechanism. It's securely delivered to the Secure Element by leveraging the pairing key. On receipt of a new AR value, the Secure Element marks any previously added cards as terminated.

Using a payment cryptogram for dynamic security

Payment transactions originating from the payment applets include a payment cryptogram along with a Device Account Number. This cryptogram, a one-time code, is computed using a transaction counter and a key. The transaction counter is incremented for each new transaction. The key is provisioned in the payment applet during personalization and is known by the payment network or the card issuer or both. Depending on the payment scheme, other data may also be used in the calculation, including:

- A Terminal Unpredictable Number, for near-field-communication (NFC) transactions
- An Apple Pay server anti-replay value, for transactions within apps and at websites
- User verification results, such as Cardholder Verification Method (CVM) information

These security codes are provided to the payment network and to the card issuer, which allows the issuer to verify each transaction. The length of these security codes may vary based on the type of transaction.

Paying with cards using Apple Pay

Apple Pay can be used to pay for purchases in stores, within apps, and at websites.

Paying with cards in stores

If iPhone or Apple Watch is on and detects an NFC field and if Apple Wallet is set as the default Payments and Contactless app, the device presents the user with the requested card (if automatic selection is turned on for that card) or the default card from Apple Wallet (which is managed in Settings). The user can also go to Apple Wallet and choose a card or, when the device is locked, can:

- Double-click the side button on devices with Face ID (if Apple Wallet is the default app)
- Double-click the Home button on devices with Touch ID (if Apple Wallet is the default app)
- Using Accessibility features that allow Apple Pay from the Lock Screen

Next, before information is transmitted, the user needs to confirm their intent to pay and authenticate themselves using one of the following methods:

- Biometric authentication
- Device passcode or password
- Double-clicking the side button of an unlocked Apple Watch

No payment information is sent without user authentication.

After the user's identity is verified, the Device Account Number and a unique security code are used to process the payment. Neither Apple nor the user's device shares full card numbers with merchants. However, Apple may get anonymous data, like the transaction's time and place. This information helps improve Apple Pay and other Apple services.

Paying with cards within apps

Apple Pay can also be used to make payments in iOS, iPadOS, macOS, visionOS, and watchOS apps. When users pay within apps using Apple Pay, Apple receives the encrypted transaction information to route to the specific developer or merchant to which the user is making a payment. Before that information is sent to the developer or merchant, Apple reencrypts the transaction with a developer-specific key. This is to help ensure that only an authorized developer with the key-pair can decrypt the information. Apple Pay retains anonymous transaction information, such as approximate purchase amount. This information can't be tied to the user and never includes what the user is buying.

Besides the use of Apple Pay in stores, it also works within iOS, iPadOS, macOS, visionOS, and watchOS, apps. When users pay in apps, Apple gets the encrypted transaction data. It then sends this data to the right developer or merchant. Before this, Apple reencrypts the data with a developer-specific key. This ensures that only authorized developers can access it. Apple Pay keeps anonymous data, like the purchase amount. However, this data isn't linked to users and doesn't reveal what they buy.

When an app initiates an Apple Pay payment transaction, the Apple Pay servers receive the encrypted transaction from the device prior to the merchant receiving it. The Apple Pay servers then reencrypt the transaction with a merchant-specific key before relaying it to the merchant.

When an app requests a payment, it calls an API to determine whether the device supports Apple Pay and whether the user has credit or debit cards that can make payments on a payment network accepted by the merchant. The app requests information it needs to process and fulfill the transaction, such as the billing and shipping address, and contact information. The app then asks iOS, iPadOS, macOS, watchOS, or visionOS to present the Apple Pay. This sheet then requests information for the app and other necessary information, such as the card to use.

At this time, the app is presented with city, state, and postal code information to calculate the final shipping cost. The full set of requested information isn't provided to the app until the user authorizes the payment using one of the following methods:

- Biometric authentication
- Device passcode or password
- Double-clicking the side button of an unlocked Apple Watch

After the payment is authorized, the information presented in the Apple Pay sheet is transferred to the merchant.

Paying with cards within App Clips

An App Clip is a small part of an app that allows a user do a task quickly (such as renting a bike or paying for parking) without downloading the full app. If the App Clip supports payments, the user can use Sign in with Apple (if configured by the app developer) and then make a payment using Apple Pay. When a user makes a payment from within an App Clip, all security and privacy measures are the same as when a user pays within an app.

How users authorize, and merchants verify, app payments

Users and merchants ensure secure app payments by passing information to the Apple Pay servers, the Secure Element, the device, and the app's API. First, the user authorizes an app payment. The app then requests a cryptographic anti-replay value from the Apple Pay servers. The servers send this value and other transaction data to the Secure Element, which creates a payment credential, one that's encrypted with an Apple key. The Secure Element then returns the payment credential to the Apple Pay servers, so that they can decrypt it, verify its anti-replay value against the anti-replay value that the Apple Pay servers originally sent, and reencrypt it with the merchant's key. The servers then return the payment to the device, which hands it back to the app API, with the API passing it along to the merchant system for processing. Finally, the merchant verifies the payment credential to confirm the transaction.

The APIs require a [Merchant ID Entitlement](#) that specifies the supported Merchant IDs. An app can also include additional data (such as an order number or customer identity) to send to the Secure Element to be signed, ensuring that the transaction can't be diverted to a different customer. This is accomplished by the app developer, who can specify application-specific data (`applicationData`) on the request for payment. A hash of this data is included in the encrypted payment data. The merchant is then responsible for verifying that their `applicationData` hash matches what's included in the payment data.

Paying with cards at websites

Apple Pay can be used to make payments at websites with the following:

- Devices that use biometric authentication
- Apple Watch
- Mac computers with Apple silicon that use the Magic Keyboard with Touch ID

Apple Pay transactions can also start on a Mac and be completed on an Apple Pay-enabled iPhone or Apple Watch using the same Apple Account. If the user is transmitting payment-related information this way, Apple Pay Handoff uses the end-to-end encrypted [Apple Identity Service \(IDS\)](#) protocol to transmit payment-related information between the user's Mac and the authorizing device. The IDS client on Mac uses the user's device keys to perform encryption so no other device can decrypt this information. These keys aren't available to Apple.

Device discovery for Apple Pay Handoff contains the type and unique identifier of the user's credit cards along with some metadata. The Device Account Number of the user's card isn't shared, and it continues to remain stored securely on the user's iPhone or Apple Watch. Apple also securely transfers the user's recently used contact, shipping, and billing addresses over iCloud Keychain.

After the user authorizes a payment, a payment token—uniquely encrypted to each website's merchant certificate—is securely transmitted from the user's iPhone or Apple Watch to their Mac and then delivered to the merchant's website.

Note: Only devices in proximity to each other may request and complete payment. Proximity is determined through Bluetooth Low Energy (BLE) advertisements.

Apple Pay on the web also requires that all participating websites register with Apple. After the domain is registered, domain name validation is performed only after Apple issues a TLS client certificate. Websites supporting Apple Pay are required to:

- Serve their content over HTTPS
- Obtain a secure and unique merchant session (for each payment transaction) with an Apple server using the Apple-issued TLS client certificate

Merchant session data is signed by Apple. After a merchant session signature is verified, a website may query whether the user has an Apple Pay-capable device and whether they have a credit, debit, or prepaid card activated on the device. No other details are shared. If the user doesn't want to share this information, they can turn off Apple Pay queries in Safari privacy settings on iPad, iPhone, and Mac devices.

If the website is using the latest version of the Apple Pay JS SDK, then Apple Pay transactions can also be started using any third-party web browser on any operating system, and be completed on an Apple Pay-enabled iPhone or iPad with iOS 18, iPadOS 18, or later. For this to occur, a code needs to be scanned using the device's camera to establish a connection with the website. When the website presents this code, a secure WebSocket connection is made between the website and Apple's servers. Upon scanning this code, an additional separate secure WebSocket connection is made between the Apple Pay-enabled device and Apple's servers. This completes the bidirectional connection required between the website and Apple Pay-enabled device, using Apple's servers as a relay. Any communication made between these two parties then follows the usual process for Apple Pay web transactions.

After a merchant session is validated, all privacy and security measures are the same as when a user pays within an app.

Automatic payments and Merchant Tokens

Devices with iOS 16, iPadOS 16, macOS 13, or later, can use Apple Pay merchant tokens, which ensure secure payments across a user's devices. The updated Apple Pay payment sheet optimizes preauthorized payments. The Apple Pay API also supports new transaction types, allowing developers to customize the payment sheet for specific uses, such as subscriptions, recurring bills, installment payments, and automatic reloading of card balances.

Merchant tokens aren't device specific, and therefore allow for continuity of recurring payments if the user removes a payment card from the device.

Payments to multiple merchants

In iOS 16 or later, Apple Pay includes the ability to specify purchase amounts for multiple merchants within a single Apple Pay payment sheet. This allows the flexibility to let customers make a bundled purchase, such as a travel package with flight, rental car, and hotel, then send payments to individual merchants.

Contactless passes in Apple Pay

To transmit data from supported passes to compatible NFC terminals, Apple uses the Apple Value Added Services (Apple VAS) protocol. The VAS protocol can be implemented on third-party contactless terminals or in iPhone apps and uses NFC to communicate with supported Apple devices. The VAS protocol works over a short distance and can be used to present contactless passes independently or as part of an Apple Pay transaction.

When the device is held near the NFC terminal, the terminal initiates receiving the pass information by sending a request for a pass. If the user has a pass with the pass provider's identifier, the user is asked to authorize its use using one of the following methods:

- Biometric authentication
- Device passcode or password
- Double-clicking the side button of an unlocked Apple Watch

The pass information, a timestamp, and a single-use random ECDH P-256 key are used with the pass provider's public key to derive an encryption key for the pass data, which is sent to the terminal.

From iOS 12.0.1 through iOS 13, users may manually select a pass before presenting it to the merchant's NFC terminal. On devices with iOS 13.1 or later, pass providers can configure manually selected passes to either require user authentication or be used without authentication.

Rendering cards unusable with Apple Pay

Credit, debit, and prepaid cards added to the Secure Element can be used only if the Secure Element is presented with authorization using the same pairing key and Authorization Random (AR) value from when the card was added. On receipt of a new AR value, the Secure Element marks any previously added cards as terminated. This allows the operating system to instruct the Secure Enclave to render cards unusable by marking its copy of the AR as invalid under the following scenarios:

Card termination method	Device
The passcode is inactive.	iPad, iPhone, Apple Watch, Apple Vision Pro
The password is inactive.	Mac
The user signs out of iCloud.	iPad, iPhone, Mac, Apple Watch, Apple Vision Pro
The user selects Erase All Content and Settings.	iPad, iPhone, Mac, Apple Watch, Apple Vision Pro
The device is restored from Recovery Mode.	iPad, iPhone, Mac, Apple Watch, Apple Vision Pro
Unpairing	Apple Watch

When a user erases the entire device—using Erase All Content and Settings, using Find My, or restoring their device—that device instructs the Secure Element to mark all cards as terminated. This has the effect of immediately changing the cards to an unusable state until the Apple Pay servers can be contacted to fully erase the cards from the Secure Element. Independently, the Secure Enclave marks the AR value as invalid so that further payment authorizations for previously enrolled cards aren't possible. When the device is online, it attempts to contact the Apple Pay servers to help ensure that all cards in the Secure Element are erased.

Suspending, removing and erasing cards

Users can suspend Apple Pay, or remove and erase their cards on devices using the following methods.

Device	Method to suspend Apple Pay	Methods to remove and erase cards from Apple Pay
iPad, iPhone, Mac	Place device in Lost Mode using Find My	Find My Apple Wallet Remove device from icloud.com Remove device from the Apple Account Delete the Apple Account from the Apple Data and Privacy page
Apple Watch	Place device in Lost Mode using Find My	Find My Apple Wallet Remove device from icloud.com Remove device from the Apple Account Delete the Apple Account from the Apple Data and Privacy page Remove from the Watch app on paired iPhone
Apple Vision Pro	NA	Apple Wallet Remove device from icloud.com Remove device from the Apple Account Delete the Apple Account from the Apple Data and Privacy page

When a user suspends or removes cards from Apple Pay, the ability to make payments using those cards is suspended or revoked by the card issuer or respective payment network, even if the device is offline and not connected to a Wi-Fi or cellular network. Users can also contact their card issuer to suspend or remove cards from Apple Pay.

Apple Card security

On supported models of iPhone and Mac, a user can securely apply for an Apple Card.

Apple Card application

On devices with iOS 12.4, iPadOS 13.1, macOS 10.14.6, watchOS 5.3, visionOS 1, or later, Apple Card can be used with Apple Pay to make payments in stores, in apps, and on the web.

To apply for Apple Card, the user needs to be signed into their Apple Account on an Apple Pay-compatible iPad or iPhone and have two-factor authentication set up on the Apple Account, or they can apply at apply.applecard.apple after signing in with their Apple Account. When the application is approved, Apple Card is available in Apple Wallet or within Settings > Wallet & Apple Pay across any of the eligible devices the user has signed in with their Apple Account.

When a user applies for Apple Card, user identity information is securely verified by Apple's identity provider partners and then shared with Goldman Sachs Bank USA for the purposes of identity and credit evaluation.

Information such as the social security number or ID document image provided during the application is securely transmitted to Apple's identity provider partners and/or Goldman Sachs Bank USA encrypted with their respective keys. Apple can't decrypt this data.

The income information provided during the application, and the bank account information used for bill payments, are securely transmitted to Goldman Sachs Bank USA encrypted with their key. The bank account information is saved in the keychain. Apple can't decrypt this data.

When adding Apple Card to Apple Wallet, the same information as when a user adds a credit or debit card may be shared with Apple's partner bank Goldman Sachs Bank USA, and with Apple Payments Inc. This information is used only for troubleshooting, fraud prevention, and regulatory purposes.

On devices with iOS 14.6, iPadOS 14.6, and watchOS 7.5, or later, the organizer of an iCloud family with an Apple Card can share their card with their iCloud Family members over the age of 13. User authentication is required to confirm the invitation. Apple Wallet uses a key in the Secure Enclave to compute a signature that binds the owner and the invitee. That signature is validated on Apple servers.

Optionally, the organizer can set a transaction limit for the participants. Participant cards can also be locked to pause their spending at any time through Apple Wallet. When a co-owner or participant over the age of 18 accepts the invitation and applies, they go through the same application process as defined in the Apple Card application section in Apple Wallet.

Apple Card usage

A physical card can be ordered from Apple Card in Apple Wallet. After the user receives the physical card, it's activated using the NFC tag that's in the bifold envelope of the physical card. The tag is unique per card and can't be used to activate another user's card. Alternatively, the card can be manually activated in Apple Wallet settings. Additionally, the user can also choose to lock or unlock the physical card at any time from Apple Wallet.

Apple Card payments and Apple Wallet pass details

Payments due on the Apple Card account can be made from a web browser or Apple Wallet in iOS with Apple Cash and a bank account. Bill payments can be scheduled as recurring or as a one-time payment at a specific date with Apple Cash and a bank account. When a user makes a payment, a call is made to the Apple Pay servers to obtain a cryptographic anti-replay value similar to Apple Cash. The anti-replay value, along with the payment setup details, is passed to the Secure Element to compute a signature. The signature is then returned to the Apple Pay servers. The authentication, integrity, and correctness of the payment are verified through the signature and the anti-replay value by Apple Pay servers, and the order is passed on to Goldman Sachs Bank USA for processing.

The Apple Card number is retrieved by Apple Wallet by presenting a certificate. The Apple Pay server validates the certificate to confirm the key was generated in the Secure Enclave. It then uses this key to encrypt the Apple Card number before returning it to Apple Wallet, so that only the iPhone that requested the Apple Card number can decrypt it. After decryption, the Apple Card number is saved in iCloud Keychain.

Displaying the Apple Card number details in the pass using Apple Wallet requires user authentication with Face ID, Touch ID, or a passcode. It can be replaced by the user in the card information section and turns off the previous one.

Advanced Fraud Protection

On devices with iOS 15 and iPadOS 15 or later, the Apple Card user can enable Advanced Fraud Protection in Apple Wallet. When enabled, the Card Security Code refreshes every few days.

Apple Cash security

On devices with iOS 11.2, iPadOS 13.1, visionOS 1, watchOS 4.2, or later, Apple Cash can be used on an iPad, iPhone, Apple Vision Pro, or Apple Watch to send, receive, and request money from other users. When a user receives money, it's added to an Apple Cash account that can be accessed in Apple Wallet or within Settings > Wallet & Apple Pay across any of the eligible devices the user has signed in with their Apple Account.

Note: Apple Cash is currently available only to users in the United States.

On devices with iOS 14, iPadOS 14, visionOS 1, watchOS 7, or later, the organizer of an iCloud family who has verified their identity with Apple Cash can enable Apple Cash for their family members under the age of 18. Optionally, the organizer can restrict the money sending capabilities of these users to family members only or contacts only. If the family member under the age of 18 goes through an Apple Account recovery, the organizer of the family needs to manually reenable the Apple Cash card for that user. If the family member under the age of 18 is no longer part of the iCloud family, their Apple Cash balance is automatically transferred to the organizer's account.

When the user sets up Apple Cash, the same information as when the user adds a credit or debit card may be shared with Apple's partner bank in the United States, Green Dot Bank, and with Apple Payments Inc., a wholly owned subsidiary created to protect the user's privacy by storing and processing information separately from the rest of Apple, and in a way that the rest of Apple doesn't know. This information is used only for troubleshooting, fraud prevention, and regulatory purposes.

Using Apple Cash in iMessage

To use person-to-person payments and Apple Cash, a user needs to be signed in with their personal Apple Account on an Apple Cash-compatible device and have two-factor authentication enabled. Money requests and transfers between users are initiated from within the Messages app (in both individual and group messages), from the Wallet app, and by asking Siri. When a user attempts to send money, the Apple Cash payment sheet is displayed within Messages. The user can elect to send from the Apple Cash balance, or from a debit card added to Apple Wallet. If the Apple Cash balance doesn't cover the full payment amount, the debit card is used for the remaining balance.

Using Tap to Cash

When using Tap to Cash, a user needs to be signed in to their Apple Account on iPhone or Apple Watch and have two-factor authentication set up on the account. To send money, customers need to enter the amount to send, hold their device near another device, and authenticate using the following methods:

- Biometric authentication
- Device passcode
- Double-clicking the side button of an unlocked Apple Watch

After authentication, devices need to be held together for several seconds to establish a connection that results in the money being sent.

Using Apple Cash in stores, apps, and on the web

The Apple Cash card in Apple Wallet can be used with Apple Pay to make payments in stores, in apps, and on the web. Money in the Apple Cash account can also be transferred to a bank account. In addition to money being received from another user, money can be added to the Apple Cash account from a debit or prepaid card in Apple Wallet.

Apple Payments Inc. stores, and may use, the user's transaction data for troubleshooting, fraud prevention, and regulatory purposes once a transaction is completed. The rest of Apple doesn't know who the user sent money to, received money from, or where the user made a purchase with their Apple Cash card.

When the user sends money with Apple Pay, adds money to an Apple Cash account, or transfers money to a bank account, a call is made to the Apple Pay servers to obtain a cryptographic anti-replay value, which is similar to the value returned for Apple Pay within apps. The anti-replay value, along with other transaction data, is passed to the Secure Element to compute a payment signature. The signature is returned to the Apple Pay servers. The authentication, integrity, and correctness of the transaction is verified through the payment signature and the anti-replay value by Apple Pay servers. Money transfer is then initiated, and the user is notified of a completed transaction.

If the transaction involves:

- A debit card for adding money to Apple Cash
- Providing supplemental money if the Apple Cash balance is insufficient

An encrypted payment credential is also produced and sent to Apple Pay servers, similar to how Apple Pay works within apps and websites.

After the balance of the Apple Cash account exceeds a certain amount or if unusual activity is detected, the user is prompted to verify their identity. Information provided to verify the user's identity—such as social security number or answers to questions (for example, to confirm a street name the user lived on previously)—is securely transmitted to the Apple partner and encrypted using their key. Apple can't decrypt this data. The user is prompted to verify their identity again if they perform an Apple Account recovery, before regaining access to their Apple Cash balance.

Tap to Pay on iPhone security

With Tap to Pay on iPhone and a supported payment app, merchants can accept all types of in-person, contactless payments right on their iPhone—from physical debit and credit cards to Apple Pay and other digital wallets. No extra payment terminals, readers, or hardware needed.

Tap to Pay on iPhone uses the built-in security and privacy features of iPhone to help protect the business and customer data. Apple doesn't store card numbers on Apple servers. Tap to Pay on iPhone doesn't collect transaction information that can be tied back to the payer. Payment card data such as Credit/Debit Card Number (PAN) is secured by the Secure Element and isn't visible to the acceptance device. Apple only gives access to the payment card data to the merchant's Payment Service Provider. In addition, the Tap to Pay on iPhone doesn't collect payer's names, addresses or phone numbers.

Note: Encrypted card numbers are temporarily stored on iPhone only for transactions made in Store and Forward mode.

Tap to Pay on iPhone has been evaluated by an accredited security laboratory and approved for use by all accepted payment networks in the territories it's available. On an iPhone with iOS 18.4 or later, Tap to Pay on iPhone is a PCI MPoC validated solution, and listed on the [PCI website](#).

Contactless payment component security

- *Secure Element:* The Secure Element hosts the payment acceptance applets and kernel configurations which read and secure the contactless payment card data.
- *NFC Controller:* The NFC controller handles Near Field Communication protocols and routes communication between the Application Processor and the Secure Element, and between the Secure Element and the contactless payment card.
- *Tap to Pay on iPhone servers:* The Tap to Pay on iPhone servers manage the setup and provisioning of the payment kernels in the device. The servers also monitor the security of the Tap to Pay on iPhone devices in a manner compatible with the Mobile Payments on COTS (MPoC) standard from the Payment Card Industry Security Standards Council (PCI SSC) and are PCI DSS compliant.

How Tap to Pay on iPhone reads credit, debit, and prepaid cards

Provisioning security overview

Upon first use of Tap to Pay on iPhone using a sufficiently entitled app, the Tap to Pay on iPhone server determines whether the device meets the eligibility criteria such as Device Model, iOS version, and whether a passcode has been set. After this verification is complete, the payment acceptance applets are downloaded from the Tap to Pay on iPhone server and installed on the Secure Element, along with the associated kernel configurations. This operation is performed securely between the Tap to Pay on iPhone servers and the Secure Element. The Secure Element validates the integrity and authenticity of this data prior to installation.

Card read security overview

When a Tap to Pay on iPhone app requests a card read from the ProximityReader framework, a sheet—controlled by iOS—is displayed and prompts the user to tap a payment card. iOS initializes the Payment Card Reader and then requests the payment kernels in the Secure Element to initiate a card read.

At this point, the Secure Element assumes control of the NFC controller in Reader Mode. This mode allows card data to be exchanged only between the payment card and the Secure Element through the NFC controller.

After the payment acceptance applet on the Secure Element has completed the payment card read, it encrypts and signs the payment card data. The payment card data remains encrypted and authenticated until it reaches the Payment Service Provider. Only the Payment Service Provider used by the app to request the card read can decrypt the payment card data. The Payment Service Provider needs to request the payment card data decryption key from the Tap to Pay on iPhone server. The Tap to Pay on iPhone server emits decryption keys to the Payment Service Provider after validation of the integrity and authenticity of the data, and after verifying that the card read was performed just prior to of the request for the payment card data decryption key.

This model helps ensure that the payment card data cannot be decrypted by anyone other than the Payment Service Provider, which processes this transaction for the merchant.

Store and Forward Mode

To support use cases with limited connectivity, Tap to Pay on iPhone can be configured to run in *Store and Forward* mode. This allows Tap to Pay on iPhone to read and encrypt payment card data while not connected to the internet. For Store and Forward transactions the Payment Service Provider may request the payment card data decryption key for up to a few weeks after the card was read. With encryption on the Secure Element, Tap to Pay on iPhone is designed to protect the integrity and confidentiality of the payment card data while it is temporarily stored on iPhone. PIN entry isn't supported for Store and Forward transactions.

Card PIN entry security overview

PIN entry allows the payer to enter their PIN on the acceptance device to authorize the transaction. The PIN entry screen may be initiated immediately after the tap based on the information exchanged with the payment card. Alternatively, the Payment Service Provider can initiate the PIN screen by providing a signed token, which is valid for one specific transaction only.

The PIN entry mechanism has been evaluated by an accredited security laboratory and is approved for use by all accepted payment networks in the territories where it's available. Tap to Pay on iPhone is designed to prevent all photo, video, screenshot and screen-recording features from capturing PIN information.

The PIN digits entered are securely captured by the Secure Element. Using these PIN digits, the Secure Element creates a payment industry standard compliant encrypted PIN block. To decrypt the PIN block, the Payment Service Provider needs to request the PIN block decryption key from the Tap to Pay on iPhone server.

The PIN value is:

- Never available to the merchant on their acceptance device.
- Never decrypted by Apple.
- Never stored by Apple.

Securing the merchant device during PIN entry

During the PIN entry process, the device will be facing the payer, and may be held away from the merchant. To ensure the protection of the merchant device and data, the merchant has the option to enable the Tap to Pay on iPhone Screen Lock setting. This option is found in the settings for each App that supports Tap to Pay on iPhone. Enabling this option will lock the merchant's device while showing the PIN entry screen. After the payer enters their card PIN, the merchant will need to perform a Face ID, Touch ID or Passcode unlock to further operate the device, ensuring the payer cannot access the merchant's device.

Using Apple Wallet

Access using Apple Wallet

In Apple Wallet on supported iPhone and Apple Watch devices, users can store [multiple types of keys](#). When a user arrives at a door, the right key can even be automatically presented (if Express Mode is supported by that key and was turned on), allowing them to enter with just a tap using Near Field Communication (NFC).

User convenience

Express Mode

When a key is added to Apple Wallet, Express Mode is turned on by default. Keys in Express Mode interact with accepting terminals without Face ID, Touch ID, passcode authentication, or double-clicking the side button on Apple Watch. To turn off this feature, users can turn off Express Mode by tapping the More button on the front of the card representing key in Apple Wallet. To turn Express Mode back on, the user needs to use Face ID, Touch ID, or a passcode.

Key sharing

On an iPhone with iOS 16 or later, key sharing is available for certain key types.

Users can share access to a key (for example a house or car key), with security and privacy enforced from the key owner's iPhone to the invited key recipient's iPhone. Keys are shared by tapping on the share icon of the key in Apple Wallet and can be shared using methods that appear in the share sheet. Key owners can also choose the access level and valid time period for each shared key. The key owner has visibility over all the keys that they shared and can revoke access for any shared key, including any instances where the key is shared again to another user by the initial key recipient.

The key sharing invitation is stored anonymized and secured by a dedicated server inside a mailbox, and protected with an AES 128 or 256 encryption key. The encryption key is never shared with the server or anyone, apart from intended key recipient, and only the key recipient can decrypt the invitation. Upon mailbox creation, the key owner's iPhone provides a device claim that's bound only to that mailbox by the server. When the key recipient's iPhone initially accesses this mailbox, it presents a key recipient device claim. Only the key owner and key recipient iPhone devices that present valid device claims can access that mailbox. Each iPhone device claim has its unique UUID value as per [RFC 4122](#).

As an additional security measure, the key owner can turn on a 6-digit randomly-generated activation code that's required on the key recipient's iPhone. The number of code retries is enforced and validated either by key owner or partner server. This activation code needs to be communicated by the key owner to the key recipient and the key recipient needs to present that code when prompted for its validation either by key owner or partner server.

After an invitation is redeemed by the key recipient, it's immediately wiped from the server by the receiving iPhone. The mailbox containing the key sharing invitation also has a limited lifetime, which is set upon mailbox creation and enforced by the server. Any expired invitations are automatically erased by the server.

Depending on the original manufacturer, keys may also be shared with non-Apple devices however, their method of securing key sharing may be different than Apple's.

Privacy and security

Access keys in Apple Wallet take full advantage of the privacy and security built into iPhone and Apple Watch. When or where a person uses their keys in Apple Wallet is never shared with Apple or stored on Apple servers, and credentials are securely stored inside the Secure Element (SE) of supported devices. The Secure Element hosts specially designed applets to securely manage keys, ensuring that they can't be extracted or leak.

Before provisioning any keys, a user needs to be signed in to their Apple Account on a compatible iPhone and have two-factor authentication turned on for their Apple Account.

During the provisioning process:

- The device connects to Apple servers to request Access pass provisioning using mTLS to protect the data in that session from being intercepted or altered.
- Before provisioning starts, Apple servers confirm the device is eligible by performing a cryptographic device attestation.
- Apple servers, hosted in the U.S., fetch the relevant Access pass data (for example, images and attributes shown on the pass) from the issuing partner's servers, then send it to the device.

- All interactions between Apple's servers and Access partner servers are authenticated using mTLS or JWT and all data exchanged is encrypted using ECC-based encryption.
- This data is then used to create the visual representation of the Access pass in Apple Wallet.
- When needed, the device downloads an Access credential (applet) and installs it in the SE. It then provisions the credential with data (for example, apps, keys, configuration settings, access rights, Access document, and so on) received from the Access Partner's servers. This data goes from the Access Partner to Apple's Hardware Security Modules (HSM) first, where it's rewrapped with a key unique to the user's device. Apple servers then send it to the user's device. Only the on-device SE can decrypt this data.
- Credential data never appears in plain text anywhere outside the HSM's memory.
- Encrypted credential data is logged on Apple servers for 15 days for troubleshooting purposes.
- Other eligible devices signed in with the same Apple Account (for example, Apple Watch) may get notified in a push notification by an Apple server, which would separately trigger a provisioning process for each eligible device. The number of eligible devices are defined and enforced by Access partners.

During a transaction, the reader communicates with a credential stored in the Secure Element through the near-field-communication (NFC) controller using an established secure channel.

The number of devices, including iPhone and Apple Watch, that can be provisioned with a key is defined and controlled by each partner and can vary from one partner to another. This approach allows each partner to have control over the maximum number of provisioned keys per device type to suit their specific needs. For this purpose, Apple supplies partners with device type and anonymized device identifiers. Identifiers are different for every partner for privacy and security reasons.

Partners also receive user identifiers, which are anonymized and unique-per-partner. This lets them securely bind the key to the user Apple Account during the initial provisioning. This measure protects keys from being provisioned by a different user in case a user account created with the partner was compromised, for example, in account takeover attack scenario.

Keys can be made inactive or removed by:

- Erasing the device remotely with Find My
- Enabling Lost Mode with Find My
- Receiving a device management remote wipe command
- Removing all cards from their Apple Account account page
- Removing all cards from iCloud.com
- Removing all cards from Apple Wallet
- Removing the card in the issuer's app

On an iPhone with iOS 15.4 or later, when a user double-clicks the side button on an iPhone with Face ID or double-clicks the Home button on an iPhone with Touch ID, their passes and access key details aren't displayed until they authenticate to the device. Either Face ID, Touch ID, or passcode authentication is required before pass specific information including hotel booking details are displayed in Apple Wallet.

Access key types

There are different types of access from Apple Wallet, such as hospitality, corporate badges, student IDs, home keys, and car keys.

Hospitality

Hotel room keys in Apple Wallet help deliver an easy and contactless experience from check-in to check-out, while providing additional privacy and security benefits for guests on top of traditional hotel key cards. Hotel guests at supported locations can tap to unlock with room keys in Apple Wallet on their compatible iPhone and Apple Watch Series 4 or later.

The capabilities in Apple Wallet are specifically designed to reduce friction for the customer:

- Prearrival provisioning from the hotel's app, to add a pass to Apple Wallet ahead of a stay
- Check-in pass tiles, to initiate check-ins and room assignments from Apple Wallet
- Post-provisioning key updates, to support extending or modifying current stays
- Multiroom key support for a single pass in Apple Wallet
- Auto-archiving of expired keys in Apple Wallet

Disney MagicMobile passes

Users can add a Disney MagicMobile Pass to Apple Wallet on iPhone or Apple Watch to enter participating Disney theme parks. MagicMobile Passes can be used for entry into the park as well as at other applicable readers throughout the parks.

To add a Disney MagicMobile Pass, in addition to having two-factor authentication enabled on their Apple Account, a user needs to have tickets or reservations to a participating theme park associated with a valid My Disney Experience account. From the My Disney Experience app on their iPhone, the user can select one or more passes to add to Apple Wallet. If the user has a paired Apple Watch, the selected passes are automatically provisioned onto both the user's iPhone and their paired Apple Watch. Express Mode is turned on by default for passes added to both iPhone and Apple Watch devices. For ease of use, when Express Mode is turned on, it's turned on for all MagicMobile passes currently on the device.

Multiple passes can be added to a single device so users can manage the passes for all members in their party. Users may also choose to use the My Disney Experience app to share passes with other users. Recipients can add the shared passes to their Apple Wallet.

Corporate badges

Employee badges of supported partners can be added to Apple Wallet on iPhone and Apple Watch, allowing employees around the world contactless access to their workplaces. To add a badge, an employee needs to have multifactor authentication enabled for their account used to sign in to the app provided by their employer.

Employee badge takes advantage of Apple's access capabilities, allowing users to:

- Automatically add an employee badge to their paired Apple Watch through push provisioning that doesn't require installing a partner's app

- Seamlessly access office amenities utilizing Express Mode
- Gain access to the workplace even after their iPhone runs out of battery

Student ID cards

On an iPhone with iOS 12 or later, students, faculty, and staff at participating campuses can add their student ID card to Apple Wallet on supported models of iPhone and Apple Watch to access locations and pay wherever their card is accepted.

A user adds their student ID card to Apple Wallet through an app provided by the card issuer or participating school. The technical process by which this occurs is the same as the one described in [Adding credit or debit cards from a card issuer's app](#). In addition, issuing apps needs to support two-factor authentication on the accounts that guard access to their student IDs. A card may be set up simultaneously on a user's iPhone and a paired Apple Watch.

Multifamily homes

Tenants and staff of supported partner facilities can use their home key in Apple Wallet to access their building, unit, and common areas. The home key can be provisioned from the app provided by the partner. For partners that support frictionless provisioning, property managers can send tenants a link to initiate provisioning using their preferred messaging channel (for example, email or SMS) so that the tenant only needs to click the link to redeem the key. App Clips also provide a secure and seamless experience, making it possible to provision a key without installing a partner's app. For more information, see the Apple Support article [Use App Clips on iPhone](#).

Multifamily home key can also be used in Express Mode and can be securely shared with friends and family members. For more information, see [Key sharing](#).

Home key

A home key in Apple Wallet can be used with supported NFC-enabled door locks with a simple tap of an iPhone or Apple Watch. For more information about how a user can set up and use a home key, see the Apple Support article [Unlock your door with a home key on iPhone](#).

When a user sets up a home key, all residents in their household also automatically receive the home key. To further share a home key or remove a member of a shared home, the owner of a home can use the Home app to manage invitations and members. When a user chooses to accept an invitation to join a home with a home key, this initiates provisioning of the home key into Apple Wallet on their devices. If a user chooses to leave a home or if the home owner withdraws their access, these actions also remove the home key from Apple Wallet.

Car key

Storing car keys digitally in Apple Wallet is available natively in supported iPhone devices and paired Apple Watch devices. Car keys are represented as passes (created by Apple on behalf of the automaker) in Apple Wallet and support the full Apple Pay card life cycle (iCloud Lost Mode, Remote Wipe, local pass deletion, and Erase All Content and Settings). As with standard Apple Pay cards, shared car keys can be deleted from the owner's iPhone, Apple Watch, and in the vehicle's Human Machine Interface (HMI).

Car keys can be used, for example, to unlock and lock the vehicle, open and close the trunk, turn the alarm on and off, start the engine, or set the vehicle into drive mode. The standard transaction offers mutual authentication and is mandatory for engine start. Unlock and lock transactions might use the fast transaction when required for performance reasons.

Keys are created by connecting (or pairing) an iPhone with an owned and supported vehicle. All keys are created inside Secure Element based on elliptic curve (NIST P-256) on-board key generation (ECC-OBKG), and the private keys never leave the Secure Element. Communication between devices and the vehicle uses either NFC or a combination of Bluetooth LE and Ultra-WideBand (UWB). Key management uses an Apple to automaker server API with mutually authenticated TLS. After a key is paired with an iPhone, any Apple Watch paired with that iPhone can also receive a key. When a key is deleted either in the vehicle or on the device, it can't be restored. Keys on lost or stolen devices can be suspended and resumed, but reprovisioning them on a new device requires a new pairing or sharing.

Car keys can also be used in Express Mode and can be securely shared with friends and family members. For more information, see [Key sharing](#). For more information on security and privacy of the digital car keys, see [Car key security](#).

Scooter key

On an iPhone with iOS 17 or later and in certain countries or regions with supported partners, users can get a scooter key provisioned from the partner app directly into Apple Wallet on supported an iPhone and paired Apple Watch for the following purposes:

- Tap to lock or unlock the scooter
- Tap to lock or unlock the scooter's trunk (if available)

A dedicated applet in the Secure Element securely handles cryptographic credentials associated with the scooter key and allows secure transactions the with scooter.

On the back of the pass users can access additional information about their scooters, such as the last four digits of vehicle identification number (VIN) and its license or number plate. Such information may be considered private and can be accessed only when using biometric authentication or the device passcode.

The scooter key can also be used in Express Mode and can be securely shared with friends and family members. For more information, see [Key sharing](#).

Car key security

Car key security in iOS

Developers can support secure keyless access to a vehicle with a supported iPhone and a paired Apple Watch.

Owner pairing

The owner needs to prove possession of the vehicle (the method is dependent on the automaker, often they have to bring one or two key fobs) and can start the pairing process in the automaker's app, by using an email link received from the automaker or from the vehicle menu. In all cases, the owner needs to present a confidential one-time pairing password to the iPhone, which is used to generate a secure pairing channel using the SPAKE2+ protocol with the NIST P-256 curve to create an Global Platform SCP03 secure channel to transfer data between the device and the vehicle. When using the app or the email link, the password is automatically transferred to the iPhone, where it needs to be entered manually when pairing is started from the vehicle.

In the owner pairing process, the vehicle sends a request through the SCP03 channel instructing the device's Secure Element to generate an Elliptic Curve Cryptography (ECC) key pair. The vehicle identifier and the vehicle public key are then securely bound to this key pair. The device public key (`device.PK`) is sent back to the vehicle in an X.509 certificate format together with a certificate chain that's verifiable with the automaker root certificate public key (`root.PK`), which is embedded in the vehicle at production. This allows for verification and acceptance of the device public key as the owner key.

The owner key in the device Secure Element also securely binds the automaker `root.PK`, which is provided by the vehicle through the SCP03 channel, to the `device.PK` for key sharing.

If required for insurance purposes, each owner's key needs to be registered with the automaker's server. The device sends its `device.PK` certificate chain to the key tracking server (KTS) which sends back a signature to confirm the registration. The device provides the signature to the vehicle at the end of the owner pairing process or at the next standard transaction with the vehicle (see below), and the vehicle enables the owner key if the KTS signature has been successfully verified. The KTS signature keys are proprietary to the automaker.

Key sharing

The sender's iPhone can share keys to eligible devices by sending an invitation URL to the recipient through any communication channel for example email, iMessage, WhatsApp, WeChat or AirDrop. The URL directs the receiver to a privacy-protected mailbox on a relay server which is implemented based on an IETF specification. The privacy encryption key is sent as a fragment in the sharing URL.

During this process, the sender's iPhone requests a user authentication (Face ID, Touch ID, or passcode entry) and a secure user intent described in [Uses for Optic ID, Face ID, and Touch ID](#). The sender authorization is stored temporarily in the Secure Element for consumption later when the recipient device returns the signing request.

For added security, to activate the shared key, the sender may require recipient to use one of the available activation options supported by their automaker or passcode entered on recipient device.

Upon acceptance of the invitation, the recipient's device retrieves key creation data from the relay server mailbox and creates a digital key according to the CCC Digital Key Specification. As a part of key creation process, the key is signed by the sender. The recipient device sends the key creation certificate chain back using the relay server to the sender's iPhone. The sender's iPhone then uses the `root.PK` embedded in the Secure Element to verify that the recipient key was created using the expected certificate chain. If successful, the sender's iPhone signs the ECC-public key of the recipient device and sends the signature back to the recipient using the relay server. The signature is authorized by consuming the sender authorization previously stored in the Secure Element.

The key entitlements and the sender signature are provided to the vehicle during the first use of the shared key on the vehicle (see [Standard transactions](#)). Entitlements describe the following:

- *Access level*: For example, unlock or drive.
- *Resharing policy*: For example, no resharing, limited resharing, or unlimited resharing (in terms of sharing chain length).

Some automakers require each shared key to be registered with the automaker's server. If required, the recipient device sends its `device.PK` certificate chain to the automaker's KTS which returns a signature to confirm the registration. The recipient device presents that signature to the vehicle in the first transaction with the vehicle (see [Standard transactions](#)). The vehicle validates the signature and—if the KTS signature has been successfully verified—enables the shared key. These signature keys are proprietary to the automaker.

Key deletion

Keys can be deleted the following ways:

- On the keyholder device
- From the owner's device
- From a sharer device with the proper authorizations
- While inside the vehicle

Deletions on the keyholder iPhone are effective immediately, even if the keyholder is using the key or if the device isn't connected to the internet.

Deletion of keys in the vehicle might be possible, at discretion of the automaker policy:

- Anytime
- Only when the vehicle is online
- Only when a key fob is present (to avoid being stranded without key)

In each case, the deletion on keyholder device or vehicle is reported to the KTS on the automaker side, which registers issued keys to a vehicle for insurance purposes.

The owner and eligible users can request a deletion using a remote termination request signed by the requester's private key (`device.SK`) by selecting the users in the shared keyholder accounts list on the back of their car key pass in Apple Wallet. The request is first sent to the automaker for key removal in the vehicle. The conditions for removing the key from the vehicle are defined by the automaker (as listed above). Only when the key is removed from the vehicle will the automaker server send a signed remote termination request to the keyholder device.

When a key is terminated in a device, the applet that manages the digital car keys creates a cryptographically signed termination attestation, which is used as proof of deletion by the automaker and used to remove the key from the KIS.

Privacy

The automaker's KTS doesn't store the device ID, SEID, or Apple Account. It stores only a mutable identifier—the instance CA identifier. This identifier isn't bound to any private data in the device or by the server, and it's deleted when the user initiates an Erase All Contents and Settings.

The CCC standard also mandates that the KTS information is kept confidential on the automaker side and is only used in case of insurance or other eligible requests.

Key registration data (key identifier, vehicle identifier, vehicle public key, device public key certificate chain, key entitlements) that's sent by the device to the KTS is encrypted against a KTS privacy encryption public key. Apple servers transmitting the key registration data from the device to the automaker server aren't able to decrypt this data. Apple doesn't know who the owner shared keys for their vehicle with.

Any other accessory to iPhone or Apple Watch which isn't able to authenticate to the device is unable to receive a stable identifier through a standard or fast transaction over NFC or Bluetooth. If the device doesn't recognize the vehicle identifier (for example, attempting by mistake to access a vehicle that looks similar to the keyholder's vehicle) then the applet in the Secure Element uses a substitute key that generates random values instead of meaningful keys or cryptograms.

The initial Bluetooth connection is protected by the use of temporary keys that are provided when sharing the car key or by the automaker in the response to the key tracking request.

Car key transactions

Car key can use various types of transactions.

Fast transactions

The fast transaction is used only to unlock the vehicle over NFC.

The iPhone generates an AES-128 cryptogram based on a secret previously generated on both sides during a standard transaction. This cryptogram allows the vehicle to quickly authenticate the device in performance sensitive scenarios as only symmetric cryptography is used and only one transfer between the NFC reader and the processing Electronic Control Unit (ECU) is required. If the vehicle requires read or write access to either the private mailbox, the confidential mailbox, or both (each car key in the Secure Element has it's pair of mailboxes), a Global Platform SCP03 secure channel between the vehicle and the device is established by deriving session keys from a secret previously shared during a standard transaction and a new ephemeral key pair. The ability of the vehicle to establish the secure channel authenticates the vehicle to the iPhone. If the vehicle stores keys for multiple devices, it needs to use a "try and error" method with a "most used first" scheme to find the right symmetric key, as the device doesn't give out a trackable identifier in the fast transaction where the vehicle isn't authenticated by the device.

If no symmetric keys have been established for the transacting device, the vehicle uses the ephemeral vehicle public key (`vehicle.PK`) to continue with a standard transaction (see Standard transactions below).

Standard transactions

The standard transaction is used to unlock the vehicle over NFC if no symmetric keys for a fast transaction are established through a prior standard transaction or when a newly shared key transacts with the vehicle for the first time (first friend key transaction). It's always used to authorize driving.

The vehicle initiates a secure channel between the vehicle and an iPhone by generating ephemeral ECC key pairs on the reader and the iPhone side. Using a ECKA-DH key agreement method, a shared secret can be derived on both sides and used for generation of a shared symmetric key for the fast transaction using Diffie-Hellman, a HKDF key derivation function ([RFC 5869](#)), and signatures from the long-term key established during pairing.

The ephemeral public key generated on the vehicle side is signed with the reader's long-term private key, which results in an authentication of the vehicle by the iPhone. From the iPhone perspective, this protocol is designed to prevent privacy-sensitive data from being revealed to an adversary intercepting the communication or impersonating a vehicle.

The iPhone uses the established, nonauthenticated secure channel to encrypt its public key identifier along with the signature computed on a reader's data-derived challenge and some additional app-specific data. This verification of the iPhone signature by the vehicle allows the reader to authenticate the device.

The NFC standard transaction is also used for the first approach (FFT) of a newly shared key to the vehicle, as the vehicle doesn't yet know the new device .PK). When the user approaches the door handle, the vehicle starts a fast transaction but wouldn't be able to verify the cryptogram. It then continues with a standard transaction (the reader command for the fast transaction, AUTH0, is also the first command for the standard transaction), creates a GP SCP03 secure channel and reads the private mailbox content that should contain an attestation of the new device .PK, signed by the sharer (sharer .PK or owner .PK are already known to the vehicle) and signed by the KTS using automaker proprietary cryptography. After verification of the signatures the vehicle accepts the device .PK as a new key.

BLE/UWB standard transactions

For vehicles using a ultra-wideband-enabled (UWB) key, a Bluetooth LE session is established between the vehicle and the iPhone. Similar to the NFC transaction, a shared secret is derived on both sides and used for the establishment of a secure session. This session is used to subsequently derive and agree a UWB Ranging Secret Key (URSK). The URSK is provided to UWB radios in the user's device and, in a vehicle OEM-proprietary secure channel, to UWB anchors.

Note: Anchors are generally placed on the four corners of a vehicle, inside, and optionally in the trunk of the vehicle to enable accurate and authenticated localization of the user's device to a specific position near to or inside the vehicle. The vehicle then uses the device position to make decisions about allowing unlocking or starting of the vehicle.

URSKs have a predefined expiration time (time-to-live, or TTL). To avoid interruption of ranging when a TTL expires, URSKs can be prederived in the device Secure Element and the vehicle hardware security module (HSM)/Secure Element while secure ranging isn't active but Bluetooth is connected. This avoids the need for a standard transaction to derive a new URSK in a time-critical situation. The prederived URSK can be transferred very quickly to the UWB radios of car and device to avoid interrupting the UWB ranging.

Adding transit and eMoney cards to Apple Wallet

In many global markets, users can add supported transit and eMoney cards to Apple Wallet on supported models of iPhone and Apple Watch. Depending on the operator, this may be done by transferring the value or commuter pass (or both) from a physical card into its digital Apple Wallet representation, or by provisioning a new transit or eMoney card from Apple Wallet or the card issuer's app. After transit cards are added to Apple Wallet, users can ride transit simply by holding their iPhone or Apple Watch near the transit reader. Some transit cards can also be used to make payments.

How transit and eMoney cards work

Added transit and eMoney cards are associated with a user's Apple Account. If the user adds more than one card to Apple Wallet, Apple or the card issuer may be able to link the user's personal information and the associated account information between cards. Transit and eMoney cards and transactions are protected by a set of hierarchical cryptographic keys.

During the process of transferring the balance from a physical card to Apple Wallet, users are required to enter card-specific information. Users may also need to provide personal information for proof of card possession. When transferring passes from iPhone to Apple Watch, both devices need to be online.

The balance can be recharged with funds from credit, debit, and prepaid cards through Apple Wallet or from the transit or eMoney card issuer's app. To understand the security of reloading the balance when using Apple Pay, see [Paying with cards within apps](#). To learn how the card is provisioned from within the card issuer's app, see [Adding credit or debit cards from a card issuer's app](#).

If provisioning from a physical card is supported, the transit or eMoney card issuer has the cryptographic keys needed to authenticate the physical card and verify the user's entered data. After the data is verified, the system can create a Device Account Number for the Secure Element and activate the newly added pass in Apple Wallet with the transferred balance. For some cards, after provisioning from the physical card is complete, the physical card is inactive.

At the end of either type of provisioning, if the card balance is stored on the device, it's encrypted and stored to a designated applet in the Secure Element. The operator has the keys to perform cryptographic operations on the card data for balance transactions.

By default, transit card users benefit from the seamless Express Transit experience that allows them to pay and ride without requiring Face ID, Touch ID, or a passcode. Information such as recently visited stations, transaction history, and additional tickets may be accessed by any nearby contactless card reader with Express Mode enabled. Users can turn on the Face ID, Touch ID, or the passcode authorization requirement in the Wallet & Apple Pay settings by turning off Express Transit. Express Mode isn't supported for eMoney cards.

As with other Apple Pay cards, users can suspend or remove eMoney cards by:

- Erasing the device remotely with Find My
- Enabling Lost Mode with Find My
- Entering a [device management service](#) remote wipe command
- Removing all cards from their Apple Account page
- Removing all cards from iCloud.com
- Removing all cards from Apple Wallet
- Removing the card in the issuer's app

Apple Pay servers notify the card operator to suspend or make those cards inactive. If a user removes a transit or eMoney card from an online device, the balance can be recovered by adding it back to a device signed in with the same Apple Account. If a device is offline, powered off, or unusable, recovery may not be possible.

Adding transit and eMoney cards to a family member's Apple Watch

On devices with iOS 15 and watchOS 8, or later, the organizer of an iCloud family can add transit and eMoney cards to their family members' Apple Watch devices through the Watch app on their iPhone. When provisioning one of these cards to a family member's Apple Watch, the watch is required to be nearby and connected to the organizer's iPhone using Wi-Fi or Bluetooth. Family members are required to have two-factor authentication enabled for their Apple Account for this to occur.

Family members can send a request to add money to a transit or eMoney card from their Apple Watch using iMessage. The content of the message is protected by end-to-end encryption, as described in [iMessage security overview](#). Adding money to a card on a family member's Apple Watch can be done remotely using a Wi-Fi or cellular connection. Proximity isn't required.

Note: This feature may not be available in all countries or regions.

Credit and debit cards

In some cities, transit readers accept EMV (smart) cards to pay for transit rides. When users present an EMV credit or debit card to those readers, user authentication is required, just as with "Pay with credit and debit cards in the stores."

On devices with iOS 12.3 and watchOS 5.2.1, or later, some existing EMV credit or debit cards in Apple Wallet can be enabled for Express Transit. Express Transit lets users pay for a trip at supported transit operators without requiring Face ID, Touch ID, or a passcode. When a user provisions an EMV credit or debit card, the first card provisioned to Apple Wallet is enabled for Express Transit. The user can tap the More button on the front of the card in Apple Wallet and turn off Express Transit for that card by setting Express Transit Settings to None. The user can also select a different credit or debit card as their Express Transit card using Apple Wallet. To reenable or select a different card for Express Transit, the user needs to use one of the following methods:

- Biometric authentication
- Device passcode
- Double-clicking the side button of an unlocked Apple Watch

Note: Apple Card and Apple Cash are eligible for Express Transit.

IDs in Apple Wallet

IDs in Apple Wallet

On iPhone 8 or later with iOS 15.4 or later and Apple Watch Series 4 or later with watchOS 8.4 or later, users in participating U.S. states can add their state ID or driver's license to Apple Wallet and tap their iPhone or Apple Watch to seamlessly and securely present it at participating locations.

On iPhone 8 or later with iOS 18.5 or later, users can add their Japanese My Number Card to Apple Wallet on iPhone, and with iOS 26.1 or later and watchOS 26.1 or later, users with a valid U.S. passport can create a Digital ID in Apple Wallet on iPhone or Apple Watch.

IDs in Apple Wallet use security features built into the hardware and software of the user's device to help protect their identity and help keep their personal information secure.

Add a driver's license or state ID to Apple Wallet

On iPhone, users can tap the Add button (+) at the top of the screen in Apple Wallet to begin adding their license or ID. If users have an Apple Watch paired at the time of setup, they are prompted to also add their driver's license or ID to their Apple Wallet on Apple Watch.

Users are first asked to use their iPhone to scan the front and back of their physical driver's license or state ID card. The iPhone evaluates the quality and type of images to help ensure that the images provided are acceptable by the state issuing authority. These identity card images are encrypted to the state issuing authority's key on the device and then sent to the state issuing authority.

To help ensure that the person adding the identity card to Apple Wallet is the same person to whom the identity card belongs, the user is asked to take a Live Photo or a selfie. In some cases, the user might be prompted to also complete a series of facial or head movements. These movements are evaluated by the user's device and by Apple to help reduce the risk of someone using a photograph, video, or mask to add someone else's ID to Apple Wallet. Results from the analysis of these movements are sent to the state issuing authority, but not the video of the movements themselves.

Before submitting the user's selfie to the state issuing authority, Apple servers and the user's device compare the selfie with the likeness of the person who performed the series of facial and head movements. After completing the comparison, the device encrypts the selfie and sends it to the state issuing authority, which compares it against their image on file for the ID.

Last, users are asked to perform a Face ID or Touch ID authentication. The user's device ties this single matched Face ID or Touch ID biometric to the state ID to help ensure that only the person who added the ID to this iPhone can present it; other enrolled biometric information can't be used to authorize presentation of the ID. This occurs strictly on device and isn't sent to the state issuing authority.

The state issuing authority will receive information necessary to set up the digital ID. This includes images of the front and back of the user's ID, data read from the PDF417 barcode as well as the selfie the user took as part of the ID verification process. The issuing state also receives a single-digit value, used to help prevent fraud, that's based on the user's device use patterns, settings data, and information about their personal Apple Account. It's then ultimately the issuing state's decision to approve or deny the ID being added to Apple Wallet.

After the state issuing authority authorizes adding the state ID or Driver's License to Apple Wallet, a key pair is generated in the Secure Element by iPhone that anchors the user's ID to that specific device. If adding to Apple Watch, a key pair is generated in the Secure Element by Apple Watch.

After the ID is on iPhone, the information reflected on the user's ID in Apple Wallet is stored in an encrypted format protected by the Secure Enclave.

Add a My Number Card to Apple Wallet

On iPhone, users can add their My Number Card to Apple Wallet through the Mynaportal app. My Number Card in Apple Wallet supports both mobile identity document and Japan Public Key Infrastructure (JPKI) functionality within a single ID pass. The setup process includes security measures enforced by Apple and the card issuer to help ensure that only the legitimate cardholder can add and present their ID.

In the Mynaportal app, users need to enter the 4-digit PIN and alphanumeric password associated with their physical My Number Card. These credentials are required to authenticate to the card, allowing the app to read data from the contactless integrated circuit (IC) which the issuer verifies to help ensure that the person adding it has legitimate access. At this time, users are also asked to input a PIN and password for use with the JPKI applet backed functionality of their My Number Card in Apple Wallet. These values never leave the user's device and are independent from the PIN and password for the user's physical card.

Note: Too many incorrect attempts on a PIN or password block the physical card and prevent it from being used or added to Apple Wallet until it's unlocked.

To help ensure that the person adding the My Number Card to Apple Wallet is the rightful owner, users are asked to take a selfie and perform a liveness check. The selfie and liveness data is evaluated by the issuer along with the physical card information to help reduce the risk of someone fraudulently adding another person's ID to Apple Wallet.

Users perform a Face ID or Touch ID authentication to tie a single biometric to the My Number Card in Apple Wallet. This helps ensure that only the same person who added a card to an iPhone can present it; other enrolled biometric information can't be used to either authorize presentation of the ID or perform JPKI user authentication operations.

The issuing authority validates all submitted evidence and makes the decision whether to authorize adding the My Number Card to Apple Wallet. If authorized, they create and digitally sign the mobile identity document to protect against tampering or forgery. A key pair generated by the iPhone in the Secure Element anchors the user's My Number Card in Apple Wallet to that specific device, ensuring it cannot be copied and used by someone else.

The JPKI applet, which protects the user's digital signing and user identification certificates, associated private keys, and PIN and password, is provisioned within the Secure Element. This ensures that cryptographic operations for digital signing in addition to PIN or password comparison operations are performed within the hardware-protected environment's boundary, and private keys are never available outside the Secure Element.

After the My Number Card is on iPhone, both the identity information reflected on the user's ID in Apple Wallet and the JPKI applet data are stored in an encrypted format protected by the Secure Enclave or Secure Element, respectively. The issuer ensures that users can only add one My Number Card to a device and that each unique identity can only be added to one device, providing additional protection against fraud.

Add a Digital ID to Apple Wallet

On iPhone, users can tap the Add button (+) at the top of the screen in Apple Wallet to begin creating their Digital ID. If users have an Apple Watch paired at the time of setup, they are prompted to also add their Digital ID to their Apple Wallet on Apple Watch.

Users are first asked to use their iPhone to scan the machine-readable portion of their physical passport photo page. If the data indicates that the passport is from a supported region, not expired, and eligible, the iPhone then guides the user to read the passport chip using NFC. The relevant machine-readable fields are used to establish a session with the chip, in accordance with ICAO 9303 specified protocols. Data read from the chip includes data elements like the document holder's name and portrait, along with the Document Security Object. To verify the data's authenticity, Apple servers validate the Document Security Object, including data group hashes, the issuing government's signature, and Document Signer Certificate using Passive Authentication. The Document Signer Certificate is validated against the corresponding Country Signing Certificate Authority Certificate to ensure authenticity and confirm it hasn't been revoked.

To help ensure that the person creating the Digital ID in Apple Wallet is the same person the passport belongs to, the user is asked to take a selfie and complete a liveness check. The device coaches the user to successfully complete the required steps, which may include a series of facial and head movements. The resulting selfie photo and liveness video are encrypted on device and uploaded for evaluation by Apple servers. The selfie photo is compared with the likeness of the person who performed the liveness check and against the validated portrait retrieved from the passport chip.

Users also need to perform a Face ID or Touch ID authentication to tie the biometric to the ID pass. This mechanism, supported by all IDs in Apple Wallet, and helps ensure that only the person who added the ID can present it.

Apple evaluates the submitted evidence including data read from the passport chip, the selfie and liveness video collected during the ID verification process, and a confidence assessment score, to make the decision to approve or deny adding the Digital ID to Apple Wallet. The confidence assessment score, used to help prevent fraud, is based on the user's device use patterns, settings data, and information about their personal Apple Account.

If approved, Apple creates and signs a Digital ID derived from the verified passport data. A key pair is generated in the Secure Element in the iPhone that anchors the user's ID to that specific device. If the user adds the ID to Apple Watch, a key pair is generated in the Secure Element in the Apple Watch. Although the expiry date and other identity data are common, the Digital ID in Apple Wallet is independent from the passport used to create it. This means that the user needs to delete the Digital ID from their devices and go through the steps to add a new one in any situation where their physical passport is revoked or reissued.

After creating the Digital ID, it's encrypted and sent to the device. After the device receives it, the information reflected on the user's Digital ID in Apple Wallet is reencrypted with a device specific key and stored and protected by the Secure Enclave.

Using an ID in Apple Wallet with an identity reader

To use their ID in Apple Wallet, users need to authenticate with the Face ID or Touch ID device associated with the ID in Apple Wallet before iPhone presents the information to the identity reader.

To use their ID in Apple Wallet on Apple Watch, users need to unlock their iPhone using the associated Face ID appearance or Touch ID fingerprint each time they put on their Apple Watch. Then, they can use their ID in Apple Wallet without authenticating until they take their Apple Watch off again. This capability leverages foundational Auto Unlock capabilities detailed in [Automatically unlock Apple devices](#).

When users hold their iPhone or Apple Watch near the identity reader, users see a prompt on device displaying which specific information is being requested, by whom, and if they intend on storing it. After authorizing with the associated Face ID or Touch ID on iPhone or double-clicking the side button on Apple Watch, the requested identity information is released from the device.

Important: Users don't need to show, or hand over their device to present their ID.

If users have an accessibility feature like Voice Control, Switch Control, or Assistive Touch instead of having Face ID or Touch ID enabled, they can use their passcode to access and present their information.

Transmission of identity data to the identity reader follows the ISO/IEC 18013-5 standard, which provides for multiple security mechanisms available that are able to detect, deter and mitigate security risks. These consist of identity data integrity and antiforgery, device binding, informed consent, and user data confidentiality over radio links.

Presentation history is stored on device and available for users to view and delete in Apple Wallet, and—for IDs on Apple Watch—in the Watch app on the paired iPhone. Each presentation record includes the fields requested, the geolocation of the presentation, and whether the business intends to store the data or not. For authenticated readers, the business's name and icon are also included.

Using an ID in Apple Wallet with iOS apps

Users can also share their ID information in Apple Wallet with iOS apps. When a user shares their ID with an app, Apple Wallet fetches and validates an encryption certificate that's registered with the app developer. This certificate is used to encrypt the information that the user has agreed to share.

A presentation sheet displays which specific information is being requested, by which app, if the app intends on storing it and for how long, and the reason for requesting it. After authorizing with the associated Face ID or Touch ID, the requested information is encrypted by Apple Wallet using HPKE and is never made available to Apple. Apple Wallet periodically queries Apple servers to verify that the ID authentication key hasn't been revoked and that the ID is still valid. If no check has been performed recently, one may occur when the user shares their ID with an app.

Presentation history is stored on device and available for users to view and delete in Apple Wallet. Each presentation record includes the app, reason, fields requested, and whether it intended to store the data and for how long.

Using an ID in Apple Wallet on websites

Users can share their ID information from Apple Wallet with websites. Participating websites use the W3C Digital Credentials API to request identity information from mobile document providers. To support acceptance of IDs in Apple Wallet, in addition to the document type and data elements, the request needs to contain an anti-replay value, encryption information for response encryption, and a signature for the device to authenticate the request. The website's server generates an encryption key pair for the request and is responsible for securely retaining the private key for response decryption. Apple Wallet utilizes the reader authentication mechanism to authenticate the request (as defined by ISO/IEC 18013-5 and ISO/IEC 18013-7 Annex C), leveraging a signing certificate the website owner obtains from Apple. It also performs domain validation on the requesting website to help ensure the request is coming from a valid source.

A presentation sheet displays which specific information is being requested, by which website, if the website intends on storing it, and the reason for requesting it. After the user authorizes presentation of the requested identity information, it is encrypted by Apple Wallet directly to the website's server using HPKE and is never made available to Apple nor the browser. Apple Wallet periodically queries Apple servers to verify that the ID authentication key hasn't been revoked and the ID is still valid. If no check has been performed recently, one may occur when the user shares their ID with a website.

Upon receiving the encrypted response, the website's server needs to verify the authenticity of the identity data. This includes performing issuer authentication by validating the document signer certificate, verifying the cryptographic signature from the issuer, and checking hashes over the requested data elements. Additionally, mobile document authentication needs to be performed to ensure the document originated from the specific device it was issued to, preventing unauthorized duplication.

To allow users to present their ID in Apple Wallet on iPhone to other devices where they may want to verify their identity on the web, both handoff between Apple devices and cross-platform presentment are supported.

Handoff between Apple devices allows identity presentation to a website on an iPad or Mac using an ID in Apple Wallet on iPhone. In this case the system:

- Verifies that the iPhone is associated with the same personal Apple Account.
- Supports the functionality.
- Is compatible with the request.

A secure connection is then established between the devices, and a notification appears on the iPhone, allowing the user to view and authorize the request from the iPhone. Cross-platform support also allows users to present their ID in Apple Wallet to other standards compliant devices and browsers. This leverages the FIDO CTAP protocol to protect the integrity and confidentiality of the communication channel for request and response data transfer between devices.

With both handoff between Apple devices and cross-platform support, the same security capabilities as described above exist for request validation, response data encryption, identity data authentication, and device authentication.

Presentation history is stored on device and available for users to view and delete in Apple Wallet. Each presentation record includes the website, reason, fields requested, and whether the website intended to store the data.

Security of IDs in Apple Wallet

The following features help increase the security of using IDs in Apple Wallet.

Identity data integrity and antiforgery

IDs in Apple Wallet use an issuer-provided signature to allow any ISO/IEC 18013-5 compliant reader to verify a user's ID in Apple Wallet. In addition, all data elements on ID in Wallet are individually protected against forgery. This allows the identity reader to request a specific subset of the data elements present on the ID in Apple Wallet and for the ID in Apple Wallet to respond with that same subset, thus only sharing the requested data and maximizing the user's privacy.

Device binding

IDs in Apple Wallet authentication use a device signature to protect against the cloning of an ID and the replay of an identity presentation. Apple Wallet stores the private key for ID authentication in the iPhone device's Secure Element, so the ID is bound to the same device that the state issuing authority created the ID for.

Informed consent

IDs in Apple Wallet may use authentication to identify the reader using the protocol defined in the ISO/IEC 18013-5 standard. During presentation, if the reader has its own certificate that's trusted by Apple Wallet, an icon is shown to them to give the user an assurance that they're interacting with the intended party.

User data confidentiality over radio links

Session encryption helps ensure that all personally identifiable information (PII) exchanged between the ID in Apple Wallet and that the identity reader is encrypted. Encryption is performed by the application layer. The security of session encryption is therefore not reliant on the security provided by the transmission layer (for example, NFC, Bluetooth, and Wi-Fi).

IDs in Apple Wallet help keep users' information private

Apple doesn't have access to the identity data stored in IDs in Apple Wallet. Identity data is only released from a device when a user explicitly authorizes each presentation. The following features further help protect users' privacy during identity presentations.

Device retrieval

IDs in Apple Wallet adhere to the "device retrieval" process outlined in ISO/IEC 18013-5. Device retrieval obviates the need to make server calls during presentation, thereby protecting users from being tracked by Apple and the issuer.

Unlinkable presentation

As described in [Identity data integrity and antiforgery](#), elements like the issuer-provided signature are included in the Mobile Security Object (MSO) for a reader to verify an ID. In order to reduce traceability of these static elements and help prevent tracking ID presentations between relying parties, the device manages multiple ID authentication keys for each ID in Apple Wallet. The ID issuer generates unique MSO payloads during ID issuance and updates for each of these ID authentication keys which are sent to and stored encrypted on the user's device. During presentation, the device selects a random ID authentication key and its associated MSO to use for the transaction. This is beneficial when the user is presenting non-identifiable information, such as `age_over_18` or other age elements, as it reduces the potential for multiple parties to reliably match elements and identify a specific user. To reduce overhead, keys may be reused in some situations, such as during presentation to the same relying party or when presentation already includes inherently linkable information. When all ID authentication keys are used at least once, if the issuer has opted in to support early rotation, the device initiates the process to generate a new batch of keys and request refreshed payloads.

ID verifier security

On an iPhone with iOS 17 or later, U.S. businesses and organizations can use iPhone to seamlessly and securely read ISO/IEC 18013-5 or ISO/IEC 23220 compliant mobile IDs in person—without the need for external hardware. ID Verifier can be used in two different ways, depending on the verification use case:

- *ID Verifier Display Only*: This enables use of an iOS user interface to display Name, Age, ID Photo, and Age Over N data for use cases that only require visual confirmation. It is also designed to prevent all screenshot and screen-recording features from capturing the displayed information. This service doesn't permit *personally identifiable information (PII)* collection that can be tied back to the presenter.
- *ID Verifier Data Transfer*: This enables apps to request additional data elements, such as date of birth and address, in order to meet legal verification requirements. Access to the ID Verifier Data Transfer API is managed with entitlements, and apps need to conform to requirements with respect to how data is used. For example, apps need to demonstrate a legal requirement to request identity data. Apps are also required to maintain a privacy policy that details its processing, storage, or other use of the requested identity data.

Reading a mobile ID

ID Verifier follows the protocol defined in the ISO/IEC 18013-5 standard. When an app using the ID Verifier API requests to read a mobile ID, a sheet—controlled by iOS—is displayed and prompts the mobile ID holder to hold their device near the identity reader. That initial NFC engagement (as defined by the ISO/IEC 18013-5 standard, a QR code can be used to initiate a Bluetooth handover process instead of NFC) establishes a secure Bluetooth Low Energy (BLE) connection between both devices. At that point, the mobile ID holder can review on their device the information that's being requested. After the mobile ID holder consents, the requested identity data is transferred to the reading device. Apps using the ID Verifier Data Transfer API receive the response data for processing, while apps using the ID Verifier Display Only API see data displayed by iOS directly.

The ISO/IEC 18013-5 standard provides for multiple security mechanisms to detect, deter, and mitigate security risks. Among those, ID Verifier performs both issuer signature and device signature validation. On an iPhone with iOS 26.4 or later, ID Verifier also supports Message Authentication Code (MAC) based device authentication as defined in the standard, which uses a key agreement mechanism to verify that identity data originates from the intended device. Additionally, ID Verifier supports reader authentication using the protocol defined in the ISO/IEC 18013-5 standard. Apps can choose to display an icon and name to deliver assurance that the ID holder is interacting with the intended party using the reader's certificate.

Issuer and device validation

As a protection against forgery, ID Verifier validates the signature of the Mobile Security Object by the trusted issuer of the mobile identity. ID Verifier Data Transfer also provides an API that enables apps to conduct their own signature validation, instead of iOS, if desired. To deliver an assurance to the business or organization that the mobile ID has not been copied from one device to another, ID Verifier validates the device signature or MAC over session data.

Reader authentication

At the time of presentment, the ID Verifier reader request is signed by the private key associated with the reader authentication certificate that chains up to the Apple Root certificate authority (CA), which contains relevant x509 custom extensions to indicate to the holder if the business intends to store the data. If an application would like to display name and icon to the ID holder, the app administrator is required to register using Apple Business Register and provide accurate branding information. After the submitted information is successfully verified, at the time of a transaction, the reader authentication certificate provides the ID holder with the information about the entity from Apple Register via the reader authentication certificate.

iMessage

iMessage security overview

Apple iMessage is a messaging service for iPad, iPhone, Mac, Apple Watch, and Apple Vision Pro. Relying on the [Apple Push Notification service \(APNs\)](#), iMessage lets users send texts and attachments like photos, contacts, locations, links, and emoji. Messages sync across all devices, enabling seamless conversations. Apple doesn't store message content or attachments, which are all secured with end-to-end encryption so that no one but the sender and receiver can access them. Apple can't decrypt the data.

When a user turns on iMessage on a device, the device generates encryption and signing pairs of keys for use with the service. The public keys are sent to [Apple Identity Service \(IDS\)](#), where they are associated with the user's phone number or email address, along with the device's APNs address.

As users enable additional devices for use with iMessage, their encryption and signing public keys, APNs addresses, and associated phone numbers are added to the directory service. Users can also add more email addresses, which are verified by sending a confirmation link. Phone numbers are verified by the carrier network and SIM. With some networks, this requires using SMS (the user is presented with a confirmation dialog if the SMS isn't zero rated). Phone number verification may be required for several system services in addition to iMessage, such as FaceTime and iCloud. All of the user's registered devices display an alert message when a new device, phone number, or email address is added.

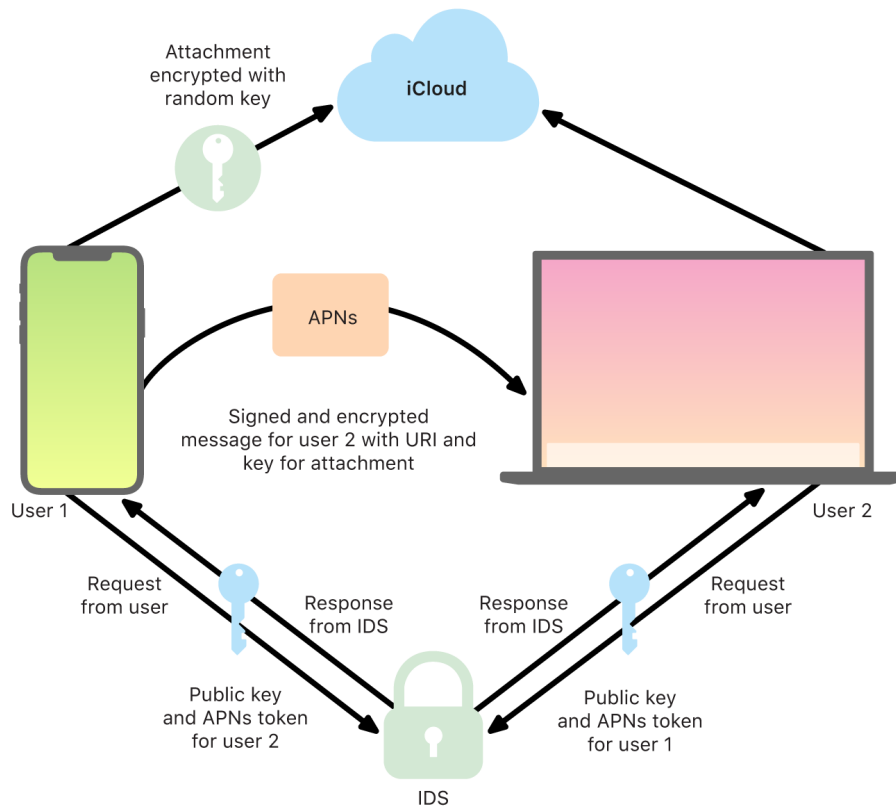
How iMessage sends and receives messages securely

Users start a new iMessage conversation by entering an address or name. If they enter a phone number or email address, the device contacts the [Apple Identity Service \(IDS\)](#) to retrieve the public keys and APNs addresses for all of the devices associated with the addressee. If the user enters a name, the device first uses the user's Contacts app to gather the phone numbers and email addresses associated with that name and then gets the public keys and APNs addresses from IDS.

The user's outgoing message is individually encrypted for each of the receiver's devices. The public encryption keys and signing keys of the receiving devices are retrieved from IDS. For each receiving device, the sending device generates a random 88-bit value and uses it as an [HMAC-SHA-256](#) key to construct a 40-bit value derived from the sender and receiver public key and the plaintext. The concatenation of the 88-bit and 40-bit values makes a 128-bit key, which encrypts the message with it using AES in Counter (CTR) Mode. The 40-bit value is used by the receiver side to verify the integrity of the decrypted plaintext. This per-message AES key is encrypted using RSA-OAEP to the public key of the receiving device. The combination of the encrypted message text and the encrypted message key is then hashed with SHA-1, and the hash is signed with the [Elliptic Curve Digital Signature Algorithm \(ECDSA\)](#) using the sending device's private signing key. On an iPhone with iOS 13 or later and an iPad with iPadOS 13.1 or later, devices may use an Elliptic Curve Integrated Encryption Scheme (ECIES) encryption instead of RSA encryption.

The resulting messages, one for each receiving device, consist of the encrypted message text, the encrypted message key, and the sender's digital signature. They are then dispatched to the APNs for delivery. Metadata, such as the timestamp and APNs routing information, isn't encrypted. Communication with APNs is encrypted using a forward-secret TLS channel.

APNs can only relay messages up to 4 or 16 KB in size, depending on the iOS or iPadOS version. If the message text is too long or if an attachment such as a photo is included, the attachment is encrypted using AES in CTR mode with a randomly generated 256-bit key and uploaded to iCloud. The AES key for the attachment, its [Uniform Resource Identifier \(URI\)](#), and an SHA-1 hash of its encrypted form are then sent to the recipient as the contents of an iMessage, with their confidentiality and integrity protected through normal iMessage encryption, as shown in the following diagram.



For group conversations, this process is repeated for each recipient and their devices.

On the receiving side, each device receives its copy of the message from APNs and, if necessary, retrieves the attachment from iCloud. The incoming phone number or email address of the sender is matched to the receiver's contacts so that a name can be displayed when possible.

As with all push notifications, the message is deleted from APNs when it's delivered. Unlike other APNs notifications, however, iMessage messages are queued for delivery to offline devices. Messages are stored on Apple servers for up to 30 days.

Check In security

Check In allows a user to choose a contact that's notified if the user's supported device doesn't arrive at a predetermined destination. When the user chooses a contact to start Check In with, an iMessage is sent to that contact with details about the session including their destination. The initiating user's device also generates an access token and Check In encryption key. The Check In encryption key is sent in a second iMessage; however, this message is held by iMessage servers rather than being immediately delivered. Because this message is end-to-end encrypted between the user and their designated contact, its contents are not available to Apple.

While the initiating user's device is making progress toward its destination, it periodically sends a heartbeat message to iMessage servers, which extends the expiry time before the Check In encryption key message is delivered. If the device is turned off or loses connectivity, iMessage servers automatically release the iMessage containing the encryption key to the contact when the expiry time is reached. The Check In encryption key message can also be released if the device isn't making progress toward its destination. When released in this fashion, the access token is included.

During a Check In session, the initiating user's device periodically collects relevant Check In data (for example, location and battery level), encrypts it with the Check In key, and uploads it to iCloud. Apple doesn't have access to the Check In key and can't access the uploaded data.

Ending a Check In session requires user authentication. If the user cancels a session, the encrypted data and iMessage containing the Check In encryption key are securely deleted from the server.

If the Check In device hasn't arrived at its destination as expected, or the configured timer expires, the chosen contact receives the iMessage containing the Check In encryption key. There are two scenarios where this could happen:

- *After a loss of connectivity:* In this case, the designated contact doesn't have the access token and the contact's device performs a second check to determine whether enough time has elapsed since the last heartbeat. The contact's device requests the encrypted Check In data from the server, and the server performs a third check to further confirm if sufficient time has passed since the last heartbeat. If so, the server provides the contact with the encrypted Check In data, and the contact can decrypt it with their Check In key.
- *After the user's device determines they aren't making progress toward their destination:* In this case, unless the user cancels or extends the Check In when prompted, the designated contact is provided both the access token and Check In encryption key. The contact's device provides the access token to the server, allowing it to download the encrypted Check In data. The data can then be decrypted with the Check In key.

Secure iMessage name and photo sharing

iMessage Name and Photo Sharing allows a user to share a Name and Photo using iMessage. The user may select their My Card information, or customize the name and include any image they choose. iMessage Name and Photo sharing uses a two-stage system to distribute the name and photo.

The data is subdivided in fields, each encrypted and authenticated separately as well as authenticated together with the process below. There are three fields:

- Name
- Photo
- Photo filename

One of the first steps in creating the data is to randomly generate a record 128-bit key on the device. This record key is then derived with HKDF-HMAC-SHA-256 to create three subkeys: Key 1:Key 2:Key 3 = HKDF(record key, "nicknames"). For each field, a random 96-bit Initialization Vector (IV) is generated and the data is encrypted using AES-CTR and Key 1. A message authentication code (MAC) is then computed with HMAC-SHA-256 using Key 2 and covering the field name, the field IV, and the field ciphertext. Finally, the set of individual field MAC values are concatenated and their MAC is computed with HMAC-SHA-256 using Key 3. The 256-bit MAC is stored along side the encrypted data. The first 128 bits of this MAC is used as RecordID.

This encrypted record is then stored in the CloudKit public database under the RecordID. This record is never mutated, and when the user chooses to change their name and photo, a new encrypted record is generated each time. When user 1 chooses to share their name and photo with user 2, they send the record key along with the RecordID inside their iMessage payload, which is [encrypted](#).

When user 2's device receives this iMessage payload, it notices that the payload contains a Nickname and Photo RecordID and key. User 2's device then goes out to the public CloudKit database to retrieve the encrypted name and photo at the record ID and sends it across using iMessage.

After the message is retrieved, user 2's device decrypts the payload and verifies the signature using the RecordID itself. If this passes, user 2 is presented with the name and photo and they can choose to add this to their contacts, or use it for Messages.

Secure Apple Messages for Business

Apple Messages for Business is a messaging service that allows users to communicate with businesses using the Messages app. With Apple Messages for Business, the user is always in control of the conversation. They can also delete the conversation and block the business from messaging them in the future. For privacy, the business doesn't receive the user's phone number, email address, or Apple Account information. Instead, a custom unique identifier called the *Opaque ID* is generated by the [Apple Identity Service \(IDS\)](#) and shared with the business. The Opaque ID is unique to the relationship between the user's Apple Account and the business's Business ID. A user has a different Opaque ID for every business they contact using Apple Messages for Business. The user decides if and when to share personal identifying information with the business and Apple Messages for Business service never stores conversation history.

Apple Messages for Business supports Managed Apple Accounts from [Apple Business](#) and determines whether they are turned on for iMessage and FaceTime in [Apple School Manager](#).

Messages sent to the business are encrypted between the user's device and Apple's messaging servers, using the same security and Apple messaging servers as iMessages. Apple messaging servers decrypt these messages in RAM, and relay them to the business over an encrypted link using TLS 1.2. Messages are never stored in unencrypted form while transiting through the Apple Messages for Business service. Businesses' replies are also sent using TLS 1.2 to the Apple messaging servers, where they are encrypted using the unique public keys of each recipient device.

If user devices are online, the message is delivered immediately and isn't cached on the Apple messaging servers. If a user's device isn't online, the encrypted message is cached for up to 30 days to enable the user to receive it when the device is back online. As soon as the device is back online, the message is delivered and deleted from cache. After 30 days, an undelivered cached message expires and is permanently deleted.

FaceTime security

FaceTime is Apple's video and audio calling service. Like iMessage, FaceTime calls use the [Apple Push Notification service \(APNs\)](#) to establish an initial connection to the user's registered devices. The audio/video contents of FaceTime calls are protected by end-to-end encryption, so no one but the sender and receiver can access them. Apple can't decrypt the data.

The initial FaceTime connection is made through an Apple server infrastructure that relays data packets between the users' registered devices. Using APNs notifications and Session Traversal Utilities for NAT (STUN) messages over the relayed connection, the devices verify their identity certificates and establish a shared secret for each session. The shared secret is used to derive session keys for media channels streamed using the Secure Real-time Transport Protocol (SRTP). SRTP packets are encrypted using AES-256 in Counter Mode and authenticated with HMAC-SHA-1. After the initial connection and security setup, FaceTime uses STUN and Internet Connectivity Establishment (ICE) to establish a peer-to-peer connection between devices, if possible.

Group FaceTime extends FaceTime to support up to 33 concurrent participants. As with classic one-to-one FaceTime, calls are end-to-end encrypted among the invited participants' devices. Even though Group FaceTime reuses much of the infrastructure and design of one-to-one FaceTime, these group calls feature a key-establishment mechanism built on top of the authenticity provided by [Apple Identity Service \(IDS\)](#). This protocol provides forward secrecy, meaning that the compromise of a user's device won't leak the contents of past calls. Session keys are wrapped using AES-SIV and are distributed among participants using an Elliptic Curve Integrated Encryption Scheme (ECIES) construction with ephemeral P-256 ECDH keys.

When a new phone number or email address is added to an ongoing Group FaceTime call, active devices establish new media keys and never share previously used keys with the newly invited devices.

SharePlay security

SharePlay is built on top of the end-to-end encrypted infrastructure of FaceTime, so the data exchanged between participants during a SharePlay activity is protected such that no one but the participants can access it. Apple can't decrypt the data.

SharePlay uses the [Apple Push Notification service \(APNs\)](#) for signaling, reusing the same push channel FaceTime establishes between registered devices, and traffic is carried over the same end-to-end encrypted data channel used by FaceTime. The key material protecting SharePlay activity traffic is re-established when the call's membership changes, so that newly added members can't decrypt earlier traffic and removed members are unable to decrypt subsequent traffic. Unlike the session keys protecting Group FaceTime media, SharePlay keys aren't rotated temporally and persist for the duration of the activity.

To keep concurrent or sequential activities on the same call cryptographically isolated, SharePlay never uses this key material directly. Each activity instead derives its own key by diversifying the shared key material with the activity's unique identifier, so apps using SharePlay can't decrypt one another's traffic. Starting a new activity produces a new identifier and therefore a different key. Participant identifiers are similarly diversified per activity, so one activity can't correlate its participants with those of another app, even when the underlying FaceTime membership is identical.

Nearby sharing security

Nearby sharing is a visionOS feature that allows people to share content from their apps with other nearby visionOS users. Nearby sharing is set to Contacts Only by default, which means both parties need to be in each other's Contacts to allow automatic discovery. Non-contacts can pair using a manually-exchanged PIN displayed on the initiator's device and entered on the joiner's. Users can choose to turn off the feature entirely by deselecting "Allow Nearby People" in Settings or Control Center.

When a user chooses to share content, a secure [SharePlay](#) session is established and used for sharing the content between the participants. Content actively being shared in a Nearby Sharing session is highlighted to the user with a green visual indicator.

To allow virtual objects to be placed in the same location for all users, Collaborative Map Building (CMB) takes place when devices first connect. They exchange information about reference objects within their environment to establish a shared coordinate system for virtual object placement. As CMB can be used to infer location, it's initially performed only after a user accepts an invitation to join a Nearby Sharing session with a given participant. However, for up to 45 days after that initial invite is accepted, CMB runs opportunistically for subsequent invites that occur in that same location with that participant. CMB data is also transmitted using end-to-end encryption, and map merging happens locally on each device; no centralized spatial map is ever stored or shared beyond the participants.

Find My

Find My security

The Find My app for Apple devices is built on a foundation of advanced public key cryptography.

Overview

The Find My app combines Find My iPhone and Find My Friends into a single app in iOS, iPadOS, and macOS. Find My can help users locate a missing device, even an offline Mac. An online device can simply report its location to the user via iCloud. Find My works offline by sending out short range Bluetooth signals from the missing device that can be detected by other Apple devices in use nearby. Those nearby devices then relay the detected location of the missing device to iCloud so users can locate it in the Find My app—all while protecting the privacy and security of all the users involved. Find My even works with a Mac that's offline and asleep.

Using Bluetooth and the hundreds of millions of iPad, iPhone, and Mac devices in active use around the world, a user can locate their missing device even if it can't connect to a Wi-Fi or cellular network. Any iPad, iPhone, or Mac with "offline finding" turned on in Find My settings can act as a "finder device." This means the device can detect the presence of another missing offline device using Bluetooth and then use its network connection to report an approximate location back to the owner. When a device has offline finding enabled, it also means that it can be located by other participants in the same way. This entire interaction is end-to-end encrypted, anonymous, and designed to be battery and data efficient. There is minimal impact on battery life and cellular data plan usage, and user privacy is better protected.

Note: Find My may not be available in all countries or regions.

End-to-end encryption

Find My is built on a foundation of advanced public key cryptography. When offline finding is turned on in Find My settings, an elliptic curve (EC) P-224 private encryption key pair noted $\{d, P\}$ is generated directly on the device where d is the private key and P is the public key. Additionally, a 256-bit secret SK_0 and a counter i is initialized to zero. This private key pair and the secret are never sent to Apple and are synced only among the user's other devices in an end-to-end encrypted manner using iCloud Keychain. The secret and the counter are used to derive the current symmetric key SK_i with the following recursive construction: $SK_i = \text{KDF}(SK_{i-1}, \text{"update"})$.

Based on the key SK_i , two large integers u_i and v_i are computed with $(u_i, v_i) = \text{KDF}(SK_i, \text{"diversify"})$. Both the P-224 private key denoted d and corresponding public key referred to as P are then derived using an affine relation involving the two integers to compute a short-lived key pair: The derived private key is d_i , where $d_i = u_i * d + v_i$ (modulo the order of the P-224 curve) and the corresponding public part is P_i and verifies that $P_i = u_i * P + v_i * G$.

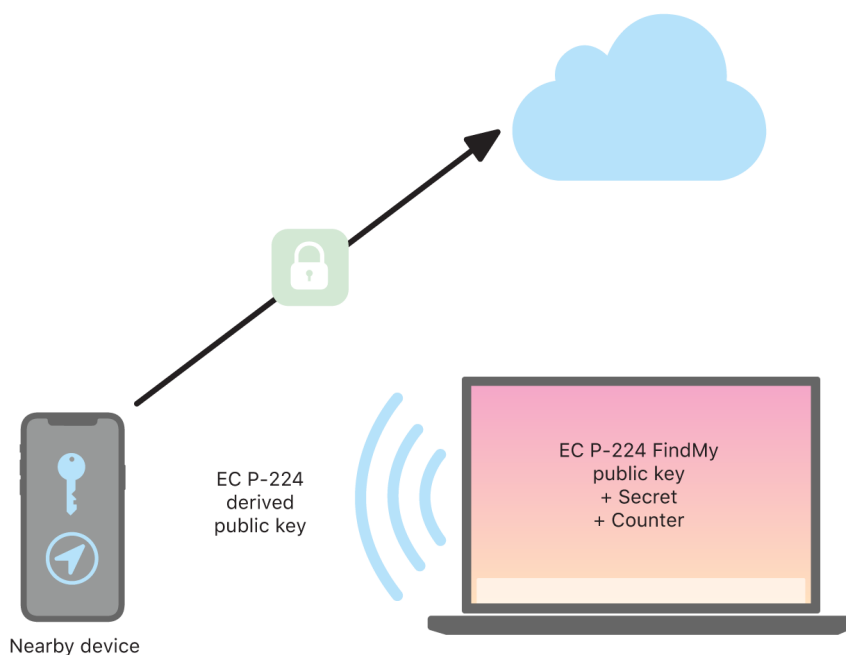
When a device goes missing and can't connect to Wi-Fi or cellular—for example, a MacBook Pro is left on a park bench—it begins periodically broadcasting the derived public key P_i for a limited period of time in a Bluetooth payload. By using P-224, the public key representation can fit into a single Bluetooth payload. The surrounding devices can then help in the finding of the offline device by encrypting their location to the public key. Approximately every 15 minutes, the public key is replaced by a new one using an incremented value of the counter and the process above so that the user can't be tracked by a persistent identifier. The derivation mechanism is designed to prevent the various public keys P_i from being linked to the same device.

Keeping users and devices anonymous

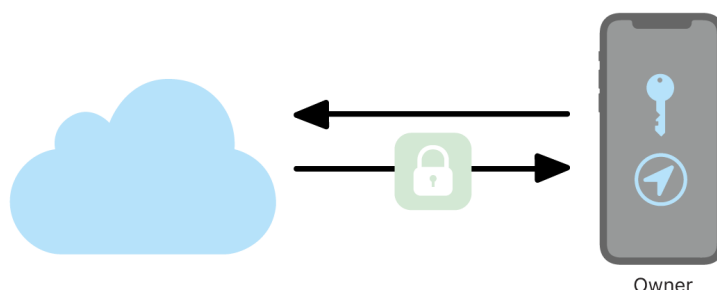
In addition to making sure that location information and other data are fully encrypted, participants' identities remain private from each other and from Apple. The traffic sent to Apple by finder devices contains no authentication information in the contents or headers. As a result, Apple doesn't know who the finder is or whose device has been found. Further, Apple doesn't log information that would reveal the identity of the finder and retains no information that would allow anyone to correlate the finder and owner. The device owner receives only the encrypted location information that's decrypted and displayed in the Find My app with no indication as to who found the device.

Using Find My to locate missing Apple devices

Any Apple devices within Bluetooth range that have offline finding enabled can detect a signal from another Apple device configured to allow Find My and read the current broadcast key P_i . Using an ECIES construction and the public key P_i from the broadcast, the finder devices encrypt their current location information and relay it to Apple. The encrypted location is associated with a server index which is computed as the SHA-256 hash of the P-224 public key P_i obtained from the Bluetooth payload. Apple never has the decryption key, so Apple can't read the location encrypted by the finder. The owner of the missing device can reconstruct the index and decrypt the encrypted location.



When trying to locate the missing device, an expected range of counter values is estimated for the location search period. With the knowledge of the original private P-224 key d and secret values SK_i in the range of counter values of the search period the owner can then reconstruct the set of values $\{d_i, \text{SHA-256}(P_i)\}$ for the entire search period. The owner device used to locate the missing device can then perform queries to the server using the set of index values $\text{SHA-256}(P_i)$ and download the encrypted locations from the server. The Find My app then locally decrypts the encrypted locations with the matching private keys d_i and shows an approximate location of the missing device in the app. Location reports from multiple finder devices are combined by the owner's app to generate a more precise location.



Locating devices that are offline

If a user has Find My iPhone enabled on their device, offline finding is turned on by default when they upgrade a device to iOS 13, iPadOS 13.1, and macOS 10.15, or later. This is designed to ensure that every user has the best possible chance to locate their device if it goes missing. However, if at any time the user prefers not to participate, they can turn off offline finding in Find My settings on their device. When offline finding is turned off, the device no longer acts as a finder nor is it detectable by other finder devices. However, the user can still locate the device as long as it can connect to a Wi-Fi or cellular network.

When a missing offline device is located, the user receives a notification and email message to let them know the device has been found. To view the location of the missing device, the user opens the Find My app and selects the Devices tab. Rather than showing the device on a blank map as it would have prior to the device being located, Find My shows a map location with an approximate address and information on how long ago the device was detected. If more location reports come in, the current location and time stamp both update automatically. Although users can't play a sound on an offline device or erase it remotely, they can use the location information to retrace their steps or take other actions to help them recover it.

Continuity

Continuity security overview

Continuity takes advantage of technologies like iCloud, Bluetooth, and Wi-Fi to enable users to continue an activity from one device to another, make and receive phone calls, send and receive text messages, and share a cellular internet connection.

Handoff security

Apple handles handoffs securely, whether from one device to another, between a native app and a website—even handoffs of large amounts of data.

How Handoff works securely

With Handoff, when a user's iOS, iPadOS, and macOS devices are near each other, the user can automatically pass whatever they're working on from one device to the other. Handoff lets the user switch devices and instantly continue working.

When a user signs in to iCloud on a second Handoff-capable device, the two devices establish a Bluetooth Low Energy (BLE) 4.2 pairing out-of-band using APNs. The individual messages are encrypted much like messages in iMessage are. After the devices are paired, each device generates a symmetric 256-bit AES key that gets stored in the device's [keychain](#). This key can encrypt and authenticate the BLE advertisements that communicate the device's current activity to other iCloud paired devices using AES-256 in GCM mode, with replay protection measures.

The first time a device receives an advertisement from a new key, it establishes a BLE connection to the originating device and performs an advertisement encryption key exchange. This connection is secured using standard BLE 4.2 encryption as well as encryption of the individual messages, which is similar to how iMessage is encrypted. In some situations, these messages are sent using APNs instead of BLE. The activity payload is protected and transferred in the same way as an iMessage.

Handoff between native apps and websites

Handoff allows an iOS, iPadOS, or macOS native app to resume user activity on a webpage in domains legitimately controlled by the app developer. It also allows the native app user activity to be resumed in a web browser.

To help prevent native apps from claiming to resume websites not controlled by the developer, the app needs to demonstrate legitimate control over the web domains it wants to resume. Control over a website domain is established using the mechanism for shared web credentials. For details, see [App access to saved passwords](#). iOS, iPadOS, and macOS need to validate an app's domain name control before the app is permitted to accept user activity Handoff.

The source of a webpage Handoff can be any browser that has adopted the Handoff APIs. When the user views a webpage, the system advertises the domain name of the webpage in the encrypted Handoff advertisement bytes. Only the user's other devices can decrypt the advertisement bytes.

On a receiving device, the system detects that an installed native app accepts Handoff from the advertised domain name and displays that native app icon as the Handoff option. When launched, the native app receives the full URL and the title of the webpage. No other information is passed from the browser to the native app.

In the opposite direction, a native app may specify a fallback URL when a Handoff receiving device doesn't have the same native app installed. In this case, the system displays the user's default browser as the Handoff app option (if that browser has adopted Handoff APIs). When Handoff is requested, the browser is launched and given the fallback URL provided by the source app. There is no requirement that the fallback URL be limited to domain names controlled by the native app developer.

Handoff of larger data

In addition to using the basic feature of Handoff, some apps may elect to use APIs that support sending larger amounts of data over Apple-created peer-to-peer Wi-Fi technology (much like AirDrop). For example, the Mail app uses these APIs to support handoff of a mail draft, which may include large attachments.

When an app uses these API's, the exchange between the two devices starts off just as in Handoff. But, after receiving the initial payload using Bluetooth Low Energy (BLE), the receiving device initiates a new connection over Wi-Fi. This connection is encrypted (with TLS), and it derives trust through an identity shared through iCloud Keychain. The identity in the certificates is verified against the user's identity. Further payload data is sent over this encrypted connection until the transfer is complete.

Universal Clipboard

Universal Clipboard leverages Handoff to securely transfer the content of a user's clipboard across devices so they can copy on one device and paste on another. Content is protected in the same way as other Handoff data and is shared by default with Universal Clipboard unless the app developer chooses to disallow sharing.

Apps have access to clipboard data regardless of whether the user has pasted the clipboard into the app. With Universal Clipboard, this data access extends to apps on the user's other devices (as established by their iCloud sign-in).

iPhone cellular call relay security

When a user's Mac, iPad, or HomePod is on the same Wi-Fi network as their iPhone, it can make and receive phone calls using the cellular connection on iPhone. Configuration requires the devices to be signed in to both iCloud and FaceTime using the same Apple Account.

When an incoming call arrives, all configured devices are notified using the [Apple Push Notification service \(APNs\)](#), with each notification using the same end-to-end encryption as iMessage. Devices that are on the same network present the incoming call notification user interface. When the user answers the call, the audio is seamlessly transmitted from the user's iPhone using a secure peer-to-peer connection between the two devices.

When a call is answered on one device, ringing of nearby iCloud-paired devices is terminated by briefly advertising using Bluetooth Low Energy (BLE). The advertising bytes are encrypted using the same method as Handoff advertisements.

Outgoing calls are also relayed to iPhone using APNs, and audio is similarly transmitted over the secure peer-to-peer link between devices. Users can turn off phone call relay on a device by turning off iPhone Cellular Calls in FaceTime settings.

iPhone Text Message Forwarding security

Text Message Forwarding automatically sends SMS text messages received on an iPhone to a user's enrolled iPad or Mac. Each device needs to be signed in to the iMessage service using the same Apple Account. When Text Message Forwarding is turned on, enrollment is automatic on devices within a user's circle of trust if two-factor authentication is enabled. Otherwise, enrollment is verified on each device by entering a random six-digit numeric code generated by iPhone.

After devices are linked, iPhone encrypts and forwards incoming SMS text messages to each device, utilizing the methods described in [iMessage security overview](#). Replies are sent back to iPhone using the same method, and then iPhone sends the reply as a text message using the carrier's SMS transmission mechanism. Text Message Forwarding can be turned on or off in Messages settings.

Instant Hotspot security

Instant Hotspot connects other Apple devices to a personal iPad or iPhone hotspot. iPad and iPhone devices that support Instant Hotspot use Bluetooth Low Energy (BLE) to discover and communicate to all devices that have signed in to the same individual personal Apple Account or other personal Apple Accounts used with Family Sharing (in iOS 13 and iPadOS 13.1). Compatible Mac computers with OS X 10.10 or later use the same technology to discover and communicate with Instant Hotspot iPad and iPhone devices.

Initially, when a user enters Wi-Fi settings on a device, it emits a BLE advertisement containing an identifier that all devices signed in to the same Apple Account agree upon. The identifier is generated from a DSID (Destination Signaling Identifier) that's tied to the Apple Account and rotated periodically. When other devices signed in to the same Apple Account are in close proximity and support Personal Hotspot, they detect the signal and respond, indicating the availability to use Instant Hotspot.

When a user who isn't part of Family Sharing chooses an iPad or iPhone for Personal Hotspot, a request to turn on Personal Hotspot is sent to that device. The request is sent across a link that's encrypted using BLE encryption, and the request is encrypted in a fashion similar to iMessage encryption. The device then responds across the same BLE link using the same per-message encryption with Personal Hotspot connection information.

For users that are part of Family Sharing, Personal Hotspot connection information is securely shared using a mechanism similar to that used by HomeKit devices to sync information. Specifically, the connection that shares hotspot information between users is secured with an ECDH (Curve25519) ephemeral key that's authenticated with the users' respective device-specific Ed25519 public keys. The public keys used are those that had previously synced between the members of Family Sharing using IDS when the Family Share was established.

Network security

Network security overview

In addition to the built-in safeguards Apple uses to protect data stored on Apple devices, there are many measures organizations can take to keep information secure as it travels to and from a device. All of these safeguards and measures fall under network security.

Because users need to be able to access corporate networks from anywhere in the world, it's important to help ensure that they are authorized and that their data is protected during transmission. To accomplish these security objectives, iOS, iPadOS, macOS, visionOS, and watchOS integrate proven technologies and the latest standards for both Wi-Fi and cellular data network connections. That's why our operating systems use—and provide developer access to—standard networking protocols for authenticated, authorized, and encrypted communications.

TLS security

iOS, iPadOS, and macOS support Transport Layer Security (TLS 1.0, TLS 1.1, TLS 1.2, TLS 1.3) and Datagram Transport Layer Security (DTLS). The TLS protocol supports both AES-128 and AES-256, and prefers cipher suites with forward secrecy. Internet apps such as Safari, Calendar, and Mail automatically use this protocol to enable an encrypted communication channel between the device and network services. High-level APIs (such as `CFNetwork`) make it easy for developers to adopt TLS in their apps, while low-level APIs (such as `Network.framework`) provide fine-grained control. `CFNetwork` disallows SSL 3, and apps that use WebKit (such as Safari) are prohibited from making an SSL 3 connection.

On devices with iOS 11, iPadOS 13.1, macOS 10.13, or later, SHA-1 certificates are no longer allowed for TLS connections unless trusted by the user. Certificates with RSA keys shorter than 2048 bits are also disallowed. The RC4 symmetric cipher suite is deprecated in iOS 10 and macOS 10.12. By default, TLS clients or servers implemented with SecureTransport APIs don't have RC4 cipher suites enabled and are unable to connect when RC4 is the only cipher suite available. To be more secure, services or apps that require RC4 should be upgraded to use secure cipher suites.

On devices with iOS 12.1, certificates issued after October 15, 2018, from a system-trusted root certificate need to be logged in a trusted Certificate Transparency log to be allowed for TLS connections.

On devices with iOS 12.2 or later, TLS 1.3 is enabled by default for `Network.framework` and `NSURLSession` APIs. TLS clients using the SecureTransport APIs can't use TLS 1.3.

On devices with iOS 26, iPadOS 26, or later, TLS 1.3 with quantum-secure encryption (with the X25519MLKEM768 key exchange algorithm) is enabled by default for URLSession framework and the Network APIs. For more information about this feature and steps for troubleshooting connections, see the Apple Support article [Prepare your network for quantum-secure encryption in TLS](#).

App Transport Security

App Transport Security provides default connection requirements so that apps adhere to best practices for secure connections when using NSURLConnection, CFURL, or URLSession APIs. By default, App Transport Security limits cipher selection to include only suites that provide forward secrecy, specifically:

- ECDHE_ECDSA_AES and ECDHE_RSA_AES in Galois/Counter Mode (GCM)
- Cipher Block Chaining (CBC) mode

Apps are able to turn off the forward secrecy requirement per domain, in which case RSA_AES is added to the set of available ciphers.

Servers need to support TLS 1.2 and forward secrecy, and certificates need to be valid and signed using SHA-256 or stronger with a minimum 2048-bit RSA key or 256-bit elliptic curve key.

Network connections that don't meet these requirements fail unless the app overrides App Transport Security. Invalid certificates always result in a hard failure and no connection. App Transport Security is automatically applied to apps that are compiled for iOS 9, iPadOS 13.1, OS X 10.11, tvOS 9, watchOS 2, visionOS 1, or later.

Certificate validity checking

Evaluating the trusted status of a TLS certificate is performed in accordance with established industry standards, as set out in [RFC 5280](#), and incorporates emerging standards such as [RFC 6962](#) (Certificate Transparency). On devices with iOS 11 or later and macOS 10.13 or later, Apple devices are periodically updated with a current list of revoked and constrained certificates. The list is aggregated from certificate revocation lists (CRLs), which are published by each of the built-in root certificate authorities trusted by Apple, as well as by their subordinate CA issuers. The list may also include other constraints at Apple's discretion. This information is consulted whenever a network API function is used to make a secure connection. If there are too many revoked certificates from a CA to list individually, a trust evaluation may instead require that an online certificate status response (OCSP) is needed, and if the response isn't available, the trust evaluation will fail.

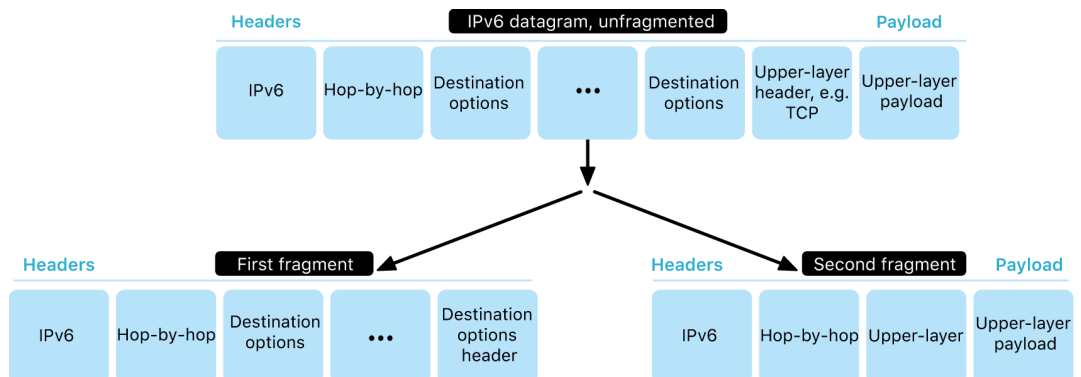
IPv6 security

All Apple operating systems support IPv6, implementing several mechanisms to protect the privacy of users and the stability of the networking stack. When Stateless Address Autoconfiguration (SLAAC) is used, the IPv6 addresses of all interfaces are generated in a way that helps prevent tracking devices across networks and at the same time allows for a good user experience by ensuring address stability when no network changes take place. The address generation algorithm is based on cryptographically generated addresses as of [RFC 3972](#), enhanced by an interface-specific modifier to warrant that even different interfaces on the same network eventually have different addresses. Furthermore, temporary addresses are created with a preferred lifetime of 24 hours, and these are used by default for any new connections. The Private Wi-Fi address feature, in which unique link-local address is generated for every Wi-Fi network that a device joins, was introduced in:

- iOS 14
- iPadOS 14
- macOS 12
- visionOS 1
- watchOS 7

The network's SSID is incorporated as an additional element for the address generation, similar to the Network_ID parameter as of [RFC 7217](#).

To protect against attacks based on IPv6 extension headers and fragmentation, Apple devices implement protection measures specified in [RFC 6980](#), [RFC 7112](#), and [RFC 8021](#). Among other measures, these inhibit attacks where the upper-layer header can be found only in the second fragment (as shown below), which in turn could cause ambiguities for security controls like stateless packet filters.



In addition, to help ensure the reliability of the IPv6 stack of Apple operating systems, Apple devices enforce various limits on IPv6-related data structures, such as the number of prefixes per interface.

Virtual private network (VPN) security

Secure network services such as virtual private networking (VPN) typically require minimal setup and configuration to work with iPad, iPhone, Mac, Apple Vision Pro, and Apple Watch devices.

Protocols supported

These devices work with VPN servers that support the following protocols and authentication methods:

- IKEv2/IPsec with authentication by shared secret, RSA Certificates, [Elliptic Curve Digital Signature Algorithm \(ECDSA\)](#) Certificates, EAP-MSCHAPv2, or EAP-TLS. Devices with iOS 26, iPadOS 26, macOS 26, tvOS 26, visionOS 26, and watchOS 26, or later can be configured to perform additional key exchanges using ML-KEM to provide quantum security.
- IKEv2/IPsec with authentication by shared secret, RSA Certificates, Elliptic Curve Digital Signature Algorithm (ECDSA) Certificates, EAP-MSCHAPv2, or EAP-TLS.
- SSL-VPN using the appropriate client app from the App Store.
- L2TP/IPsec with user authentication by MS-CHAPV2 password and machine authentication by shared secret (iOS, iPadOS, macOS, tvOS, and visionOS) and RSA SecurID or CRYPTOCARD (macOS only).
- Cisco IPsec with user authentication by password, RSA SecurID or CRYPTOCARD, and machine authentication by shared secret and certificates (macOS only).

VPN deployments supported

iOS, iPadOS, macOS, tvOS, and visionOS support the following:

- *VPN On Demand*: For networks that use certificate-based authentication. IT policies specify which domains require a VPN connection by using a VPN configuration profile.

iOS, iPadOS, macOS, visionOS, and supervised watchOS support the following:

- *Per-app VPN*: For facilitating VPN connections on a much more granular basis. A [device management service](#) can specify a connection for each managed app and specific domains in Safari. This helps ensure that secure data always goes to and from the corporate network—and that a user's personal data doesn't.

iOS, iPadOS, and visionOS support the following:

- *Always On VPN*: For devices managed through a device management service and supervised using Apple Configurator for Mac, [Apple School Manager](#), or [Apple Business](#). Always On VPN eliminates the need for users to turn on VPN to enable protection when connecting to cellular and Wi-Fi networks. It also gives an organization full control over device traffic by tunneling all IP traffic back to the organization. The default exchange of parameters and keys for the subsequent encryption, IKEv2, secures traffic transmission with data encryption. The organization can monitor and filter traffic to and from its devices, secure data within its network, and restrict device access to the internet.

Wi-Fi security and privacy

Security features when connecting to wireless networks

All Apple platforms support industry-standard Wi-Fi authentication and encryption protocols, to provide authenticated access and confidentiality when connecting to the following secure wireless networks:

- WPA2 Personal
- WPA2 Enterprise
- WPA2/WPA3 Transitional
- WPA3 Personal
- WPA3 Enterprise
- WPA3 Enterprise 192-bit Security

WPA2 and WPA3 authenticate each connection and provide 128-bit AES encryption to help ensure confidentiality of data sent over the air. This grants users the highest level of assurance that their data remains protected when they're sending and receiving communications over a Wi-Fi network connection.

WPA3 support

WPA3 is supported on the following Apple devices:

- All iPhone models starting with iPhone 7 or later
- All iPad models starting with iPad (5th generation) or later
- All Mac computers (late 2013 or later, with 802.11ac or later)
- All Apple TV models starting with Apple TV 4K (1st generation) or later
- All Apple Watch models starting with Apple Watch Series 3 or later
- Apple Vision Pro
- All HomePod models

Newer devices support authentication with WPA3 Enterprise 192-bit security, which includes support for 256-bit AES encryption when connecting to compatible wireless access points (APs). This encryption provides even stronger confidentiality protections for traffic sent over the air. WPA3 Enterprise 192-bit security is supported in all iPhone 11 models or later, all iPad models starting with the iPad (7th generation), and all Mac computers with Apple silicon.

WPA3 R3

The R3 WPA3 Personal update (WPA R3) was launched in order to enhance the security and privacy of Wi-Fi, with a focus on certain handshake vulnerabilities. On Apple platforms, the following WPA3 R3 features are supported:

- Transition Terminated Indication
- Hash-To-Element (H2E) Support and H2E Fast Transition
- Mitigation for Group Downgrade Attacks

- Anticlogging Token Container Element

These improvements target protection against downgrade attacks (for example, Transition Terminated Indication) and password element generation (for example, the use of Hash-To-Element, which uses a non-iterative algebraic algorithm to derive the secret key, an improvement to the “Hunt-and-Peck” method). These WPA3 R3 features were introduced in iOS 16, iPadOS 16, macOS 13, and tvOS 16, and are supported on the following Apple devices:

- All iPhone models starting with iPhone 11 or later
- All iPad models from late 2020 or later
- All Mac computers from late 2020 or later
- All Apple TV models starting with Apple TV 4K (2nd generation) or later

Protected Management Frame support

In addition to protecting data sent over the air, Apple platforms extend WPA2 and WPA3 level protections to unicast and multicast management frames through the Protected Management Frame (PMF) service defined in 802.11w. PMF support is available on the following Apple devices:

- All iPhone models starting with the iPhone 6 or later
- All iPad models starting with the iPad Air 2 or later
- All Mac computers (late 2013 or later, with 802.11ac or later)
- All Apple TV models starting with the Apple TV HD or later
- All Apple Watch models starting with the Apple Watch Series 3 or later
- Apple Vision Pro
- All HomePod models

With support for 802.1X, Apple devices can be integrated into a broad range of RADIUS authentication environments. 802.1X wireless authentication methods supported include EAP-TLS, EAP-TTLS, EAP-FAST, EAP-SIM, PEAPv0, and PEAPv1.

Platform protections

Apple operating systems protect the device from vulnerabilities in network processor firmware. This means that network controllers with Wi-Fi have limited access to Application Processor memory. Each network processor is on its own isolated PCIe bus. An [Input/Output Memory Management Unit \(IOMMU\)](#) on each PCIe bus further limits the network processor’s DMA access to only memory and resources containing its network packets and control structures.

Deprecated protocols

Apple products support the following deprecated Wi-Fi authentication and encryption protocols:

- WEP Open, with both 40-bit and 104-bit keys
- WEP Shared, with both 40-bit and 104-bit keys
- Dynamic WEP

- Temporal Key Integrity Protocol (TKIP)
- WPA
- WPA/WPA2 Transitional

These protocols are no longer considered secure, and their use is strongly discouraged for compatibility, reliability, performance, and security reasons. They are supported for backward compatibility purposes only and may be removed in future software versions.

It's recommended that all Wi-Fi implementations be migrated to WPA3 Personal or WPA3 Enterprise, to provide the most robust, secure, and compatible Wi-Fi connections possible.

Wi-Fi security with Apple devices

Apple devices have features designed to help devices maintain secure Wi-Fi connections.

Auto-join

In order to enhance user security, open and captive portal networks are not auto-joined if they haven't been joined within the past two weeks. For more information about auto-join policies, see the Apple Support article [How iOS, iPadOS, and macOS decide which wireless network to auto-join](#).

Opportunistic Wireless Encryption

Opportunistic Wireless Encryption (OWE), was introduced in an effort to improve security on open networks *encrypting the air* model to prevent traffic snooping and other related attacks. It provides protection against passive surveillance by encrypting over-the-air traffic on unauthenticated networks. Support was added on the following devices with iOS 16.1, iPadOS 16, macOS 13, tvOS 16, or later:

- All iPhone models starting with iPhone 11 or later
- All iPad models from late 2020 or later
- All Mac computers from late 2020 or later
- All Apple TV models starting with Apple TV 4K (1st generation) or later

Personal Hotspot and Internet Sharing security

By default, all Apple platforms provide a system-suggested, default secure & random password for access point modes (Personal Hotspot and Internet Sharing).

Personal Hotspot on iOS and iPadOS support:

- WPA2/WPA3 Personal Security (default)
- WPA2 Personal (if maximize compatibility switch is enabled)

Personal Hotspot on macOS supports:

- WPA2/WPA3 Personal Security (default)
- WPA3 Personal

Maximize Compatibility Mode

On iPhone 12 or later, users can turn on *Maximize Compatibility Mode* for Personal Hotspot. For compatibility with client devices, Maximize Compatibility Mode does the following:

- Turns off 5GHz (and, if applicable, 6GHz)
- Restricts the device to the use of only 2.4GHz
- Enforces a WPA2 Personal Security Mode

Privacy features when connecting to wireless networks

Scan privacy

Apple platforms use a randomized Media Access Control address (MAC address) when performing Wi-Fi scans when not associated with a Wi-Fi network. These scans can be performed to find and connect to a known Wi-Fi network or to assist Location Services for apps that use geofences, such as location-based reminders or fixing a location in Apple Maps. Note that Wi-Fi scans that happen while trying to connect to a preferred Wi-Fi network aren't randomized.

Because a device's MAC address changes when disconnected from a Wi-Fi network, it can't be used to persistently track a device by passive observers of Wi-Fi traffic, even when the device is connected to a cellular network. Apple has informed Wi-Fi manufacturers that iOS and iPadOS Wi-Fi scans use a randomized MAC address and that neither Apple nor manufacturers can predict these randomized MAC addresses.

Wi-Fi frame privacy

Wi-Fi frames include a sequence number, which is used by the low-level 802.11 protocol to enable efficient and reliable Wi-Fi communications. Because these sequence numbers increment on each transmitted frame, they could be used to correlate information transmitted during Wi-Fi scans with other frames transmitted by the same device.

To guard against this, Apple devices randomize the sequence numbers whenever a MAC address is changed to a new randomized address. This includes randomizing the sequence numbers for each new scan request that's initiated while the device is unassociated. This randomization was introduced in iOS 16.1, iPadOS 16.1, macOS 13.1, tvOS 16.1, watchOS 9.1, and visionOS 1, and is supported on the following devices:

- All iPhone models starting with the iPhone 7 or later
- All iPad models starting with the iPad (5th generation) or later
- All Mac computers from late 2018 or later
- iMac Pro (2017) or later
- All Apple TV models starting with the Apple TV 4K (1st generation) or later
- All Apple Watch models starting with the Apple Watch Series 3 or later
- Apple Vision Pro
- All HomePod models

Additionally, in an effort to increase user privacy and mitigate device fingerprinting, Apple devices randomize the seed value used for scrambling. This value is changed for every probe request and discovery frame, whenever an interface is started or stopped, on association and disassociation, on roam, and at random intervals.

Apple devices also randomize dialogue token fields in Information Elements (IEs) by selecting a new random value in each transaction. This helps ensure that there's no predictable pattern to the value of these fields across multiple transactions, and that said pattern can't be used to distinguish a given device. This applies to IEs across all interfaces (client, Access Point modes, and P2P).

Wi-Fi privacy with Apple devices

Apple devices have features designed to help devices maintain private Wi-Fi connections.

Private Wi-Fi address

On devices with iOS 14, iPadOS 14, macOS 14, visionOS 1, watchOS 7, or later, when connecting to a Wi-Fi network, the device has the ability to identify itself with a unique and random Wi-Fi (MAC) address. This assists in reducing tracking of devices and enhances user privacy.

Note: This feature can be turned off by a [device management service](#) (requires visionOS 1.1 or later). If the feature is turned off, the operating system shows a privacy warning in Settings indicating that the network has reduced privacy protections.

On devices with iOS 18, iPadOS 18, macOS 15, visionOS 2, watchOS 11, or later, the Private Wi-Fi Address feature has been updated to support three operating modes, selectable on a per-network basis:

- *Off:* Uses the device's hardware Wi-Fi address, allowing tracking by networks and nearby Wi-Fi devices.
- *Fixed:* A fixed private address reduces cross-network tracking by using a unique Wi-Fi address on the network. Used as default for secure network connections, such as WPA2 Personal, WPA2 Enterprise, WPA3 Personal, WPA3 Enterprise, WPA3 Enterprise 192-bit Security, and R3 WPA3 Personal.
- *Rotating:* A rotating private address reduces tracking by periodically changing this device's Wi-Fi address on the network. Used as default for connections that use weaker authentication or encryption methods, such as WPA, OWE, WEP, captive portals, and open networks.

For more information, see the Apple Support article [Use private Wi-Fi addresses on iPad, iPhone, and Apple Watch](#).

Hidden networks

Wi-Fi networks are identified by their network name, known as a *service set identifier* (SSID). Some Wi-Fi networks are configured to hide their SSID, which results in the wireless access point not broadcasting the network's name. These are known as *hidden networks*. Apple devices automatically detect when a network is hidden. If a network is hidden, the device sends a probe with the SSID included in the request—not otherwise. This helps prevent the device from broadcasting the name of previously hidden networks a user was connected to, thereby further ensuring privacy.

Personal Hotspot, Internet Sharing, and peer-to-peer privacy

Apple generates randomized MAC addresses for the peer-to-peer Wi-Fi connections that are used for AirDrop and AirPlay. Randomized addresses are also used for Personal Hotspot in iOS and iPadOS (with a SIM card) and Internet Sharing in macOS. New random addresses are generated whenever these network interfaces are started, and unique addresses are independently generated for each interface as needed.

Timing synchronization function offset randomization

To increase user privacy and mitigate device fingerprinting, Apple devices randomize the starting *Timing Synchronization Function* (TSF) offset for all peer-to-peer and Access Point modes. This 56-bit random offset helps prevent an attacker from using the TSF value to defeat MAC address randomization. The TSF offset for an interface is randomized whenever the interface MAC address changes. TSF offset randomization support was added on the following devices with iOS 15, iPadOS 15, macOS 12.0.1, tvOS 15, visionOS 1, or later:

- All iPhone models starting with the iPhone 8 or later
- All iPhone SE models starting with the iPhone SE (2rd generation) or later
- All iPad models starting with iPad Air (3rd generation) or later
- All Mac computers from late 2020 or later
- All Apple TV models starting with Apple TV 4K (1st generation) or later
- Apple Vision Pro
- All HomePod models

Bluetooth security

There are two types of Bluetooth in Apple devices, Bluetooth Classic and Bluetooth Low Energy (BLE). The Bluetooth security model for both versions includes the following distinct security features:

- *Pairing*: The process for creating one or more shared secret keys
- *Bonding*: The act of storing the keys created during pairing for use in subsequent connections to form a trusted device pair
- *Authentication*: Verifying that the two devices have the same keys
- *Encryption*: Message confidentiality
- *Message integrity*: Protection against message forgeries
- *Secure Simple Pairing*: Protection against passive eavesdropping and protection against man-in-the-middle attacks

Bluetooth version 4.1 added the Secure Connections feature to Bluetooth Classic (BR/EDR) physical transport.

The security features for each type of Bluetooth are listed below.

Support	Bluetooth Classic	Bluetooth Low Energy
Pairing	P-256 elliptic curve	FIPS-approved algorithms (AES-CMAC and P-256 elliptic curve)
Bonding	Pairing information stored in a secure location in iOS, iPadOS, macOS, tvOS, and watchOS devices	Pairing information stored in a secure location in iOS, iPadOS, macOS, tvOS, and watchOS devices
Authentication	FIPS-approved algorithms (HMAC-SHA-256 and AES-CTR)	FIPS-approved algorithms
Encryption	AES-CCM cryptography, performed in the Controller	AES-CCM cryptography, performed in the Controller
Message integrity	AES-CCM, used for message integrity	AES-CCM, used for message integrity
Protection against passive eavesdropping	Elliptic Curve Diffie-Hellman Exchange Ephemeral (ECDHE)	Elliptic Curve Diffie-Hellman Exchange (ECDHE)
Protection against man-in-the-middle (MITM) attacks	Two user-assisted numeric methods: numerical comparison or passkey entry	Two user-assisted numeric methods: numerical comparison or passkey entry Pairings require a user response, including all non-MITM pairing modes
Bluetooth 4.1 or later	iMac Late 2015 or later MacBook Pro Early 2015 or later	iPadOS 13.1 or later iOS 9 or later macOS 10.12 or later tvOS 9 or later watchOS 2 or later
Bluetooth 4.2 or later	iPhone 6 or later	iPadOS 13.1 or later iOS 9 or later macOS 10.12 or later tvOS 9 or later watchOS 2 or later

Bluetooth Low Energy privacy

To help secure user privacy, BLE includes the following two features: address randomization and cross-transport key derivation.

Address randomization is a feature that reduces the ability to track a BLE device over a period of time by changing the Bluetooth device address on a frequent basis. For a device using the privacy feature to reconnect to known devices, the device address, referred to as the *private address*, needs to be resolvable by the other device. The private address is generated using the device's identity resolving key exchanged during the pairing procedure.

On devices with iOS 13 and iPadOS 13.1 or later have the ability to derive link keys across transports, a feature known as *cross-transport key derivation*. For example, a link key generated with BLE can be used to derive a Bluetooth Classic link key. In addition, Apple added Bluetooth Classic to BLE support for devices that support the Secured Connections feature that was introduced in the Bluetooth Core Specification 4.1 (see the [Bluetooth Core Specification 5.1](#)).

Ultra Wideband security in iOS

The new Apple-designed U1 chip uses Ultra Wideband technology for spatial awareness—allowing iPhone 11, iPhone 11 Pro, and iPhone 11 Pro Max or later iPhone models to precisely locate other U1-equipped Apple devices. Ultra Wideband technology uses the same technology to randomize data found in other supported Apple devices:

- MAC address randomization
- Wi-Fi frame sequence number randomization

Single Sign-on security

iOS, iPadOS, macOS and visionOS support authentication to enterprise networks through Single Sign-on (SSO). SSO works with an identity provider (IdP) to authenticate users to services they are authorized to access. SSO can be used for a range of network activities, from secure Safari sessions to third-party apps.

iOS, iPadOS, macOS, and visionOS include a built-in Kerberos SSO extension. App developers can also provide their own Single Sign-on implementations using SSO extensions.

Kerberos Single Sign-on

iOS, iPadOS, macOS, and visionOS support authentication to enterprise networks using Kerberos. SSO support is based on the open source Heimdal project and uses SPNEGO tokens and the HTTP Negotiate protocol to work with Kerberos-based authentication gateways and Windows Integrated Authentication systems that support Kerberos tickets. Certificate-based authentication such as PKINIT is also supported.

The Kerberos SSO extension supports the following encryption types:

- AES-128-CTS-HMAC-SHA1-96
- AES-256-CTS-HMAC-SHA1-96

iOS, iPadOS, and visionOS provide flexible support for SSO to any app that uses the class `NSURLSession` or `URLSession` to manage network connections and authentication. Apple provides all developers with these classes to seamlessly integrate network connections within their apps.

Any Mac app that supports Kerberos authentication works with SSO. This includes many of the apps built into macOS, such as Calendar, Mail, and Safari, and includes services like file sharing, screen sharing, and secure shell (SSH). Many third-party apps also support Kerberos.

To configure SSO, a declarative configuration allows a device management service to push the necessary settings to the device. This includes setting the user principal name (the Active Directory user account) and Kerberos realm settings, as well as configuring which apps and Safari web URLs should be allowed to use SSO.

Extensible Single Sign-on

App developers can provide their own Single Sign-on implementation using SSO extensions for iOS, iPadOS, macOS, and visionOS. SSO extensions are invoked when a native or web app needs to use an IdP for user authentication. Developers can provide two types of extensions:

- Redirect extensions, which redirect to HTTPS
- Challenge/response extensions, such as Kerberos

This allows Open ID Connect (OIDC), OAuth, SAML 2.0, and Kerberos authentication schemes to be supported by extensible Single Sign-on. SSO extensions may also support authentication at the FileVault and Login Window on macOS by adopting Platform SSO, which allows the Mac to retrieve SSO tokens during login.

To use a Single Sign-on extension, an app can either use the AuthenticationServices API or can rely on the URL interception mechanism offered by the operating system. WebKit and CFNetwork provide an interception layer that permits seamless support of Single Sign-on for any native or WebKit app. For a Single Sign-on extension to be invoked, a configuration needs to be installed through a device management service. In addition, redirect type extensions need to use Associated Domains to prove that the IdP they support is aware of their existence.

Platform Single Sign-on

With Platform Single Sign-on (Platform SSO), developers can build SSO extensions for macOS that extend to the macOS FileVault and Login Window. Platform SSO supports different authentication mechanisms with an IdP:

- Password
 - Password authentication supports WS-Trust. This allows a user to authenticate even when the IdP managing their account is federated.
 - Optionally, users can also synchronize their local account credentials with an IdP. The local account password is automatically kept in sync with the password of their IdP account.
- Secure Enclave Key
 - A key is created and stored in the Secure Enclave during the user registration process and used to authenticate with the IdP without a password.
- Smart card
 - A smart card is used to authenticate with the IdP.
- Access Key
 - A pass stored in Apple Wallet is used to authenticate with the IdP.

Registration

To use Platform SSO, the Mac and each user register with the IdP. Depending on IdP support and the applied configuration, the Mac can perform device registration silently in the background using:

- A registration token provided in the device management configuration.
- An attestation, which provides a strong assurance about device identifiers (UDID and serial number).

After a successful device registration, the user registers (unless the user account is using Authenticated Guest Mode). If the IdP requires it, user registration can involve prompting the user to confirm their registration. For local user accounts that Platform SSO creates on demand, user registration happens automatically in the background.

During the registration process, the Mac creates two keys: a device signing key, and a device encryption key. Depending on the SSO extension, the keys can use the following:

- NIST P-256 ECDH-ES
- HKPE with NIST P-256, NIST P-384, or X25519

These maintain a trusted connection with the IdP independent of the user. Platform SSO supports shared device keys. This enables features like Platform SSO during Automated Device Enrollment, on-demand creation of user accounts based on information from the IdP, network authorization, and Authenticated Guest Mode and uses the same signing and encryption key for all users on a specific device.

Note: The SSO extension needs to support the requested method to perform the registration. Switching methods is also supported. For example, when creating a new user account with a user name and password during login, that account can then switch to use a Secure Enclave-backed key or smart card after the login is successful.

Token retrieval and refresh

To authenticate the user to their apps and websites as needed, the SSO extension uses SSO tokens. To help ensure the SSO extension is always ready to perform an authentication, Platform SSO attempts to refresh or retrieve new tokens from the IdP if the tokens are:

- Missing
- Expired
- More than four hours old

To request or renew tokens from the IdP, a JSON Web Token (JWT) login or refresh request is sent and—depending on the method defined by the SSO extension—signed with the device signing key using ES256, ES384, or Ed25519. The created tokens are part of the response payload which is AES-GCM 256 encrypted using the public key of the device encryption key.

If password encryption is requested by the SSO extension, the password isn't sent in the login request and replaced by an AES-GCM 256 encrypted embedded assertion containing the password. The associated encryption key is then created (as defined by the SSO extension using one of the mechanisms also available for the device encryption key).

When the authentication mechanism requires a digital signature, Platform SSO uses an embedded JWT assertion for Secure Enclave Key and smart card authentication. The assertion is either:

- Signed with the authenticator when using a smart card.
- Encrypted using ES256, ES384, or Ed25519 when using a Secure Enclave Key as defined by the SSO extension.

Regardless of the authentication method, SSO tokens are:

- Stored in the keychain using the keychain data protection attribute `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`
- Shared only with the SSO extension
- Nonmigratory

AirDrop security

Apple devices that support AirDrop use Bluetooth Low Energy (BLE) and Apple-created peer-to-peer Wi-Fi technology to send files and information to nearby devices, including AirDrop-capable iPad and iPhone devices with iOS 7 or later and Mac computers with OS X 10.11 or later. The Wi-Fi radio is used to communicate directly between devices without using any internet connection or wireless access point (AP). This connection is encrypted with TLS.

AirDrop is set to share with Contacts Only by default. Users can also choose to use AirDrop to share with everyone, or turn off the feature entirely. Organizations can restrict the use of AirDrop for devices or apps being managed by using a [device management service](#).

AirDrop operation

AirDrop uses iCloud services to help users authenticate. When a user signs in to iCloud, a 2048-bit RSA identity is stored on the device, and when the user turns on AirDrop, an AirDrop short identity hash is created based on the email addresses and phone numbers associated with the user's Apple Account.

When a user chooses AirDrop as the method for sharing an item, the sending device emits an AirDrop signal over BLE that includes the user's AirDrop short identity hash. Other Apple devices that are awake, in close proximity, and have AirDrop turned on, detect the signal and respond using peer-to-peer Wi-Fi, so that the sending device can discover the identity of any responding devices.

In Contacts Only mode, the received AirDrop short identity hash is compared with hashes of people in the receiving device's Contacts app. If a match is found, the receiving device responds over peer-to-peer Wi-Fi with its identity information. If there is no match, the device doesn't respond.

In Everyone mode, the same overall process is used. However, the receiving device responds even if there is no match in the device's Contacts app.

The sending device then initiates an AirDrop connection using peer-to-peer Wi-Fi, using this connection to send a long identity hash to the receiving device. If the long identity hash matches the hash of a known person in the receiver's Contacts, then the receiver responds with its long identity hashes.

If the hashes are verified, the recipient's first name and photo (if present in Contacts) are displayed in the sender's AirDrop share sheet. On devices with iOS and iPadOS, they are shown in the "People" or "Devices" section. Devices that aren't verified or authenticated are displayed in the sender's AirDrop share sheet with a silhouette icon and the device's name, as defined in Settings > General > About > Name. They're placed in the "Other People" section of the AirDrop share sheet.

The sending user may then select whom they want to share with. Upon user selection, the sending device initiates an encrypted (TLS) connection with the receiving device, which exchanges their iCloud identity certificates. The identity in the certificates is verified against each user's Contacts app.

If the certificates are verified, the receiving user is asked to accept the incoming transfer from the identified user or device. If multiple recipients have been selected, this process is repeated for each destination.

Wi-Fi password sharing security

Apple devices that support Wi-Fi password sharing use a mechanism similar to AirDrop to send a Wi-Fi password from one device to another.

When a user selects a Wi-Fi network (requestor) and is prompted for the Wi-Fi password, the Apple device starts a Bluetooth Low Energy (BLE) advertisement indicating that it wants the Wi-Fi password. Other Apple devices that are awake, in close proximity, and have the password for the selected Wi-Fi network connect using BLE to the requesting device.

The device that has the Wi-Fi password (grantor) requires the contact information of the requestor, and the requestor needs to prove their identity using a process that's similar to AirDrop. After identity is proven, the grantor sends the requestor the passcode, which can be used to join the network.

Organizations can restrict the use of Wi-Fi password sharing for devices or apps being managed through a [device management service](#).

Firewall security in macOS

macOS includes a built-in firewall to protect the Mac from network access and denial-of-service attacks. It can be configured by going to System Settings > Privacy & Security (macOS 13 or later), the Security & Privacy pane of System Preferences (macOS 12 or earlier), or by using a configuration profile with the Firewall payload installed manually or provided by a [device management service](#). The following configurations are supported:

- Block all incoming connections, regardless of app.
- Automatically allow built-in software to receive incoming connections.
- Automatically allow downloaded and signed software to receive incoming connections.
- Add or deny access based on user-specified apps.
- Prevent the Mac from responding to ICMP (Internet Control Message Protocol) probing and portscan requests.

Developer kit security

Developer kit security overview

Apple provides a number of “kit” frameworks to enable third-party developers to extend Apple services. These frameworks are built with user privacy and security at their core:

- [ARKit](#)
- [DriverKit](#)
- [EnergyKit](#)
- [HomeKit](#)
- [NFC & SE Platform](#)
- [ReplayKit](#)
- [SiriKit](#)
- [WidgetKit](#)

ARKit security in iOS and iPadOS

ARKit is a framework that lets developers produce augmented reality experiences in their app or game. Developers can add 2D or 3D elements using the front or rear camera of an iOS or iPadOS device.

Apple designed cameras with privacy in mind, and third-party apps need to obtain the user’s consent before accessing the camera. On devices with iOS and iPadOS, when a user grants an app access to their camera, that app can access real-time images from the front and rear cameras. Apps aren’t allowed to use the camera without transparency that the camera is in use.

Photos and videos taken with the camera may contain other information, such as where and when they were taken, the depth of field, and overcapture. If users don’t want photos and videos taken with the Camera app to include location, they can control this at any time by going to Settings > Privacy > Location Services > Camera. If users don’t want photos and video to include location when shared, they can turn location off in the Options menu in the share sheet.

To better position the user’s AR experience, apps that use ARKit can use world- or face-tracking information from the other camera. World tracking uses algorithms on the user’s device to process information from these sensors to determine their position relative to a physical space. World tracking enables features such as Optical Heading in Maps.

DriverKit security for macOS

DriverKit allows developers to create device drivers that the user installs on their Mac. Drivers built with DriverKit run in user space, rather than as kernel extensions, for improved system security and stability. This makes for easier installation and increases the stability and security of macOS.

The user downloads the app (installers aren't necessary when using system extensions or DriverKit) and the extension is enabled only when required.

IT administrators who use device drivers, cloud storage solutions, networking, and security apps that require kernel extensions are encouraged to move to newer versions that are built on system extensions. These newer versions greatly reduce the possibility of kernel panics on the Mac as well as reduce the attack surface. These new extensions run in the user space, won't require special privileges required for installation, and are automatically removed when the bundling app is moved to the Trash.

The DriverKit framework provides C++ classes for I/O services, device matching, memory descriptors, and dispatch queues. It also defines I/O-appropriate types for numbers, collections, strings, and other common types. The user uses these with family-specific driver frameworks like USBDriverKit and HIDDriverKit. Use the System Extensions framework to install and upgrade a driver.

EnergyKit data security

iCloud and iCloud Keychain securely synchronize EnergyKit data between a user's Apple devices. End-to-end encryption through the entire process ensures the data isn't accessible to anyone, even Apple.

Users can control the access of their energy data in the system Privacy settings. When apps want to access energy data, the operating system prompts a user for approval, in a way that's similar to Home data, Contacts, Photos, and other iOS, iPadOS, and macOS data. If a user approves the prompt, apps can access the names of Homes, Electricity Guidance, and other home information. In addition, EnergyKit can receive electricity usage data from authorized apps.

Local data storage

EnergyKit stores data on a user's Apple devices. The framework secures the data using the Data Protection class Protected Until First User Authentication and a data vault. However, local backups exclude EnergyKit data.

HomeKit security

HomeKit communication security

HomeKit provides a home automation infrastructure that uses iCloud and device security features to protect and sync private data without exposing it to Apple.

HomeKit identity and security are based on Ed25519 public-private key pairs. An Ed25519 key pair is generated on the user's device, which becomes their HomeKit identity. The key pair is used as part of the HomeKit Accessory Protocol (HAP) to authenticate direct communication between user's Apple devices and their HomeKit accessories.

For homes where there is a home hub, members of the shared home can send commands to accessories through the hub. These commands are sent, end-to-end encrypted and authenticated, from the user's device to the home hub using [Apple Identity Service \(IDS\)](#), where they are forwarded on to the relevant accessory using the HomeKit Accessory Protocol (HAP) or Matter, a smart home connectivity standard.

The keys—stored in [keychain](#) and are included only in encrypted keychain backups—are kept up to date between devices using iCloud Keychain.

Communication between HomeKit accessories

HomeKit accessories generate their own Ed25519 key pair for use in communicating with Apple devices. If the accessory is restored to factory settings, a new key pair is generated.

To establish a relationship between an Apple device and a HomeKit accessory, keys are exchanged using the Secure Remote Password (3072-bit) protocol with an eight-digit code provided by the accessory's manufacturer, entered on the user's device, and then encrypted using ChaCha20-Poly1305 AEAD with HKDF-SHA-512 derived keys. The accessory's MFi certification is also verified during setup. Accessories without an MFi chip can build in support for software authentication in iOS 11.3 or later.

When the device and HomeKit accessory communicate during use, each authenticates the other using the keys exchanged in the above process. Each session is established using the Station-to-Station protocol and is encrypted with HKDF-SHA-512 derived keys based on per-session Curve25519 keys. This applies to both IP-based and Bluetooth Low Energy (BLE) accessories.

For BLE devices that support broadcast notifications, the accessory is provisioned with a broadcast encryption key by a paired device over a secure session. This key is used to encrypt the data about state changes on the accessory, which are notified using the BLE advertisements. The broadcast encryption key is an HKDF-SHA-512 derived key, and the data is encrypted using the ChaCha20-Poly1305 AEAD algorithm. The broadcast encryption key is periodically changed and updated on other devices using iCloud, as described in [Data security](#).

Communication with Matter accessories

Identity and security with Matter accessories are certificate based. For Apple homes, the root of trust Certificate Authority (CA) is generated on the initial user's device (the "owner"), and the private key for the CA is stored in their iCloud Keychain. Each Apple device in the home generates a Certificate Signing Request (CSR) using NIST P256. This CSR is ingested by the owner's device, which creates a Matter identity certificate for the device using their CA private key. This certificate is subsequently used to authenticate communication between users' devices and their accessories.

Matter accessories generate their own NIST P256 key pair and CSR and receive a certificate from the CA during accessory pairing. Before the key pairs are generated, the Matter accessory and the home owner's devices exchange keys—using the SPAKE2+ protocol with a PIN provided by the accessory's manufacturer—and a Device Attestation process is performed. The CSR and certificate are then exchanged over this channel encrypted using AES-CCM with HKDF-SHA-256 derived keys. If the accessory is restored to factory settings, a new key pair and CSR are generated and a new certificate is issued for the accessory during pairing.

When an Apple device and Matter accessory communicate during use, each authenticates the other using their own certificates. Each session is established using a three-phase (sigma) protocol and is encrypted with HKDF-SHA-256 derived keys based on per-session P256 keys.

For more information about how Apple devices interact securely with Matter accessories, see [Matter support in iOS 16](#) on the Apple Developer website.

HomeKit and Siri

Siri can be used to query and control accessories, and to activate scenes. Minimal information about the configuration of the home is provided anonymously to Siri, to provide names of rooms, accessories, and scenes that are necessary for command recognition. Audio sent to Siri may denote specific accessories or commands, but such Siri data isn't associated with other Apple features such as HomeKit.

Siri-enabled HomeKit accessories

Users can enable new features like Siri, and other HomePod features like timers, alarms, intercom, and doorbell, on Siri-enabled accessories using the Home app. When these features are enabled, the accessory coordinates with a paired HomePod on the local network that hosts these Apple features. Audio is exchanged between the devices over encrypted channels using both HomeKit and AirPlay protocols.

When Listen for Hey Siri is turned on, the accessory listens for the "Hey Siri" phrase using a locally running trigger-phrase detection engine. If this engine detects the phrase, it sends the audio frames directly to a paired HomePod using HomeKit. The HomePod does a second check on the audio and may cancel the audio session if the phrase doesn't appear to contain the trigger phrase.

When Touch for Siri is turned on, the user can press a dedicated button on the accessory to start a conversation with Siri. The audio frames are sent directly to the paired HomePod.

After a successful invocation of Siri is detected, the HomePod sends the audio to Siri servers and fulfills the user's intent using the same security, privacy, and encryption safeguards that the HomePod applies to user invocations made to the HomePod itself. If Siri has an audio reply, then Siri's response is sent over an AirPlay audio channel to the accessory. Some Siri requests require additional information from the user (for example, asking if the user wants to hear more options). In that case, the accessory receives an indication that the user should be prompted, and the additional audio is streamed to the HomePod.

The accessory is required to have a visual indicator to signal to a user when it's actively listening (for example, an LED indicator). The accessory has no knowledge of the intent of the Siri request, except for access to the audio streams, and no user data is stored on the accessory.

HomeKit data security

For homes that have been upgraded to the new HomeKit architecture (available in iOS 16.2 and iPadOS 16.2), HomeKit data is securely synchronized between a user's Apple devices using iCloud and iCloud Keychain. During this process, the HomeKit data is encrypted using iCloud end-to-end encryption and isn't accessible by Apple.

The user who initially created the home in the Home app (the "owner") or another user with editing permissions can add new users. The owner's device configures the accessories with the public key of the new user so that the accessory can authenticate and accept commands from the new user. When a user with editing permissions adds a new user, the process is delegated to a home hub to complete the operation.

Home data and apps

Access to home data by third-party apps is controlled by users in Privacy settings and users are asked to grant access when these apps request home data (similar to access Contacts, Photos, and other iOS, iPadOS, and macOS data sources). If the user approves, apps have access to the names of rooms, names of accessories, the room each accessory is in, and other information as detailed in the HomeKit developer documentation at <https://developer.apple.com/homekit/>.

Local data storage

HomeKit stores data about the homes, accessories, scenes, and users on a user's Apple devices. This data is stored using the [Data Protection](#) class Protected Until First User Authentication and within a data vault. HomeKit data isn't backed up in local backups.

Securing routers with HomeKit

Users can improve the security of their home network by using routers that support HomeKit. With these routers, users can manage the Wi-Fi access that HomeKit accessories have to their local network and to the internet. The routers also support Private PSK (PPSK) authentication, so accessories can be added to the Wi-Fi network using a key that's specific to the accessory and that can be revoked when needed. PPSK authentication improves security by not exposing the main Wi-Fi password to accessories, as well as by allowing the router to securely identify an accessory even if it changes its MAC address.

Using the Home app, a user can configure access restrictions for groups of accessories as follows:

- *No restriction:* Allow unrestricted access to the internet and the local network.
- *Automatic:* This is the default setting. Allow access to the internet and the local network based on a list of internet sites and local ports provided to Apple by the accessory manufacturer. This list includes all sites and ports needed by the accessory to function properly. (No Restriction is in place until such a list is available.)

- *Restrict to Home*: No access to the internet or the local network except for the connections required by HomeKit to discover and control the accessory from the local network (including from the home hub to support remote control).

A PPSK is a strong, accessory-specific WPA2 Personal pass-phrase that's automatically generated by HomeKit and revoked if and when the accessory is later removed from the Home. A PPSK is used when an accessory is added to the Wi-Fi network by HomeKit in a Home that has been configured with a HomeKit router; this addition is reflected as Wi-Fi Credential: HomeKit-managed on the settings screen for the accessory in the Home app. Accessories that were added to the Wi-Fi network before adding the router are reconfigured to use a PPSK if the accessory supports this; otherwise, they retain their existing credentials.

As an additional security measure, users need to configure the HomeKit router using the router manufacturer's app, so that the app can validate that users have access to the router and can add it to the Home app.

HomeKit camera security

Cameras that have an IP address (IP camera) in HomeKit send video and audio streams directly to the iOS, iPadOS, macOS, and tvOS device on the local network accessing the stream. The streams are encrypted using randomly generated keys on the device and the IP camera, and they're exchanged over the secure HomeKit session to the camera.

When a device isn't on the local network, the encrypted streams are relayed through the home hub to the device. The home hub doesn't decrypt the streams; it functions only as a relay between the device and the IP camera. When an app displays the IP camera video view to the user, HomeKit renders the video frames securely from a separate system process. As a result, the app is unable to access or store the video stream. In addition, apps aren't permitted to capture screenshots from this stream.

HomeKit secure video

HomeKit provides an end-to-end secure and private mechanism to record, analyze, and view clips from HomeKit IP cameras without exposing that video content to Apple or any third party. When motion is detected by the IP camera, video clips are sent directly to an Apple device acting as a home hub, using a dedicated local network connection between that home hub and the IP camera. The local network connection is encrypted with a per-session HKDF-SHA-512 derived key-pair that's negotiated over the HomeKit session between home hub and IP camera. HomeKit decrypts the audio and video streams on the home hub and analyzes the video frames locally for any significant event. If a significant event is detected, HomeKit encrypts the video clip using AES-256-GCM with a randomly generated AES-256 key. HomeKit also generates poster frames for each clip and these poster frames are encrypted using the same AES-256 key. The encrypted poster frame and audio and video data are uploaded to iCloud servers. The related metadata for each clip including the encryption key are uploaded to CloudKit using iCloud end-to-end encryption.

For face classification, HomeKit stores all data used to classify a particular person's face in CloudKit using iCloud end-to-end encryption. The data stored includes information about each person, such as name, as well as images representing that person's face. These face images can be sourced from a user's Photos if they opt in, or they can be collected from previously analyzed IP camera video. A HomeKit Secure Video analysis session uses this classification data to identify faces in the secure video stream it receives directly from the IP camera and includes that identification information in the clip metadata mentioned previously.

When the Home app is used to view the clips for a camera, the data is downloaded from iCloud and the keys to decrypt the streams are unwrapped locally using iCloud end-to-end decryption. The encrypted video content is streamed from the servers and decrypted locally on the iOS device before displaying it in the viewer. Each video clip session maybe broken down into sub-sections with each sub-section encrypting the content stream with its own unique key.

HomeKit security with Apple TV

HomeKit securely connects some third-party remote accessories to Apple TV and supports adding user profiles to the owner of the home's Apple TV.

Using third-party remote accessories with Apple TV

Some third-party remote accessories provide Human Interface Design (HID) events and Siri audio to an associated Apple TV added using the Home app. The remote sends the HID events over the secure session to the Apple TV. A Siri-capable TV remote sends audio data to Apple TV when the user explicitly activates the microphone on the remote using a dedicated Siri button. The remote sends the audio frames directly to the Apple TV using a dedicated local network connection. A per-session HKDF-SHA-512 derived key-pair that's negotiated over the HomeKit session between Apple TV and the TV remote is used to encrypt the local network connection. HomeKit decrypts the audio frames on Apple TV and forwards them to the Siri app, where they are treated with the same privacy protections as all Siri audio input.

Apple TV profiles for HomeKit homes

When a user of a HomeKit home adds their profile to the owner of the home's Apple TV, it gives that user access to their TV shows, music, and podcasts. Settings for each user regarding their profile use on the Apple TV are shared to the owner's Apple Account using iCloud end-to-end encryption. The data is owned by each user and is shared as read-only to the owner. Each user of the home can change these values in the Home app and the Apple TV of the owner uses these settings.

When a setting is turned on, the iTunes account of the user is made available on the Apple TV. When a setting is turned off, all account and data pertaining to that user is deleted on the Apple TV. The initial CloudKit share is initiated by the user's device and the token to establish the secure CloudKit share is sent over the same secure channel that's used to sync data between users of the home.

NFC & SE Platform security

The NFC & SE Platform is a secure solution developed by Apple that enables authorized developers to provide capabilities to support transactions from within their iOS app. On an iPhone with iOS 18.1 or later, developers can store credentials in the Secure Element. These credentials support NFC transactions for payments, access, transit, loyalty programs, tickets, and government IDs. Developers need to protect user privacy and data, including card details. The NFC & SE Platform provides security features from iPhone hardware, software, and Apple servers. Developers can use the Secure Element, biometric sensors, Secure Enclave, and Apple servers to protect credentials during the following phases of use:

- [Credential provisioning](#)
- [Secure storage and access](#)
- [Transaction authorization](#)
- [Management](#)

Note: Developers need to be granted an entitlement to use the NFC & SE Platform. This helps ensure that only authorized developers that are committed to upholding privacy and security standards can use the NFC & SE Platform.

NFC & SE Platform component security

The NFC & SE Platform provides access to hardware and software features that enable developers to provide secure transactions for iPhone users.

Secure Element

The Secure Element is an industry-standard integrated circuit that runs the Java Card platform. Certified by both EMVCo and Common Criteria, it supports standard Java Card applets, including those approved for the NFC & SE Platform. It also has a special applet for managing NFC & SE Platform applets' authorization and activation. Credential data can be encrypted and sent to these applets using unique keys. This data is stored in the applets and secured by the Secure Element's security features. During transactions, the terminal communicates directly with the Secure Element through the near-field-communication (NFC) controller.

NFC controller

The NFC controller handles NFC protocols and routes communication between the Application Processor and the Secure Element, and between the Secure Element and the point-of-sale terminal. The NFC controller helps ensure that contactless transactions are conducted using a terminal that's in close proximity to the device. Only requests arriving from an in-field terminal are marked by the NFC controller as contactless transactions.

After a transaction is authorized to proceed by the user using Face ID, Touch ID, or the phone's passcode, contactless responses prepared by the NFC & SE Platform applet within the Secure Element are exclusively routed by the NFC controller to the NFC field. Consequently, contactless transaction details are contained to the local NFC field and aren't exposed to the Application Processor.

Secure Enclave

The Secure Enclave manages the user authentication and secure intent processes on the device, allowing authorized transactions to proceed. Communication between the Secure Enclave and the Secure Element takes place over a serial interface, with the Secure Element connected to the NFC controller, which in turn is connected to the Application Processor. Though not directly connected, the Secure Enclave and Secure Element can communicate securely using a shared secret generated at runtime, which can be used to provide confidentiality and integrity over the communication link as needed.

Apple servers

Apple servers host NFC & SE Platform partners' approved applet packages. They also manage the setup and creation of security domains and applets in the Secure Element, including those used by the NFC & SE Platform.

Applets and configurations

To use the NFC & SE Platform, developers need to have an approved applet bundle and product configuration to support their credentials. Before being delivered to Apple for secure installation on the Secure Element, all applets need to go through a security review by an accredited third-party security evaluation laboratory. After delivery to Apple, the applet bundle and associated product configuration are reviewed and need to be approved before they can be used with the NFC & SE Platform. After approval, the applet package is signed and hosted on Apple servers.

Developers are also independently responsible for obtaining any other necessary certifications or qualifications for their applets based on their use case and operating plans, such as those required by law, regulation, or payment network operators.

Credential provisioning

The NFC & SE Platform developer is responsible for protecting the security of the process for a user to provision new credentials. This may include steps like authentication of the user in their iOS app, protection and validation of sensitive data entered by the user, communication with servers, granting approval for adding a credential, initiation of credential provisioning, and handling of personalization data. The developer is also responsible for ensuring their solution meets any applicable regulations and industry standards related to the security of provisioning and use of credentials.

Upon request from the iOS app to create a new credential, the NFC & SE Platform sends the request to Apple servers. If the applet associated with the requested product configuration hasn't yet been downloaded to the Secure Element, an Apple server initiates the download of the signed package into the Secure Element. Next, a new applet instance is installed within an isolated Security Domain in the Secure Element to be used for storage of the new credential. After installation, the developer's chosen Trusted Service Manager (TSM) can confidentially personalize the applet instance with any necessary credential data, such as keys and account numbers.

Secure storage and access

The Secure Element's hardware and software security functionality are available to protect NFC & SE Platform credentials both at rest and during use. Developers ensure that their Secure Element applets are implemented in a secure manner, follow the security guidelines, and fully utilize the available security features provided by the platform in order to adequately protect credential data.

The NFC & SE Platform restricts access to product configurations and applet instances to their associated iOS apps. As a result, no unauthorized apps can interact with them to modify or use them. Only associated iOS apps are allowed to:

- Request applet instance creation
- Personalize, update and send commands to applet instances
- Initiate transactions
- Request applet instance deletion

Transaction authorization

Apps need to get user authorization for all transactions; the NFC & SE Platform provides a way for app developers to ensure this happens. Users authorize transactions with biometrics or a passcode, combined with a physical gesture to the Secure Enclave. After approval, the Secure Enclave sends authentication data to the Secure Element. The Secure Element then verifies this and notifies the applet to activate the NFC interface. The NFC & SE Platform developer needs to implement their iOS app and Secure Element applet according to the specifications provided by Apple in order to securely utilize the transaction authorization mechanism and conduct transactions.

To initiate a transaction, the iOS app needs to be in the foreground and the iPhone needs to be unlocked. When an iOS app is set as the default contactless app in Settings, the app is automatically launched when the user double-clicks the side button (for Face ID devices) or the Home button (for Touch ID) and after user authentication (if the iPhone is locked).

Additionally, the developer's NFC & SE Platform iOS app is responsible for clearly informing the user of the credential to be used for the transaction and displaying any relevant transaction details.

Management

Using the NFC & SE Platform APIs, developers can update credential data or delete credentials and then send the appropriate commands to their applet from their iOS app. They can also add features to suspend or unlink their credentials, but this functionality is independent of the NFC & SE Platform.

All Secure Element credentials are securely erased when:

- The user signs out of iCloud
- Removes their device passcode
- Erases their device using Erase All Content and Settings or remotely with Find My
- Requests deletion of their Apple Account from the [Apple Data and Privacy page](#)

Users also have the ability to remove specific credentials from their iPhone by deleting the associated iOS app.

ReplayKit security in iOS and iPadOS

ReplayKit allows developers to add recording and live broadcasting capabilities to their apps. In addition, it allows users to annotate their recordings and broadcasts using the device's front camera and microphone.

Movie recording

There are several layers of security built into recording a movie:

- *Permissions dialog*: Before recording starts, ReplayKit presents a user consent alert requesting that the user acknowledge their intent to record the screen, the microphone, and the front camera. This alert is presented once per app process, and it's presented again if the app is left in the background for longer than eight minutes.
- *Screen and audio capture*: Screen and audio capture occurs out of the app's process in the ReplayKit daemon `replayd`. This is designed to ensure the recorded content is never accessible to the app process.
- *In-app screen and audio capture*: This allows an app to get video and sample buffers, which is guarded by the permissions dialogue.
- *Movie creation and storage*: The movie file is written to a directory that's only accessible to the ReplayKit subsystems and is never accessible to any apps. This helps prevent recordings being used by third parties without the user's consent.
- *End-user preview and sharing*: The user has the ability to preview and share the movie with a user interface vended by ReplayKit. The user interface is presented out-of-process through the iOS Extension infrastructure and has access to the generated movie file.

ReplayKit broadcasting

There are several layers of security built into broadcasting a movie:

- *Screen and audio capture*: The screen and audio capture mechanism during broadcasting is identical to movie recording and occurs in `replayd`.
- *Broadcast extensions*: For third-party services to participate in ReplayKit broadcasting, they're required to create two new extensions that are configured with the `com.apple.broadcast-services` endpoint:
 - A user interface extension that allows the user to set up their broadcast
 - An upload extension that handles uploading video and audio data to the service's back-end servers

The architecture helps ensure that hosting apps have no privileges to the broadcasted video and audio contents. Only ReplayKit and the third-party broadcast extensions have access.

- *Broadcast picker:* With the broadcast picker, users initiate system broadcasts directly from their app using the same system-defined user interface that's accessible using Control Center. The user interface is implemented using a private API and is an extension that lives within the ReplayKit framework. It is out-of-process from the hosting app.
- *Upload extension:* The extension that third-party broadcast services implement to handle video and audio content during broadcasting uses raw unencoded sample buffers. During this mode of handling, video and audio data is serialized and passed to the third-party upload extension in real time through a direct XPC connection. Video data is encoded by extracting the `IOSurface` object from the video sample buffer, encoding it securely as an XPC object, sending it over through XPC to the third-party extension, and decoding it securely back into an `IOSurface` object.

SiriKit security for iOS, iPadOS, and watchOS

Siri uses the app extension system to communicate with third-party apps. On a device, Siri can access the user's contact information and the device's current location. But before it provides protected data to an app, Siri checks the app's user-controlled access permissions. According to those permissions, Siri passes only the relevant fragment of the original user utterance to the app extension. For example, if an app doesn't have access to contact information, Siri won't resolve a relationship in a user request such as "Pay my mother 10 dollars using Payment App." In this case, the app would see only the literal term "my mother."

However, if the user has granted the app access to contact information, the app would receive resolved information about the user's mother. If a relationship is referenced in the body portion of a message—for example, "Tell my mother on MessageApp that my brother is awesome"—Siri doesn't resolve "my brother" regardless of the app's permissions.

SiriKit-enabled apps can send app-specific or user-specific vocabulary to Siri, such as the names of the user's contacts. This information allows Siri's speech recognition and natural language understanding to recognize vocabulary for that app and is associated with a random identifier. The custom information remains available as long as the identifier is in use, or until the user turns off the app's Siri integration in Settings, or until the SiriKit-enabled app is uninstalled.

For an utterance like "Get me a ride to my mom's home using RideShareApp," the request requires location data from the user's contacts. For that request only, Siri provides the required information to the app's extension, regardless of the user permission settings for location or contact information for the app.

WidgetKit security

WidgetKit is the framework developers use to offer widgets and watch complications. Both may show sensitive information and can be highly visible, especially on devices with an Always On display.

On an iPhone, users can configure whether to show sensitive data on the Lock Screen and while in Always On. In Settings, they can deactivate data access for Lock Screen widgets in the "Allow Access When Locked" section of Settings > Face ID & Passcode.

On Apple Watch, users can configure whether to show sensitive data during Always On by choosing Settings > Display & Brightness > Always On > Hide Sensitive Complications. They can also choose to show redacted content for all or individual complications.

If a user chooses to hide content they consider private, WidgetKit renders a placeholder or redactions. To configure redactions, a developer needs to:

1. Implement the `redacted(reason:)` callback.
2. Read out the `privacy` property.
3. Provide custom placeholder views.

Developers can also render a view as unredacted with the `unredacted()` view modifier.

As an alternative to marking individual views as privacy sensitive, for example, if an entire widget content is privacy sensitive, the developer can add the Data Protection capability to a widget extension. Until a user unlocks their device to match the privacy level selected, WidgetKit displays placeholders instead of the widget content. The developer needs to enable the Data Protection capability for the widget extension in Xcode and then set the Data Protection entitlement to the value that fits the level of privacy they want to offer:

- `NSFileProtectionComplete`
- `NSFileProtectionCompleteUnlessOpen`

WidgetKit hides these widgets' content when the device is passcode locked and displays a placeholder until a user authenticates after they restart their device. Additionally, these iOS widgets aren't available as iPhone widgets on Mac.

Secure device management

Secure device management overview

iOS, iPadOS, macOS, tvOS, visionOS, and watchOS support flexible security policies and configurations that are easy to enforce and manage. Through them, organizations can protect corporate information and help ensure that employees meet enterprise requirements, even if they are using devices they've provided themselves—for example, as part of a “bring your own device” (BYOD) program.

Organizations can use the device management framework implemented by a device management service developer to enforce passcode requirements, configure settings, restrict functionality, and even remotely wipe corporate data on managed devices. This helps keep corporate data secure, even when employees use their personal devices to access this data.

Device management

Device management security overview

Apple operating systems support device management, which allows organizations to securely configure and manage scaled Apple device deployments.

How device management works securely

Device management capabilities are built on operating system technologies, such as configurations, over-the-air enrollment, and the [Apple Push Notification service \(APNs\)](#). For example, APNs is used to wake the device and trigger it to communicate directly with its device management service over a secured connection. No confidential or proprietary information is transmitted over APNs.

Using a device management service, IT departments can enroll Apple devices in an enterprise or educational environment, wirelessly configure and update settings, monitor compliance, manage software updates, and even remotely wipe or lock managed devices.

On devices with iOS 13, iPadOS 13.1, macOS 10.15, visionOS 1.1, or later, Apple added support an additional enrollment option specifically designed for “bring your own device” BYOD programs. User Enrollment provides more autonomy for users on their own devices, while increasing the security of enterprise data by cryptographically separating managed data. This provides a better balance of security, privacy, and user experience for BYOD programs. A similar data separation mechanism has been added for account-driven Device Enrollments in iOS 17, iPadOS 17, macOS 14, visionOS 1.1, or later.

Enrollment types

- *User Enrollment:* User Enrollment is designed for devices owned by the user and is integrated with Managed Apple Accounts to establish a user identity on the device. Managed Apple Accounts are required to initiate the enrollment, and the user needs to successfully authenticate for the enrollment to succeed. Managed Apple Accounts can be used alongside a personal Apple Account that the user has already signed in with. Managed apps and accounts use the Managed Apple Account, and personal apps and accounts use the personal Apple Account.
- *Device Enrollment:* Device Enrollment allows organizations to have users manually enroll devices and then manage many different aspects of device use, including the ability to erase the device. Device Enrollment also has a larger set of configurations and restrictions that can be applied to the device. When a user removes an enrollment profile, all configurations, settings, and managed apps based on that enrollment profile are removed. Similar to User Enrollment, Device Enrollment can also be integrated with a Managed Apple Account. This account-driven Device Enrollment also provides the ability to use a Managed Apple Account alongside a personal Apple Account and cryptographically separates corporate data.
- *Automated Device Enrollment:* Automated Device Enrollment lets organizations configure and manage devices from the moment the devices are removed from the box. These devices are known as *supervised*, and users have the option to prevent the device management service profile from being removed by the user. Automated Device Enrollment is designed for devices owned by the organization.

Device restrictions

Restrictions can be added—or in some cases, removed—by administrators to help prevent users from accessing a specific app, service, or function of an Apple device that’s enrolled in a device management service. Restrictions are sent to devices in a restrictions payload, which is part of a configuration. Certain restrictions on an iPhone may be mirrored on a paired Apple Watch.

Passcode and password settings management

By default, the user’s passcode can be defined as a numeric PIN on iOS, iPadOS, visionOS, and watchOS. On iPad, iPhone, and Apple Vision Pro devices with biometric authentication, the default passcode length is six digits, with a minimum of four digits. Because longer and more complex passcodes are harder to guess or attack, they are recommended.

Administrators can enforce complex passcode requirements and other policies using a device management service on iOS, iPadOS, and visionOS, or Microsoft Exchange. An administrator password is needed when the macOS passcode policy payload is installed manually. Passcode policies can require a certain passcode length, composition, or other attributes.

Apple Watch uses numeric passcodes by default. If a passcode policy applied to a managed Apple Watch requires the use of non-numeric characters, the paired iPhone needs to be used to unlock the device.

Managed Device Attestation for Apple devices

Managed Device Attestation is available in iOS 16, iPadOS 16, macOS 14, tvOS 16, visionOS 1.1, watchOS 9, or later. It uses the Secure Enclave to provide cryptographic assurances about the identity of a device and its security posture. iPad, iPhone, Apple TV devices require the A11 Bionic chip or later, and only Mac computers with Apple silicon are supported. Managed Device Attestation helps to protect against the following threats:

- A compromised device lying about its properties
- A compromised device providing an outdated attestation
- A compromised device sending a different device's identifiers
- Private key extraction for use on a rogue device
- An attacker hijacking a certificate request to trick the certificate authority (CA) into issuing the attacker a certificate

With managed device attestation, a device can request an attestation from the Apple attestation servers, which return an array of data consisting of a leaf and intermediate certificate which is rooted in the Apple Enterprise Attestation Root CA. Depending on the device type, the leaf certificate can contain specific properties as shown in the following table.

OID and value	Minimum supported operating system versions	Description
1.2.840.113635.100.8.9.1 Serial number	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Represents the serial number of the device and can be used to identify a device. To help protect the user's privacy, this value isn't included when using Managed Device Attestation with User Enrollment.
1.2.840.113635.100.8.9.2 UDID	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Represents the unique hardware ID and can be used to identify a device. On a Mac, the UDID matches the provisioning UDID of the device. To help protect the user's privacy, this value isn't included when using Managed Device Attestation with User Enrollment.
1.2.840.113635.100.8.10.2 sepOS version	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Represents the version of the Secure Enclave firmware.

OID and value	Minimum supported operating system versions	Description
1.2.840.113635.100.8.11.1 Freshness code	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	A unique and non-predictable code identifying a particular attestation. This indicates that the attestation was generated after the code was created.
1.2.840.113635.100.8.13.1 SIP status	macOS 14	Represents the SIP enablement status on a Mac with Apple silicon.
1.2.840.113635.100.8.13.2 Secure boot status	macOS 14	Represents the selected secure boot configuration of a Mac with Apple silicon.
1.2.840.113635.100.8.13.3 Third-party kernel extensions allowed	macOS 14	Represents whether or not third-party kernel extensions are allowed on a Mac with Apple silicon.
1.2.840.113635.100.8.10.3 LLB version	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Represents the version of the Low-Level Bootloader.
1.2.840.113635.100.8.10.1 Operating system version	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Represents the operating system and iBoot version.
1.2.840.113635.100.8.9.4 Software update device ID	iOS 16 iPadOS 16 macOS 14 tvOS 16 visionOS 1.1 watchOS 9	Attests the SoftwareUpdateDeviceID returned in the DeviceInformation query which can be used by a device management service to determine whether a particular update is applicable to the device.

A request for an attestation is initiated either by a DeviceInformation request sent by the device management service or by installing an ACME configuration. In both cases, the device receives a *freshness code* from either the device management service or ACME solution. When using DeviceInformation attestation, the freshness code is the value of the DeviceAttestationNonce specified in the request. When using ACME attestation, the freshness code is the SHA-256 hash of the token from the device-attest-01 challenge. The resulting value is included as a property in the leaf certificate and allows the device management service or ACME solution to verify that the attestation matches the request.

Upon receiving the attestation, a backend service needs to carefully perform validation checks. These checks include ensuring the leaf cert was issued by the Apple Enterprise Attestation Root CA, comparing the hash of the freshness code to the expected value, and examining other properties in the attestation.

Depending on an organization's deployment model, Managed Device Attestation can be an important foundation of a modern and secure deployment and used in different ways:

- Use an ACME-issued certificate to authenticate the connection from the client to the device management service and leverage DeviceInformation attestation to verify properties of a device on an ongoing basis.
- Verify the identity of a device and its security posture and have the ACME solution perform a trust evaluation before issuing a certificate. In this way it ensures that only devices meeting the required standards receive a certificate.
- Embed the device's properties from the attestation in an ACME certificate, and perform the trust evaluation on the relying parties.

Hardware-bound keys

As part of issuing a certificate using the ACME protocol, a device can be requested to provide an attestation which also causes the associated key pair to be protected by the Secure Enclave so that it benefits from its strong hardware security and helps to prevent exporting the private key.

To create a hardware-bound key, the ACME configuration needs to use the ECSECPPrimeRandom algorithm with a key size of 256 or 384 bit. This specifies a key pair on the P-256 or P-384 curves as defined in NIST SP 800-186.

To generate the private key, the Secure Enclave first uses the following to create a wrapped cryptographic secret:

- A combination of its hardware True Random Number Generator (TRNG)
- Entropy stored within the secure non-volatile storage of the Secure Enclave
- A Data Protection key

The operating system stores the wrapped secret however, the wrapped secret is useless by itself. To derive the private key the Secure Enclave needs to use the same secret, the same stored entropy, and the same Data Protection key. Because the stored entropy and Data Protection key are unique to that specific Secure Enclave, only that Secure Enclave is capable of generating the private key which effectively binds the hardware-bound key to that Secure Enclave.

When the device is erased, the Secure Enclave effaces the stored entropy. Because of this, a hardware-bound key can't be regenerated after a backup and restore, even when restoring to the same device.

Any configurations containing an ACME payload with a hardware-bound key are removed upon restore. If a hardware-bound key is used as the device management service client identity, the device unenrolls. In this scenario if the device was enrolled through Automated Device Enrollment, the device fetches its enrollment profile again and reenrolls.

Attestation process security

Attestation process security

When the ACME configuration specifies a hardware-bound key, or when the device management service requests a DeviceInformation attestation, the device uses the following steps:

1. The operating system requests a hardware-bound key from the Secure Enclave, called the *attested key*. For ACME attestation, this key is part of the issued certificate identity. For DeviceInformation attestation the key is generated by the operating system and reused for all subsequent DeviceInformation attestations for the lifetime of the enrollment.
2. The operating system requests a hardware attestation from the Secure Enclave. It specifies the attested key, the freshness code, and which properties it wants attested.
3. The Secure Enclave generates and returns a new hardware attestation.
4. The operating system sends the hardware attestation to the Apple attestation servers.
5. The attestation servers validate the request and either refuse to issue an attestation certificate or generate an attestation containing only the properties and values that it was able to confirm. The public key of the attestation certificate matches the attested key.
6. The operating system provides the resulting attestation certificate to the device management service or ACME solution.

For more information about validating the provided attestation certificate, see [Validating a Managed Device Attestation](#) on the Apple Developer website.

About hardware attestation

When the Secure Enclave generates a hardware attestation, it includes the following elements, among others:

- Identifiers of the main logic board.
- The public key of the attested key.
- Hashes of the following software components which are calculated during the boot process and stored in hardware registers (similar to Sealed Key Protection):
 - The Secure Enclave firmware (sepOS).
 - The Image4 manifest attached to LLB which also contains a measurement of all other system-paired firmware.
 - The operating system Image4 manifest which contains measurements of all of the OS-paired firmware.
- Other device-specific values like ECID and ChipID.
- On Mac computers, details of the LocalPolicy.
- The freshness code.

If the Secure Enclave is unable to unwrap the attested key, the Secure Enclave refuses to generate a hardware attestation. This ensures that the attested key is bound to that specific Secure Enclave and all other attested properties are associated with that key.

All of these elements are compiled into a custom binary structure and signed with the UIK. This ties the hardware attestation with a specific identity that Apple's attestations can verify, allowing them to associate the request with a specific activation record, and therefore with a specific manufacturing record.

Generating the attestation certificate

When Apple's attestation servers receive an attestation request they:

- Validate the User Identity Key (UIK) signature of the hardware attestation. If the signature is invalid, no attestation is generated.
- Look up `ucrt` in Apple's activation records which corresponds to the public key of UIK. If no `ucrt` is found, no attestation is generated.
- Look up the `scrt` which corresponds to the `ucrt` of the device. The `scrt` is a certificate issued by Apple servers during manufacturing based on the Silicon Identity Key (SIK). The SIK is generated by the Secure Enclave and its public key is collected during manufacturing from the device. The SIK is used during activation to verify that the Secure Enclave of the activating device is in possession of the private key. The SIK persists throughout the lifetime of a device and acts as a unique identifier. Using the `scrt`, the manufacturing record of the device is retrieved. If no record is found, no attestation is generated.
- Verify the identifiers of hardware components match manufacturing records. If the component identifiers don't match manufacturing records, no attestation is generated.
- Copies the freshness code into the attestation, if present.
- If requested by the attestation:
 - Compare calculated hashes of the software components to known hashes from build records. If there's a match, the official version number is inserted into the attestation; if not, the OID is omitted or its value is blank.

When images for LLB, Secure Enclave, or the main operating system are built, hashes are calculated and recorded in a build records database, along with the official version numbers for those images.
 - Calculate the UDID from the ECID and ChipID.
 - Copy values for the requested OIDs from the LocalPolicy into the attestation if the device is a Mac.

The practical result of these validations is that no attestation is generated if the attestation servers can't recognize the device as genuine Apple hardware. Each property in the attestation certificate is included only if the attestation servers are able to verify the property's value, with the exception of the freshness code which is included without any validation.

Configuration enforcement

Configurations are the primary way that a device management service delivers and manages policies and restrictions on managed devices. If organizations need to configure a large number of devices—or to provide lots of custom email settings, network settings, or certificates to a large number of devices—configurations are a safe and secure way to do it.

Configurations

A *configuration* is an XML profile or json-formatted file following a certain structure and consists of payloads that load settings and authorization information onto Apple devices. Configurations automate the configuration of settings, accounts, restrictions, and credentials. These files can be created by a device management service or Apple Configurator for Mac, or they can be created manually. Before organizations send a configuration to an Apple device, they need to enroll the device in the device management service using an enrollment profile.

Enrollment profiles

An *enrollment profile* is a configuration with a device management service payload that enrolls the device in the device management service specified for that device. This allows the service to send commands and configurations to the device and to query certain aspects of the device. When a user removes an enrollment profile, all configurations, their settings, and depending on the enrollment type and used configuration also managed apps based on that enrollment profile are removed with it. There can be only one enrollment profile on a device at a time.

Example configurations

A configuration contains a number of settings in specific payloads that can be specified, including (but not limited to):

- Passcode and password policies
- Restrictions on device features
- Network and VPN settings
- Microsoft Exchange settings
- Mail settings
- Account settings
- LDAP directory service settings
- CalDAV calendar service settings
- Credentials and identities
- Certificates
- Software updates

Profile signing and encryption

Configuration profiles can be signed, to validate their origin, and encrypted, to help ensure their integrity and protect their contents. Configuration profiles for Apple devices are encrypted using the Cryptographic Message Syntax (CMS) specified in [RFC 5652](#), supporting 3DES and AES-128.

Profile installation

Configurations can be installed on devices using a device management service or manually by the users. Alternatively, Apple Configurator for Mac can be used to deploy configurations to iPad, iPhone, and Apple TV devices. Some configurations require the installation using a device management service. For information on how to remove profiles, see [Intro to device management profiles](#) in Apple Platform Deployment.

Note: On supervised devices, configuration profiles can also be locked to a device. This is designed to prevent their removal or to allow removal only with a passcode.

Automated Device Enrollment

Organizations can automatically enroll iPad, iPhone, Mac, Apple TV, and Apple Vision Pro devices in a [device management service](#) without having to physically touch or prepare the devices before users get them. After an organization signs up for Apple School Manager or Apple Business, administrators sign in to the web portal and link to their device management service. The devices they purchased can then be assigned to users through that service. During the device configuration process, the device queries Apple servers for an assigned device management service and if so, reaches out to it to perform the enrollment. Using Automated Device Enrollment and a compatible device management service allows organizations to implement the following security measures:

- Have users authenticate as part of the initial setup flow in the Apple device's Setup Assistant during activation.
- Provide a preliminary configuration with limited access and require additional device configuration to access sensitive data.
- Require devices to run a minimum operating system version before enrolling.
- Enforce FileVault enablement on a Mac.

After a device has enrolled with a device management service, any configurations, restrictions, or controls are automatically installed.

The setup process for users can be further simplified by removing specific steps in the Setup Assistant for devices so that users are up and running quickly. If steps are skipped, the more privacy-preserving setting is used. For example, if the pane to configure location services is skipped, the service is not enabled during Setup Assistant.

Administrators can also control whether users can remove the device management service profile from the device and help ensure that configurations and restrictions are in place throughout the life cycle of the device.

Apple School Manager and Apple Business

Apple School Manager and Apple Business are services for IT administrators to deploy Apple devices that an organization has purchased directly from Apple or through participating Apple Authorized Resellers and carriers.

When used with a device management service, administrators can simplify the setup process for users, configure device settings, and distribute apps and books purchased in these three services. Apple School Manager also integrates with Student Information Systems (SISs) directly or using SFTP, and all three services support directory synchronization and federated authentication, so accounts can be automatically provisioned, updated, and deprovisioned based on an organization's identity provider (IdP).

Apple maintains certifications in compliance with the ISO/IEC 27001 and 27018 standards to enable Apple customers to address their regulatory and contractual obligations. These certifications provide our customers with an independent attestation over Apple's Information Privacy and Security practices for in-scope systems. For more information, see [Apple internet services security certifications](#) in Apple Platform Certifications.

Note: To learn whether an Apple program is available in a specific country or region, see the Apple Support article [Availability of Apple programs and payment methods for education and business](#).

Device supervision

Supervision generally denotes that the device is owned by the organization, giving them additional control over the device's configuration and restrictions. For more information, see [About Apple device supervision](#) in Apple Platform Deployment.

Supervision gets automatically enabled on a device when using Automated Device Enrollment.

Activation Lock security

Activation Lock helps to prevent unauthorized users from reactivating an iPad, iPhone, Mac, Apple Vision Pro, and Apple Watch if it's lost or stolen. It remains enabled even if the device is erased. This makes it more difficult for someone to use or sell a missing device. How Apple enforces Activation Lock varies depending on the device.

Activation Lock on iPhone parts

Apple is extending Activation Lock for iPhone to cover individual parts to help deter stolen parts from entering the market. During a repair, if an iPhone detects that a supported part came from another iPhone with Activation Lock or Lost Mode turned on, calibration is restricted for that part. This enhancement to the Activation Lock feature further extends Apple's commitment to protecting users while increasing consumer choice when it comes to repairs.

Behavior on iPad, iPhone, and Apple Vision Pro

On an unsupervised iPad, iPhone, and Apple Vision Pro, Activation Lock is enabled automatically when the user signs in to their Apple Account and turns on Find My.

On a supervised device, Activation Lock is disallowed by default, but a device management service can allow the user to enable it. This allows the service to escrow a bypass code from the device. That bypass code can then be used to turn off Activation Lock at a later time. A device generates a new bypass code when:

- Setting up the device the first time

- Setting up the device after an erase and not restoring the device from a backup of that same device
- Setting up the device after an erase and restoring the device from a backup of a different device

Alternatively, for managed and supervised devices, a device management service can contact Apple servers directly to enable Activation Lock. This is done entirely server-side and has no dependencies on user actions or the state of the device. The device management service needs to create a 31-byte bypass code, then send it to Apple servers when it wants to enable Activation Lock for the device. The service's bypass code should be randomly created and unique for each device.

Activation Lock is enforced through the activation process after the Wi-Fi selection screen in Setup Assistant. When a device indicates that it's activating, it sends a request to the activation server to get an activation certificate.

Unsupervised iPad, iPhone, and Apple Vision Pro devices that are locked with Activation Lock can be unlocked by:

- The credentials of the personal Apple Account used to enable Activation Lock
- The previously used device passcode

Supervised iPad, iPhone, and Apple Vision Pro devices that are locked with Activation Lock can be unlocked by:

- The credentials of the personal Apple Account used to enable Activation Lock
- The credentials of the Managed Apple Account used to link the device management service with Apple School Manager or Apple Business
- The bypass code escrowed by the device management service
- The device management service making a server-side call to Apple servers with the same bypass code it used to enable Activation Lock

Note: The Setup Assistant in iOS, iPadOS, and visionOS won't progress unless a valid certificate can be obtained.

Behavior on Apple Watch

Activation Lock on an unsupervised Apple Watch is related to the Activation Lock status of the paired iPhone. If the iPhone has Activation Lock enabled, the Apple Watch is instructed to contact Apple servers at the end of the pairing process to turn on Activation Lock. If Activation Lock isn't enabled on the iPhone at the time of pairing, but enabled at a later time, the iPhone:

- Prompts all paired Apple Watch devices to contact Apple servers
- Can enable Activation Lock on the Apple Watch

As part of the initial pairing process, the iPhone sends a request to the activation server to get an activation certificate for the Apple Watch

If the Apple Watch is locked with Activation Lock, the user is prompted for the credentials of the Apple Account which was used to enable Activation Lock at that time to unpair, erase, or reactivate an Apple Watch.

Note: Pairing can't be completed unless a valid certificate can be obtained.

Behavior on Mac

On an unsupervised Mac, Activation Lock is enabled automatically when the user signs in with their Apple Account and turns on Find My. For a Mac that's supervised, Activation Lock is disallowed by default, but a device management service can allow the user to enable it. This allows the service to escrow a bypass code from the device. That bypass code can then be used to turn off Activation Lock at a later time. A device generates a new bypass code when:

- Setting up the device the first time
- Setting up the device after an erase

Additional behavior on a Mac with Apple silicon

On a Mac with Apple silicon, the Low-Level Bootloader (LLB) verifies that a valid LocalPolicy for the device exists and that the LocalPolicy policy anti-replay values match the values stored in the Secure Storage Component. The LLB boots to recoveryOS if:

- There is no LocalPolicy for the current macOS
- The LocalPolicy is invalid for that version of macOS
- The LocalPolicy anti-replay value hash values don't match the hashes of values stored in the Secure Storage Component

recoveryOS detects that the Mac isn't activated and contacts the activation server to get an activation certificate.

If the device is locked using Activation Lock while in recoveryOS, Activation Lock can be unlocked by:

- The credentials of the personal Apple Account used to enable Activation Lock
- The previously used device password of the local user who enabled Activation Lock
- The bypass code escrowed by the device management service

After a valid activation certificate is obtained, that activation certificate key is used to obtain a RemotePolicy certificate. The Mac uses the LocalPolicy key and RemotePolicy certificate to produce a valid LocalPolicy.

Note: The LLB won't allow booting of macOS unless a valid LocalPolicy is present.

Additional behavior on a Mac with the T2 chip

On a Mac with the T2 chip, the T2 chip firmware verifies that a valid activation certificate is present before allowing the computer to boot to macOS. UEFI firmware loaded by the T2 chip is responsible for querying the activation status of the device from the T2 chip. The Mac boots to recoveryOS if:

- A valid activation certificate isn't present.
- recoveryOS detects that the Mac isn't activated and contacts the activation server to get an activation certificate.

If the Mac is locked using Activation Lock while in recoveryOS, Activation Lock can be unlocked by:

- The credentials of the personal Apple Account used to enable Activation Lock
- The previously used device password of the local user who enabled Activation Lock

- The bypass code escrowed by the device management service

Note: UEFI firmware won't allow macOS to boot unless a valid activation certificate is present.

Managing Activation Lock in Apple School Manager or Apple Business

If an Apple device is registered with an Apple School Manager or Apple Business organization, users with a role that has Manage Device privileges can turn off Activation Lock for organization-owned devices. This option is available only for devices registered with the organization before Activation Lock was enabled and that haven't been released. Because Activation Lock is turned off using server-side calls, a device doesn't need to be managed by a device management service.

Note: Devices that are currently locked with Activation Lock can't be added to an Apple School Manager or Apple Business organization.

Managed Lost Mode and remote wipe

Managed Lost Mode is used to locate supervised devices when they are stolen. After they are located, they can be remotely locked or erased.

Managed Lost Mode

If a supervised iPad or iPhone is lost or stolen, a [device management service](#) administrator can remotely enable Lost Mode (called Managed Lost Mode) on that device. When Managed Lost Mode is enabled, the current user is logged out and the device can't be unlocked. The screen displays a message that can be customized by the administrator, such as displaying a phone number to call if the device is found. The administrator can also request the device to send its current location (even if Location Services are off) and, optionally, play a sound. When an administrator turns off Managed Lost Mode, which is the only way the mode can be exited, the user is informed of this action through a message on the Lock Screen or an alert on the Home Screen.

Remote wipe

iPad, iPhone, Mac, Apple TV, Apple Vision Pro, and Apple Watch devices can be erased remotely by an administrator or user, rendering all data unreadable.

When a remote wipe command is initiated by a device management service or iCloud, the device sends an acknowledgment back to the service and performs the wipe. For remote wipe through Microsoft Exchange ActiveSync, the device checks in with the Microsoft Exchange Server before performing the wipe.

Remote wipe isn't possible in the following situations:

- With User Enrollment
- Using Microsoft Exchange ActiveSync when the account that was installed with User Enrollment
- Using Microsoft Exchange ActiveSync if the device is supervised

Users can also wipe supported devices in their possession using Settings (iPad, iPhone, and Apple Vision Pro) or System Settings (Mac). And as mentioned, iPad, iPhone, Apple Vision Pro, and Apple Watch, devices can be set to automatically wipe after a series of failed passcode attempts.

Instant remote wipe is available on a Mac with Apple silicon and a Mac with an Apple T2 Security Chip or if FileVault is turned on. Instant remote wipe is achieved by securely discarding the [media key](#).

Shared iPad security in iPadOS

Shared iPad is a multiuser mode for use in iPad deployments. It allows users to share an iPad while maintaining separation of documents and data for each user. Each user gets their own private, reserved storage location, which is implemented as an [Apple File System \(APFS\)](#) volume protected by the user's credential. Shared iPad requires the use of a Managed Apple Account that's issued and owned by the organization.

With Shared iPad, a user can sign in to any organizationally owned device that's configured for use by multiple users. User data is partitioned into separate directories, each in their own data protection domains and protected by both UNIX permissions and sandboxing. In iPadOS 13.4 or later, users can also sign in to a temporary session. When the user signs out of a temporary session, their APFS volume is deleted and its reserved space is returned to the system.

Signing in to Shared iPad

Both native and federated Managed Apple Accounts are supported when signing in to Shared iPad. When using a federated account for the first time, the user is redirected to the Identity Provider's (IdP) sign-in portal. After authenticated, a short-lived access token is issued for the backing Managed Apple Accounts and the login process proceeds similarly to the native Managed Apple Accounts sign-in process. After the user signs in, Setup Assistant on Shared iPad prompts the user to establish a passcode (credential) used to secure the local data on the device and to authenticate to the login screen in the future. Like a single-user device, in which the user would sign in once to their Managed Apple Account using their federated account and then unlock their device with their passcode, on Shared iPad the user signs in once using their federated account and from then on uses their established passcode.

When a user signs in without federated authentication, the Managed Apple Account is authenticated with [Apple Identity Service \(IDS\)](#) using the SRP protocol. If authentication is successful, a short-lived access token specific to the device is granted. If the user has used the device before, they already have a local user account, which is unlocked using the same credential.

If the user hasn't used the device before or is using the temporary session feature, Shared iPad provisions a new UNIX user ID, an APFS volume to store the user's personal data, and a local keychain. Because storage is allocated (reserved) for the user at the time the APFS volume is created, there may be insufficient space to create a new volume. In such an event, the system identifies an existing user whose data has finished syncing to the cloud and evicts that user from the device so that the new user can sign in. In the unlikely event that all existing users haven't completed uploading their cloud data, the new user sign in fails. To sign in, the new user needs to wait for the previous user's data to finish syncing, or have an administrator forcibly delete an existing user account, thereby risking data loss.

If the device isn't connected to the internet (for example, if the user has no Wi-Fi access point), authentication can occur against the local account for a limited number of days. In that situation, only users with previously existing local accounts or a temporary session can sign in. After the time limit has expired, users are required to authenticate online, even if a local account already exists.

After a user's local account has been unlocked or created, if it's remotely authenticated, the short-lived token issued by Apple's servers is converted to an iCloud token that permits signing in to iCloud. The users' settings are then restored and their documents and data are synced from iCloud.

While a user session is active and the device remains online, documents and data are stored on iCloud as they are created or modified. In addition, a background syncing mechanism helps ensure that changes are pushed to iCloud, or to other web services using NSURLSession background sessions, after the user signs out. After background syncing for that user is complete, the user's APFS volume is unmounted and can't be mounted again without the user signing back in.

Temporary sessions don't sync data with iCloud, and although a temporary session can sign into a third-party syncing service such as Box or Google Drive, there's no facility to continue syncing data when the temporary session ends.

Signing out of Shared iPad

When a user signs out of Shared iPad, that user's [keybag](#) is immediately locked and all apps are shut down. To accelerate the case of a new user signing in, iPadOS defers some ordinary sign-out actions temporarily and presents a sign-in window to the new user. If a user signs in during this time (approximately 30 seconds), Shared iPad performs the deferred cleanup as part of signing in to the new user account. However, if Shared iPad remains idle, it initiates the deferred cleanup. During the cleanup phase, the Sign-in Window is restarted as if another sign-out had occurred.

When a temporary session is ended, Shared iPad performs the full logout sequence and deletes the temporary session's APFS volume immediately.

Apple Configurator security

Apple Configurator for Mac features a flexible, secure, device-centric design that lets an administrator quickly and easily configure one or dozens of iPad, iPhone, and Apple TV (Wi-Fi and Ethernet only) devices connected to a Mac through USB (or for Apple TV, through Ethernet) before giving them to users. With Apple Configurator for Mac, an administrator can update software, install apps and configuration profiles, rename and change wallpaper on devices, export device information and documents, and much more.

Apple Configurator for Mac can also revive or restore Mac computers with Apple silicon, those with the Apple T2 Security Chip, and Apple Vision Pro (with a Developer Strap). When a Mac or Apple Vision Pro is revived or restored in this manner, the file containing the latest minor updates to the operating systems (macOS, recoveryOS for Apple silicon, or sepOS for T2) is securely downloaded from Apple servers and installed directly on the Mac. After a successful revive or restore, the file is deleted from the Mac with Apple Configurator. At no time can the user inspect or use this file outside of Apple Configurator.

Administrators can also choose to add devices to Apple School Manager or Apple Business using Apple Configurator for Mac or Apple Configurator for iPhone, even if the devices weren't purchased directly from Apple, an Apple Authorized Reseller, or an authorized cellular carrier. When the administrator sets up a device that has been manually enrolled, it behaves like any other device in one of those services, with mandatory supervision and device management service enrollment. For devices that weren't purchased directly, the user has a 30-day provisional period to release the device from one of those services, supervision, and a device management service.

Organizations can also use Apple Configurator for Mac to activate iPad, iPhone, and Apple TV devices that have absolutely no internet connection by connecting them to a host Mac with an internet connection while the devices are being set up. Administrators can restore, activate, and prepare devices with their necessary configuration including apps, profiles, and documents without ever needing to connect to either Wi-Fi or cellular networks. This feature doesn't allow an administrator to bypass any existing Activation Lock requirements normally required during nontethered activation.

Screen Time security

Screen Time is a built-in feature for seeing and managing how much time adults and their children spend on apps, websites and more. There are two types of users: adults and (managed) children.

Although Screen Time isn't a new system security feature, it's important to understand how it protects the privacy and security of the data gathered and shared between devices. Screen Time is available in iOS 12, iPadOS 13.1, and macOS 10.15, or later, and some features of watchOS 6 or later.

The table below describes the main features of Screen Time.

Feature	Supported operating system
Configure Downtime	iPadOS iOS macOS watchOS
Enforce additional restrictions	iPadOS iOS macOS watchOS
Set app limits	iPadOS iOS macOS watchOS
Set web usage limits	iPadOS iOS macOS
View usage data	iPadOS iOS macOS

For users managing their own device usage, Screen Time controls and usage data can be synced across devices associated to the same Apple Account using CloudKit end-to-end encryption. This requires that the user's account have two-factor authentication enabled (syncing is on by default). Screen Time replaces the Restrictions feature found in previous versions of iOS and iPadOS, and the Parental Controls feature found in previous versions of macOS.

On devices with iOS 13, iPadOS 13.1, and macOS 10.15, or later, Screen Time users and managed children automatically share their usage across devices if their Apple Account has two-factor authentication enabled. When a user clears Safari history or deletes an app, the corresponding usage data is removed from the device and all synced devices.

Parents and Screen Time

Parents can also use Screen Time in iOS, iPadOS, and macOS devices to understand and control their children's use. If the parent is a family organizer (in iCloud Family Sharing), they can view usage data and manage Screen Time settings for their children. Children are informed when their parents turn on Screen Time, and they can monitor their own usage as well. When parents turn on Screen Time for their children, the parents set a passcode so their children can't make changes. When they reach age of majority (age will vary depending on country or region), the children can turn this monitoring off.

Usage data and configuration settings are transferred between the parent's and child's devices using the end-to-end encrypted [Apple Identity Service \(IDS\)](#) protocol. Encrypted data may be briefly stored on IDS servers until it's read by the receiving device (for example, as soon as the iPad or iPhone is turned on, if it was off). This data isn't readable by Apple.

Screen Time analytics

If the user turns on Share iPhone & Watch Analytics, only the following anonymized data is collected so that Apple can better understand how Screen Time is being used:

- Was Screen Time turned on during Setup Assistant or later in Settings
- Change in Category usage after creating a limit for it (within 90 days)
- Is Screen Time turned on
- Is Downtime turned on
- Number of times the "Ask for more" query was used
- Number of app limits
- Number of times users viewed usage in the Screen Time settings, per user type and per view type (local, remote, widget)
- Number of times users ignore a limit, per user type
- Number of times users delete a limit, per user type

No specific app or web usage data is gathered by Apple. When a user sees a list of apps in Screen Time usage information, the app icons are pulled directly from the App Store, which doesn't retain any data from these requests.

Glossary

Address Space Layout Randomization (ASLR) A technique employed by operating systems to make the successful exploitation by a software bug much more difficult. By ensuring memory addresses and offsets are unpredictable, exploit code can't hard code these values.

AES (Advanced Encryption Standard) A popular global encryption standard used to encrypt data to keep it private.

AES cryptographic engine A dedicated hardware component that implements AES.

AES-XTS A mode of AES defined in IEEE 1619-2007 meant to work for encrypting storage media.

Apple File System (APFS) The default file system for iOS, iPadOS, tvOS, Mac computers using macOS 10.13 or later, visionOS, and watchOS. APFS features strong encryption, space sharing, snapshots, fast directory sizing, and improved file system fundamentals.

Apple Business A unified platform designed to empower organizations with seamless device management, robust support, secure cloud storage, and powerful tools to enhance customer engagement and brand presence across Apple's ecosystem.

Apple Identity Service (IDS) Apple's directory of iMessage public keys, APNs addresses, and phone numbers and email addresses that are used to look up the keys and device addresses.

Apple Push Notification service (APNs) A worldwide service provided by Apple that delivers push notifications to Apple devices.

Apple School Manager A simple, web-based portal for IT administrators that provides a fast, streamlined way for organizations to deploy Apple devices that they have purchased directly from Apple or from a participating Apple Authorized Reseller or carrier. They can automatically enroll devices in their device management service without having to physically touch or prepare the devices before users get them.

Apple Security Bounty A reward given by Apple to researchers who report a vulnerability that affects the latest shipping operating systems and, where relevant, the latest hardware.

Boot Camp A Mac utility that supports the installation of Microsoft Windows on supported Mac computers.

Boot Progress Register (BPR) A set of system-on-chip (SoC) hardware flags that software can use to track the boot modes the device has entered, such as Device Firmware Update (DFU) mode and Recovery mode. After a Boot Progress Register flag is set, it can't be cleared. This allows later software to get a trusted indicator of the state of the system.

Boot ROM The very first code executed by a device's processor when it first boots. As an integral part of the processor, it can't be altered by either Apple or an attacker.

CKRecord A dictionary of key-value pairs that contain data saved to or fetched from CloudKit.

Data Protection A file and keychain protection mechanism for supported Apple devices. It can also refer to the APIs that apps use to protect files and keychain items.

Data Vault A mechanism—enforced by the kernel—to protect against unauthorized access to data regardless of whether the requesting app is itself sandboxed.

Device Firmware Upgrade (DFU) mode A mode in which a device's Boot ROM code waits to be recovered over USB. The screen is black when in DFU mode, but upon connecting to a computer with the Finder or iTunes, the following prompt is presented: "Finder (or the iTunes) has detected an [iPad][iPhone] in Recovery mode. The user needs to restore this [iPad][iPhone] before it can be used with Finder (or the iTunes)."

device management service A service that lets an administrator remotely manage enrolled devices. After a device is enrolled, the administrator can use the device management service over the network to configure settings and perform other tasks on the device without user interaction.

direct memory access (DMA) A feature that allows hardware subsystems to access main memory directly, bypassing the CPU.

Effaceable Storage A dedicated area of NAND storage, used to store cryptographic keys, that can be addressed directly and wiped securely. While it doesn't provide protection if an attacker has physical possession of a device, keys held in Effaceable Storage can be used as part of a key hierarchy to facilitate fast wipe and forward security.

Elliptic Curve Diffie-Hellman Exchange Ephemeral (ECDHE) A key exchange mechanism based on elliptic curves. ECDHE allows two parties to agree on a secret key in a way that prevents the key from being discovered by an eavesdropper watching the messages between the two parties.

Elliptic Curve Digital Signature Algorithm (ECDSA) A digital signature algorithm based on elliptic curve cryptography.

Enhanced Serial Peripheral Interface (eSPI) An all-in-one bus designed for synchronous serial communication.

Exclusive Chip Identification (ECID) A 64-bit identifier that's unique to the processor in each iPad or iPhone that's as part of the personalization process, it's not considered a secret.

file system key The key that encrypts each file's metadata, including its class key. This is kept in Effaceable Storage to facilitate fast wipe, rather than confidentiality.

Gatekeeper On devices with macOS, a technology designed to help ensure that only trusted software runs on a user's Mac.

group ID (GID) Like the UID, but common to every processor in a class.

hardware security module (HSM) A specialized tamper-resistant computer that safeguards and manages digital keys.

HMAC A hash-based message authentication code based on a cryptographic hash function.

iBoot The stage 2 boot loader for all Apple devices. Code that loads XNU, as part of the secure boot chain. Depending on the system-on-chip (SoC) generation, iBoot may be loaded by the Low-Level Bootloader or directly by the Boot ROM.

Input/Output Memory Management Unit (IOMMU) An input/output memory management unit. A subsystem in an integrated chip that controls access to address space from other input/output devices and peripherals.

integrated circuit (IC) Also known as a *microchip*.

Joint Test Action Group (JTAG) A standard hardware debugging tool used by programmers and circuit developers.

keybag A data structure used to store a collection of class keys. Each type (user, device, system, backup, escrow, or iCloud Backup) has the same format.

A header containing: Version (set to four in iOS 12 or later), Type (system, backup, escrow, or iCloud Backup), Keybag UUID, an HMAC if the keybag is signed, and the method used for wrapping the class keys—tangling with the UID or PBKDF2, along with the salt and iteration count.

A list of class keys: Key UUID, Class (which file or Keychain Data Protection class), wrapping type (UID-derived key only; UID-derived key and passcode-derived key), wrapped class key, and a public key for asymmetric classes.

keychain The infrastructure and a set of APIs used by Apple operating systems and third-party apps to store and retrieve passwords, keys, and other sensitive credentials.

key wrapping Encrypting one key with another. iOS and iPadOS use NIST AES key wrapping, in accordance with [RFC 3394](#).

Low-Level Bootloader (LLB) On a Mac with a two-stage boot architecture, the LLB contains the code that's invoked by the Boot ROM and that in turn loads iBoot, as part of the secure boot chain.

media key Part of the encryption key hierarchy. The media key helps ensure quick, secure data wiping of Apple devices. On iPad, iPhone, Apple TV, and Apple Watch, it does this by wrapping the metadata on the data volume. Without the media key, file keys are locked away, making files protected with Data Protection inaccessible. On Mac, the media key wraps key material, all metadata, and FileVault data. In both cases, wiping the media key blocks access to encrypted data.

memory controller The subsystem in a system on a chip that controls the interface between the system on a chip and its main memory.

NAND Nonvolatile flash memory.

Passcode-derived key (PDK) The encryption key derived from the entangling of the user password with the long-term SKP key and the UID of the Secure Enclave.

per-file key The key used by Data Protection to encrypt a file on the file system. The per-file key is wrapped by a class key and is stored in the file's metadata.

provisioning profile A property list (.plist file) signed by Apple that contains a set of entities and entitlements allowing apps to be installed and tested on an iOS or iPadOS device. A development provisioning profile lists the devices that a developer has chosen for ad hoc distribution, and a distribution provisioning profile contains the app ID of an enterprise-developed app.

Recovery mode A mode used to restore many Apple devices if it doesn't recognize the user's device so the user can reinstall the operating system.

ridge flow angle mapping A mathematical representation of the direction and width of the ridges extracted from a portion of a fingerprint.

Sealed Key Protection (SKP) A technology in Data Protection that protects, or seals, encryption keys with measurements of system software and keys available only in hardware (such as the UID of the Secure Enclave).

Secure Storage Component A chip designed with immutable RO code, a hardware random number generator, cryptography engines, and physical tamper detection. On supported devices, the Secure Enclave is paired with a Secure Storage Component for anti-replay value storage. To read and update anti-replay values, the Secure Enclave and storage chip employ a secure protocol that helps ensure exclusive access to the anti-replay values. There are multiple generations of this technology with differing security guarantees.

sepOS The Secure Enclave firmware, based on an Apple-customized version of the L4 microkernel.

software seed bits Dedicated bits in the Secure Enclave AES Engine that get appended to the UID when generating keys from the UID. Each software seed bit has a corresponding lock bit. The Secure Enclave Boot ROM and operating system can independently change the value of each software seed bit as long as the corresponding lock bit hasn't been set. After the lock bit is set, neither the software seed bit nor the lock bit can be modified. The software seed bits and their locks are reset when the Secure Enclave reboots.

SSD controller A hardware subsystem that manages the storage media (solid-state drive).

System Coprocessor Integrity Protection (SCIP) A mechanism Apple uses designed to prevent modification of coprocessor firmware.

system on a chip (SoC) An integrated circuit (IC) that incorporates multiple components into a single chip. The Application Processor, the Secure Enclave, and other coprocessors are components of the SoC.

system software authorization A process that combines cryptographic keys built into hardware with an online service to check that only legitimate software from Apple, appropriate to supported devices, is supplied and installed at upgrade time.

tangling The process by which a user's passcode is turned into a cryptographic key and strengthened with the device's UID. This process helps ensure that a brute-force attack needs to be performed on a given device, and thus is rate limited and can't be performed in parallel. The tangling algorithm is PBKDF2, which uses AES keyed with the device UID as the pseudorandom function (PRF) for each iteration.

Unified Extensible Firmware Interface (UEFI) firmware A replacement technology for BIOS to connect firmware to a computer's operating system.

Uniform Resource Identifier (URI) A string of characters that identifies a web-based resource.

unique ID (UID) A 256-bit AES key that's burned into each processor at manufacture. It can't be read by firmware or software, and it's used only by the processor's hardware AES Engine. To obtain the actual key, an attacker would have to mount a highly sophisticated and expensive physical attack against the processor's silicon. The UID isn't related to any other identifier on the device including, but not limited to, the UDID.

xART An abbreviation for eXtended Anti-Replay Technology. A set of services that provide encrypted, authenticated persistent storage for the Secure Enclave with anti-replay capabilities based on the physical storage architecture. See Secure Storage Component.

XProtect On devices with macOS, an antivirus technology for the signature-based detection and removal of malware.

XNU The kernel at the heart of the Apple operating systems. It's assumed to be trusted, and it enforces security measures such as code signing, sandboxing, entitlement checking, and Address Space Layout Randomization (ASLR).

Document revision history

Document revision history

August 2026

Topics added:

- [Terminal and script protections](#)
- [Search on iCloud.com security](#)
- [SharePlay security](#)
- [Nearby sharing security](#)

Topics updated:

- [Intro to Apple platform security](#)
- [Microsoft Windows on an Intel-based Mac with a T2 chip](#)
- [Data Protection overview](#)
- [Tap to Pay on iPhone](#)
- [Access using Apple Wallet](#)

March 2026

Topics added:

- [Mac on-screen camera indicator light](#)

January 2026

Topics added:

- [Automatically unlock Apple devices](#)
- [Communicating emergency information using satellites](#)
- [Quantum-secure cryptography with Apple devices](#)
- [Platform Single Sign-on](#)
- [EnergyKit data security](#)
- [Attestation process security](#)

Topics updated:

- [Intro to Apple platform security](#)
- [Apple SoC security](#)
- [Background Security Improvements](#)
- [Operating system integrity](#)
- [System security for watchOS](#)
- [Protecting user data in the face of attack](#)
- [Protecting access to user's health data](#)
- [Volume encryption with FileVault](#)
- [Managing FileVault](#)
- [App security overview](#)
- [Apple Cash security](#)
- [Tap to Pay on iPhone](#)
- [Car key security](#)
- [Car key transactions](#)
- [IDs in Apple Wallet](#)
- [TLS security](#)
- [VPN security](#)
- [Managed Device Attestation](#)

December 2024

Topics added:

- [Optic ID security](#)
- [Optic ID matching security](#)
- [Automatically unlock Apple devices](#)
- [Check In security](#)
- [Privacy features when connecting to wireless networks](#)
- [Wi-Fi privacy with Apple devices](#)
- [NFC & SE Platform security](#)
- [Managed Device Attestation](#)

Topics updated:

- [Apple SoC security](#)
- [Uses for Optic ID, Face ID, and Touch ID](#)
- [Boot process for iPad and iPhone devices](#)
- [Gatekeeper and runtime protection](#)
- [Protecting against malware](#)
- [How Apple Pay keeps users' purchases protected](#)
- [Card provisioning security overview](#)

- [Adding credit or debit cards to Apple Pay](#)
- [Payment authorization with Apple Pay](#)
- [Contactless passes in Apple Pay](#)
- [Rendering cards unusable with Apple Pay](#)
- [Tap to Pay on iPhone](#)
- [Adding transit and eMoney cards to Apple Wallet](#)
- [Security features when connecting to wireless networks](#)
- [Wi-Fi security with Apple devices](#)
- [Wi-Fi privacy with Apple devices](#)
- [Wi-Fi password sharing security](#)
- [Developer kit security overview](#)
- [Activation Lock security](#)

Topics updated that include *only* Optic ID, Apple Vision Pro, or visionOS

Optic ID additions:

- [Hardware security overview](#)
- [Root Cryptographic Keys](#)
- [Secure nonvolatile storage](#)
- [The Secure Neural Engine](#)
- [Biometric security](#)
- [Optic ID, Face ID, Touch ID, passcodes, and passwords](#)
- [Uses for Optic ID, Face ID, and Touch ID](#)
- [Passcodes and passwords](#)
- [Data Protection classes](#)
- [Keychain data protection](#)
- [Secure features in the Notes app](#)

Apple Vision Pro additions:

- [Hardware security overview](#)
- [Apple SoC security](#)
- [The Secure Enclave](#)
- [Biometric security](#)
- [Optic ID, Face ID, Touch ID, passcodes, and passwords](#)
- [Uses for Optic ID, Face ID, and Touch ID](#)
- [Secure intent and connections to the Secure Enclave](#)
- [System security overview](#)
- [Encryption and Data Protection overview](#)
- [Passcodes and passwords](#)
- [Data Protection overview](#)

- [Data Protection classes](#)
- [Role of Apple File System](#)
- [App code signing process](#)
- [App protection and app groups](#)
- [Secure features in the Notes app](#)
- [Passcode security overview](#)
- [iCloud Keychain security overview](#)
- [Apple Pay security overview](#)
- [Rendering cards unusable with Apple Pay](#)
- [iMessage security overview](#)
- [VPN security](#)
- [Security features when connecting to wireless networks](#)
- [Privacy features when connecting to wireless networks](#)
- [Wi-Fi privacy with Apple devices](#)
- [Device management security overview](#)
- [Managed Device Attestation](#)
- [Automated Device Enrollment](#)
- [Managed Lost Mode and remote wipe](#)

visionOS additions:

- [Intro to Apple platform security](#)
- [Hardware security overview](#)
- [Operating system integrity](#)
- [Random number generation](#)
- [Passcodes and passwords](#)
- [Keybags for Data Protection](#)
- [Protecting keys in alternate boot modes](#)
- [Role of Apple File System](#)
- [Keychain data protection](#)
- [Protecting app access to user data](#)
- [Digital signing and encryption](#)
- [App security overview](#)
- [App code signing process](#)
- [Intro to app security for iOS, iPadOS, and visionOS](#)
- [About App Store security](#)
- [Security of runtime process](#)
- [App protection and app groups](#)
- [Supporting extensions](#)

- [iCloud Private Relay security](#)
- [Passcode security overview](#)
- [Automatic strong passwords](#)
- [Password AutoFill security](#)
- [App access to saved passwords](#)
- [Password security recommendations](#)
- [iCloud Keychain security overview](#)
- [Paying with cards using Apple Pay](#)
- [Apple Card security](#)
- [Apple Cash security](#)
- [Network security overview](#)
- [IPv6 security](#)
- [VPN security](#)
- [Privacy features when connecting to wireless networks](#)
- [Wi-Fi privacy with Apple devices](#)
- [Single Sign-on security](#)
- [Secure device management overview](#)
- [Device management security overview](#)
- [Managed Device Attestation](#)

May 2024

Topics added:

- [Cryptex1 Image4 Manifest Hash \(spih\)](#)
- [Cryptex1 Generation \(stng\)](#)
- [Background Security Improvements](#)
- [BlastDoor for Messages and IDS](#)
- [Lockdown Mode security](#)
- [About App Store security](#)
- [WidgetKit security](#)

Topics updated:

- [Intro to Apple platform security](#)
- [Apple SoC security](#)
- [The Secure Enclave](#)
- [Optic ID, Face ID, Touch ID, passcodes, and passwords](#)
- [Facial matching security](#)
- [Uses for Optic ID, Face ID, and Touch ID](#)
- [Express Cards with power reserve](#)

- [Operating system integrity](#)
- [Activating data connections securely](#)
- [Verifying accessories](#)
- [System security for watchOS](#)
- [Passcodes and passwords](#)
- [Data Protection overview](#)
- [Keybags for Data Protection](#)
- [Protecting keys in alternate boot modes](#)
- [Protecting user data in the face of attack](#)
- [Managing FileVault](#)
- [Intro to app security for iOS, iPadOS, and visionOS](#)
- [Gatekeeper and runtime protection](#)
- [Managed Apple Account security](#)
- [iCloud encryption](#)
- [Account recovery contact security](#)
- [Legacy Contact security](#)
- [iCloud Keychain security overview](#)
- [Secure keychain syncing](#)
- [Escrow security for iCloud Keychain](#)
- [Card provisioning security overview](#)
- [Adding credit or debit cards to Apple Pay](#)
- [Paying with cards using Apple Pay](#)
- [Apple Card security](#)
- [Tap to Pay on iPhone](#)
- [Access using Apple Wallet](#)
- [Access key types](#)
- [IDs in Apple Wallet](#)
- [Security of IDs in Apple Wallet](#)
- [Developer kit security overview](#)
- [Communication security](#)
- [Device management security overview](#)
- [Configuration enforcement](#)

December 2022

Topics added:

- [Advanced Data Protection for iCloud](#)

Topics updated:

- [iCloud security overview](#)
- [iCloud encryption](#)
- [iCloud Backup security](#)
- [Account recovery contact security](#)
- [Legacy Contact security](#)

May 2022

Updated for:

- iOS 15.4
- iPadOS 15.4
- macOS 12.3
- tvOS 15.4
- watchOS 8.5

Topics added:

- [Paired recoveryOS restrictions](#)
- [Local Operating System Version \(losv\)](#)
- [Health sharing](#)
- [Account recovery contact security](#)
- [Legacy Contact security](#)
- [Tap to Pay on iPhone](#)
- [Access using Apple Wallet](#)
- [Access key types](#)
- [IDs in Apple Wallet](#)
- [Siri-enabled HomeKit accessories](#)

Topics updated:

- [Magic Keyboard with Touch ID](#)
- [Optic ID, Face ID, Touch ID, passcodes, and passwords](#)
- [Facial matching security](#)
- [Express Cards with power reserve](#)
- [Boot modes](#)
- [Contents of a LocalPolicy file for a Mac with Apple silicon](#)
- [Signed system volume security](#)
- [System security for watchOS](#)
- [Apple Security Research Device](#)
- [Role of Apple File System](#)
- [Protecting app access to user data](#)
- [Intro to app security for macOS](#)

- [Protecting against malware](#)
- [iCloud security overview](#)
- [Secure keychain syncing](#)
- [Secure iCloud Keychain recovery](#)
- [Paying with cards using Apple Pay](#)
- [Contactless passes in Apple Pay](#)
- [Rendering cards unusable with Apple Pay](#)
- [Apple Card application](#)
- [Apple Cash security](#)
- [Adding transit and eMoney cards to Apple Wallet](#)
- [Secure Apple Messages for Business](#)
- [FaceTime security](#)
- [Car key security](#)
- [Apple Configurator security](#)

Topics removed:

- HomeKit accessories and iCloud

May 2021

Updated for:

- iOS 14.5
- iPadOS 14.5
- macOS 11.3
- tvOS 14.5
- watchOS 7.4

Topics added:

- [Magic Keyboard with Touch ID.](#)
- [Secure intent and connections to the Secure Enclave.](#)
- [CustomOS Image4 Manifest Hash \(coih\).](#)

Topics updated:

- Added two new Express Mode transactions in [Express Cards with power reserve](#).
- Edited [Secure Enclave feature summary](#).
- Software update content added to [Secure Multi-Boot \(smb3\)](#).
- Additional content for [Sealed Key Protection \(SKP\)](#).

February 2021

Updated for:

- iOS 14.3

- iPadOS 14.3
- macOS 11.1
- tvOS 14.3
- watchOS 7.2

Topics added:

- [Memory safe iBoot implementation](#)
- [Boot process](#)
- [Boot modes](#)
- [Startup Disk security policy control](#)
- [LocalPolicy signing-key creation and management](#)
- [Contents of a LocalPolicy file for a Mac with Apple silicon](#)
- [Signed system volume security](#)
- [Apple Security Research Device](#)
- [Password Monitoring](#)
- [IPv6 security](#)
- [Car key security](#)

Topics updated:

- [The Secure Enclave](#)
- [Hardware microphone disconnect](#)
- [recoveryOS and diagnostics environments](#)
- [Direct memory access protections](#)
- [Securely extending the kernel](#)
- [System Integrity Protection](#)
- [System security for watchOS](#)
- [Managing FileVault](#)
- [App access to saved passwords](#)
- [Password security recommendations](#)
- [Apple Cash security](#)
- [Secure Apple Messages for Business](#)
- [Privacy features when connecting to wireless networks](#)
- [Activation Lock security](#)
- [Apple Configurator security](#)

April 2020

Updated for:

- iOS 13.4
- iPadOS 13.4

- macOS 10.15.4
- tvOS 13.4
- watchOS 6.2

Updates:

- iPad microphone disconnect added to [Hardware microphone disconnect](#).
- Data vaults added to [Protecting app access to user data](#).
- Updates to [Managing FileVault](#) and Command-line tools.
- Malware Removal Tool additions in [Protecting against malware](#).
- Updates to [Shared iPad security](#).

December 2019

Merged the iOS Security Guide, macOS Security Overview, and the Apple T2 Security Chip Overview

Updated for:

- iOS 13.3
- iPadOS 13.3
- macOS 10.15.2
- tvOS 13.3
- watchOS 6.1.1

Privacy Controls, Siri and Siri Suggestions, and Safari Intelligent Tracking Prevention have been removed. See <https://www.apple.com/privacy/> for the latest on those features.

May 2019

Updated for iOS 12.3

- Support for TLS 1.3
- Revised description of AirDrop security
- DFU mode and [Recovery mode](#)
- Passcode requirements for accessory connections

November 2018

Updated for iOS 12.1

- Group FaceTime

September 2018

Updated for iOS 12

- Secure Enclave
- OS Integrity Protection

- Express Card with power reserve
- DFU mode and [Recovery mode](#)
- HomeKit TV Remote accessories
- Contactless passes
- Student ID cards
- Siri Suggestions
- Shortcuts in Siri
- Shortcuts app
- User password management
- Screen Time
- Security Certifications and programs

July 2018

Updated for iOS 11.4

- Biometric policies
- HomeKit
- Apple Pay
- Business Chat
- Messages in iCloud
- Apple Business Manager

December 2017

Updated for iOS 11.2

- Apple Pay Cash

October 2017

Updated for iOS 11.1

- Security Certifications and programs
- Touch ID/Face ID
- Shared Notes
- CloudKit end-to-end encryption
- TLS update
- Apple Pay, Paying with Apple Pay on the web
- Siri Suggestions
- Shared iPad

July 2017

Updated for iOS 10.3

- Secure Enclave
- File Data Protection
- Keybags
- Security Certifications and programs
- SiriKit
- HealthKit
- Network Security
- Bluetooth
- Shared iPad
- Lost Mode
- Activation Lock
- Privacy Controls

March 2017

Updated for iOS 10

- System Security
- Data Protection classes
- Security Certifications and programs
- HomeKit, ReplayKit, SiriKit
- Apple Watch
- Wi-Fi, VPN
- Single Sign-on
- Apple Pay, Paying with Apple Pay on the web
- Credit, debit, and prepaid card provisioning
- Safari Suggestions

May 2016

Updated for iOS 9.3

- Managed Apple Account
- Two-factor authentication for Apple Account
- Keybags
- Security Certifications
- Lost Mode, Activation Lock
- Secure Notes
- Apple School Manager

- Shared iPad

September 2015

Updated for iOS 9

- Apple Watch Activation Lock
- Passcode policies
- Touch ID API support
- Data Protection on A8 uses AES-XTS
- Keybags for unattended software update
- Certification updates
- Enterprise app trust model
- Data Protection for Safari bookmarks
- App Transport Security
- VPN specifications
- iCloud Remote Access for HomeKit
- Apple Pay Rewards cards, Apple Pay card issuer's app
- Spotlight on-device indexing
- iOS Pairing Model
- Apple Configurator 2
- Restrictions

© 2026 Apple Inc. All rights reserved.

Use of the “keyboard” Apple logo (Option-Shift-K) for commercial purposes without the prior written consent of Apple may constitute trademark infringement and unfair competition in violation of federal and state laws.

Apple, the Apple logo, AirDrop, AirPlay, App Clips, Apple Books, Apple Card, Apple Cash, Apple Music, Apple Pay, Apple TV, Apple Vision Pro, Apple Wallet, Apple Watch, AppleScript, ARKit, Boot Camp, CarPlay, Face ID, FaceTime, FileVault, Find My, Finder, FireWire, Handoff, HealthKit, HomeKit, HomePod, HomePod mini, iMac, iMac Pro, iMessage, iPad, iPadOS, iPad Air, iPadOS, iPhone, iTunes, Keychain, Lightning, Mac, Mac Catalyst, Mac Pro, MacBook, MacBook Air, MacBook Pro, Macintosh, macOS, Magic Keyboard, Objective-C, Optic ID, OS X, QuickType, Rosetta, Safari, Siri, Siri Remote, SiriKit, Swift, Spotlight, Touch Bar, Touch ID, TrueDepth, tvOS, visionOS, watchOS, and Xcode are trademarks of Apple Inc., registered in the U.S. and other countries and regions.

EnergyKit and SharePlay are trademarks of Apple Inc.

App Store, AppleCare, Apple Messages for Business, CloudKit, iCloud, iCloud Drive, iCloud Keychain, iCloud+, iTunes Store, and Tap to Cash are service marks of Apple Inc., registered in the U.S. and other countries and regions.

Apple
One Apple Park Way
Cupertino, CA 95014
apple.com

IOS is a trademark or registered trademark of Cisco in the U.S. and other countries and is used under license.

The Bluetooth® word mark and logos are registered trademarks owned by Bluetooth SIG, Inc. and any use of such marks by Apple is under license.

Java is a registered trademark of Oracle and/or its affiliates.

UNIX® is a registered trademark of The Open Group.

Other product and company names and logos mentioned herein may be trademarks of their respective companies.

Every effort has been made to ensure that the information in this manual is accurate. Apple is not responsible for printing or clerical errors. Information about products not manufactured by Apple, or independent websites not controlled or tested by Apple, is provided without recommendation or endorsement. Apple assumes no responsibility with regard to the selection, performance, or use of third-party websites or products. Apple makes no representations regarding third-party website accuracy or reliability. [Contact the vendor](#) for additional information.

Some apps are not available in all areas. App and feature availability is subject to change.

028-00856